Ước lượng cao độ F_0 kiểu Praat trên DE10-Standard

Từ file WAV 16 kHz qua HPS, FPGA và trở về HPS, so với Praat 7.0.03 trên laptop
(Intel Quartus Prime — hai đường: IP Catalog và HDL tự viết)

Bản v3 — viết lại theo đề cương chốt v2 (10/11/2026)

Mục lục

Chương 0. Bài toán, mục tiêu so sánh, và Praat thật sự làm gì **3**
0.1. Một file, hai câu trả lời . . . 3
0.2. Ý tưởng của Boersma: Chuẩn hóa bằng tự tương quan của cửa sổ . . . 5
0.3. Tám bước của Praat đúng như mã nguồn và các lựa chọn thiết kế phần cứng . . . 7
0.4. Kiến trúc hệ thống và bản đồ các chương . . . 12
0.5. Các phép đo đánh giá: GPE, VDE, FFE và sai số cent . . . 15
0.6. Ngân sách chu kỳ, hệ số thời gian thực RTF, và mạch phát cặp mẫu phần cứng . . . 17
0.7. Bản đồ kiểm chứng ba tầng và bộ golden v2 . . . 22
Bài tập cuối chương . . . 26

Chương 1. WAV trên HPS và cầu HPS–FPGA, chiều đi và chiều về **32**
1.1. Vấn đề truyền dữ liệu giữa hai miền HPS và FPGA . . . 32
1.2. Đọc tệp WAV đúng chuẩn và tính toán tham số toàn cục trên HPS . . . 33
1.3. Các phương án truyền dữ liệu, đánh đổi và lựa chọn . . . 39
1.4. Đường (A): Xây dựng bộ đệm bằng Intel On-Chip FIFO IP . . . 42
1.5. Đường (B): Hiện thực phần cứng bằng HDL tự viết . . . 44
1.6. Bản đồ thanh ghi và giao tiếp điều khiển lõi xử lý . . . 50
1.7. Phần mềm điều khiển trên HPS và kiểm chứng toàn diện . . . 52

Chương 2. Chuẩn giao tiếp Avalon-ST, ngân sách chu kỳ khi xử lý tệp và kỹ thuật mã hóa FSM **59**
2.1. Chuẩn giao thức Avalon-ST: Cơ chế valid/ready và cạm bẫy readyLatency . . . 59
2.2. Ngân sách chu kỳ khi đầu vào là tệp âm thanh và thực tế thời gian thực . . . 62
2.3. Kỹ thuật mã hóa FSM: Đánh đổi tài nguyên và độ trễ trên kiến trúc Cyclone V . . . 64
2.4. Khối đệm trượt (Skid Buffer): Cắt đứt đường phản áp tới hạn . . . 70
2.5. Kiểm chứng bắt tay luồng và thực nghiệm mã hóa FSM . . . 74

Chương 3. Bước làm mềm mép Nyquist: khối FIR tùy chọn **81**
3.1. Bản chất hiện tượng rò rỉ phổ mép Nyquist và dao động đổi dấu trong tự tương quan . . . 81
3.2. Các phương án, đánh đổi, ta chọn gì, vì sao (có số liệu kiểm chứng) . . . 83
3.3. Đường (A): Khối IP FIR Compiler II của Intel . . . 86
3.4. Đường (B): Khối HDL tự viết: Bộ lọc FIR đối xứng gấp thời gian . . . 88
3.5. Kiểm chứng mô hình song sinh Python và mô phỏng chu kỳ HDL . . . 95

Chương 4. Chia khung (638 mẫu, hop 160) và RAM vòng **103**
4.1. Vì sao 3 chu kỳ, vì sao 10 ms: Cơ sở vật lý và quy chuẩn kích thước cửa sổ Praat . . . 103
4.2. Căn lưới giống Praat: Dẫn xuất t_1, định vị mẫu bắt đầu và cạm bẫy số lẻ . . . 105
4.3. Các phương án lưu trữ và chia khung: Đánh đổi tài nguyên và cấu trúc RAM vòng . . . 107
4.4. Đường (A): Khối IP RAM: 2-PORT của Intel trong Quartus Prime . . . 109
4.5. Đường (B): Khối HDL tự viết: RAM vòng đồng bộ suy diễn . . . 111
4.6. Kiểm chứng mô hình song sinh Python và mô phỏng chu kỳ HDL . . . 114
Bài tập cuối chương . . . 116

Chương 0. Bài toán, mục tiêu so sánh, và Praat thật sự làm gì

▶ MỤC TIÊU HỌC TẬP

- Nắm vững bản chất bài toán ước lượng tần số cơ bản (F_0) và lý do vì sao hai phương pháp trực giác (đếm điểm qua không và tìm đỉnh phổ FFT) thất bại trên tín hiệu tiếng nói thực tế.
- Hiểu rõ nguyên lý cốt lõi của Boersma: chuẩn hóa hàm tự tương quan bằng cách chia cho hàm tự tương quan của cửa sổ Hanning để triệt tiêu độ suy giảm giả tạo ở các độ trễ lớn.
- Nắm tường tận 8 bước thuật toán trong mã nguồn Praat 7.0.03 cùng 10 tham số chuẩn, đối chiếu chính xác giữa các khâu phần mềm của Praat và các khối xử lý phần cứng trên FPGA.
- Hiểu rõ kiến trúc toàn hệ thống: luồng dữ liệu từ tệp WAV qua HPS, cầu nối Lightweight HPS-to-FPGA, lõi tính toán trên FPGA, và đường trả kết quả F0 về HPS để ghi tệp nhị phân.
- Nắm chắc định nghĩa 4 chỉ số sai số chuẩn trong so sánh cao độ: VDE, GPE, FFE và sai số cent mịn, cùng cách tính tay trên từng khung.
- Phân tích ngân sách chu kỳ từ nguyên lý đầu tiên: 50 MHz, 3.125 chu kỳ/mẫu, 500.000 chu kỳ/bước nhảy 10 ms; dẫn xuất chính xác 98.540 phép nhân-cộng MAC và đo thực tế 98.543 chu kỳ/khung, từ đó xác lập hệ số thời gian thực (RTF).
- Hiểu kiến trúc kiểm chứng ba tầng và vai trò của bộ mô hình tương đương (twin) số nguyên `golden_v2/fixed_twin.py` làm đặc tả phần cứng cho toàn bộ dự án.

0.1. Một file, hai câu trả lời

Một tệp âm thanh WAV chuẩn 16 kHz, kênh đơn (mono), định dạng mẫu PCM 16-bit có dấu được đưa vào hệ thống. Trên máy tính cá nhân, phần mềm Praat đọc tệp âm thanh này và xuất ra một đường cao độ F_0 với chu kỳ phân tích 10 ms. Trên bo mạch DE10-Standard, hệ thống vi xử lý HPS (Hard Processor System, Cortex-A9 lõi kép) đọc cùng tệp WAV đó từ thẻ nhớ SD, truyền các mẫu âm thanh qua cầu nối bus sang vùng logic FPGA (Cyclone V 5CSXFC6D6F31C6N). Lõi phần cứng trên FPGA tính toán cao độ từng khung và trả kết quả về HPS dưới dạng tệp nhị phân `pitch_f0.bin` ở định dạng số cố định Q12.4.

Câu hỏi trung tâm mà anh Tú cùng bất kỳ kỹ sư phần cứng nào cũng phải đối mặt là: hai đường cong cao độ đó lệch nhau bao nhiêu, và nguyên nhân vật lý hay số học nào tạo ra sự chênh lệch ở từng mili-giây? Để trả lời thấu đáo câu hỏi này, trước hết ta phải hiểu vì sao việc ước lượng cao độ tiếng nói lại khó, và vì sao những cách tiếp cận trực giác thông thường đều sụp đổ khi áp dụng vào thực tế.

Sự sụp đổ của phương pháp đếm điểm qua không (Zero-Crossing Counting)

Phương pháp đơn giản nhất mà người mới học xử lý tín hiệu thường nghĩ đến là đếm số lần tín hiệu đổi dấu qua mức 0 (zero-crossing) trong một khoảng thời gian, từ đó suy ra tần số. Tuy nhiên, một sự thật vật lý cơ bản là: tiếng nói có thanh (voiced speech) của con người hầu như không bao giờ là một sóng hình sin đơn âm. Khi dây thanh âm rung động, luồng khí đóng mở theo chu kỳ tạo ra dạng sóng có chứa rất nhiều hài âm (harmonics). Nếu hài âm bậc hai ($2f_0$) có biên độ mạnh và lệch pha so với sóng cơ bản, nó sẽ uốn dạng sóng tổng hợp cắt qua trục 0 nhiều lần trong cùng một chu kỳ lặp lại.

Để thấy rõ sự thất bại này bằng số liệu định lượng, ta xét một đoạn tín hiệu tổng hợp được tạo bởi kịch bản kiểm chứng `verify_v3/ch00_naive_methods.py`:

$$x(t) = \sin(2\pi \cdot 100t) - 0{,}75\sin(2\pi \cdot 200t) \tag{1}$$

Tần số cơ bản thực sự là $f_0 = 100{,}0$ Hz, tần số lấy mẫu $f_s = 16.000$ Hz. Một chu kỳ cơ bản kéo dài $T_0 = 1/100 = 10$ ms, tương ứng đúng $P = 160$ mẫu. Nếu tín hiệu là sóng sin thuần, một chu kỳ 160 mẫu sẽ cắt qua trục 0 đúng 2 lần. Nhưng khi kiểm tra trên đoạn tín hiệu dài 3 chu kỳ (480 mẫu), bộ đếm điểm qua không ghi nhận tổng cộng 12 lần đổi dấu, tức trung bình 4 điểm qua không trong mỗi chu kỳ. Tần số ước lượng bị thổi phồng thành:

$$f_{\text{est}} = \frac{12}{2 \times 3 \times 0{,}01\ \text{s}} = 200{,}0\ \text{Hz} \tag{2}$$

Thuật toán bị đánh lừa hoàn toàn bởi hài âm bậc hai, báo sai lệch cao độ gấp đôi (100% lỗi) ngay trên một tín hiệu hoàn toàn tuần hoàn và không hề có nhiễu bên ngoài.

Ví dụ tính tay thu nhỏ. Để nhìn thấy từng bước ở mức mẫu, xét dãy 16 mẫu rời rạc đại diện cho một chu kỳ rút gọn được sinh theo công thức $x_{16}[n] = \text{round}\left(100 \cdot \left(\sin\left(\frac{2\pi n}{16}\right) - 0{,}8\sin\left(\frac{4\pi n}{16}\right)\right)\right)$ (trích xuất trực tiếp từ kịch bản `verify_v3/ch00_naive_methods.py`):

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$x[n]$	0	-18	-9	36	100	149	151	95	0	-95	-151	-149	-100	-36	9	18

Quan sát dãy mẫu: từ $n = 2$ (giá trị -9) sang $n = 3$ ($+36$), tín hiệu đổi dấu lần thứ nhất. Đến $n = 8$, tín hiệu bằng 0 và giảm xuống -95 ở $n = 9$, đổi dấu lần thứ hai. Từ $n = 13$ (-36) sang $n = 14$ ($+9$), tín hiệu lại đổi dấu lần thứ ba. Cùng với điểm 0 ban đầu ở $n = 0$, ta có 4 lần cắt qua 0 trong chỉ một chu kỳ cơ bản. Mạch đếm phần cứng nếu dựa vào hiện tượng này sẽ nhận diện nhầm chu kỳ thành 8 mẫu thay vì 16 mẫu.

Sự sụp đổ của phương pháp chọn đỉnh phổ FFT (FFT Peak Picking)

Cách tiếp cận thứ hai là chuyển tín hiệu sang miền tần số bằng phép biến đổi Fourier nhanh (FFT, Fast Fourier Transform) rồi tìm vị trí bin có biên độ cực đại. Để bám sát ngữ điệu biến đổi liên tục của tiếng nói, ta bắt buộc phải phân tích trên từng khung thời gian ngắn khoảng vài chục mili-giây (thường từ 20 ms đến 40 ms). Tại tần số lấy mẫu $f_s = 16.000$ Hz, một khung 40 ms tương ứng với 640 mẫu (hoặc 638 mẫu theo quy ước làm tròn đối xứng của Praat).

Độ phân giải tần số của phép phân tích FFT bị chặn cứng bởi chiều dài khung N:

$$\Delta f = \frac{f_s}{N} = \frac{16.000}{640} = 25{,}00 \text{ Hz} \quad (\text{với } N = 638, \Delta f = \frac{16.000}{638} \approx 25{,}08 \text{ Hz}) \tag{3}$$

Khoảng cách giữa hai vạch phổ kề nhau lên tới 25 Hz. Hãy xét một giọng nam trầm có tần số cơ bản thực là $f_0 = 112{,}5$ Hz. Tần số này rơi vào chính giữa bin 4 ($4 \times 25 = 100$ Hz) và bin 5 ($5 \times 25 = 125$ Hz). Bộ chọn đỉnh phổ dù lý tưởng nhất cũng chỉ có thể chỉ định bin 4 hoặc bin 5, gây ra sai số tuyệt đối ít nhất là $12{,}5$ Hz.

Trong âm nhạc học và ngữ âm học, tai người cảm nhận cao độ theo hàm logarit của tần số. Đơn vị đo độ chênh lệch cao độ chuẩn là **cent**, trong đó 1 quãng tám (tần số gấp đôi) tương ứng với 1.200 cent, và 1 nửa cung (semitone) tương ứng với 100 cent:

$$\Delta\text{cent} = 1.200 \cdot \left|\log_2\left(\frac{f_{\text{est}}}{f_0}\right)\right| \tag{4}$$

Áp dụng cho trường hợp chọn nhầm bin 100 Hz khi tần số thực là 112,5 Hz:

$$\Delta\text{cent} = 1.200 \cdot \left|\log_2\left(\frac{100{,}0}{112{,}5}\right)\right| \approx 203{,}91 \text{ cent} \tag{5}$$

(kết quả được kiểm chứng trong `verify_v3/ch00_naive_methods.log`). Sai số tương ứng 204 cent tương đương với hai nửa cung (một cung nguyên) trong âm nhạc. Một bộ ước lượng F_0 với sai số thô như vậy hoàn toàn không đáp ứng được yêu cầu trong các ứng dụng phân tích ngữ điệu hoặc đối sánh khoa học. Muốn giảm Δf xuống dưới 1 Hz, FFT cần cửa sổ dài hơn 1 giây, nhưng trong 1 giây đó người nói đã phát âm qua 3 đến 5 âm tiết khác nhau, tính dừng của tín hiệu bị phá vỡ hoàn toàn.

Bản chất của phép tự tương quan (Autocorrelation)

Để khắc phục triệt để hai nhược điểm trên, ta quay về nguyên lý đầu tiên: **một tín hiệu tuần hoàn luôn có hình dạng giống hệt chính nó khi bị dịch trễ đi đúng một chu kỳ cơ bản**. Phép đo mức độ tương đồng giữa tín hiệu $x[n]$ và bản sao dịch trễ của nó $x[n + \tau]$ ở độ trễ τ (lag) được định nghĩa qua hàm tự tương quan ngắn hạn:

$$R[\tau] = \sum_{n=0}^{N-1-\tau} x[n]x[n+\tau] \tag{6}$$

Ví dụ tính tay số nguyên thu nhỏ. Xét một tín hiệu rời rạc tuần hoàn lý tưởng với chu kỳ cơ bản $P = 3$ mẫu, độ dài khung $N = 9$ mẫu:

$$x = [3, -1, -2,\ 3, -1, -2,\ 3, -1, -2] \tag{7}$$

Ta tính trực tiếp giá trị tích vô hướng $R[\tau]$ theo từng bước cho các độ trễ từ $\tau = 0$ đến $\tau = 5$ (trích xuất từ kịch bản kiểm chứng `verify_v3/ch00_naive_methods.py`):

- Tại $\tau = 0$: lấy 9 cặp mẫu tự nhân với chính nó:

$$\begin{aligned} R[0] &= 3^2 + (-1)^2 + (-2)^2 + 3^2 + (-1)^2 + (-2)^2 + 3^2 + (-1)^2 + (-2)^2 \\ &= 9 + 1 + 4 + 9 + 1 + 4 + 9 + 1 + 4 = 42 \end{aligned}$$

- Tại $\tau = 1$: nhân 8 cặp mẫu kề nhau:

$$\begin{aligned} R[1] &= 3(-1) + (-1)(-2) + (-2)3 + 3(-1) + (-1)(-2) + (-2)3 + 3(-1) + (-1)(-2) \\ &= -3 + 2 - 6 - 3 + 2 - 6 - 3 + 2 = -15 \end{aligned}$$

- Tại $\tau = 2$: nhân 7 cặp mẫu cách nhau 2 vị trí:

$$\begin{aligned} R[2] &= 3(-2) + (-1)3 + (-2)(-1) + 3(-2) + (-1)3 + (-2)(-1) + 3(-2) \\ &= -6 - 3 + 2 - 6 - 3 + 2 - 6 = -20 \end{aligned}$$

- Tại $\tau = 3$: nhân 6 cặp mẫu cách nhau đúng một chu kỳ $P = 3$:

$$\begin{aligned} R[3] &= 3(3) + (-1)(-1) + (-2)(-2) + 3(3) + (-1)(-1) + (-2)(-2) \\ &= 9 + 1 + 4 + 9 + 1 + 4 = 28 \end{aligned}$$

- Tại $\tau = 4$: nhân 5 cặp mẫu, thu được $R[4] = -8$.
- Tại $\tau = 5$: nhân 4 cặp mẫu, thu được $R[5] = -13$.

Chỗ dễ vướng: Vì sao $R[3] = 28$ lại bé hơn $R[0] = 42$ dù tín hiệu tuần hoàn lý tưởng? Đây là bẫy tư duy kinh điển mà người thiết kế phần cứng hay gặp. Ở độ trễ $\tau = 3$, số cặp mẫu còn lại để nhân với nhau chỉ là $N - \tau = 9 - 3 = 6$ cặp, trong khi ở $\tau = 0$ ta cộng dồn tới 9 cặp. Nếu ta tính giá trị trung bình không chệch bằng cách chia tổng tích lũy cho số cặp mẫu tham gia:

$$\frac{R[0]}{9 - 0} = \frac{42}{9} \approx 4{,}67 \tag{8}$$

$$\frac{R[3]}{9 - 3} = \frac{28}{6} \approx 4{,}67 \tag{9}$$

Hai con số hoàn toàn trùng khớp! Đỉnh tương quan tại $\tau = 3$ đã chỉ ra chính xác chu kỳ cơ bản của tín hiệu là 3 mẫu mà không bị nhiễu hài âm đánh lừa như phương pháp đếm điểm qua 0, cũng không bị lượng tử hóa thô thiển như phương pháp chọn đỉnh FFT.

Tóm lại: Đếm điểm qua không thất bại vì hài âm bậc hai tạo thêm điểm cắt giả, lấy đỉnh FFT thất bại vì độ phân giải bin thô tới 204 cent ở 112,5 Hz (khi làm tròn về bin 100 Hz); phương pháp tự tương quan đo mức độ tương đồng theo miền thời gian, mở ra con đường xác định chu kỳ cơ bản chính xác và bền vững trước cấu trúc hài âm phức tạp của tiếng nói.

0.2. Ý tưởng của Boersma: Chuẩn hóa bằng tự tương quan của cửa sổ

Khi áp dụng phương pháp tự tương quan lên các đoạn tiếng nói thực tế, một bài toán kỹ thuật mới lập tức nảy sinh: ta không thể tính toán trên toàn bộ tệp âm thanh dài hàng phút mà bắt buộc phải cắt tín hiệu thành từng khung ngắn hữu hạn (khoảng 40 ms) để theo dõi sự biến đổi cao độ theo thời gian. Việc cắt khung bằng một hàm chữ nhật đột ngột (rectangular window) sẽ tạo ra các điểm gián đoạn sắc nhọn ở hai biên khung, dẫn đến hiện tượng rò rỉ năng lượng phổ (spectral leakage) nghiêm trọng. Để khắc phục hiện tượng này, giải pháp kinh điển là nhân khung tín hiệu với một hàm cửa sổ êm dịu, tiêu biểu là cửa sổ Hanning (hoặc Hamming), có biên độ vuốt dần về 0 ở hai đầu mút.

Tuy nhiên, việc nhân cửa sổ lại kéo theo một hậu quả tiêu cực khác: các mẫu âm thanh nằm xa tâm khung bị suy giảm biên độ mạnh mẽ. Khi ta tính hàm tự tương quan trên tín hiệu đã nhân cửa sổ $x_w[n] = x[n] \cdot w[n]$:

$$R_{xw}[\tau] = \sum_{n=0}^{N-1-\tau} x_w[n] x_w[n+\tau] = \sum_{n=0}^{N-1-\tau} x[n] w[n] x[n+\tau] w[n+\tau] \tag{10}$$

Độ trễ τ càng lớn thì khoảng chồng lấn giữa $w[n]$ và $w[n+\tau]$ càng thu hẹp, và tích $w[n]w[n+\tau]$ càng bị đè bẹp về gần 0. Đồ thị $R_{xw}[\tau]$ xuất hiện một sườn dốc suy thoái nhân tạo theo τ. Hệ quả là đỉnh tương quan ứng với chu kỳ cơ bản thực sự T_0 của một giọng nam trầm (nằm ở các độ trễ lớn, ví dụ $\tau \approx 160$ mẫu tại 100 Hz) bị lùn đi nghiêm trọng. Nếu dùng hàm tương quan chuẩn hóa thông thường $R_{xw}[\tau]/R_{xw}[0]$, giá trị đỉnh này thường xuyên rơi xuống dưới ngưỡng hữu thanh (voicing threshold mặc định của Praat là 0,45) khiến khung âm thanh bị nhận diện nhầm thành vô thanh, hoặc bị lép vế trước một đỉnh phụ ở độ trễ nhỏ hơn ($\tau = T_0/2$), gây ra lỗi kinh điển là **nhân đôi quãng tám (octave doubling)**.

Cơ chế chia cho tự tương quan cửa sổ $R_w[\tau]$

Đóng góp đột phá của Paul Boersma (1993) nằm ở nhận định giải tích: hàm tự tương quan của tín hiệu đã nhân cửa sổ xấp xỉ bằng tích giữa hàm tự tương quan thực sự của tín hiệu $R_x[\tau]$ với hàm tự tương quan của chính hàm cửa sổ $R_w[\tau]$:

$$R_{xw}[\tau] \approx R_x[\tau] \cdot R_w[\tau] \tag{11}$$

Trong đó hàm tự tương quan của cửa sổ được định nghĩa hoàn toàn tương tự:

$$R_w[\tau] = \sum_{n=0}^{N-1-\tau} w[n]w[n+\tau] \tag{12}$$

Từ mối liên hệ nhân này, Boersma đề xuất thuật toán chuẩn hóa: muốn tìm lại hàm tự tương quan chuẩn hóa không thiên vị $r[\tau]$ của bản thân tín hiệu gốc (với giá trị lý tưởng $r[0] = 1$ và $r[T_0] = 1$ đối với tín hiệu tuần hoàn lý tưởng), ta chỉ cần lấy $R_{xw}[\tau]$ chia cho $R_w[\tau]$, rồi chuẩn hóa tỉ lệ theo năng lượng tại $\tau = 0$:

$$r[\tau] = \frac{R_{xw}[\tau]/R_w[\tau]}{R_{xw}[0]/R_w[0]} = \frac{R_{xw}[\tau]}{R_w[\tau]} \cdot \frac{R_w[0]}{R_{xw}[0]} \tag{13}$$

Phép chia này đóng vai trò như một bộ bù độ dốc: vì $R_w[\tau]$ giảm dần về 0 khi τ tăng theo đúng quy luật hình học của cửa sổ, việc chia cho $R_w[\tau]$ sẽ nâng bổng các đỉnh ở độ trễ xa lên đúng biên độ thực sự của chúng, triệt tiêu hoàn toàn sự thiên lệch về phía các độ trễ nhỏ.

Vì sao phép xấp xỉ $R_{xw}[\tau] \approx R_x[\tau] \cdot R_w[\tau]$ lại đúng?

Để trả lời thuyết phục người kỹ sư phần cứng, ta xét bản chất vật lý của hai chuyển động: dao động nhanh của dây thanh âm và sự biến thiên chậm của đường bao cửa sổ.

Trong thuật toán Praat, chiều dài khung phân tích được ấn định bằng đúng 3 chu kỳ của tần số thấp nhất (pitch floor $f_{\text{min}} = 75$ Hz):

$$T_{\text{window}} = \frac{3}{f_{\text{min}}} = \frac{3}{75} = 0{,}04 \text{ s} = 40 \text{ ms} \quad (N = 638 \text{ mẫu tại } f_s = 16 \text{ kHz}) \tag{14}$$

Trong khi đó, tần số cơ bản của người nói dao động từ 75 Hz đến 600 Hz, tương ứng với chu kỳ cơ bản T_0 từ 26 mẫu (600 Hz) đến 213 mẫu (75 Hz). Điều này có nghĩa là bên trong một khung 638 mẫu, tín hiệu tiếng nói thực hiện từ 3 đến hơn 24 chu kỳ dao động tuần hoàn nhanh.

Đường bao cửa sổ Hanning $w[n]$ là một hàm cosin trơn dốc êm dịu trải dài trên toàn bộ 638 mẫu. Trong phạm vi một chu kỳ đơn lẻ T_0, giá trị của $w[n]$ và $w[n+\tau]$ gần như là một hằng số cục bộ. Khi ta lấy tổng tích lũy $\sum x[n]x[n+\tau] \cdot (w[n]w[n+\tau])$, đại lượng dao động nhanh $x[n]x[n+\tau]$ tạo ra giá trị kỳ vọng trung bình tuần hoàn $R_x[\tau]$, trong khi đại lượng biến thiên chậm $w[n]w[n+\tau]$ có thể đưa ra ngoài dấu tích phân cục bộ dưới dạng trọng số bao $R_w[\tau]$. Phép nhân chập giữa phổ tín hiệu hẹp và phổ cửa sổ rộng trong miền tần số tương ứng với phép nhân đường bao trực tiếp trong miền thời gian.

Ví dụ tính tay thu nhỏ: Sóng sin thuần qua cửa sổ Hanning 8 mẫu

Để minh chứng tường minh bằng số học cụ thể, ta xét một thí nghiệm mô phỏng thu nhỏ được thực thi bởi kịch bản `verify_v3/ch00_norm_sine.py`. Ta sử dụng một sóng sin tuần hoàn lý tưởng có chu kỳ $P = 4$ mẫu (tương ứng tần số $f_0 = f_s/4 = 4.000$ Hz), chiều dài phân tích $N = 8$ mẫu:

$$x[n] = \sin\left(\frac{2\pi n}{4}\right) = [0{,}000,\ 1{,}000,\ 0{,}000,\ -1{,}000,\ 0{,}000,\ 1{,}000,\ 0{,}000,\ -1{,}000] \tag{15}$$

Hàm cửa sổ Hanning 8 mẫu của Praat tuân thủ công thức $N+1$ (được định nghĩa trong mã nguồn Praat `Sound_to_Pitch.cpp` dòng 422–425):

$$w[i] = 0{,}5 - 0{,}5\cos\left(\frac{2\pi i}{N+1}\right), \quad i = 1, \ldots, N \quad (i = n+1) \tag{16}$$

Với $N = 8$, các mẫu cửa sổ tính ra là:

$$w = [0{,}116978,\ 0{,}413176,\ 0{,}750000,\ 0{,}969846,\ 0{,}969846,\ 0{,}750000,\ 0{,}413176,\ 0{,}116978]$$

Tín hiệu sau khi nhân cửa sổ $x_w[n] = x[n] \cdot w[n]$ là:

$$x_w = [0{,}0000,\ 0{,}4132,\ 0{,}0000,\ -0{,}9698,\ -0{,}0000,\ 0{,}7500,\ 0{,}0000,\ -0{,}1170]$$

Bảng 1 liệt kê kết quả tính tay chi tiết hàm tự tương quan của cửa sổ $R_w[\tau]$, hàm tự tương quan của tín hiệu nhân cửa sổ $R_{xw}[\tau]$, hệ số tương quan chưa chuẩn hóa $r_{\text{unnorm}}[\tau] = R_{xw}[\tau]/R_{xw}[0]$, và hệ số chuẩn hóa Boersma $r_{\text{Boersma}}[\tau]$ cho các độ trễ từ $\tau = 0$ đến $\tau = 4$ (trích xuất nguyên văn từ tệp nhật ký `verify_v3/ch00_norm_sine.log`).

Bảng 1: Đối chiếu giá trị tự tương quan trước và sau khi chuẩn hóa cửa sổ Boersma trên sóng sin thuần ($N = 8, P = 4$)

Độ trễ τ	$R_w[\tau]$	$R_{xw}[\tau]$	$r_{\text{unnorm}}[\tau]$	$r_{\text{Boersma}}[\tau]$	Đánh giá vật lý
0	3,375000	1,687500	1,000000	1,000000	Gốc chuẩn hóa năng lượng
1	3,111800	0,000000	0,000000	0,000000	Trực giao ở góc pha $90°$
2	2,431670	$-1{,}215835$	$-0{,}720495$	$-1{,}000000$	Ngược pha hoàn toàn ($180°$)
3	1,590835	0,000000	0,000000	0,000000	Trực giao ở góc pha $270°$
4	**0,846665**	**0,423332**	**0,250864**	**1,000000**	**Chu kỳ cơ bản** $P = 4$

Đọc bảng và kết luận định lượng. Hãy nhìn thẳng vào dòng $\tau = 4$, chính là chu kỳ lặp lại thực tế của sóng sin:

- Nếu **không chuẩn hóa cửa sổ**, hệ số tương quan chỉ đạt $r_{\text{unnorm}}[4] = 0{,}250864 \approx 0{,}251$. Tín hiệu tuần hoàn lý tưởng nhưng giá trị đỉnh tương quan bị sụt giảm tới gần 75% năng lượng! Với ngưỡng voicing mặc định của Praat là 0,45, thuật toán sẽ kết luận ngay khung này là *vô thanh* (unvoiced) do $0{,}251 < 0{,}45$.
- Ngược lại, khi áp dụng **chuẩn hóa Boersma**: ta lấy $R_{xw}[4]/R_w[4] = 0{,}423332/0{,}846665 = 0{,}500000$. Chia cho tỉ số tại gốc $R_{xw}[0]/R_w[0] = 1{,}687500/3{,}375000 = 0{,}500000$, ta thu được kết quả chính xác:

$$r_{\text{Boersma}}[4] = \frac{0{,}500000}{0{,}500000} = 1{,}000000 \tag{17}$$

Giá trị 1,000000 phản ánh đúng tính tuần hoàn của sóng sin, không hề bị ảnh hưởng bởi việc cửa sổ bị thót hẹp ở hai đầu.

Tóm lại: Cửa sổ cắt khung làm suy hao giả tạo năng lượng ở các độ trễ lớn khiến đỉnh cao độ thật bị lùn đi; ý tưởng cốt lõi của Boersma là chia hàm tự tương quan của tín hiệu cho hàm tự tương quan của cửa sổ $R_{xw}[\tau]/R_w[\tau]$, khôi phục trọn vẹn độ mạnh của chu kỳ cơ bản và triệt tiêu định kiến thiên lệch về các độ trễ nhỏ.

0.3. Tám bước của Praat đúng như mã nguồn và các lựa chọn thiết kế phần cứng

Mọi so sánh định lượng giữa bộ đo trên FPGA và phần mềm Praat chỉ có giá trị khoa học khi và chỉ khi ta hiểu chính xác từng dòng mã nguồn của Praat thực sự làm gì. Để tránh các sai lệch thường gặp về kích thước cửa sổ, số lượng tham số hay phạm vi tính toán, tài liệu này đối chiếu trực tiếp với mã nguồn chuẩn của bản phát hành **Praat 7.0.03** (ghim làm mốc tham chiếu xuyên suốt), cụ thể là các tệp `fon/Sound_to_Pitch.cpp`, `fon/Pitch.cpp`, `fon/Sampled.cpp` và `fon/praat_Sound.cpp`.

Tám bước thuật toán theo đúng thứ tự thực thi trong mã nguồn Praat

Thuật toán ước lượng cao độ bằng tự tương quan thô của Praat vận hành tuần tự qua tám bước sau:

Bước 1: Tính đỉnh biên độ toàn cục (`globalPeak`). Trong tệp `Sound_to_Pitch.cpp` (dòng 375–385), Praat quét qua toàn bộ tệp âm thanh đầu vào, tính giá trị trung bình DC của cả tệp, rồi xác định đỉnh biên độ cực đại:

$$\text{globalPeak} = \max_t |x(t) - \bar{x}_{\text{file}}| \tag{18}$$

Giá trị này phản ánh mức biên độ đỉnh tuyệt đối của người nói, được giữ nguyên làm mốc quy chiếu xuyên suốt để thiết lập ngưỡng yên lặng động cho từng khung ở Bước 8.

Bước 2: Phân bố lưới khung thời gian (`Sampled_shortTermAnalysis`). Trong tệp `Sampled.cpp` (hàm `Sampled_getWindowSamples` từ dòng 53, cùng hàm nội tuyến `Sampled_xToLowIndex` trong `Sampled.h` dòng 34; dòng 71–82 định nghĩa `Sampled_shortTermAnalysis`), Praat không đặt khung đầu tiên tại mẫu 0 mà căn các khung đối xứng quanh tâm của tệp âm thanh. Với độ dài tệp T, số khung phân tích được tính bởi công thức làm tròn sàn $\lfloor (T - 0{,}04)/0{,}01 \rfloor + 1$. Vị trí tâm của khung đầu tiên t_1 phụ thuộc vào độ dài tổng thể của tệp. Vì vậy, khung thứ nhất không bắt đầu ở mẫu 0 mà lệch đi một số mẫu nhất định; lõi phần cứng FPGA bắt buộc phải nhận tọa độ t_1 và chỉ số mẫu bắt đầu do bộ vi xử lý HPS tính toán để đảm bảo hai bên phân tích trên đúng từng mẫu âm thanh.

Bước 3: Trừ trung bình cục bộ trên 426 mẫu. Trong tệp `Sound_to_Pitch.cpp` (dòng 56–66), Praat tính giá trị trung bình cục bộ của tín hiệu trên đoạn dài đúng hai chu kỳ dài nhất về hai phía tâm khung ("one longest period to both sides"). Tại $f_{\text{min}} = 75$ Hz, chu kỳ dài nhất là $T_{0,\text{max}} = 16.000/75 = 213{,}33$ mẫu, làm tròn thành 213 mẫu. Do đó, trung bình cục bộ được tính trên đúng $2 \times 213 = 426$ mẫu quanh tâm khung ($106 \ldots 531$), hoàn toàn không phải trên toàn bộ 638 mẫu. Sau đó, tại dòng 68–84, Praat sao chép khung và trừ trung bình cục bộ này trước khi nhân với cửa sổ Hanning nhằm triệt tiêu triệt để thành phần một chiều (DC offset) cục bộ, ngăn ngừa sai lệch hàm tự tương quan.

Bước 4: Nhân cửa sổ Hanning $N + 1$ với $N = 638$ mẫu. Trong tệp `Sound_to_Pitch.cpp` (dòng 335–339 và dòng 422–425), kích thước cửa sổ phân tích được làm tròn đối xứng: từ 640 mẫu lý thuyết, Praat lấy nửa cửa sổ half $= 640/2 - 1 = 319$ mẫu, suy ra chiều dài cửa sổ thực tế là $N = 2 \times 319 = 638$ mẫu. Tín hiệu sau khi trừ trung bình cục bộ được nhân với hàm cửa sổ Hanning dạng $N + 1$:

$$w[i] = 0{,}5 - 0{,}5 \cos\left(\frac{2\pi i}{639}\right), \quad i = 1, \ldots, 638 \tag{19}$$

Bước 5: Tính đỉnh biên độ cục bộ (`localPeak`). Trong tệp `Sound_to_Pitch.cpp` (dòng 87–102), Praat tìm giá trị biên độ đỉnh tuyệt đối lớn nhất của tín hiệu đã nhân cửa sổ bên trong khoảng cách nửa chu kỳ dài nhất ± 107 mẫu quanh tâm khung ($j \in [212, 425]$):

$$\text{localPeak} = \max_{j \in [212,425]} |x_w[j]| \tag{20}$$

Tỉ số cường độ tương đối intensity được xác lập qua biểu thức (dòng 102):

$$\text{intensity} = \begin{cases} 1{,}0, & \text{nếu localPeak} > \text{globalPeak}, \\ \dfrac{\text{localPeak}}{\text{globalPeak}}, & \text{ngược lại}, \end{cases} \tag{21}$$

phục vụ trực tiếp cho việc xác định ngưỡng chuyển đổi có thanh/vô thanh.

Bước 6: Tự tương quan và chuẩn hóa Boersma bằng FFT 1024 điểm.

Trong tệp `Sound_to_Pitch.cpp` (dòng 147–157), Praat tính hàm tự tương quan khung $R_{xw}[\tau]$ bằng phép biến đổi FFT 1024 điểm qua hàm `NUMfft_forward` (dòng 150, với độ dài nhân chập $638 \times 1{,}5 = 957$ mẫu, chọn lũy thừa 2 gần nhất là 1024).

Đồng thời, tại dòng 430–445, Praat biến đổi FFT và tính tự tương quan của chính cửa sổ $R_w[\tau]$ (mảng `windowR`), chuẩn hóa mảng này và xác định độ trễ tối đa `brent_ixmax` $= 319$ mẫu. Sau đó, tại dòng 163–165, kết quả được chuẩn hóa và chia theo công thức Boersma:

$$r[i] = \frac{ac[i+1]}{ac[1] \cdot \texttt{windowR}[i+1]}$$

Bước 7: Tìm kiếm ứng viên cao độ và tinh chỉnh sinc70 + Brent. Trong tệp `Sound_to_Pitch.cpp` (dòng 185–200), Praat quét tìm các đỉnh cực đại cục bộ thỏa mãn đồng thời: $r[i] > 0{,}5 \times$ voicingThreshold, $r[i] > r[i-1]$ và $r[i] \geq r[i+1]$ với $i \in [2, \text{maximumLag} - 1]$. Với $N = 638$, maximumLag $= \lfloor 638/3 \rfloor + 2 = 214$, nên Praat quét qua $i = 2 \ldots 213$. Các độ trễ $i \in [2, 26]$ vẫn có thể trở thành ứng viên cực đại, nhưng do chúng tương ứng với tần số $f \geq 600$ Hz (vượt quá trần `pitch ceiling` 600 Hz), bộ dò đường Viterbi coi chúng là vô thanh (`Pitch.cpp` dòng 558, hàm `Pitch_util_frequencyIsVoiced`). Trong khi đó, mạch số FPGA của ta quét trực tiếp trên dải $[27, 213]$ (với độ trễ 26 và 214 đóng vai trò làm hàng xóm biên cho nội suy parabol 3 điểm; xem `golden_v2/README` độ lệch 6). Vị trí đỉnh ban đầu được ước lượng bằng nội suy parabol (dòng 195–200), nhưng sau đó **luôn luôn được tinh chỉnh chính xác bằng phép nội suy hàm sinc 70 điểm kết hợp thuật toán tìm kiếm Brent** (dòng 254–270, hàm `NUMimproveMaximum` ở dòng 258), bất kể chế độ `Very accurate` bật hay tắt. Độ mạnh (`strength`) của đỉnh nếu vượt quá 1 sẽ được phản xạ thành $1/\text{strength}$. Praat lưu lại tối đa 15 ứng viên có độ mạnh lớn nhất trong mỗi khung.

Bước 8: Thuật toán quy hoạch động Viterbi (Path Finder). Trong tệp `Pitch.cpp` (dòng 550–565), một bộ tìm đường tối ưu toàn cục (Viterbi) xâu chuỗi các ứng viên qua tất cả các khung trong tệp âm thanh. Ứng viên vô thanh có độ mạnh phụ thuộc vào cường độ biên độ:

$$\text{strength}_{\text{unvoiced}} = 0{,}45 + \max\left(0,\ 2 - \frac{\text{intensity}}{0{,}03/1{,}45}\right) \tag{22}$$

Ứng viên hữu thanh bị trừ đi một lượng chi phí quãng tám:

$$\text{octave_cost} = 0{,}01 \cdot \log_2\left(\frac{\text{Pitch ceiling}}{f}\right) \tag{23}$$

Đặc biệt, trong tệp `Pitch.cpp` (dòng 543–545), Praat nhân hệ số tỉ lệ thời gian timeStepCorrection $= 0{,}01/\Delta t$ vào hai chi phí chuyển tiếp: chi phí nhảy quãng tám (`octave-jump cost` $= 0{,}35 \cdot (0{,}01/\Delta t)$) và chi phí chuyển đổi hữu thanh/vô thanh (`voiced/unvoiced cost` $= 0{,}14 \cdot (0{,}01/\Delta t)$). Với bước nhảy mặc định $\Delta t = 10$ ms $= 0{,}01$ s, hệ số tỉ lệ này đúng bằng $1{,}0$; đối với các bước nhảy tổng quát khác, công thức này bảo đảm chi phí chuyển tiếp luôn tương thích theo thang thời gian vật lý.

Mười tham số chuẩn của Praat và hành vi ở $f_s = 16$ kHz

Lệnh chuẩn trong giao diện Praat là `Sound: To Pitch (raw autocorrelation)...` tiếp nhận đúng **10 tham số** cấu hình:

- **Time step** (mặc định 0,0 s, tự động): Praat tự tính $\Delta t = 0{,}75/f_{\text{min}} = 0{,}75/75 = 10$ ms, tương ứng bước nhảy khung (`hop`) đúng 160 mẫu tại 16 kHz.
- **Pitch floor** (mặc định 75 Hz): Đặt giới hạn tần số thấp nhất, quyết định cửa sổ phân tích dài 3 chu kỳ ($3/75 = 40$ ms $= 638$ mẫu) và độ trễ tối đa quét là $\tau_{\text{max}} = 214$ mẫu.
- **Pitch ceiling** (mặc định 600 Hz): Đặt giới hạn tần số cao nhất, quyết định độ trễ tối thiểu quét là $\tau_{\text{min}} = \lfloor 16000/600 \rfloor = 26$ mẫu; mọi ứng viên có tần số vượt 600 Hz bị coi là vô thanh.
- **Max. number of candidates** (mặc định 15): Số lượng ứng viên cao độ tối đa được giữ lại ở mỗi khung cho bộ giải thuật Viterbi; mã nguồn tự động ép giá trị này tối thiểu bằng $\lfloor 600/75 \rfloor = 8$.
- **Very accurate** (mặc định `off`): Khi tắt, thuật toán dùng cửa sổ Hanning 3 chu kỳ và nội suy sinc 70 điểm; nếu bật, Praat chuyển sang cửa sổ Gaussian 6 chu kỳ và nội suy sinc 700 điểm.
- **Silence threshold** (mặc định 0,03): Ngưỡng tỉ số biên độ đỉnh tuyệt đối cục bộ so với đỉnh toàn cục (tỉ số localPeak/globalPeak, không phải đo bằng năng lượng RMS); khi intensity giảm xuống dưới ngưỡng, Praat kích hoạt vùng dốc phi tuyến nhằm nâng cao độ mạnh của ứng viên vô thanh qua công thức $0{,}45 + \max(0,\ 2 - \text{intensity}/(0{,}03/1{,}45))$ (`Pitch.cpp` dòng 553–555), giúp ứng viên vô thanh thắng thế trong các khung thì thầm hoặc yên lặng.
- **Voicing threshold** (mặc định 0,45): Ngưỡng sức mạnh tương quan để ứng viên có thanh được chấp nhận cạnh tranh với ứng viên vô thanh.
- **Octave cost** (mặc định 0,01): Trọng số phạt theo logarit tần số, ưu tiên nhẹ cho các tần số cao hơn để chống lại xu hướng rơi vào bẫy chia đôi quãng tám.

- **Octave-jump cost** (mặc định 0,35): Hệ số phạt khi đường cao độ nhảy vọt đột ngột quá một quãng tám giữa hai khung liên tiếp trong thuật toán Viterbi (nhân tỉ lệ $0{,}01/\Delta t$).
- **Voiced/unvoiced cost** (mặc định 0,14): Hệ số phạt khi có sự chuyển trạng thái giữa có thanh và vô thanh giữa hai khung kề nhau trong thuật toán Viterbi (nhân tỉ lệ $0{,}01/\Delta t$).

Lưu ý về bước lọc mềm mép Nyquist (Soft Upsampling): Trong bài báo gốc của Paul Boersma năm 1993, tác giả có đề xuất một bước tiền xử lý gọi là "soft upsampling": làm mềm phổ biên độ dốc tuyến tính về 0 từ 95% đến 100% tần số Nyquist (tức từ 7,6 kHz đến 8,0 kHz ở $f_s = 16$ kHz) bằng biến đổi FFT nhằm giảm hiện tượng gợn sóng bậc cao trong hàm tương quan. Tuy nhiên, khi kiểm tra kỹ lưỡng toàn bộ mã nguồn của Praat 6.4.12 và Praat 7.0.03, **bước xử lý này hoàn toàn không tồn tại trong đường thực thi `raw autocorrelation`**. Tài liệu hướng dẫn sử dụng hiện hành của Praat cũng đã loại bỏ bước này. Do đó, trong thiết kế phần cứng ở Chương 3, ta bố trí một khối lọc FIR 31-tap tùy chọn nhưng đặt công tắc mặc định là **bypass** khi tiến hành đo đạc đối sánh với Praat để tránh tự tạo ra sai lệch không đáng có.

Bảng đối chiếu thiết kế: Praat phần mềm so với FPGA phần cứng

Bảng 2 tổng hợp toàn diện các quyết định kiến trúc: khâu nào ta bám sát Praat nguyên bản, khâu nào ta chủ động thay thế bằng giải thuật thân thiện với phần cứng, lý do kỹ thuật và tác động định lượng đến phép so sánh.

Bảng 2: Đối chiếu toàn diện giữa thuật toán Praat 7.0.03 và kiến trúc phần cứng FPGA trên DE10-Standard

Khâu xử lý	**Praat 7.0.03 (SW)**	**Thiết kế FPGA (HW)**	**Lý do kỹ thuật phần cứng**	**Ảnh hưởng tới phép so sánh**
1. Cửa sổ và lưới thời gian	$N = 638$ mẫu, hop 160, căn đối xứng cả tệp	Bám đúng $N = 638$, hop 160, nhận lưới thời gian từ HPS	Cùng lưới là điều kiện tiên quyết bắt buộc để so sánh từng khung	Sai lệch thời gian bằng 0; khung phân tích trùng khít hoàn toàn.
2. Trung bình cục bộ	Lấy trên 426 mẫu ($106 \dots 531$), số thực double	426 mẫu, nhân hằng số nghịch đảo $M = 10.082.083$, dịch 32 bit	Tránh bộ chia số nguyên phức tạp, vừa vặn trong bộ nhân cứng DSP 25×24	Trừ trung bình khớp bit chính xác (bit-exact) với mô hình số nguyên.
3. Tính tự tương quan	FFT 1024 điểm, số thực double 64-bit	Nhân-cộng MAC trực tiếp, toán hạng 17-bit, cần 43 bit, dùng 48 bit	1 khối DSP 18×18 chạy nối tiếp 98.543 nhịp, tiết kiệm hàng nghìn ALM so với lõi FFT	Khớp về mặt toán học; mô hình số nguyên twin vs R2 đạt sai số cent mịn trung bình $0{,}21$ cent ($p_{95} = 0{,}48$ cent) trên tín hiệu sạch, và $3{,}07$ cent ($p_{95} = 9{,}78$ cent) trên tín hiệu ồn 5 dB.
4. Chi phí quãng tám (`octave cost`)	Trừ $0{,}01 \log_2(\text{ceiling}/f)$ vào độ mạnh ứng viên	Chuẩn hóa $R[\tau]$ về 27 bit, trừ số hạng tra bảng $K[\tau]$	Thay thế phép chia và hàm logarit phức tạp bằng phép trừ số nguyên	Khớp hoàn toàn với công thức Praat, không làm méo dạng đỉnh parabol.
5. Tinh chỉnh đỉnh cực đại	Nội suy Parabol sơ bộ rồi tinh chỉnh sinc 70 điểm + Brent	Nội suy Parabol 3 điểm, gộp hai phép chia thành một phép chia duy nhất	Phép nội suy sinc70 lặp Brent quá cồng kềnh cho FPGA; parabol chỉ tốn 1 bộ chia	Lệch cực đại $1{,}08$ cent trên tín hiệu sạch; lệch cực đại $32{,}91$ cent trên tín hiệu ồn 5 dB.
6. Xếp hạng ứng viên	Quét $i \in [2, 213]$ (lags $2 \dots 26$ thành vô thanh), lưu 15 ứng viên	Quét trực tiếp dải $[27, 213]$, chỉ lưu 1 ứng viên có điểm số S_c cao nhất trên lưới	Tiết kiệm thanh ghi và logic quản lý bảng ứng viên phức tạp	Ở tín hiệu ồn có hai đỉnh gần hòa, FPGA có thể chọn đỉnh khác Praat (3/207 khung).
7. Lựa chọn cao độ toàn cục	Quy hoạch động Viterbi (Path Finder) duyệt toàn bộ tệp âm thanh	Bộ lọc trung vị 3 điểm (median-3) xử lý trôi (streaming)	Viterbi cần bộ nhớ lưu trữ toàn bộ tệp và giải thuật đệ quy quay lui; median-3 chỉ trễ 1 khung	Median-3 sửa được các đỉnh gai đơn lẻ nhưng không thay thế được Viterbi; phần chênh lệch đo bằng $R_1 - R_2$.

Đọc bảng và định vị mốc tham chiếu. Bảng đối chiếu trên cho thấy một nguyên tắc thiết kế mạch lạc: các khâu biến đổi tín hiệu vật lý cơ bản (lưới khung, độ dài cửa sổ 638 mẫu, trung bình 426 mẫu, chuẩn hóa cửa sổ và chi phí quãng tám dạng trừ) được hiện thực bám sát Praat để loại trừ mọi sai số tiên nghiệm.

Ba sự khác biệt có chủ ý nằm ở: (1) nội suy parabol thay cho sinc70 + Brent, (2) chọn đỉnh tốt nhất trên lưới thay vì 15 ứng viên, và (3) bộ lọc trung vị median-3 thay cho Viterbi. Chính vì FPGA không chứa bộ giải thuật Viterbi, trong toàn bộ tài liệu này ta định nghĩa hai mốc tham chiếu Praat chuẩn:

- **Praat** R_1: Chạy Praat với toàn bộ 10 tham số mặc định (gồm cả thuật toán Viterbi với `octave-jump cost` = 0,35 và `V/UV cost` = 0,14).
- **Praat** R_2: Chạy Praat với cấu hình tương tự nhưng đặt `octave-jump cost` = 0 và `V/UV cost` = 0. Khi triệt tiêu hai chi phí chuyển tiếp này, thuật toán Viterbi tự động suy biến thành phép chọn cực đại độc lập trên từng khung (argmax cục bộ).

Nhờ định nghĩa này, phép so sánh giữa FPGA (trước bộ lọc median) và Praat R_2 trở thành một phép so sánh **ngang hàng (apples-to-apples)**: cả hai đều chọn ứng viên tốt nhất trên từng khung độc lập. Hiệu số giữa Praat

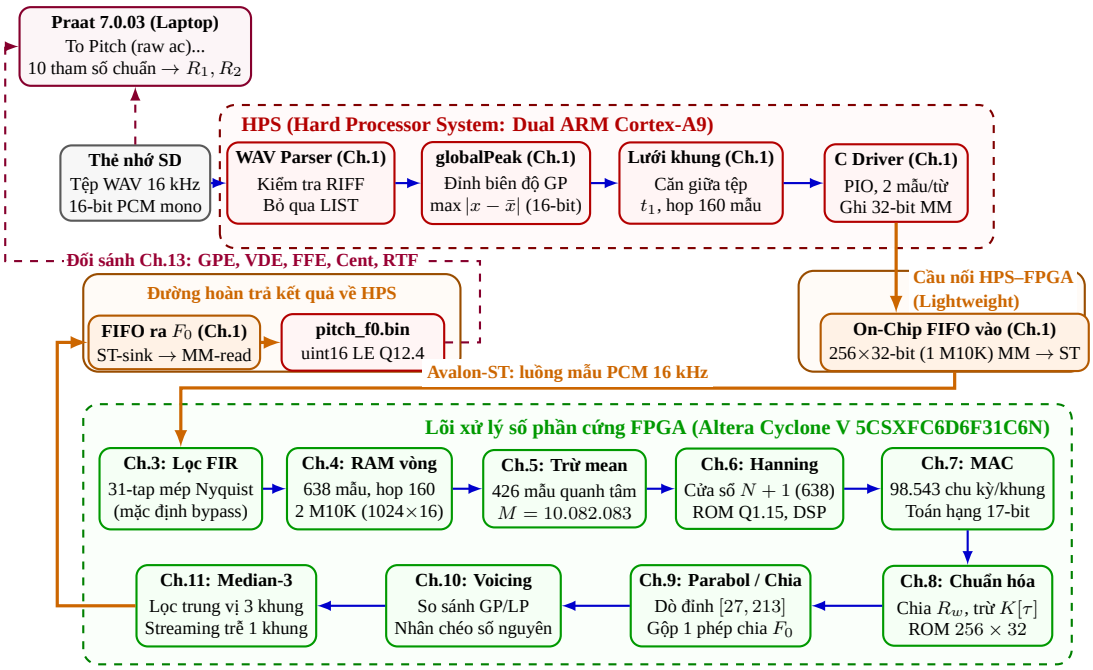
R_1 và Praat R_2 sẽ bóc tách và định lượng chính xác phần đóng góp riêng biệt của thuật toán Viterbi, giúp anh Tú trả lời trọn vẹn câu hỏi: phần chênh lệch nào do phần cứng số nguyên gây ra, và phần nào do sự vắng mặt của thuật toán quy hoạch động toàn cục.

Tóm lại: Praat vận hành qua 8 bước chặt chẽ với 10 tham số chuẩn; trên FPGA ta bám sát Praat ở kích thước cửa sổ 638 mẫu, trung bình 426 mẫu và chuẩn hóa cửa sổ, đồng thời thay thế sinc70+Brent bằng parabol và Viterbi bằng median-3, sử dụng mốc Praat R_2 làm chuẩn đối sánh ngang hàng trực tiếp.

0.4. Kiến trúc hệ thống và bản đồ các chương

Để hiện thực hóa thuật toán ước lượng cao độ kiểu Praat trên hệ thống phần cứng dị thể SoC (System-on-Chip) Altera Cyclone V (DE10-Standard), toàn bộ quy trình xử lý được phân bổ khoa học giữa hệ thống vi xử lý ARM Cortex-A9 (HPS) và vùng logic khả trình FPGA.

Hình 1 phác họa toàn cảnh kiến trúc luồng dữ liệu hai chiều: đi từ tệp âm thanh WAV trên thẻ nhớ SD, qua phần mềm HPS, đổ vào bộ đệm FIFO thông qua cầu nối bus Lightweight, chạy qua chuỗi xử lý số tín hiệu nối tiếp trong FPGA, rồi quay trở lại HPS để ghi tệp nhị phân kết quả, song song với nhánh đối chứng Praat 7.0.03 chạy trên máy tính cá nhân.



Hình 1: Kiến trúc tổng thể hệ thống ước lượng cao độ F_0: luồng dữ liệu từ tệp WAV qua HPS, cầu Lightweight, lõi xử lý phần cứng FPGA, đường hoàn trả kết quả và nhánh đối sánh với Praat trên máy tính.

Đọc hình. Hãy quan sát kỹ từng phân vùng chức năng trên sơ đồ kiến trúc:

1. **Phân vùng HPS (màu đỏ):** Bắt đầu từ tệp âm thanh WAV chuẩn nằm trên thẻ nhớ SD. Khối `WAV Parser` đọc và bóc tách cấu trúc nhị phân RIFF/WAVE, kiểm tra tần số lấy mẫu 16 kHz, định dạng 16-bit PCM mono và bỏ qua các khối mở rộng (như khối `LIST`). Khối tiếp theo tính toán đỉnh biên độ toàn cục globalPeak (GP) của cả tệp. Khối `Lưới khung` tính số khung, mốc thời gian t_1 và chỉ số mẫu bắt đầu của từng khung theo công thức đối xứng của Praat. Cuối cùng, khối điều khiển `C Driver` thực hiện ánh xạ bộ nhớ (`mmap`) thanh ghi cầu nối và phát các lệnh ghi mẫu PCM theo cơ chế PIO (Programmed I/O).

2. **Vùng cầu nối và hàng đợi (màu cam):** Cầu nối `Lightweight HPS-to-FPGA` (32-bit Avalon-MM) tiếp nhận các giao dịch ghi từ vi xử lý. Dữ liệu PCM được nạp vào khối bộ nhớ đệm `On-Chip FIFO` (dung lượng 512 từ 16-bit, chiếm đúng 1 khối RAM M10K) có chức năng đệm mẫu và chuyển đổi giao thức từ bus nhớ Avalon-MM sang giao tiếp luồng Avalon-ST (`valid/ready`).

3. **Phân vùng lõi xử lý phần cứng FPGA (màu xanh lá):** Các mẫu âm thanh lần lượt đi qua chuỗi khối chuyên biệt:

   - Khối lọc `FIR 31-tap` làm mềm mép phổ Nyquist (có công tắc bypass khi đo đối chiếu Praat);

- Bộ đệm `RAM vòng` 1024 mẫu (sử dụng 2 khối M10K) tích lũy đủ 638 mẫu cho mỗi khung và dịch chuyển mốc đọc `hop` = 160 mẫu;
- Khối `Trừ trung bình` cục bộ trên 426 mẫu quanh tâm khung, sử dụng phép nhân hằng số nghịch đảo $M = 10.082.083$ trên bộ nhân DSP cứng 25×24;
- Khối `Cửa sổ Hanning` nhân khung tín hiệu với mảng trọng số 638 mẫu lưu trong bộ nhớ ROM Q1.15;
- Khối tích lũy `MAC tự tương quan` sử dụng 1 khối DSP 18×18 chạy liên tục 98.543 chu kỳ để tính năng lượng $R[0]$ và 189 giá trị $R[\tau]$ ($\tau = 26 \dots 214$);
- Khối `Chuẩn hóa và Octave` tra bảng ROM 256×32 để nhân hệ số bù $C[\tau]$ và trừ chi phí quãng tám $K[\tau]$;
- Khối `Parabol và Bộ chia` dò tìm đỉnh cục bộ lớn nhất trên dải $[27, 213]$ và thực thi một phép chia số nguyên duy nhất để tính F_0 định dạng Q12.4;
- Khối `Quyết định Voicing` so sánh độ mạnh đỉnh với năng lượng và biên độ GP, LP qua bất đẳng thức nhân chéo số nguyên;
- Khối `Bộ lọc Median-3` nhận chuỗi kết quả theo cơ chế dòng (streaming), loại bỏ các gai cao độ đơn lẻ với độ trễ cố định 1 khung.

4. **Đường trả kết quả và nhánh đối chứng (màu tím):** Giá trị F_0 tính được cùng trạng thái có thanh/vô thanh được đẩy vào FIFO ngõ ra F0. Bộ điều khiển HPS đọc từ FIFO này qua cầu nối và ghi tuần tự vào tệp nhị phân `pitch_f0.bin` kèm tệp mô tả meta. Song song trên máy tính, phần mềm Praat 7.0.03 xử lý cùng tệp WAV với 10 tham số chuẩn, xuất ra hai đường tham chiếu R_1 và R_2 để phục vụ phân tích sai số toàn diện ở Chương 13.

Phân tích phương án truyền dữ liệu HPS–FPGA và kích thước bộ đệm FIFO

Để đưa dữ liệu từ vi xử lý sang FPGA một cách tối ưu, ta tiến hành phân tích so sánh giữa các phương án phần cứng:

1. Cơ chế truyền dữ liệu: PIO qua Lightweight Bridge so với mSGDMA qua Full Bridge.

- *Phương án A (Cơ chế PIO qua Lightweight HPS-to-FPGA):* Bộ điều khiển C trên HPS sử dụng ánh xạ bộ nhớ (`mmap`) trực tiếp vào không gian địa chỉ 2 MB của cầu nối Lightweight 32-bit Avalon-MM và ghi từng từ vào FIFO ngõ vào.
- *Phương án B (Cơ chế DMA qua HPS-to-FPGA toàn phần):* Sử dụng bộ điều khiển DMA phân tán `altera_avalon_msgdma` kết nối qua cầu nối HPS-to-FPGA AXI 64/128-bit.
- *Đánh đổi định lượng:* Âm thanh chuẩn 16 kHz, 16-bit PCM mono có lưu lượng dữ liệu vật lý rất thấp:

$$B = 16.000 \text{ mẫu/s} \times 2 \text{ byte/mẫu} = 32.000 \text{ byte/s} = 32 \text{ KB/s} \tag{24}$$

Khi ghép 2 mẫu 16-bit thành một từ 32-bit (chẵn khung), HPS chỉ cần phát 8.000 thao tác ghi/giây. Trên xung nhịp bus 50 MHz, một chu kỳ ghi bus tốn 20 ns. Ngay cả khi ước lượng với con số cực kỳ bi quan là $1\ \mu$s cho mỗi thao tác ghi trong không gian người dùng của Linux kernel, tổng thời gian CPU tiêu tốn cho việc truyền dữ liệu trong mỗi 1 giây âm thanh chỉ là $8.000 \times 1\ \mu\text{s} = 8$ ms, tương ứng hệ số thời gian thực truyền dữ liệu:

$$\text{RTF}_{\text{transfer}} = \frac{8 \text{ ms}}{1.000 \text{ ms}} = 0{,}008 \tag{25}$$

(thời gian ghi thực tế trên Linux cần được đo đạc chính xác trên bo mạch ở Chương 13 – phải đo). Hệ số $0{,}008$ chỉ chiếm chưa đầy 1% thời gian của hệ thống. Ngược lại, cấu hình mSGDMA đòi hỏi phân bổ bộ nhớ vật lý liên tục, khởi tạo bảng mô tả descriptor, quản lý ngắt hoàn tất và tiêu tốn hàng nghìn logic ALM trên Cyclone V. Với băng thông 32 KB/s, mSGDMA là sự phức tạp hóa không cần thiết.

2. Kích thước On-Chip FIFO: 512×16 bit so với 1024×16 bit.

- Một khối RAM nhúng M10K trên Cyclone V có dung lượng 10.240 bit. Ở độ rộng 16-bit, cấu hình sâu nhất của 1 khối M10K là 512 từ $\times$ 16 bit $= 8.192$ bit. FIFO 512×16 chiếm vừa vặn đúng **1 khối M10K duy nhất**.

- Dung lượng 512 mẫu chứa được $512/160 = 3{,}2$ bước nhảy hop (tương đương 32 ms âm thanh), hoàn toàn đủ để hấp thụ các dao động trễ lập lịch (scheduling jitter) của hệ điều hành Linux HPS (thường dưới $2 \ldots 5$ ms).
- Nếu nâng lên 1024×16, thiết kế bắt buộc phải tiêu tốn **2 khối M10K**. Do HPS phát dữ liệu đều đặn mỗi 10 ms và FPGA xử lý một khung chỉ mất khoảng 2 ms, bộ đệm 512 mẫu hầu như luôn ở mức rỗng; việc tăng lên 1024 mẫu không đem lại lợi ích thiết thực mà gây lãng phí tài nguyên RAM nhúng.

Lựa chọn thiết kế: Sử dụng cơ chế **PIO qua cầu nối Lightweight HPS-to-FPGA** kết hợp **On-Chip FIFO** 512×16 **(chiếm đúng 1 M10K)**. Đây là lựa chọn tối giản, vững chắc và tiết kiệm tài nguyên nhất (phù hợp với thiết kế chi tiết ở Chương 1, Mục 1.3).

Bản đồ phân công nội dung chi tiết theo từng chương

Mỗi khối chức năng trong kiến trúc tổng thể trên được nghiên cứu, thiết kế và kiểm chứng chi tiết trong từng chương chuyên biệt của tài liệu này:

- **Chương 1 (WAV trên HPS và Cầu giao tiếp):** Phân tích định dạng WAV nhị phân, giải thuật tính GP và lưới khung Praat bằng C; phân tích đánh đổi lưu lượng và thiết kế On-Chip FIFO (IP `altera_avalon_fifo` và HDL tự viết).
- **Chương 2 (Giao thức Avalon-ST và Mã hóa FSM):** Bản chất giao tiếp bắt tay `valid/ready`, quy tắc `readyLatency` 0 và 1, ngân sách chu kỳ chế độ tệp (file-driven), và so sánh tài nguyên các kiểu mã hóa máy trạng thái FSM (One-Hot, Binary, Gray).
- **Chương 3 (Lọc mép Nyquist FIR):** Hiện thực bộ lọc FIR 31-tap đối xứng bằng FIR Compiler II và HDL gấp tap qua pre-adder; phân tích đáp ứng tần số và cơ chế bypass khi so với Praat.
- **Chương 4 (Chia khung và RAM vòng):** Kỹ thuật đệm trượt 638 mẫu trên RAM vòng 1024×16 (2 khối M10K) với chế độ đọc trước ghi (read-during-write) và quản lý con trỏ cơ sở `rd_base`.
- **Chương 5 (Trừ trung bình cục bộ và DC-Blocker):** Thuật toán trừ trung bình 426 mẫu bằng phép nhân nghịch đảo khớp bit $M = 10.082.083$, $k = 32$; bộ lọc IIR DC-blocker tùy chọn $\alpha = 1 - 2^{-8}$.
- **Chương 6 (Cửa sổ Hanning $N + 1$):** Thiết kế bộ nhớ ROM chứa 638 hệ số Hanning định dạng Q1.15, tối ưu hóa một nửa ROM đối xứng và bộ nhân có chốt thanh ghi.
- **Chương 7 (Tự tương quan bằng MAC):** Trục tính toán nặng nhất của hệ thống: dẫn xuất 98.540 phép nhân-cộng 17-bit, cấu hình khối DSP cứng 18×18 kết hợp cờ `sload` để chuyển độ trễ với 0 nhịp nghỉ.
- **Chương 8 (Chuẩn hóa cửa sổ và Chi phí quãng tám):** Bộ nhớ ROM 256×32 chứa hệ số $C[\tau]$ và $K[\tau]$, kỹ thuật chuẩn hóa $R[\tau]$ về 27 bit để phép nhân vừa vặn trong 1 khối DSP.
- **Chương 9 (Dò đỉnh Parabol và Bộ chia gộp):** Điều kiện tìm đỉnh cục bộ trên dải $[27, 213]$, công thức nội suy parabol đúng dấu, và kiến trúc bộ chia số nguyên LPM_DIVIDE gộp cả nội suy lẫn đổi ra Hz thành một phép chia duy nhất.
- **Chương 10 (Quyết định Voicing):** Bất đẳng thức nhân chéo số nguyên kiểm tra năng lượng và biên độ GP, LP, thay thế hoàn toàn phép chia số thực của Praat.
- **Chương 11 (Bộ lọc trung vị Median-3):** Mạch so sánh lọc trung vị 3 khung nối tiếp, đánh giá khả năng sửa lỗi nhảy quãng tám và tác động tới sai số phân loại vô thanh.
- **Chương 12 (Tích hợp hệ thống và FSM đỉnh):** Kết nối liên hoàn toàn bộ các khối phần cứng, lập bảng phân bổ chu kỳ từng khung, bộ đếm chu kỳ hiệu năng và đóng gói tệp `pitch_f0.bin`.
- **Chương 13 (Kiểm chứng và Đo đạc đối sánh):** Thực hiện đo đạc 3 chiều: sai số chất lượng (GPE, VDE, FFE, cent) so với Praat R_1, R_2; đo thời gian ba phía (chu kỳ FPGA, HPS end-to-end, Praat trên máy tính); bảng so sánh tài nguyên IP và HDL từ báo cáo Fitter.
- **Chương 14 (Bảng kiểm tra lỗi Checklist):** Bảng rà soát 25 tiêu chí kỹ thuật: độ rộng bit, tràn số, độ trễ nhịp, tham số IP và bảo đảm tính tái lập của mã nguồn.
- **Phụ lục A (Bộ chuyển đổi tần số lấy mẫu Resampling):** Thiết kế khối lọc đa pha chuyển đổi tần số âm thanh từ 48 kHz và 44,1 kHz về chuẩn 16 kHz.

Tóm lại: Hệ thống phân chia rành mạch giữa HPS (quản lý tệp, căn lưới thời gian, giao tiếp bus) và FPGA (xử lý số tín hiệu nối tiếp theo dòng); từng khối phần cứng được ánh xạ tương ứng vào từng chương độc lập với kiểm chứng song song hai đường IP và HDL tự viết.

0.5. Các phép đo đánh giá: GPE, VDE, FFE và sai số cent

Để so sánh đường cao độ do mạch số FPGA ước lượng với đường cao độ tham chiếu chuẩn của phần mềm Praat (hoặc cao độ thực *ground-truth* của tệp thử nghiệm), ta không thể dùng các độ đo sai số trung bình bình phương (MSE) thông thường. Tiếng nói gồm các đoạn có thanh (voiced, dây thanh rung) xen kẽ các đoạn vô thanh (unvoiced, âm ma sát hoặc khoảng lặng). Khi một khung bị đoán sai trạng thái thanh tính, giá trị tần số hoàn toàn không xác định, khiến phép trừ số học trực tiếp mất ý nghĩa.

Dẫn xuất các tình huống xảy ra trên một khung phân tích

Xét một khung phân tích bất kỳ tại mốc thời gian t. Gọi F_{ref} là tần số cơ bản của Praat (hoặc ground-truth) và F_{test} là tần số do FPGA tính toán (quy ước quy đổi giá trị 0 đại diện cho trạng thái vô thanh). So sánh giữa hai giá trị này chỉ có thể rơi vào đúng **bốn tình huống vật lý** sau:

- **Tình huống 1: Cả hai bên cùng hữu thanh và gần nhau ($F_{\text{ref}} > 0, F_{\text{test}} > 0$, sai số tương đối $\leq 20\%$).** Đây là các khung ước lượng thành công về mặt cao độ cơ bản. Phần chênh lệch nhỏ giữa hai bên phản ánh sai số làm tròn số học, sự khác biệt giữa thuật toán nội suy đỉnh parabol và nội suy hàm sinc. Tình huống này được đưa vào tập đo **sai số cent mịn (fine-cent error)**.
- **Tình huống 2: Cả hai bên cùng hữu thanh nhưng lệch nhau rất xa ($F_{\text{ref}} > 0, F_{\text{test}} > 0$, sai số tương đối $> 20\%$).** Đây là các lỗi thô nghiêm trọng về mặt giải thuật, thường do mạch số chọn nhầm hài âm bậc hai (nhân đôi quãng tám: $F_{\text{test}} \approx 2F_{\text{ref}}$) hoặc chọn nhầm độ trễ gấp đôi (chia đôi quãng tám: $F_{\text{test}} \approx 0{,}5F_{\text{ref}}$). Hiện tượng này được định nghĩa là **Lỗi cao độ thô (Gross Pitch Error – GPE)**.
- **Tình huống 3: Quyết định thanh tính không đồng nhất (một bên có thanh, bên kia vô thanh).** Hoặc Praat phát hiện có thanh mà FPGA bỏ sót ($F_{\text{ref}} > 0, F_{\text{test}} = 0$), hoặc ngược lại FPGA nhận diện nhầm tiếng ồn thành có thanh ($F_{\text{ref}} = 0, F_{\text{test}} > 0$). Hiện tượng này được định nghĩa là **Lỗi quyết định thanh tính (Voicing Decision Error – VDE)**.
- **Tình huống 4: Cả hai bên cùng vô thanh ($F_{\text{ref}} = 0, F_{\text{test}} = 0$).** Hai bên hoàn toàn đồng thuận về khoảng lặng hoặc âm vô thanh; khung phân tích hoàn toàn chính xác và không có lỗi nào phát sinh.

Từ bốn tình huống trên, cộng đồng xử lý tiếng nói quốc tế chuẩn hóa bốn chỉ số định lượng:

1. Tỉ lệ lỗi quyết định thanh tính (VDE – Voicing Decision Error): Là tỉ số giữa số khung có quyết định thanh tính khác nhau (N_{VDE}, tức Tình huống 3) trên tổng số khung phân tích N:

$$\text{VDE} = \frac{N_{\text{VDE}}}{N} = \frac{\sum_{i=1}^{N} \left[(F_{\text{ref},i} > 0) \oplus (F_{\text{test},i} > 0) \right]}{N} \tag{26}$$

2. Tỉ lệ lỗi cao độ thô (GPE – Gross Pitch Error): Chỉ tính riêng trên tập hợp các khung mà *cả hai bên đều nhận định là có thanh* (N_{both} khung). GPE là tỉ lệ số khung bị lệch tần số quá 20% (N_{GPE}, tức Tình huống 2) trên N_{both}:

$$\text{GPE} = \frac{N_{\text{GPE}}}{N_{\text{both}}} = \frac{\sum_{i \in \text{both}} \mathbb{I}\left(\left| \frac{F_{\text{test},i}}{F_{\text{ref},i}} - 1 \right| > 0{,}20 \right)}{N_{\text{both}}} \tag{27}$$

3. Tỉ lệ lỗi tổng hợp toàn khung (FFE – F_0 Frame Error): Phản ánh bức tranh toàn cảnh về độ chính xác của hệ thống, bằng tổng số khung bị lỗi thanh tính cộng số khung bị lỗi cao độ thô, chia cho tổng số khung N:

$$\text{FFE} = \frac{N_{\text{VDE}} + N_{\text{GPE}}}{N} \tag{28}$$

4. Thống kê sai số cent mịn (Fine-Cent Statistics): Được tính riêng biệt trên các khung ở Tình huống 1 (cùng có thanh và không phạm lỗi thô GPE). Độ lệch cao độ tính bằng cent:

$$C_i = 1.200 \cdot \log_2 \left(\frac{F_{\text{test},i}}{F_{\text{ref},i}} \right) \tag{29}$$

Ta báo cáo bốn đại lượng thống kê: trung bình độ lệch tuyệt đối (Mean $|C|$), độ lệch chuẩn (Std C), phân vị 95 (P95 $|C|$) và sai số mịn cực đại (max $|C|$).

Ý nghĩa của ngưỡng 20% **và vì sao phải loại bỏ lỗi thô khi đo cent?** Một ngưỡng lệch tương đối 20% về mặt tần số tương ứng với bao nhiêu trong cảm thụ âm nhạc? Ta quy đổi trực tiếp ra cent:

$$1.200 \cdot \log_2(1{,}20) \approx 315{,}64 \text{ cent} \approx 3{,}16 \text{ nửa cung (semitones)} \tag{30}$$

Khoảng cách hơn 3 nửa cung vượt xa ngưỡng nghe nhận biết cao độ (JND – Just Noticeable Difference) của con người (khoảng $5 \ldots 10$ cent đối với âm thanh đơn âm hoặc người nghe được huấn luyện, và $10 \ldots 25$ cent đối với tiếng nói tự nhiên trong ngữ cảnh phức tạp). Khi một khung bị lỗi nhảy quãng tám (nhân đôi tần số: $F_{\text{test}} = 2F_{\text{ref}}$), sai số cent lên tới đúng 1.200 cent! Nếu gộp các khung lỗi quãng tám này vào thống kê cent mịn, chỉ cần một khung lỗi 1.200 cent duy nhất cũng đủ làm biến dạng giá trị trung bình từ $0{,}2$ cent tăng lên hàng chục cent, che lấp độ chính xác vi mô của bộ nội suy đỉnh parabol. Do đó, việc tách rời GPE (đánh giá lỗi giải thuật) và cent mịn (đánh giá độ tinh xảo số học) là nguyên tắc bắt buộc cần tuân thủ.

Ví dụ tính tay thu nhỏ trên 10 khung mô phỏng

Để người đọc tự tay kiểm tra từng con số, xét kịch bản mẫu gồm 10 khung phân tích đại diện đầy đủ cho 4 tình huống vật lý (được sinh và tính toán bởi kịch bản `verify_v3/ch00_metrics.py`):

Bảng 3: Bảng tính tay chi tiết các chỉ số chất lượng trên 10 khung mô phỏng sư phạm

Khung	F_{ref} (Hz)	F_{test} (Hz)	Phân loại tình huống	Sai số tương đối	Sai số cent	Trạng thái lỗi
0	0,0	0,0	Cả hai vô thanh (TH 4)	—	—	Không lỗi
1	120,0	120,0	Cùng có thanh (TH 1)	$0{,}00\%$	$+0{,}00$	Khớp hoàn toàn
2	150,0	152,0	Cùng có thanh (TH 1)	$1{,}33\%$	$+22{,}93$	Sai số mịn
3	200,0	196,0	Cùng có thanh (TH 1)	$2{,}00\%$	$-34{,}98$	Sai số mịn
4	220,0	224,0	Cùng có thanh (TH 1)	$1{,}82\%$	$+31{,}19$	Sai số mịn
5	0,0	130,0	Lệch thanh tính (TH 3)	—	—	**Lỗi VDE**
6	140,0	0,0	Lệch thanh tính (TH 3)	—	—	**Lỗi VDE**
7	100,0	200,0	Nhân đôi quãng tám (TH 2)	$100{,}00\%$	$+1.200{,}00$	**Lỗi GPE**
8	300,0	150,0	Chia đôi quãng tám (TH 2)	$50{,}00\%$	$-1.200{,}00$	**Lỗi GPE**
9	0,0	0,0	Cả hai vô thanh (TH 4)	—	—	Không lỗi

Dẫn xuất từng chỉ số từ Bảng 3:

- Tổng số khung: $N = 10$.
- Số khung cùng có thanh: $N_{\text{both}} = 6$ (Khung 1, 2, 3, 4, 7, 8).
- Lỗi thanh tính (VDE): xuất hiện ở Khung 5 và Khung 6 ($N_{\text{VDE}} = 2$). Tỉ lệ:

$$\text{VDE} = \frac{2}{10} = 20{,}0\% \tag{31}$$

- Lỗi cao độ thô (GPE): trong 6 khung cùng có thanh, Khung 7 bị nhân đôi cao độ ($100\% > 20\%$) và Khung 8 bị chia đôi ($50\% > 20\%$), nên $N_{\text{GPE}} = 2$. Tỉ lệ:

$$\text{GPE} = \frac{2}{6} \approx 33{,}3\% \tag{32}$$

- Lỗi tổng hợp toàn khung (FFE):

$$\text{FFE} = \frac{N_{\text{VDE}} + N_{\text{GPE}}}{N} = \frac{2+2}{10} = 40{,}0\% \tag{33}$$

- Thống kê sai số cent mịn trên 4 khung không lỗi thô (Khung 1, 2, 3, 4 với độ lệch tuyệt đối là $0{,}00$, $22{,}93$, $34{,}98$, $31{,}19$ cent):

$$\text{Trung bình tuyệt đối} = \frac{0{,}00 + 22{,}93 + 34{,}98 + 31{,}19}{4} = 22{,}28 \text{ cent} \tag{34}$$

$$\text{Độ lệch chuẩn (Std)} = 25{,}64 \text{ cent} \tag{35}$$

$$\text{Phân vị 95 (P95)} = 34{,}41 \text{ cent} \tag{36}$$

$$\text{Sai số mịn cực đại} = 34{,}98 \text{ cent} \tag{37}$$

Để khẳng định tính đúng đắn của các hàm đo đạc, kịch bản `verify_v3/ch00_metrics.py` đã nạp trực tiếp toàn bộ 16 tệp dữ liệu `.tsv` kết quả trong `golden_v2/out/` và chạy hàm `compute_metrics()`, xác nhận số lượng khung VDE, GPE, FFE và giá trị sai số cent mịn cực đại trùng khớp với từng con số trong `golden_v2/out/compare_summary.json`. Đồng thời, sự khác biệt có chủ ý về trung bình và phân vị 95 (do `compare.py` tính trên toàn bộ các khung cùng có thanh bao gồm cả lỗi thô, còn `ch00_metrics.py` chỉ tính trên các khung cent mịn) đã được ghi nhận và giải thích minh bạch trong mã nguồn.

Tóm lại: VDE đo sự sai lệch phân loại hữu thanh/vô thanh trên toàn tệp, GPE đo lỗi nhảy quãng tám trên các khung cùng hữu thanh, FFE gộp cả hai loại lỗi trên tổng số khung, còn sai số cent mịn đánh giá độ chính xác toán học của bộ nội suy đỉnh sau khi đã bóc tách triệt để các lỗi thô.

0.6. Ngân sách chu kỳ, hệ số thời gian thực RTF, và mạch phát cặp mẫu phần cứng

Khi chuyển dịch một thuật toán từ môi trường máy tính sang chip bán dẫn FPGA, bài toán thiết kế trọng tâm đầu tiên của người kỹ sư là: mạch số có đủ nhanh để xử lý tín hiệu theo thời gian thực (real-time) hay không, và ta cần bao nhiêu tài nguyên phần cứng (bộ nhân DSP, khối nhớ RAM M10K) để đáp ứng thời hạn đó?

Dẫn xuất ngân sách chu kỳ từ nguyên lý đầu tiên

Hệ thống phần cứng trên bo mạch DE10-Standard vận hành với xung nhịp đồng hồ hệ thống $f_{\text{clk}} = 50$ MHz (chu kỳ xung nhịp $T_{\text{clk}} = 20$ ns). Tín hiệu âm thanh chuẩn được lấy mẫu ở tần số $f_s = 16.000$ Hz (chu kỳ lấy mẫu $T_s = 62{,}5\ \mu$s).

Số chu kỳ xung nhịp đồng hồ FPGA có sẵn giữa hai mẫu âm thanh liên tiếp là:

$$N_{\text{cycles/sample}} = \frac{f_{\text{clk}}}{f_s} = \frac{50.000.000}{16.000} = 3.125 \text{ chu kỳ/mẫu} \tag{38}$$

Cứ sau một khoảng thời gian bước nhảy khung $\Delta t = 10$ ms (`hop` = 160 mẫu), hệ thống phải hoàn tất việc tính toán một giá trị F_0 mới. Ngân sách chu kỳ tối đa cho mỗi khung để đạt ngưỡng thời gian thực là:

$$N_{\text{budget/hop}} = 160 \text{ mẫu} \times 3.125 \text{ chu kỳ/mẫu} = 500.000 \text{ chu kỳ/khung} \tag{39}$$

Con số 500.000 chu kỳ ở 50 MHz chính là trần thời gian thực. Bất kỳ thiết kế nào hoàn thành một khung dưới 500.000 chu kỳ đều chạy nhanh hơn tốc độ phát âm thanh thực tế.

Dẫn xuất số phép nhân-cộng MAC và số chu kỳ của khối tự tương quan

Khối tiêu tốn chu kỳ tính toán nặng nề nhất trong toàn bộ hệ thống là khối tự tương quan. Ta xét một khung có độ dài $N = 638$ mẫu, rà soát trên dải độ trễ từ $\tau_{\min} = 26$ đến $\tau_{\max} = 214$ (tương ứng với $214 - 26 + 1 = 189$ độ trễ).

Tại một độ trễ τ bất kỳ, số cặp mẫu cần nhân và tích lũy là $N - \tau = 638 - \tau$. Tổng số phép nhân-cộng (MAC) cho toàn bộ 189 độ trễ được tính chính xác bằng công thức tổng cấp số cộng:

$$\sum_{\tau=26}^{214} (638 - \tau) = \sum_{\tau=26}^{214} 638 - \sum_{\tau=26}^{214} \tau \tag{40}$$

$$= 189 \times 638 - \frac{189 \times (26 + 214)}{2} \tag{41}$$

$$= 120.582 - \frac{189 \times 240}{2} = 120.582 - 189 \times 120 \tag{42}$$

$$= 120.582 - 22.680 = 97.902 \text{ phép MAC} \tag{43}$$

Để tính năng lượng khung chuẩn hóa tại gốc $\tau = 0$ ($R[0]$), ta cần thêm $N = 638$ phép nhân-cộng nữa:

$$N_{\text{pairs}} = 97.902 + 638 = 98.540 \text{ cặp mẫu} \tag{44}$$

Khối nhân-cộng phần cứng trong Chương 7 được thiết kế với đường ống 2 tầng thanh ghi (thanh ghi ngõ vào toán hạng trong khối DSP cứng và thanh ghi tổng tích lũy ngõ ra), cộng với 1 nhịp đọc từ bộ nhớ RAM M10K hai cổng, tạo ra độ trễ đường ống (pipeline latency) tổng cộng là 3 chu kỳ xung nhịp.

Nhờ cơ chế phát cặp mẫu liên tục không nhịp nghỉ (sẽ phân tích ở phần HDL bên dưới), tổng số chu kỳ xung nhịp từ lúc phát cặp địa chỉ đầu tiên đến khi giá trị $R[214]$ xuất hiện hợp lệ ở đầu ra là:

$$N_{\text{cycles, MAC}} = 98.540 \text{ nhịp phát} + 3 \text{ nhịp trễ} = 98.543 \text{ chu kỳ/khung} \tag{45}$$

Con số 98.543 chu kỳ này đã được đo đạc chính xác bằng bộ mô phỏng iverilog trên mã nguồn `golden_v2/hdl_check/tb_mac638.v`.

Phân tích phương án kiến trúc khối MAC: Tuần tự 1 DSP so với Song song P DSP

Trước khi ấn định thiết kế phần cứng cho khối tự tương quan, ta phân tích đánh đổi giữa hai phương án kiến trúc:

- *Phương án 1 (Kiến trúc tuần tự 1 DSP, $P = 1$):* Dùng duy nhất 1 bộ nhân-cộng MAC 18×18 bit vận hành nối tiếp qua tất cả các cặp mẫu của 190 độ trễ ($R[0]$ và 189 độ trễ τ).
- *Phương án 2 (Kiến trúc song song P DSP, $P \geq 2$):* Bố trí P bộ nhân-cộng MAC hoạt động song song để chia sẻ việc tính toán các độ trễ khác nhau.
- *Đánh đổi định lượng:*
  - **Số chu kỳ:** Số chu kỳ tính toán của khối MAC tỉ lệ nghịch với P:

$$N_{\text{cycles, MAC}} \approx \frac{98.540}{P} + 3 \text{ chu kỳ} \tag{46}$$

    Với $P = 2$, số chu kỳ giảm còn khoảng 49.273 chu kỳ; với $P = 4$, giảm còn khoảng 24.638 chu kỳ.
  - **Tài nguyên DSP:** Số khối DSP tiêu tốn tăng tuyến tính bằng đúng P khối nhân.
  - **Nghẽn cổ chai cổng đọc RAM M10K:** Đây là rào cản vật lý then chốt. Mỗi bộ nhân MAC ở mỗi chu kỳ cần đọc đồng thời 2 mẫu âm thanh $x[n]$ và $x[n+\tau]$. Một khối RAM nhúng M10K chỉ hỗ trợ tối đa 2 cổng đọc độc lập (True Dual-Port RAM). Nếu sử dụng P bộ MAC song song, hệ thống đòi hỏi $2P$ cổng đọc đồng thời tại mỗi nhịp xung nhịp. Để giải quyết, ta buộc phải nhân bản bộ nhớ RAM vòng thành P bản sao giống hệt nhau (tiêu tốn $2P$ khối M10K thay vì 2 khối M10K), hoặc phải thiết kế cơ chế phân vùng ngân hàng nhớ (multi-banking) kèm logic phân xử chuyển mạch vô cùng phức tạp để chống xung đột truy xuất.
  - **Khả năng đáp ứng thời gian thực:** Ở tần số xung nhịp chuẩn 50 MHz, ngân sách cho mỗi bước nhảy khung 10 ms là 500.000 chu kỳ. Kiến trúc tuần tự với $P = 1$ chỉ tiêu tốn 98.543 chu kỳ, tương ứng hệ số $\text{RTF}_{\text{MAC}} \approx 0{,}197$ (nhanh gấp 5 lần thời gian thực).
- *Lựa chọn thiết kế:* Sử dụng **kiến trúc tuần tự 1 khối DSP 18×18 ($P = 1$)**. Vì $P = 1$ đã thừa sức đáp ứng thời gian thực với biên an toàn gấp 5 lần, việc tăng P chỉ gây lãng phí bộ nhân DSP và khối nhớ M10K quý giá trên FPGA mà không đem lại bất kỳ giá trị thực tiễn nào.

Khái niệm Hệ số thời gian thực (RTF) và Độ trễ so với Thông lượng

Hệ số thời gian thực (**RTF – Real-Time Factor**) được định nghĩa là tỉ số giữa thời gian xử lý của phần cứng chia cho thời lượng thực tế của đoạn âm thanh:

$$\text{RTF} = \frac{T_{\text{xử lý}}}{T_{\text{âm thanh}}} = \frac{\text{Số chu kỳ FPGA}}{N_{\text{budget/hop}}} \tag{47}$$

Riêng đối với khối tự tương quan MAC, hệ số RTF đạt được là:

$$\text{RTF}_{\text{MAC}} = \frac{98.543}{500.000} \approx 0{,}197086 \approx 0{,}197 \tag{48}$$

Tổng số chu kỳ của toàn bộ lõi FPGA (bao gồm cả trừ trung bình 426 mẫu, nhân cửa sổ Hanning 638 mẫu, chuẩn hóa cửa sổ, nội suy parabol và lọc median-3) ước lượng chỉ mất khoảng 99.700 chu kỳ/khung (sẽ phân tích chi tiết ở Chương 12):

$$\text{RTF}_{\text{core}} = \frac{99.700}{500.000} \approx 0{,}1994 \approx 0{,}199 \tag{49}$$

Hệ số RTF $\approx 0{,}20$ có nghĩa là: mạch số FPGA chỉ mất 2 mili-giây để tính toán xong một khung âm thanh dài 10 mili-giây! Phần cứng chạy nhanh gấp 5 lần tốc độ phát âm thanh thời gian thực.

Chỗ dễ vướng: Độ trễ đường ống (Latency) có làm chậm 98.540 phép tính không? Câu trả lời là hoàn toàn không. Người học cần phân biệt rành mạch giữa *độ trễ (latency)* và *thông lượng (throughput)*. Hãy liên tưởng đến một băng chuyền nhà máy: món hàng đầu tiên mất 3 nhịp di chuyển mới đến điểm cuối của băng chuyền (latency = 3 chu kỳ). Nhưng kể từ thời điểm đó trở đi, cứ mỗi nhịp xung nhịp đồng hồ lại có một món hàng hoàn thiện lăn ra khỏi băng chuyền (throughput = 1 kết quả/nhịp). Vì 98.540 cặp mẫu được phát liên tiếp không nghỉ, ta thu được 98.540 kết quả chỉ sau $98.540 + 3 = 98.543$ chu kỳ. Khoảng trễ 3 chu kỳ trên tổng số gần một trăm nghìn chu kỳ chỉ chiếm tỉ lệ vỏn vẹn $3/98.543 \approx 0{,}003\%$, hoàn toàn không đáng kể.

Ba phép đo thời gian và định nghĩa RTF trong hệ thống

Trong báo cáo kiểm thử toàn tuyến (Chương 13), ta đo đạc và so sánh ba mốc thời gian độc lập:

1. **Thời gian lõi FPGA (Core Cycle Count):** Đo bằng thanh ghi đếm chu kỳ phần cứng (`cycle counter`) tích hợp trực tiếp trên FPGA, bấm giờ từ lúc cờ `start` bật đến khi cờ `done` tích cực; quy đổi $\text{RTF}_{\text{FPGA}} = \text{Số chu kỳ}/500.000$.
2. **Thời gian toàn trình HPS (End-to-End HPS Time):** Đo bằng hàm chuẩn POSIX `clock_gettime` (với nguồn xung `CLOCK_MONOTONIC`) trên hệ điều hành Linux của HPS, bao gồm toàn bộ thời gian: đọc tệp WAV từ thẻ SD, bóc tách cấu trúc, tính GP, ghi mẫu qua cầu nối bus, đợi FPGA xử lý, đọc kết quả F0 từ FIFO và ghi tệp nhị phân xuống đĩa.
3. **Thời gian Praat trên máy tính (Laptop Praat Time):** Đo trực tiếp bên trong kịch bản Praat (tệp `praat_pitch.praat`) bằng các lệnh bấm giờ nội bộ, nhằm loại trừ thời gian khởi tạo giao diện GUI và nạp thư viện hệ điều hành. Do Praat mặc định kích hoạt tính toán đa luồng (cơ chế `MelderThread`), số luồng CPU của máy tính xách tay phải được ghi nhận rõ ràng trong báo cáo đo đạc.

Mạch phần cứng Chương 0: Bộ phát cặp mẫu tự tương quan (`ch00_pair_counter`)

Để gắn kết chặt chẽ ngân sách chu kỳ lý thuyết với phần cứng thực tế, ta thiết kế một khối phần cứng tổng hợp được: `rtl/v3/ch00_pair_counter.v`. Khối này có nhiệm vụ phát tuần tự chuỗi địa chỉ $(n, n+\tau)$ để truy xuất bộ nhớ RAM hai cổng, đồng thời sinh ra các cờ điều khiển `sload` (báo nạp tích mới) và `last` (báo kết thúc lag) đúng nhịp cho khối MAC ở Chương 7.

Vì sao không có Intel IP cho khối này? Trong thư viện IP Catalog của Intel Quartus Prime, Intel cung cấp các bộ đếm chuẩn (LPM_COUNTER) hoặc bộ tạo địa chỉ DMA tuyến tính. Tuy nhiên, quy luật duyệt chỉ số tự tương quan $(n, n+\tau)$ trượt hai chiều với giới hạn $n \in [0, N-1-\tau]$ biến thiên theo từng độ trễ τ là một hàm chỉ số phi tuyến tính đặc thù. Không có bất kỳ khối IP nào hỗ trợ sẵn cấu trúc này; việc tự viết mã HDL là lựa chọn tối ưu, linh hoạt và tiết kiệm tài nguyên nhất.

Listing 1: Trích đoạn máy trạng thái và bộ phát cặp mẫu trong `rtl/v3/ch00_pair_counter.v`

```
module ch00_pair_counter #(
    parameter integer N          = 638,
    parameter integer TAU_MIN    = 26,
    parameter integer TAU_MAX    = 214,
    parameter integer ADDR_WIDTH = 10
)(
    input  wire                  clk, rst_n, start,
    input  wire [ADDR_WIDTH-1:0] cfg_n, cfg_tau_min, cfg_tau_max,
    output reg                   busy, done, valid, sload, last,
    output reg  [ADDR_WIDTH-1:0] addr_a, addr_b, tau_out,
    output reg  [31:0]           total_cycles, pair_count
);
    localparam [1:0] STATE_IDLE = 2'b00, STATE_RUN = 2'b01, STATE_DONE = 2'b10;
    reg [1:0]           state;
    reg [ADDR_WIDTH-1:0] n_cnt, tau_cnt, act_n, act_tau_min, act_tau_max;

    wire [ADDR_WIDTH-1:0] max_n = (tau_cnt == 0) ?
                                  (act_n - 1'b1) : (act_n - 1'b1 - tau_cnt);

    always @(posedge clk or negedge rst_n) begin
```

```
        if (!rst_n) begin
            state <= STATE_IDLE;
            busy <= 0; done <= 0; valid <= 0; sload <= 0; last <= 0;
            addr_a <= 0; addr_b <= 0; tau_out <= 0;
            total_cycles <= 0; pair_count <= 0;
            n_cnt <= 0; tau_cnt <= 0;
            act_n <= N; act_tau_min <= TAU_MIN; act_tau_max <= TAU_MAX;
        end else begin
            case (state)
                STATE_IDLE: begin
                    done <= 0;
                    if (start) begin
                        state <= STATE_RUN; busy <= 1; valid <= 1; sload <= 1;
                        act_n <= (cfg_n != 0) ? cfg_n : N;
                        act_tau_min <= (cfg_n != 0) ? cfg_tau_min : TAU_MIN;
                        act_tau_max <= (cfg_n != 0) ? cfg_tau_max : TAU_MAX;
                        addr_a <= 0; addr_b <= 0; tau_cnt <= 0; n_cnt <= 0;
                        last <= ((cfg_n != 0 ? cfg_n : N) == 1);
                        total_cycles <= 1; pair_count <= 1;
                    end
                end
                STATE_RUN: begin
                    total_cycles <= total_cycles + 1'b1;
                    if (n_cnt < max_n) begin
                        n_cnt <= n_cnt + 1'b1;
                        addr_a <= n_cnt + 1'b1;
                        addr_b <= (tau_cnt == 0) ?
                                  (n_cnt + 1'b1) : (n_cnt + 1'b1 + tau_cnt);
                        sload <= 0;
                        last <= (n_cnt + 1'b1 == max_n);
                        pair_count <= pair_count + 1'b1;
                    end else begin
                        if (tau_cnt == 0) begin // Xong R[0], sang TAU_MIN
                            tau_cnt <= act_tau_min; tau_out <= act_tau_min;
                            n_cnt <= 0; addr_a <= 0; addr_b <= act_tau_min;
                            sload <= 1;
                            last <= (act_n - 1'b1 - act_tau_min == 0);
                            pair_count <= pair_count + 1'b1;
                        end else if (tau_cnt < act_tau_max) begin // Sang tau ke tiep
                            tau_cnt <= tau_cnt + 1'b1; tau_out <= tau_cnt + 1'b1;
                            n_cnt <= 0; addr_a <= 0; addr_b <= tau_cnt + 1'b1;
                            sload <= 1;
                            last <= (act_n - 1'b1 - (tau_cnt + 1'b1) == 0);
                            pair_count <= pair_count + 1'b1;
                        end else begin // Hoan tat tat ca cac lag
                            valid <= 0; sload <= 0; last <= 0; busy <= 0; done <= 1;
                            state <= STATE_DONE;
                        end
                    end
                end
                STATE_DONE: begin
                    busy <= 0; valid <= 0; sload <= 0; last <= 0; done <= 1;
                    if (!start) state <= STATE_IDLE;
                end
            endcase
        end
    end
endmodule
```

Giải thích mã nguồn Verilog chi tiết từng dòng:

- **Dòng 1–12 (Khai báo cổng):** Nhận xung nhịp `clk`, reset tích cực mức thấp `rst_n`, xung kích hoạt `start`, và 3 cổng cấu hình độ dài động `cfg_n, cfg_tau_min, cfg_tau_max`. Xuất các cờ trạng thái `busy, done, valid, sload, last`, địa chỉ truy xuất RAM hai cổng `addr_a, addr_b`, chỉ số độ trễ hiện tại `tau_out`, cùng hai thanh ghi đếm chu kỳ và đếm cặp mẫu `total_cycles, pair_count`.
- **Dòng 17–18 (Tín hiệu tổ hợp `max_n`):** Biến `max_n` được tính bằng mạch logic tổ hợp (`wire`) thay vì thanh

ghi có chốt nhịp. Khi $\tau = 0$, chỉ số mẫu cuối là $N-1$; khi $\tau \geq \tau_{\min}$, chỉ số cuối là $N-1-\tau$. Việc tính tổ hợp cho phép FSM đánh giá tức thời điều kiện chạm đỉnh lag (`n_cnt == max_n`) ngay trong chu kỳ hiện tại, loại bỏ hoàn toàn độ trễ 1 nhịp xung nhịp so với việc dùng thanh ghi.

- **Dòng 30–41 (`STATE_IDLE`):** Khi nhận xung kích hoạt `start`:
  - *Dự phòng khi `cfg_n == 0` (dòng 34–36):* Biểu thức `act_n <= (cfg_n != 0) ? cfg_n : N` (tương tự cho `act_tau_min, act_tau_max`) tạo cơ chế dự phòng linh hoạt: mạch nhận tham số cấu hình động từ thanh ghi HPS khi có yêu cầu, hoặc tự động lùi về tham số tĩnh mặc định ($N = 638, \tau \in [26, 214]$) nếu phần mềm truyền giá trị 0.
  - *Khởi tạo `total_cycles` và `pair_count` bằng 1 (dòng 39):* Ngay trong chính chu kỳ nhận `start` này, cặp mẫu đầu tiên $(0, 0)$ của $R[0]$ đã bắt đầu được xuất ra bus địa chỉ cùng cờ `valid = 1, sload = 1`; do đó hai bộ đếm này phải bắt đầu từ 1 để phản ánh chính xác từng chu kỳ vật lý thực tế.
  - *Tính trước cờ `last` (dòng 38):* Biểu thức `last <= ((cfg_n != 0 ? cfg_n : N) == 1)` tính trước cờ kết thúc cho trường hợp biên suy biến $N = 1$ (chỉ có đúng 1 mẫu).
- **Dòng 42–70 (`STATE_RUN`):** Mạch thực thi phát cặp mẫu liên tục không bóng bóng:
  - *Dòng 43–51 (Quét trong cùng một độ trễ):* Tăng dần con đếm mẫu `n_cnt`. Địa chỉ cổng A nhận `addr_a <= n_cnt + 1`, và cổng B nhận `addr_b <= (tau_cnt == 0) ? (n_cnt + 1) : (n_cnt + 1 + tau_cnt)`. Việc cộng thêm 1 là kỹ thuật định địa chỉ nhìn trước (look-ahead addressing): thanh ghi chốt sẵn địa chỉ của chu kỳ xung nhịp kế tiếp để dữ liệu từ RAM M10K xuất hiện đúng thời điểm khối MAC cần. Cờ `sload` hạ về 0 để tiếp tục cộng dồn. Cờ `last` được kích hoạt sớm đúng 1 chu kỳ (`last <= (n_cnt + 1 == max_n)`) để tín hiệu kết thúc lag đi kèm đồng bộ với mẫu cuối cùng vào đường ống MAC.
  - *Dòng 52–56 (Chuyển tiếp tức thì $R[0] \to \tau_{\min}$):* Khi vừa hoàn tất mẫu cuối cùng của $R[0]$ (`tau_cnt == 0`), mạch lập tức nhảy sang `act_tau_min`, phát địa chỉ $(0, \tau_{\min})$, bật cờ `sload = 1` để khối MAC khởi tạo tổng tích lũy mới mà không cần nhịp xóa thanh ghi, và kiểm tra điều kiện biên $N-1-\tau_{\min} == 0$ (trường hợp độ trễ chỉ có đúng 1 mẫu) để chốt cờ `last` ngay.
  - *Dòng 57–64 (Chuyển sang độ trễ kế tiếp $\tau+1$):* Khi ở $\tau < \tau_{\max}$, mạch tăng τ thêm 1, reset `n_cnt = 0`, phát $(0, \tau+1)$, kích hoạt `sload = 1`, và kiểm tra trường hợp góc $N-1-(\tau+1) == 0$ cho cờ `last`. Toàn bộ quá trình chuyển lag diễn ra trong đúng 1 chu kỳ xung nhịp, không có bất kỳ nhịp chết nào.
  - *Dòng 65–68 (Nhánh hoàn tất toàn bộ các lag):* Khi hoàn thành mẫu cuối cùng của độ trễ cực đại $\tau_{\max}$, mạch hạ toàn bộ cờ `valid, sload, last`, bật cờ `done = 1` và chuyển sang `STATE_DONE`.

Mô hình tương đương Python (Twin) và Kết quả kiểm chứng iverilog: Bộ mô hình tương đương số nguyên viết bằng Python (`verify_v3/ch00_pair_counter_twin.py`) mô phỏng chính xác hành vi phát cặp mẫu và tính giá trị băm kiểm tra (checksum 32-bit):

Listing 2: Mô hình tương đương `PairCounterTwin` trong `ch00_pair_counter_twin.py`

```
class PairCounterTwin:
    def __init__(self, N, tau_min, tau_max):
        self.N, self.tau_min, self.tau_max = N, tau_min, tau_max

    def generate_pairs(self):
        pairs = []
        # Lag 0: R[0]
        for n in range(self.N):
            pairs.append({
                'a': n, 'b': n, 'tau': 0,
                'sload': int(n == 0), 'last': int(n == self.N - 1)
            })
        # Lags tau_min .. tau_max
        for tau in range(self.tau_min, self.tau_max + 1):
            max_n = self.N - 1 - tau
            for n in range(max_n + 1):
                pairs.append({
                    'a': n, 'b': n + tau, 'tau': tau,
                    'sload': int(n == 0), 'last': int(n == max_n)
```

```
                })
        return pairs
```

Giải thích cấu trúc và vận hành của mô hình Python Twin: Lớp `PairCounterTwin` đóng vai trò là quy chuẩn vàng (golden specification) độc lập về mặt giải thuật để đối soát với phần cứng:

- *Vòng lặp 1 (dòng 8–14):* Duyệt N mẫu của năng lượng $R[0]$, sinh ra các từ điển chứa địa chỉ $a = n, b = n, \tau = 0$, đánh dấu cờ nạp `sload = 1` tại mẫu đầu tiên ($n = 0$) và cờ kết thúc `last = 1` tại mẫu cuối cùng ($n = N - 1$).
- *Vòng lặp 2 (dòng 16–23):* Lần lượt duyệt qua từng độ trễ τ từ τ_{min} đến τ_{max}. Với mỗi độ trễ, số cặp mẫu tối đa là $\text{max}_n = N - 1 - \tau$. Vòng lặp trong duyệt $n \in [0, \text{max}_n]$, sinh ra cặp địa chỉ $a = n, b = n + \tau$, đánh dấu `sload = 1` tại $n = 0$ và `last = 1` tại $n = \text{max}_n$.
- Danh sách từ điển trả về chứa toàn bộ chuỗi sự kiện chu kỳ đối chu kỳ, được kịch bản kiểm thử dùng để đối chiếu bit-exact với dữ liệu mô phỏng từ testbench Verilog.

Kịch bản kiểm thử (tệp `verify_v3/ch00_test_pair_counter.py`) kết nối với testbench Verilog `tb/v3/tb_ch00_pair_counter.v`, biên dịch bằng `iverilog -g2005` và thực thi bằng `vvp`. Quá trình kiểm chứng bao gồm:

- **Khẳng định cấu hình chuẩn Praat ($N = 638, \tau \in [26, 214]$):** Mạch phát đúng 98.540 cặp mẫu trong đúng 98.540 chu kỳ, sinh đúng 190 xung `sload` và 190 xung `last`. Mã băm kiểm tra chuỗi dữ liệu (canonical checksum 32-bit) đạt giá trị `0x26518cf3`, được khẳng định bằng lệnh `assert` trực tiếp trong testbench và trùng khớp hoàn toàn với mô hình Python.
- **So sánh chuỗi vết chi tiết (Trace Diff cho $N \leq 40$):** Với các cấu hình có kích thước nhỏ ($N \leq 40$), testbench xuất toàn bộ chuỗi tín hiệu chu kỳ đối chu kỳ (`valid, sload, last, addr_a, addr_b, tau`) ra tệp vết và so sánh đối chiếu từng dòng (line-by-line diff), xác nhận chuỗi phát của RTL trùng khít hoàn toàn với mô hình Python twin.
- **Khẳng định không bóng bóng trên 200 cấu hình ngẫu nhiên ($N \in [15, 80]$):** Với toàn bộ 200 cấu hình ngẫu nhiên sinh tự động, testbench kiểm tra và khẳng định điều kiện chu kỳ đối chu kỳ không bóng bóng: tổng số chu kỳ thực tế phải thỏa mãn chính xác tuyệt đối công thức total_cycles $= N + \sum_{\tau=\tau_{\text{min}}}^{\tau_{\text{max}}} (N - \tau)$ cho từng ca thử nghiệm, bảo đảm không có chu kỳ chờ (`valid = 0`) nào chen vào giữa các cặp mẫu. Toàn bộ $201/201$ ca kiểm thử đều vượt qua với 0 lỗi (lưu tại `verify_v3/ch00_pair_counter.log`).

Tóm lại: Ở 50 MHz, ngân sách thời gian thực là 500.000 chu kỳ cho mỗi bước nhảy 10 ms; khối tự tương quan cần 98.543 chu kỳ (hệ số RTF $\approx 0{,}197$), được hiện thực bằng mạch phát cặp mẫu `ch00_pair_counter` chuyển độ trễ với 0 nhịp nghỉ, đã kiểm chứng khớp hoàn toàn 201/201 ca thử nghiệm.

0.7. Bản đồ kiểm chứng ba tầng và bộ golden v2

Vấn đề thực tế: Khi hiện thực một chuỗi xử lý tín hiệu số phức tạp lên phần cứng FPGA (DE10-Standard Cyclone V), nếu ta đưa toàn bộ kiến trúc từ xử lý mẫu đến chia nội suy và xuất dữ liệu lên chip, rồi chỉ so sánh kết quả tần số cơ bản F0 cuối cùng với phần mềm Praat chạy trên máy tính, ta sẽ rơi vào bế tắc hoàn toàn khi xuất hiện sai số. Nếu một khung tín hiệu bị lệch tần số hoặc sai phân loại hữu thanh/vô thanh (V/UV), nguyên nhân bắt nguồn từ đâu?

- Do chỉ số cắt khung lệch một mẫu tại tầng HPS?
- Do khối trừ trung bình cục bộ làm tròn sai hoặc tràn số 17-bit?
- Do bảng ROM Hanning lượng tử hóa sai hệ số tỷ lệ?
- Do khối nhân-cộng tự tương quan MAC bị trễ chu kỳ xung nhịp hoặc sai tín hiệu kích hoạt nạp?
- Hay do phép chia điểm đỉnh parabol số nguyên bị mất mát độ chính xác?

Một sai lệch dù chỉ 1 mẫu hay 1 bit thấp nhất (LSB) ở tầng trước sẽ bị tích lũy và khuếch đại thành sai số hàng trăm cent hoặc sai nhãn V/UV ở tầng sau mà không để lại bất kỳ manh mối nào. Vì vậy, ta cần một kiến trúc kiểm chứng đa tầng chặt chẽ, phân định rạch ròi giữa *tính đúng đắn logic của mạch phần cứng* và *sai số xấp xỉ thuật toán so với Praat*.

0.7.1. Kiến trúc kiểm chứng ba tầng (Three-tier Verification)

Chiến lược kiểm chứng của cuốn sách được xây dựng thành ba tầng độc lập, tạo thành ”tấm lưới bảo vệ” khép kín xuyên suốt từ mã nguồn thuật toán đến chân chip FPGA (Hình 1):

1. **Tầng 1 — Khớp bit chính xác (bit-exact) giữa mô hình Python số nguyên và mô phỏng RTL (Twin vs RTL):** Tầng này so sánh đối ứng bit-exact (khớp chính xác từng bit dữ liệu và từng chu kỳ xung nhịp) giữa mô hình tương đương Python (`golden_v2/fixed_twin.py` và các script twin cục bộ của từng chương) với mô phỏng Verilog RTL (`rtl/v3/` và `tb/v3/`) chạy bằng `iverilog/vvp`. Mỗi khối phần cứng phải vượt qua ít nhất 200 véc-tơ kiểm tra ngẫu nhiên cùng các khung tín hiệu biên dồn nén (stress vectors, biên độ cực trị ± 32.768). Tiêu chuẩn nghiệm thu ở tầng 1 là 0 lỗi: chuỗi bit đầu ra và mã băm kiểm tra (checksum) của RTL phải trùng khớp hoàn toàn với mô hình Python. Tầng này loại bỏ triệt để mọi lỗi logic, lỗi tràn số và lỗi sai lệch chu kỳ giao tiếp Avalon-ST.

2. **Tầng 2 — Khớp bit chính xác (bit-exact) giữa mô phỏng RTL và phần cứng chạy thực (RTL vs Board):** Tầng này so sánh kết quả mô phỏng RTL với dữ liệu đo thực tế trên kit DE10-Standard, được trích xuất thông qua bộ phân tích logic nhúng Quartus SignalTap II hoặc đọc trực tiếp từ bộ đệm DDR của HPS qua bus Avalon-MM. Tầng 2 nhằm phát hiện các lỗi vật lý nằm ngoài tầm kiểm soát của code RTL thuần túy: khởi tạo bộ nhớ M10K qua file `.mif`/`.hex`, cấu hình bộ chia tần và độ ổn định của khối PLL (50 MHz), ràng buộc chân IO, và sự đồng bộ tín hiệu bắt tay trên bus hệ thống Platform Designer (Qsys). Tiêu chuẩn nghiệm thu: chuỗi kết quả F0 phần cứng trả về HPS phải trùng khít hoàn toàn với chuỗi kết quả của mô phỏng RTL.

3. **Tầng 3 — Đánh giá thống kê giữa phần cứng và chuẩn Praat trên máy tính (FPGA/Twin vs Praat):** Sau khi tầng 1 và tầng 2 đảm bảo phần cứng thực thi đúng đặc tả toán học số nguyên, tầng 3 tiến hành so sánh đối chiếu giữa đầu ra của FPGA (hoặc Python Twin) với phần mềm Praat 7.0.03 gốc chạy trên máy tính thông qua công cụ `golden_v2/compare.py`. Thước đo ở tầng này không còn là khớp bit mà là các chỉ số xử lý tín hiệu khách quan: VDE, GPE, FFE và phân bố sai số cent mịn (trung bình, độ lệch chuẩn, phân vị 95, cực đại). Mục tiêu là chứng minh độ lệch âm học giữa giải pháp phần cứng và Praat luôn nằm sâu dưới ngưỡng nghe nhận biết của tai người, đồng thời giải thích tường minh nguyên nhân của từng khung sai lệch.

0.7.2. Cấu trúc bộ công cụ golden v2

Toàn bộ mã nguồn kiểm chứng tham chiếu được đặt tại thư mục `golden_v2/`, đóng vai trò ”trọng tài kỹ thuật” cho toàn bộ tài liệu:

- `constants.py`: Định nghĩa tập trung mọi hằng số suy xuất trực tiếp từ mã nguồn C++ của Praat 7.0.03 (`Sound_to_Pitch.cpp`), bao gồm kiểm tra khẳng định `assert`.
- `frames.py`: Hiện thực hóa chia lưới khung ngắn hạn (`Sampled_shortTermAnalysis`) bằng số thực kép.
- `praat_float_ref.py`: Bản chuyển đổi trực tiếp thuật toán Praat 7.0.03 sang Python số thực chuẩn: trích chọn 15 ứng viên, nội suy sơ bộ parabol, tinh chỉnh độ chính xác cao sinc70 + Brent, bộ dò đường Viterbi (R_1) và đường chi phí bằng 0 (R_2), kết hợp bộ lọc trung vị median-3.
- `fixed_twin.py`: Mô hình số nguyên tương đương bit-exact toàn tuyến. Phần tài liệu docstring của tệp này chính là bản đặc tả số nguyên 10 bước chuẩn hóa mà mạch phần cứng RTL bắt buộc phải tuân theo.
- `gen_roms.py`: Kịch bản tự động sinh bảng ROM Hanning Q1.15 (`rom/hanning638_q15.hex`), ROM bù suy giảm $C[\tau]$ và chi phí $K[\tau]$ (`rom/ck_rom.hex`), cùng tệp `rom/constants.vh`.
- `compare.py`: Kịch bản tự động thực thi chuỗi so sánh đa phương diện, xuất báo cáo đối soát chi tiết (`golden_v2/out/compare_report.md`).

0.7.3. Đặc tả 10 bước số nguyên của lõi phần cứng (fixed_twin.py)

Để mạch phần cứng FPGA có thể hoạt động hoàn toàn bằng số nguyên có dấu với tài nguyên tối thiểu mà không cần khối dấu phẩy động tốn kém, thuật toán ước lượng F0 được phân rã thành 10 bước xử lý số nguyên bit-exact như sau:

1. **Bước 1 — Trừ trung bình cục bộ:** Lấy tổng tích lũy $S = \sum_{j=106}^{531} x[j]$ trên đúng 426 mẫu quanh tâm khung (ứng với hai chu kỳ dài nhất tại floor 75 Hz). Giá trị trung bình nguyên được tính bằng phép nhân-dịch thay thế bộ chia:
$$\text{mean} = \left\lfloor \frac{(S + 213 + 426 \cdot 2^{15}) \cdot 10.082.083}{2^{32}} \right\rfloor - 2^{15} = \left\lfloor \frac{S + 213}{426} \right\rfloor \tag{50}$$

2. **Bước 2 — Nhân cửa sổ Hanning:** Độ lệch $d[j] = x[j] - \text{mean}$ là số nguyên có dấu 17-bit (nằm trong khoảng $[-65.536, 65.535]$). Mẫu sau khi nhân cửa sổ Hanning định dạng Q1.15 được làm tròn và dịch phải:
$$xw[j] = (d[j] \cdot w[j] + 2^{14}) \gg 15 \tag{51}$$
với $w[j] \in [1, 32.767]$ đọc từ ROM. Khung biên cực đại có thể đạt $|xw| = 59.379$, vượt quá 16-bit nên yêu cầu thanh ghi 17-bit có dấu.

3. **Bước 3 — Đỉnh biên độ cục bộ (LP):** Tìm giá trị cực đại tuyệt đối của mẫu trong vùng trung tâm 214 mẫu quanh tâm khung:
$$LP = \max_{j=212}^{425} |xw[j]| \tag{52}$$

4. **Bước 4 — Khối nhân-cộng tự tương quan (MAC):** Tích lũy năng lượng khung $R[0] = \sum_{j=0}^{637} xw[j]^2$ và tương quan chập cho 189 độ trễ:
$$R[\tau] = \sum_{j=0}^{637-\tau} xw[j] \cdot xw[j+\tau], \quad \tau \in [26, 214] \tag{53}$$

5. **Bước 5 — Chuẩn hóa thang đo động:** Tìm số bit dịch động $sh = \max(0, \text{bitlen}(R[0]) - 26)$ để đưa $R[0]$ và toàn bộ các giá trị $R[\tau]$ về dải số nguyên 27-bit:
$$R_{0n} = R[0] \gg sh, \quad R_n[\tau] = R[\tau] \gg sh \tag{54}$$

6. **Bước 6 — Bù suy giảm cửa sổ và trừ chi phí quãng tám (Scoring):** Tích số chuẩn hóa $U[\tau]$ và điểm số đánh giá $S_c[\tau]$ được tính bằng phép nhân số nguyên với các hệ số đọc từ ROM:
$$U[\tau] = (R_n[\tau] \cdot C[\tau] + 2^{13}) \gg 14 \quad (\approx r(\tau) \cdot R_{0n}) \tag{55}$$
$$S_c[\tau] = U[\tau] - \big((R_{0n} \cdot K[\tau] + 2^{19}) \gg 20\big) \quad \left(\approx \left(r(\tau) - 0{,}01 \log_2 \frac{600}{f}\right) \cdot R_{0n}\right) \tag{56}$$
với $C[\tau] \in$ Q2.14 và $K[\tau] \in$ Q12.20.

7. **Bước 7 — Trích chọn ứng viên cực đại cục bộ:** Quét tìm các độ trễ $\tau \in [27, 213]$ thỏa mãn điều kiện tạo đỉnh cục bộ:
$$U[\tau] > U[\tau - 1], \quad U[\tau] \geq U[\tau + 1], \quad 1000 \cdot U[\tau] > 225 \cdot R_{0n} \tag{57}$$
Ứng viên tốt nhất được chọn là vị trí độ trễ τ_p có điểm số $S_c[\tau]$ lớn nhất (argmax).

8. **Bước 8 — Quyết định hữu thanh/vô thanh (Voicing):** Khung được xác nhận là hữu thanh khi và chỉ khi bất đẳng thức tích chéo số nguyên sau đây thỏa mãn:
$$(100 \cdot S_c[\tau_p] - 45 \cdot R_{0n}) \cdot 3 \cdot GP > 100 \cdot R_{0n} \cdot \max(0, 6 \cdot GP - 145 \cdot LP) \tag{58}$$
Bất đẳng thức này phản ánh chính xác điều kiện ngưỡng voicing của Praat: strength $> 0{,}45 + \max(0, 2 - I/(0{,}03/1{,}45))$ với cường độ tương đối $I = LP/GP$.

9. **Bước 9 — Nội suy đỉnh parabol và nghịch đảo tần số F0 gộp:** Lấy hiệu độ dốc $num = U[\tau_p{+}1] - U[\tau_p{-}1]$ và độ cong $den = 2U[\tau_p] - U[\tau_p - 1] - U[\tau_p + 1]$. Sau khi dịch chuẩn hóa $s_2 = \max(0, \text{bitlen}(den) - 24)$, mẫu số chung $D = 2 \cdot den_n \cdot \tau_p + num_n$. Giá trị tần số F0 định dạng Q12.4 được tính gọn bằng duy nhất một phép chia số nguyên:
$$F0_Q12.4 = \left\lfloor \frac{512.000 \cdot den_n + \lfloor D/2 \rfloor}{D} \right\rfloor \tag{59}$$

10. **Bước 10 — Bộ lọc trung vị median-3:** Đầu ra tức thời của khung là $F0_Q12.4$ nếu hữu thanh hoặc 0 nếu vô thanh; sau đó được đưa qua bộ lọc trung vị trượt 3 khung liên tiếp để triệt tiêu các xung nhảy quãng tám cô lập.

0.7.4. Kết quả đối chiếu trên tập tín hiệu tổng hợp chuẩn

Để đánh giá định lượng độ chính xác của bộ mô hình `golden_v2`, ta thực hiện kiểm thử toàn diện trên hai tệp âm thanh tổng hợp chuẩn (tần số lấy mẫu 16 kHz, độ dài 2,8 giây, gồm 277 khung phân tích, bước nhảy hop 160 mẫu, $t_1 = 0{,}020$ s). Tệp thứ nhất là tín hiệu sạch không tạp âm (`synth_glide_vuv_16k.wav`, $GP = 22.946$), tệp thứ hai được cộng thêm nhiễu trắng ở tỷ số $SNR = 5$ dB (`synth_glide_vuv_16k_snr5.wav`, $GP = 22.937$) để kích hoạt các tình huống nhảy quãng tám phức tạp. Kết quả tổng hợp từ công cụ `golden_v2/compare.py` kết hợp kịch bản trích xuất sai số cent mịn `verify_v3/ch00_table_golden.py` được trình bày trong Bảng 4.

Bảng 4: Kết quả đối chiếu toàn tuyến bộ công cụ golden v2 với Praat 7.0.03 trên tín hiệu tổng hợp chuẩn (277 khung, 16 kHz). Lưu ý: Các cột sai số cent phản ánh thống kê cent mịn (loại bỏ khung lỗi thô GPE theo đúng định nghĩa chuẩn tại Mục 0.5), khác với báo cáo `compare_report.md` vốn tính trung bình và phân vị 95 trên toàn bộ các khung cùng có thanh bao gồm cả lỗi thô.

Phép so sánh	**VDE**	**GPE**	**FFE**	**Cent TB**	**Cent p_{95}**	**Cent Max**
Tín hiệu sạch (Clean synth, 208 khung cùng hữu thanh):						
Port float vs Praat R2	0/277	0/208	0,0000	$7{,}9 \cdot 10^{-6}$	$5{,}8 \cdot 10^{-5}$	$7{,}3 \cdot 10^{-5}$
Port float vs Praat R1	0/277	0/208	0,0000	$7{,}9 \cdot 10^{-6}$	$5{,}8 \cdot 10^{-5}$	$7{,}3 \cdot 10^{-5}$
Praat R1 vs Praat R2	0/277	0/208	0,0000	0,00	0,00	0,00
Twin (trước median) vs Praat R2	0/277	0/208	0,0000	0,21	0,48	1,08
Twin + median-3 vs Praat R1	0/277	0/208	0,0000	0,37	0,64	12,96
Praat R1 vs Ground-Truth	4/277	0/207	0,0144	0,30	0,83	7,91
Twin + median-3 vs Ground-Truth	4/277	0/207	0,0144	0,53	1,05	14,28
Tín hiệu có ồn SNR 5 dB (207 khung cùng hữu thanh):						
Port float vs Praat R2	0/277	0/207	0,0000	$1{,}2 \cdot 10^{-5}$	$6{,}3 \cdot 10^{-5}$	$1{,}5 \cdot 10^{-4}$
Port float vs Praat R1	0/277	0/207	0,0000	$1{,}2 \cdot 10^{-5}$	$6{,}4 \cdot 10^{-5}$	$1{,}5 \cdot 10^{-4}$
Praat R1 vs Praat R2 (Viterbi)	0/277	19/207	0,0686	1,01	0,00	37,28
Twin (trước median) vs Praat R2	0/277	3/207	0,0108	3,07	9,78	32,91
Twin + median-3 vs Praat R1	0/277	15/207	0,0542	5,58	20,62	48,41
Praat R1 vs Ground-Truth	3/277	0/207	0,0108	10,56	21,80	29,32
Praat R2 vs Ground-Truth	3/277	19/207	0,0794	10,55	21,79	28,48
Twin (trước median) vs Ground-Truth	3/277	22/207	0,0903	9,12	20,53	25,85
Twin + median-3 vs Ground-Truth	3/277	15/207	0,0650	9,71	25,76	32,68

Đọc bảng. Bảng 4 làm sáng tỏ các mối quan hệ nền tảng của toàn bộ hệ thống:

- *Độ chính xác của bản port mã nguồn (Port float vs Praat):* Sai số cent giữa bản Python dấu phẩy động và phần mềm Praat 7.0.03 gốc chỉ ở mức 10^{-5} đến 10^{-4} cent, với $VDE = 0$ và $GPE = 0$. Con số cực nhỏ này khẳng định ta đã mô phỏng chuẩn xác thuật toán C++ của Praat đến từng phép toán dấu phẩy động kép.
- *Vai trò của bộ dò đường Viterbi (Praat R1 vs R2):* Trên tín hiệu sạch, cấu hình mặc định R_1 và cấu hình không chi phí R_2 cho kết quả trùng khít hoàn toàn. Tuy nhiên, khi chịu ảnh hưởng của tạp âm mạnh (SNR 5 dB), bộ dò đường Viterbi của R_1 sửa được 19 khung nhảy quãng tám mà R_2 (thuần túy chọn đỉnh lớn nhất từng khung) mắc phải.
- *Độ chính xác của lõi số nguyên (Twin vs Praat R2):* Trên tín hiệu sạch, mô hình số nguyên khớp sát với Praat R2: $VDE = 0$, $GPE = 0/208$, sai số cent mịn trung bình chỉ $0{,}21$ cent, $p_{95} = 0{,}48$ cent và sai số cực đại là $1{,}08$ cent (hoàn toàn không thể nhận biết bởi tai người). Trên tín hiệu ồn 5 dB, twin chỉ lệch 3/207 khung GPE so với R2 (các khung 63, 64, 172). Nguyên nhân không phải do sai số lượng tử hóa số nguyên, mà là hiện tượng hai đỉnh quãng tám cạnh tranh sít sao (near-ties): Praat xếp hạng ứng viên dựa trên độ mạnh tương quan sau khi đã tinh chỉnh bằng hàm sinc70, trong khi twin xếp hạng dựa trên điểm số trên lưới nguyên trước khi nội suy; chênh lệch điểm số giữa hai đỉnh tốt nhất ở 3 khung này chỉ từ $0{,}0005$ đến $0{,}007$, khiến twin chọn đỉnh quãng tám lân cận. Khi loại bỏ 3 khung lỗi thô này, sai số cent mịn của twin so với R2 trên tín hiệu ồn 5 dB đạt trung bình $3{,}07$ cent, $p_{95} = 9{,}78$ cent và cực đại $32{,}91$ cent.
- *Hiệu quả của bộ lọc trung vị phần cứng (Twin + median-3 vs Ground-Truth):* Trên tín hiệu ồn 5 dB, so với sự thật mặt đất (ground-truth), bộ lọc median-3 giúp giảm số khung lỗi thô GPE của phần cứng từ 22 khung xuống còn 15 khung (sửa được 7 khung lỗi quãng tám đơn lẻ), chứng minh hiệu quả rõ rệt của một cấu trúc phần cứng nhỏ gọn.

Ba khác biệt có chủ ý và phân tích hạn chế cố hữu

Để giữ sự minh bạch khoa học, ta cần chỉ rõ ba sai khác có chủ ý giữa thiết kế phần cứng FPGA và thuật toán gốc của Praat (đã được thống nhất tại Mục 0.3):

1. **Nội suy đỉnh parabol thay cho sinc70 + Brent:** Trên tín hiệu sạch, parabol cho sai số nội suy nhỏ hơn $1{,}1$ cent (sai số cực đại $1{,}08$ cent). Tuy nhiên, trên tín hiệu có ồn lớn (SNR 5 dB), hình dạng đỉnh tự tương quan bị méo mó hoặc chẻ đôi, khiến xấp xỉ parabol lệch tới 32,9 cent so với tinh chỉnh sinc70. Đây là cái giá phải trả để tiết kiệm hàng chục nghìn chu kỳ tính toán và tránh thuật toán tìm nghiệm Brent phi tuyến tính trên FPGA.
2. **Xếp hạng ứng viên bằng điểm số trên lưới rời rạc:** Thuật toán phần cứng chọn đỉnh ứng viên tốt nhất $\tau_p = \text{argmax}\, S_c[\tau]$ dựa trên các giá trị nguyên rời rạc trên lưới trước khi nội suy, thay vì tính điểm sức mạnh S_c tại tọa độ thực sau nội suy như Praat. Tại các khung có hai đỉnh quãng tám cạnh tranh sít sao (near-ties) với độ chênh lệch chi phí cục bộ cực nhỏ (chỉ từ $0{,}0005$ đến $0{,}007$ như ghi nhận trong `golden_v2/README`), sự khác biệt giữa điểm số trên lưới và độ mạnh sau nội suy sinc khiến thứ tự ưu tiên giữa hai đỉnh có thể bị hoán đổi (gây ra hiện tượng lệch 3/207 khung GPE ở tệp thử nghiệm SNR 5 dB).
3. **Bộ lọc trung vị median-3 thay cho bộ dò đường quy hoạch động Viterbi:** Vì FPGA xử lý luồng theo từng khung thời gian thực và không lưu toàn bộ ma trận ứng viên của cả tệp âm thanh vào bộ nhớ lớn, nó không thể chạy thuật toán Viterbi toàn cục. Median-3 chỉ quan sát cửa sổ cục bộ 3 khung nên chỉ sửa được các lỗi nhảy quãng tám đơn lẻ (1 khung), không thể khắc phục được chuỗi nhảy quãng tám kéo dài từ 2 khung trở lên.

Ghi chú về tập dữ liệu thử nghiệm: Toàn bộ các con số kiểm chứng trong chương này được tính toán nghiêm ngặt trên tập tín hiệu tổng hợp chuẩn (`synth_glide_vuv_16k.wav` và phiên bản SNR 5 dB). Tập dữ liệu các tệp WAV giọng nói thực tế của anh Tú hiện chưa sẵn sàng tại thời điểm xuất bản chương này; khi có dữ liệu, kịch bản tự động `golden_v2/compare.py` sẽ được tái thực thi để cập nhật bổ sung mà không làm thay đổi kiến trúc kiểm chứng ba tầng đã thiết lập.

Tóm lại: Bản đồ kiểm chứng ba tầng bảo đảm sự nhất quán chặt chẽ giữa mã nguồn Python twin số nguyên, mô phỏng RTL, phần cứng thực tế trên kit DE10-Standard và chuẩn Praat 7.0.03; trên tín hiệu sạch, lõi phần cứng đạt sai số cent trung bình chỉ $0{,}21$ *cent và* $GPE = 0$*; trên tín hiệu ồn 5 dB, sai số cent mịn là* $3{,}07$ *cent (*$p_{95} = 9{,}78$ *cent), và bộ lọc median-3 giúp giảm lỗi thô từ 22 khung xuống 15 khung.*

Bài tập cuối chương

Dưới đây là các bài tập vận dụng nhằm củng cố các nguyên lý nền tảng đã trình bày trong Chương 0. Tất cả các bài tập đều có lời giải chi tiết và số liệu đã được kiểm chứng bằng kịch bản `verify_v3/ch00_exercises.py`.

Bài tập 1 (Hiện tượng nhân đôi tần số khi đếm điểm qua không). Cho tín hiệu âm thanh liên tục có dạng:

$$s(t) = \sin(2\pi \cdot 120t) + 0{,}8\sin(2\pi \cdot 240t) \tag{60}$$

Tín hiệu được lấy mẫu ở tần số $f_s = 16$ kHz.

a) Xác định chu kỳ cơ bản thực tế T_0 và tần số cơ bản F_0 của tín hiệu.

b) Tìm tất cả các thời điểm tín hiệu đổi dấu ($s(t) = 0$) trong một chu kỳ $[0, T_0)$.

c) Nếu một thuật toán đếm số lần đổi dấu để ước lượng F0 trong khoảng thời gian quan sát $T_{\text{obs}} = 0{,}05$ s (50 ms), hãy tính tổng số điểm qua không ghi nhận được và tần số ước lượng $\hat{F}_0$. Nhận xét về kết quả này.

Lời giải chi tiết:

a) Tín hiệu $s(t)$ là tổng của hai thành phần tần số 120 Hz và 240 Hz. Ước số chung lớn nhất của 120 và 240 là 120 Hz. Do đó, tần số cơ bản thực tế là $F_0 = 120$ Hz, tương ứng với chu kỳ cơ bản:

$$T_0 = \frac{1}{120}\ \text{s} \approx 8{,}333\ \text{ms} \tag{61}$$

b) Đặt $\theta = 2\pi \cdot 120t = \omega_0 t$. Khi $t \in [0, T_0)$, góc pha biến thiên trong khoảng $\theta \in [0, 2\pi)$. Ta có:

$$s(t) = \sin(\theta) + 0{,}8\sin(2\theta) = \sin(\theta) + 1{,}6\sin(\theta)\cos(\theta) = \sin(\theta) \cdot (1 + 1{,}6\cos(\theta)) \tag{62}$$

Phương trình $s(t) = 0$ tương đương với hai trường hợp:

- $\sin(\theta) = 0 \implies \theta_1 = 0, \theta_3 = \pi \approx 3{,}1416$ rad.
- $1 + 1{,}6\cos(\theta) = 0 \implies \cos(\theta) = -\frac{1}{1{,}6} = -0{,}625$. Trong khoảng $[0, 2\pi)$, phương trình có hai nghiệm: $\theta_2 = \arccos(-0{,}625) \approx 2{,}2459$ rad và $\theta_4 = 2\pi - 2{,}2459 \approx 4{,}0373$ rad.

Như vậy, trong một chu kỳ T_0, tín hiệu $s(t)$ cắt trục hoành tại đúng 4 điểm pha:

$$\theta \in \{0;\ 2{,}2459;\ 3{,}1416;\ 4{,}0373\} \tag{63}$$

c) Trong khoảng thời gian $T_{\text{obs}} = 0{,}05$ s, số lượng chu kỳ cơ bản hoàn chỉnh là:

$$N_{\text{period}} = \frac{T_{\text{obs}}}{T_0} = \frac{0{,}05}{1/120} = 6{,}0 \text{ chu kỳ} \tag{64}$$

Mỗi chu kỳ có 4 điểm qua không, do đó tổng số lần đổi dấu trong $0{,}05$ s là:

$$N_{\text{zeros}} = 6{,}0 \times 4 = 24 \text{ điểm} \tag{65}$$

Vì mỗi chu kỳ sóng chuẩn chỉ có 2 lần đổi dấu (nửa chu kỳ dương và nửa chu kỳ âm), thuật toán ngây thơ ước lượng tần số theo công thức:

$$\hat{F}_0 = \frac{N_{\text{zeros}}}{2 \cdot T_{\text{obs}}} = \frac{24}{2 \times 0{,}05} = 240{,}0 \text{ Hz} \tag{66}$$

Nhận xét: Tần số ước lượng bị nhân đôi ($\hat{F}_0 = 240$ Hz thay vì 120 Hz), dẫn tới sai số 100% (1200 cent). Nguyên nhân là do thành phần hài âm bậc hai có biên độ lớn ($0{,}8$) đã làm uốn cong dạng sóng và tạo thêm hai điểm cắt giả trong mỗi chu kỳ.

Bài tập 2 (Độ phân giải phổ FFT và sai số cent ở tần số 150 Hz). Một thuật toán ước lượng F0 thực hiện lấy đỉnh biên độ phổ sau biến đổi Fourier nhanh (FFT) độ dài $N = 640$ mẫu tại tần số lấy mẫu $f_s = 16$ kHz.

a) Tính độ rộng của mỗi vạch phổ (bin spacing Δf).

b) Giả sử giọng người nói có tần số cơ bản thực tế rơi vào $F_{\text{true}} = 162{,}5$ Hz (nằm chính giữa hai bin kề nhau). Hãy xác định hai bin lân cận và sai số tần số cực đại nếu chọn đỉnh rời rạc lớn nhất mà không dùng thuật toán nội suy.

c) Tính sai số cực đại này theo đơn vị cent âm học so với tần số thực tế $162{,}5$ Hz. So sánh sai số này với ngưỡng nghe nhận biết cao độ (JND) của con người ($5 \ldots 10$ cent đối với âm thanh đơn âm hoặc người nghe được huấn luyện, và $10 \ldots 25$ cent đối với tiếng nói tự nhiên trong ngữ cảnh phức tạp).

Lời giải chi tiết:

a) Độ phân giải tần số của phép biến đổi FFT độ dài $N = 640$ mẫu là:

$$\Delta f = \frac{f_s}{N} = \frac{16.000}{640} = 25{,}0 \text{ Hz} \tag{67}$$

b) Các vạch phổ rời rạc có tần số $f_k = k \cdot \Delta f = k \cdot 25$ Hz ($k \in \mathbb{N}$). Với $F_{\text{true}} = 162{,}5$ Hz:

$$k = \frac{162{,}5}{25} = 6{,}5 \tag{68}$$

Tần số này nằm chính giữa bin $k = 6$ ($f_6 = 6 \times 25 = 150{,}0$ Hz) và bin $k = 7$ ($f_7 = 7 \times 25 = 175{,}0$ Hz). Nếu chỉ chọn bin có biên độ cực đại rời rạc, thuật toán sẽ chọn bin 6 hoặc bin 7. Sai số tần số cực đại trong trường hợp xấu nhất là nửa bin:

$$|\Delta F_{\text{max}}| = \frac{\Delta f}{2} = \frac{25{,}0}{2} = 12{,}5 \text{ Hz} \tag{69}$$

c) Sai số theo thang cent âm học khi làm tròn về bin 7 (175 Hz) là:

$$E_{\text{cent}} = 1200 \cdot \log_2\left(\frac{175{,}0}{162{,}5}\right) = 1200 \cdot \log_2(1{,}07692) \approx 128{,}30 \text{ cent} \tag{70}$$

Nếu làm tròn về bin 6 (150 Hz), sai số là:

$$E_{\text{cent}} = 1200 \cdot \left|\log_2\left(\frac{150{,}0}{162{,}5}\right)\right| = 1200 \cdot \log_2(1{,}08333) \approx 138{,}57 \text{ cent} \tag{71}$$

Nhận xét: Sai số $128 \ldots 138$ cent tương đương hơn 1,3 bán âm (vượt quá khoảng cách giữa nốt Đô và Đô thăng trên phím đàn piano). Trong khi đó, ngưỡng phân biệt cao độ của con người chỉ từ 5 đến 10 cent (với âm thanh đơn âm) và $10 \ldots 25$ cent (với tiếng nói phức tạp). Do đó, phương pháp tìm đỉnh FFT thô hoàn toàn không đáp ứng được yêu cầu phân tích ngữ điệu giọng nói.

Bài tập 3 (Bù suy giảm cửa sổ Hanning trong miền tự tương quan). Xét một chuỗi mẫu tín hiệu ngắn đối xứng gồm 6 điểm:

$$x = [2,\ 4,\ 6,\ 6,\ 4,\ 2] \tag{72}$$

được nhân với một cửa sổ đối xứng hình chuông:

$$w = [0{,}25,\ 0{,}50,\ 1{,}00,\ 1{,}00,\ 0{,}50,\ 0{,}25] \tag{73}$$

a) Xác định chuỗi mẫu sau khi nhân cửa sổ $xw[n] = x[n] \cdot w[n]$ ($n = 0 \ldots 5$).

b) Tính hàm tự tương quan không chuẩn hóa của tín hiệu $R_{xw}(\tau)$ tại $\tau = 0$ và $\tau = 2$.

c) Tính hàm tự tương quan của chính cửa sổ $R_w(\tau)$ tại $\tau = 0$ và $\tau = 2$.

d) Tính hệ số tương quan chưa chuẩn hóa $r_{\text{unnorm}}(2) = R_{xw}(2)/R_{xw}(0)$ và hệ số tương quan chuẩn hóa theo công thức Boersma $r(2) = \frac{R_{xw}(2)/R_w(2)}{R_{xw}(0)/R_w(0)}$. Nhận xét sự khác biệt.

Lời giải chi tiết:

a) Nhân từng phần tử giữa chuỗi x và cửa sổ w:

$$xw[0] = 2 \times 0{,}25 = 0{,}5, \quad xw[1] = 4 \times 0{,}50 = 2{,}0, \quad xw[2] = 6 \times 1{,}00 = 6{,}0 \tag{74}$$

$$xw[3] = 6 \times 1{,}00 = 6{,}0, \quad xw[4] = 4 \times 0{,}50 = 2{,}0, \quad xw[5] = 2 \times 0{,}25 = 0{,}5 \tag{75}$$

Như vậy $xw = [0{,}5,\ 2{,}0,\ 6{,}0,\ 6{,}0,\ 2{,}0,\ 0{,}5]$.

b) Tính tự tương quan của tín hiệu xw:

$$\begin{aligned} R_{xw}(0) &= \sum_{n=0}^{5} xw[n]^2 = 0{,}5^2 + 2{,}0^2 + 6{,}0^2 + 6{,}0^2 + 2{,}0^2 + 0{,}5^2 \\ &= 0{,}25 + 4{,}0 + 36{,}0 + 36{,}0 + 4{,}0 + 0{,}25 = 80{,}500 \end{aligned} \tag{76}$$

$$\begin{aligned} R_{xw}(2) &= \sum_{n=0}^{3} xw[n] \cdot xw[n+2] = (0{,}5 \times 6{,}0) + (2{,}0 \times 6{,}0) + (6{,}0 \times 2{,}0) + (6{,}0 \times 0{,}5) \\ &= 3{,}0 + 12{,}0 + 12{,}0 + 3{,}0 = 30{,}000 \end{aligned} \tag{77}$$

c) Tính tự tương quan của cửa sổ w:

$$\begin{aligned} R_w(0) &= \sum_{n=0}^{5} w[n]^2 = 0{,}25^2 + 0{,}50^2 + 1{,}00^2 + 1{,}00^2 + 0{,}50^2 + 0{,}25^2 \\ &= 0{,}0625 + 0{,}25 + 1{,}0 + 1{,}0 + 0{,}25 + 0{,}0625 = 2{,}6250 \end{aligned} \tag{78}$$

$$\begin{aligned} R_w(2) &= \sum_{n=0}^{3} w[n] \cdot w[n+2] = (0{,}25 \times 1{,}0) + (0{,}50 \times 1{,}0) + (1{,}0 \times 0{,}50) + (1{,}0 \times 0{,}25) \\ &= 0{,}25 + 0{,}50 + 0{,}50 + 0{,}25 = 1{,}5000 \end{aligned} \tag{79}$$

d) So sánh hai hệ số tương quan:

- Hệ số tương quan chưa chuẩn hóa (ngây thơ):

$$r_{\text{unnorm}}(2) = \frac{R_{xw}(2)}{R_{xw}(0)} = \frac{30{,}000}{80{,}500} \approx 0{,}3727 \tag{80}$$

- Hệ số tương quan chuẩn hóa Boersma (chia cho tự tương quan cửa sổ):

$$r(2) = \frac{R_{xw}(2)/R_w(2)}{R_{xw}(0)/R_w(0)} = \frac{30{,}000/1{,}5000}{80{,}500/2{,}6250} = \frac{20{,}000}{30{,}6667} \approx 0{,}6522 \tag{81}$$

Nhận xét và lưu ý quan trọng về tính tuần hoàn: Do hiệu ứng vát biên của cửa sổ, hệ số chưa chuẩn hóa bị nén giảm giả tạo xuống chỉ còn $0{,}3727$ (giảm tới $62{,}7\%$), có thể rơi xuống dưới ngưỡng hữu thanh $0{,}45$ và bị gán nhầm thành vô thanh. Phép chuẩn hóa Boersma đã bù đắp phần suy giảm và nâng giá trị lên $0{,}6522$. Cần lưu ý rằng chuỗi mẫu ngắn $x = [2, 4, 6, 6, 4, 2]$ bản thân không phải là một tín hiệu tuần hoàn (do có dạng đường bao tam giác hữu hạn); bài tập này minh họa rằng phép chia cho tự tương quan cửa sổ $R_w[\tau]$ triệt tiêu độ dốc suy hao do biên cửa sổ hữu hạn (taper bias), chứ không thể biến đổi một tín hiệu không tuần hoàn thành tuần hoàn.

Bài tập 4 (Tính toán các chỉ số đánh giá GPE, VDE, FFE và sai số cent). Một thử nghiệm so sánh giữa phần mềm chuẩn (Ref) và hệ thống FPGA (Test) trên 6 khung phân tích cho kết quả tần số ước lượng như sau:

Khung	Khung 1	Khung 2	Khung 3	Khung 4	Khung 5	Khung 6
F_{Ref} (Hz)	120,0	0,0	180,0	100,0	220,0	0,0
F_{Test} (Hz)	121,0	150,0	0,0	200,0	225,0	0,0

(Quy ước: Giá trị 0 Hz biểu diễn khung vô thanh).

a) Xác định các khung mắc lỗi quyết định hữu thanh và tính tỷ lệ lỗi VDE.

b) Xác định các khung cùng hữu thanh, phân loại các khung mắc lỗi bước nhảy thô và tính tỷ lệ lỗi GPE (với ngưỡng chuẩn 20%).

c) Tính tỷ lệ lỗi tổng hợp toàn khung FFE.

d) Tính sai số cent của từng khung hữu thanh không bị lỗi thô và sai số cent mịn trung bình trị tuyệt đối.

Lời giải chi tiết:

a) Tổng số khung phân tích là $N = 6$. Khung mắc lỗi quyết định hữu thanh (VDE) là khung có trạng thái V/UV giữa Ref và Test khác nhau:

- Khung 2: Ref = 0 Hz (UV), Test = 150 Hz (V) $\implies$ Lỗi VDE (nhận nhầm vô thanh thành hữu thanh).
- Khung 3: Ref = 180 Hz (V), Test = 0 Hz (UV) $\implies$ Lỗi VDE (bỏ sót khung hữu thanh).

Tổng cộng có 2 khung lỗi VDE. Tỷ lệ lỗi là:

$$\text{VDE} = \frac{2}{6} \approx 33{,}33\% \tag{82}$$

b) Các khung mà cả hai hệ thống cùng phân loại là hữu thanh gồm: Khung 1, Khung 4 và Khung 5 ($N_{\text{both}} = 3$ khung). Ta kiểm tra độ lệch tương đối $|F_{\text{Test}} - F_{\text{Ref}}|/F_{\text{Ref}}$:

- Khung 1: $|121 - 120|/120 = 1/120 \approx 0{,}83\% < 20\%$ (Hợp lệ, lỗi mịn).
- Khung 4: $|200 - 100|/100 = 100/100 = 100\% \geq 20\% \implies$ Lỗi bước nhảy thô GPE (nhảy quãng tám).
- Khung 5: $|225 - 220|/220 = 5/220 \approx 2{,}27\% < 20\%$ (Hợp lệ, lỗi mịn).

Trong số 3 khung cùng hữu thanh, có 1 khung bị lỗi GPE (Khung 4). Tỷ lệ lỗi bước nhảy thô là:

$$\text{GPE} = \frac{1}{3} \approx 33{,}33\% \tag{83}$$

c) Tập hợp các khung mắc lỗi phân loại hoặc lỗi tần số thô bao gồm: Khung 2, Khung 3 (lỗi VDE) và Khung 4 (lỗi GPE). Tổng số khung lỗi là $2 + 1 = 3$ khung. Tỷ lệ lỗi tổng hợp FFE là:

$$\text{FFE} = \frac{3}{6} = 50{,}00\% \tag{84}$$

d) Các khung cùng hữu thanh và không mắc lỗi thô gồm Khung 1 và Khung 5 ($N_{\text{fine}} = 2$):

$$C_1 = 1200 \cdot \log_2\left(\frac{121{,}0}{120{,}0}\right) = 1200 \cdot \log_2(1{,}008333) \approx +14{,}37 \text{ cent} \tag{85}$$

$$C_5 = 1200 \cdot \log_2\left(\frac{225{,}0}{220{,}0}\right) = 1200 \cdot \log_2(1{,}022727) \approx +38{,}91 \text{ cent} \tag{86}$$

Sai số cent mịn trung bình trị tuyệt đối là:

$$\overline{|C|} = \frac{|14{,}37| + |38{,}91|}{2} = \frac{53{,}28}{2} = 26{,}64 \text{ cent} \tag{87}$$

Bài tập 5 (Phân tích ngân sách chu kỳ và hệ số RTF ở tần số xung nhịp 25 MHz và 100 MHz). Một hệ thống ước lượng F0 xử lý tín hiệu âm thanh lấy mẫu ở $f_s = 16$ kHz, độ dài khung $N = 638$ mẫu, bước nhảy khung $hop = 160$ mẫu (10 ms). Khối nhân-cộng tự tương quan MAC cần 98.543 chu kỳ/khung và tổng toàn lõi xử lý ước tính cần 99.700 chu kỳ/khung.

a) Tính ngân sách chu kỳ tối đa của một bước nhảy 10 ms khi mạch làm việc ở tần số xung nhịp $f_{\text{clk}} = 25$ MHz và $f_{\text{clk}} = 100$ MHz.

b) Xác định hệ số thời gian thực của khối tự tương quan RTF_{MAC} và toàn bộ lõi xử lý RTF_{core} ở cả hai mức xung nhịp trên.

c) Nếu hệ thống hoạt động ở tần số 25 MHz, nó có đáp ứng được yêu cầu thời gian thực không? Giả sử cần ghép kênh để xử lý đồng thời $M = 8$ luồng âm thanh độc lập trong thời gian thực trên cùng một FPGA, hãy xác định tần số xung nhịp tối thiểu của lõi tuần tự này.

Lời giải chi tiết:

a) Thời gian vật lý của một bước nhảy khung là $T_{\text{hop}} = 10$ ms $= 0{,}010$ s. Ngân sách chu kỳ tối đa cho phép là:

- Ở tần số $f_{\text{clk}} = 25$ MHz:

$$N_{\text{budget}} = 25 \times 10^6 \times 0{,}010 = 250.000 \text{ chu kỳ} \tag{88}$$

- Ở tần số $f_{\text{clk}} = 100$ MHz:

$$N_{\text{budget}} = 100 \times 10^6 \times 0{,}010 = 1.000.000 \text{ chu kỳ} \tag{89}$$

b) Hệ số thời gian thực $\text{RTF} = N_{\text{cycles}}/N_{\text{budget}}$:

- Ở tần số 25 MHz:

$$\text{RTF}_{\text{MAC}} = \frac{98.543}{250.000} \approx 0{,}3942 \tag{90}$$

$$\text{RTF}_{\text{core}} = \frac{99.700}{250.000} \approx 0{,}3988 \tag{91}$$

- Ở tần số 100 MHz:

$$\text{RTF}_{\text{MAC}} = \frac{98.543}{1.000.000} \approx 0{,}0985 \tag{92}$$

$$\text{RTF}_{\text{core}} = \frac{99.700}{1.000.000} \approx 0{,}0997 \tag{93}$$

c) Đánh giá khả năng thời gian thực và ghép kênh:

- Vì ở 25 MHz, $\text{RTF}_{\text{core}} \approx 0{,}3988 < 1{,}0$, hệ thống hoàn toàn đáp ứng yêu cầu xử lý thời gian thực. Lõi phần cứng chỉ sử dụng khoảng $39{,}9\%$ năng lực xử lý của xung nhịp và có thời gian nghỉ rỗi hơn 60%.
- Để xử lý đồng thời $M = 8$ kênh âm thanh trong mỗi khoảng thời gian 10 ms, tổng số chu kỳ xử lý cần thiết của toàn bộ 8 kênh là:

$$N_{\text{total}} = 8 \times 99.700 = 797.600 \text{ chu kỳ} \tag{94}$$

  Tần số xung nhịp tối thiểu để hoàn thành 797.600 chu kỳ trong 10 ms là:

$$f_{\text{min}} = \frac{N_{\text{total}}}{T_{\text{hop}}} = \frac{797.600}{0{,}010 \text{ s}} = 79.760.000 \text{ Hz} = 79{,}76 \text{ MHz} \tag{95}$$

Kết luận: Một mức xung nhịp hệ thống phổ thông 100 MHz trên Cyclone V (giả định cần được xác nhận bằng đo đạc F_{max} thực tế trên bo mạch – phải đo) lớn hơn $79{,}76$ MHz, hoàn toàn đủ khả năng ghép kênh xử lý cùng lúc 8 luồng âm thanh thời gian thực chỉ với duy nhất một bộ nhân-cộng DSP tuần tự!

Chương 1. WAV trên HPS và cầu HPS–FPGA, chiều đi và chiều về

▶ MỤC TIÊU HỌC TẬP

- Hiểu rõ bản chất vấn đề phân chia nhiệm vụ và truyền nhận dữ liệu giữa hai miền xử lý độc lập trên chip Cyclone V SoC: bộ vi xử lý HPS (ARM Cortex-A9) và vùng logic khả trình FPGA.
- Nắm vững cấu trúc nhị phân của tệp RIFF/WAVE theo từng byte, hiểu rõ lý do vì sao việc nhảy cóc cố định vào độ lệch offset 44 dẫn tới sai lệch dữ liệu nghiêm trọng, và xây dựng bộ bóc tách định dạng WAV tin cậy bằng ngôn ngữ C trên nền tảng Linux.
- Dẫn xuất công thức tính đỉnh biên độ toàn cục globalPeak (GP) của Praat bằng số học nguyên chính xác, triệt tiêu hoàn toàn sự phụ thuộc vào số thực dấu phẩy động trên phần cứng.
- Phân tích tường tận thuật toán tạo lưới phân tích khung ngắn hạn (`Sampled_shortTermAnalysis` và `Sampled_xToLowIndex`), giải mã hiện tượng bẫy sàn số lẻ (odd-sample floor trap) và cách thức mã nguồn HPS bảo vệ tính đều đặn của bước nhảy khung.
- Đánh giá định lượng thông lượng truyền thông qua ba phương án phần cứng: PIO qua cầu Lightweight HPS-to-FPGA, DMA thông qua cầu Full Bridge, và bộ điều khiển mSGDMA; hiểu rõ sự đánh đổi giữa thông lượng, độ phức tạp phần mềm và việc bảo toàn tính nhất quán bộ đệm cache.
- Nắm vững kiến trúc và từng tham số cấu hình của khối IP hàng đợi Intel `altera_avalon_fifo` (On-Chip FIFO Memory), giải thích cơ chế phân rã từ dữ liệu 32-bit thành hai mẫu 16-bit theo thứ tự byte nhỏ (little-endian) và tài nguyên khối nhớ M10K.
- Tự thiết kế và hiện thực khối hàng đợi Avalon-MM write slave sang Avalon-ST source bằng ngôn ngữ Verilog-2001, hiểu rõ cơ chế con trỏ có bit vòng, hành vi tín hiệu `waitrequest` bảo vệ bus và cơ chế phản áp dòng chảy dữ liệu.
- Định nghĩa tường minh bản đồ không gian thanh ghi điều khiển cho toàn bộ lõi xử lý F0 và lập trình trình điều khiển (C driver) tương tác qua giao tiếp ánh xạ bộ nhớ `mmap` trên Linux.

1.1. Vấn đề truyền dữ liệu giữa hai miền HPS và FPGA

Một hệ thống nhúng thực tế không bao giờ bắt đầu bằng một khối logic đứng cô lập. Tín hiệu tiếng nói thu nhận từ micro hoặc lưu trữ sẵn dưới dạng tệp âm thanh WAV được đặt trong hệ thống tệp tin định dạng FAT32 hoặc ext4 trên thẻ nhớ SD. Thẻ nhớ này kết nối trực tiếp với bộ điều khiển ngoại vi của hệ thống vi xử lý cứng HPS (Hard Processor System, gồm hai lõi ARM Cortex-A9 MPCore chạy hệ điều hành Linux) trên bo mạch DE10-Standard (Cyclone V SoC 5CSXFC6D6F31C6N). Trong khi đó, toàn bộ chuỗi xử lý tín hiệu số chuyên dụng tốc độ cao—từ các phép tích lũy tự tương quan MAC, tra cứu trọng số cửa sổ Hanning, tìm đỉnh parabol cho đến bộ lọc trung vị median-3—lại được hiện thực hóa bên trong lưới logic khả trình FPGA.

Sự thật kỹ thuật đầu tiên cần đối mặt là: **mẫu âm thanh nằm trong bộ nhớ DDR của HPS, còn logic tính toán lại nằm ở FPGA**. Hai miền này thuộc về hai thế giới hoàn toàn tách biệt:

- Miền vi xử lý HPS hoạt động ở xung nhịp cao (thường là 800 MHz đến 925 MHz), có bộ quản lý bộ nhớ MMU (Memory Management Unit), cơ chế dịch địa chỉ ảo (virtual address translation) và bộ nhớ đệm đa tầng (L1/L2 cache).
- Miền logic FPGA hoạt động ở xung nhịp đồng bộ của hệ thống lõi (trong thiết kế này được ấn định là 50 MHz), giao tiếp theo từng chu kỳ xung nhịp xác định thông qua các đường bus chuẩn công nghiệp.

Để kết nối hai miền xử lý này, kiến trúc phần cứng Cyclone V SoC đúc sẵn ba cầu nối giao tiếp AXI (Advanced eXtensible Interface) chuyên dụng đi xuyên qua ranh giới vật lý giữa HPS và FPGA:

1. **Cầu HPS-to-FPGA (Full Bridge):** Đây là cầu nối master tốc độ cao do vi xử lý làm chủ, hỗ trợ bề rộng dữ liệu cấu hình được là 32, 64 hoặc 128 bit, xung nhịp độc lập với miền HPS và được đồng bộ vào miền xung nhịp FPGA. Cầu nối này chiếm một không gian địa chỉ vật lý rộng lớn (từ `0xC0000000` đến `0xFBFFFFFF`, dung lượng gần 1 GB), chuyên chở các khối dữ liệu khổng lồ như khung hình video hoặc bộ đệm mạng.
2. **Cầu Lightweight HPS-to-FPGA (LWHPS-to-FPGA Bridge):** Đây là cầu nối master thứ hai từ phía HPS, nhưng được thu gọn với bề rộng dữ liệu cố định đúng 32 bit và độ trễ truy xuất thấp. Cầu nối này được ánh xạ cố định vào cửa sổ địa chỉ vật lý `0xFF200000` với độ mở span đúng 2 MB (tới `0xFF3FFFFF`). Cầu Lightweight được sinh ra nhằm mục đích điều khiển các thanh ghi trạng thái (CSR) và các cổng ngoại vi giao tiếp nhẹ mà không làm tiêu tốn tài nguyên quản lý bus phức tạp của vi xử lý.

3. **Cầu FPGA-to-HPS:** Đây là cầu nối theo chiều ngược lại, cho phép các bộ master nằm trong vùng logic FPGA (tiêu biểu như bộ điều khiển DMA) chủ động phát lệnh đọc hoặc ghi trực tiếp vào không gian bộ nhớ DDR3 của HPS.

Câu hỏi thiết kế đặt ra là: **Ta nên chọn cầu nào và cơ chế vận chuyển nào cho bài toán ước lượng cao độ F_0?**

Nếu ta dồn toàn bộ việc đọc tệp, giải mã tệp WAV và tính toán lưới khung vào FPGA, ta sẽ phải hiện thực một bộ điều khiển hệ thống tệp (FAT32/ext4) phức tạp trên phần cứng, tiêu tốn hàng nghìn khối logic ALM vô ích mà không đem lại tốc độ nhanh hơn. Ngược lại, nếu ta dồn toàn bộ thuật toán tự tương quan vào vi xử lý ARM, việc tính toán 98.540 phép nhân-cộng trên từng khung 10 ms sẽ chiếm dụng đáng kể thời gian của CPU, làm giảm khả năng đáp ứng của các tác vụ hệ thống khác.

Giải pháp phân chia tối ưu là thực hiện một ranh giới rõ ràng:

- Phía HPS chạy chương trình bằng ngôn ngữ C trên Linux, đảm nhiệm việc bóc tách định dạng tệp WAV, kiểm tra tính hợp lệ của âm thanh, tính toán các tham số thống kê toàn cục (globalPeak, số khung, mốc thời gian t_1, mẫu bắt đầu của từng khung) và quản lý tiến trình.
- Phía FPGA đón nhận luồng dữ liệu mẫu âm thanh thông qua bộ đệm hàng đợi (FIFO), tự động thực hiện chuỗi thuật toán xử lý tín hiệu số ở tốc độ phần cứng, và gửi kết quả cao độ F_0 cùng các cờ trạng thái trở lại cho HPS.

Việc truyền tải dữ liệu giữa hai miền thông qua cầu nối không chỉ đơn thuần là đẩy dữ liệu lên dây dẫn mà còn phải đảm bảo không bao giờ làm tràn bộ nhớ, không làm rơi rụng mẫu âm thanh và không gây nghẽn bus hệ thống. Trong các tiểu mục tiếp theo, ta sẽ lần lượt bóc tách chi tiết từng khâu: từ giải mã tệp WAV, lựa chọn cầu nối với các con số định lượng, thiết kế bộ đệm FIFO ở cả hai hướng (IP và HDL tự viết), cho đến việc thiết lập bản đồ thanh ghi điều khiển thống nhất.

Tóm lại: Dữ liệu âm thanh nằm trong DDR của vi xử lý còn thuật toán tính toán nằm ở logic FPGA; bài toán trung tâm là phân định ranh giới xử lý và lựa chọn cầu nối sao cho việc truyền tải vừa đủ nhanh để đáp ứng thời gian thực, vừa giữ cấu trúc phần cứng và trình điều khiển ở mức đơn giản nhất.

1.2. Đọc tệp WAV đúng chuẩn và tính toán tham số toàn cục trên HPS

Cạm bẫy ngây thơ: Vì sao giả định dữ liệu ở offset 44 luôn thất bại?

Một sai lầm kinh điển mà người mới làm việc với âm thanh số trên FPGA thường mắc phải là mở tệp WAV, bỏ qua 44 byte đầu tiên rồi giả định rằng mẫu âm thanh PCM bắt đầu cố định từ byte thứ 44. Con số 44 byte này chỉ đúng trong trường hợp lý tưởng tối giản nhất: một tệp chỉ chứa duy nhất khối `RIFF` tiêu đề (12 byte), một khối định dạng `fmt` cơ bản không có phần mở rộng (24 byte gồm 8 byte tiêu đề và 16 byte dữ liệu), và 8 byte tiêu đề của khối `data` ($12 + 24 + 8 = 44$).

Tuy nhiên, trong thực tế sản xuất âm thanh, hầu hết các phần mềm biên tập chuẩn (như Audacity, Adobe Audition, Praat hay các công cụ ghi âm hiện đại) đều tự động chèn thêm các khối dữ liệu phụ trợ (auxiliary metadata chunks) vào tiêu đề để lưu trữ thông tin bản quyền, tên nghệ sĩ, mốc thời gian ghi âm hoặc thông số phát sóng. Tiêu biểu nhất là khối danh sách thông tin `LIST` (chứa các thẻ con như `INFO`, `INAM`, `IART`) hoặc khối thông tin phát sóng `bext` (Broadcast Audio Extension). Các khối này thường được đặt ngay sau khối `fmt` và nằm phía trước khối `data`.

Để thấy rõ sự sai lệch này bằng số liệu cụ thể, hãy xét tệp âm thanh thử nghiệm `edge_list_chunk.wav` (trong thư mục `verify_v3/data/`, do kịch bản `make_test_wavs.py` sinh ra). Tệp này chèn một khối `LIST` 18 byte mang nhãn "DE10" xen vào giữa khối `fmt` và khối `data`.

Quan sát trích xuất hex dump thực tế từ tệp `edge_list_chunk.wav`:

Offset	Dữ liệu Hex	Ký tự ASCII	Nhận diện khối
0x0000	52 49 46 46 7e 01 00 00	RIFF~...	Khối `RIFF` tiêu đề tệp
0x0008	57 41 56 45 66 6d 74 20	WAVEfmt	Định dạng WAVE + Khối `fmt`
0x0010	10 00 00 00 01 00 01 00	....	16 byte, PCM không nén, mono
0x0018	80 3e 00 00 00 7d 00 00	.>...}..	16.000 Hz, 32.000 byte/s
0x0020	02 00 10 00 4c 49 53 54	....LIST	16-bit, bắt đầu khối `LIST`
0x0028	12 00 00 00 49 4e 46 4f	....INFO	Kích thước 18 byte, nhãn `INFO`
0x0030	49 4e 41 4d 06 00 00 00	INAM....	Thẻ `INAM`, độ dài 6 byte
0x0038	44 45 31 30 00 00 64 61	DE10..da	Dữ liệu "DE10", đầu khối `data`
0x0040	74 61 40 01 00 00 00 00	ta@.....	Ký tự `ta`, kích thước 320 byte

Nếu một bộ đọc ngây thơ nhảy cóc thẳng vào vị trí byte 44 (`0x002C`), nó sẽ đọc đúng 4 byte ký tự ASCII `"INFO"` (mã hex: `49 4E 46 4F`; 4 byte kế tiếp ở offset 48 là `"INAM"`) và nhầm tưởng đó là các mẫu âm thanh PCM 16-bit!

- Hai byte đầu tiên tại offset 44 là `0x49` và `0x4E`. Theo quy ước số nguyên có dấu bù hai byte nhỏ (little-endian), chúng ghép lại thành mẫu số nguyên:

$$x_{\text{rác}}[0] = \texttt{0x4E49} = +20.041 \tag{96}$$

- Hai byte tiếp theo tại offset 46 là `0x46` và `0x4F`, ghép lại thành mẫu số nguyên:

$$x_{\text{rác}}[1] = \texttt{0x4F46} = +20.294 \tag{97}$$

Hai mẫu rác này có biên độ rất lớn (vượt quá 61% thang đo cực đại của PCM 16-bit), trong khi khối âm thanh thật sự `data` phải tới byte 62 (`0x003E`) mới xuất hiện và các mẫu âm thanh thực tế chỉ bắt đầu từ byte 70 (`0x0046`). Hệ quả là:

1. Âm thanh giải mã bị lệch pha đúng 26 byte (13 mẫu âm thanh), làm toàn bộ các khung phân tích bị trôi mốc thời gian so với phần mềm Praat.
2. Hai mẫu văn bản bị đọc nhầm thành hai xung đột biến biên độ > 20.000, tạo ra một xung nổ ở đầu tệp và làm sai lệch nghiêm trọng giá trị đỉnh biên độ toàn cục globalPeak, từ đó phá hỏng logic cổng im lặng ở Chương 10.

Cấu trúc phân cấp chuẩn của tệp RIFF/WAVE từ mức byte

Định dạng tệp WAV thuộc họ tệp hoán đổi tài nguyên RIFF (Resource Interchange File Format) do Microsoft và IBM đồng phát triển. Một tệp RIFF được tổ chức thành một cây chứa các khối (chunks) lồng nhau. Mỗi khối bắt buộc phải có cấu trúc tiêu đề 8 byte mở đầu gồm:

- **Mã nhận diện khối (Chunk ID):** 4 byte ký tự ASCII (FourCC, Four Character Code) xác định loại khối.
- **Kích thước khối (Chunk Size):** số nguyên không dấu 32-bit ở định dạng byte nhỏ (little-endian), cho biết độ dài chính xác của phần dữ liệu bên trong khối (không tính 8 byte tiêu đề của chính nó).

Tiêu đề tệp bắt đầu bằng chuỗi nhận diện `"RIFF"`, theo sau là kích thước toàn tệp trừ 8 byte đầu, và mã định dạng `"WAVE"`. Sau 12 byte đầu tiên này là một chuỗi các khối nằm liên tiếp nhau. Một trình đọc chuẩn bắt buộc phải duyệt tuần tự qua từng khối theo thuật toán nhảy cóc (chunk jumping): đọc 4 byte ID, đọc 4 byte kích thước S, kiểm tra xem khối đó có phải là khối ta cần hay không; nếu không phải, dùng hàm dịch chuyển con trỏ tệp (`fseek`) để nhảy qua đúng S byte nhằm đến đầu khối tiếp theo.

Quy tắc đệm từ của chuẩn RIFF (Word Alignment Padding): Chuẩn kỹ thuật RIFF quy định: **mọi khối bắt buộc phải bắt đầu tại một địa chỉ byte chẵn**. Do đó, nếu kích thước dữ liệu S của một khối là một số lẻ (tức $S \pmod 2 \neq 0$), bên ghi tệp bắt buộc phải chèn thêm đúng 1 byte đệm có giá trị `0x00` vào ngay sau dữ liệu của khối đó. Khi nhảy qua khối, trình đọc phải dịch chuyển con trỏ một khoảng là:

$$\Delta_{\text{jump}} = S + (S \,\&\, 1) \tag{98}$$

Nếu bỏ qua việc kiểm tra bit lẻ này, khi gặp các khối siêu dữ liệu có kích thước lẻ (như tệp thử nghiệm `edge_odd_chunk.wav`), con trỏ tệp sẽ bị lệch 1 byte. Toàn bộ các mã FourCC tiếp theo bị đọc lệch thành các chuỗi vô nghĩa và việc giải mã thất bại hoàn toàn.

Một trình đọc WAV đúng chuẩn trên vi xử lý HPS bắt buộc phải kiểm tra nghiêm ngặt 5 điều kiện hợp lệ của khối `fmt` :

1. `AudioFormat == 1`: bắt buộc là định dạng PCM tuyến tính không nén. Các định dạng nén như A-law, μ-law hay IEEE Float đều bị từ chối với mã lỗi `WAV_ERR_UNSUPPORTED_FORMAT`.
2. `NumChannels == 1`: bắt buộc là mono (kênh đơn). Tệp hai kênh (stereo, ví dụ `edge_stereo.wav`) bị từ chối với mã lỗi `WAV_ERR_UNSUPPORTED_CHANNELS` do dữ liệu đan xen làm hỏng phép tự tương quan.

3. `SampleRate == 16000`: tần số lấy mẫu bắt buộc là 16 kHz để phù hợp với thông số thiết kế cửa sổ 40 ms và dải quét độ trễ của FPGA. Tệp có tần số khác (như 44,1 kHz trong `edge_wrong_rate.wav`) bị từ chối với mã `WAV_ERR_UNSUPPORTED_RATE`.
4. `BitsPerSample == 16`: mỗi mẫu âm thanh biểu diễn bởi đúng 16 bit số nguyên có dấu bù hai (dải biểu diễn từ -32.768 đến $+32.767$).
5. `BlockAlign == 2`: mỗi mẫu chiếm đúng 2 byte dữ liệu.

Ví dụ bóc tách tệp thực tế: Tệp mẫu thu nhỏ `mini_speech.wav`

Để quan sát tường tận cấu trúc nhị phân ở mức từng byte, hãy xem xét toàn bộ nội dung của tệp âm thanh thu nhỏ `mini_speech.wav` gồm đúng 8 mẫu âm thanh:

$$s = [0, 100, -200, 300, -400, 500, -600, 700] \tag{99}$$

Địa chỉ Offset	Giá trị Hex (Little-Endian)	Ý nghĩa giải mã kỹ thuật
`0x0000 -- 0x0003`	`52 49 46 46`	Ký tự FourCC: `"RIFF"`
`0x0004 -- 0x0007`	`34 00 00 00`	Kích thước phần còn lại: 52 byte ($60 - 8 = 52$)
`0x0008 -- 0x000B`	`57 41 56 45`	Định dạng tài nguyên: `"WAVE"`
`0x000C -- 0x000F`	`66 6d 74 20`	Ký tự FourCC: `"fmt "`
`0x0010 -- 0x0013`	`10 00 00 00`	Kích thước khối định dạng: 16 byte cơ bản
`0x0014 -- 0x0015`	`01 00`	`AudioFormat = 1` (PCM không nén)
`0x0016 -- 0x0017`	`01 00`	`NumChannels = 1` (kênh đơn mono)
`0x0018 -- 0x001B`	`80 3e 00 00`	`SampleRate = 16.000 Hz` ($0x00003E80 = 16.000$)
`0x001C -- 0x001F`	`00 7d 00 00`	`ByteRate = 32.000 byte/s` ($16.000 \times 2 = 32.000$)
`0x0020 -- 0x0021`	`02 00`	`BlockAlign = 2 byte/mẫu`
`0x0022 -- 0x0023`	`10 00`	`BitsPerSample = 16 bit/mẫu`
`0x0024 -- 0x0027`	`64 61 74 61`	Ký tự FourCC: `"data"`
`0x0028 -- 0x002B`	`10 00 00 00`	Kích thước dữ liệu âm thanh: 16 byte (8 mẫu)
`0x002C -- 0x002D`	`00 00`	Mẫu 0: 0
`0x002E -- 0x002F`	`64 00`	Mẫu 1: $+100$ (0x0064)
`0x0030 -- 0x0031`	`38 ff`	Mẫu 2: -200 (0xFF38)
`0x0032 -- 0x0033`	`2c 01`	Mẫu 3: $+300$ (0x012C)
`0x0034 -- 0x0035`	`70 fe`	Mẫu 4: -400 (0xFE70)
`0x0036 -- 0x0037`	`f4 01`	Mẫu 5: $+500$ (0x01F4)
`0x0038 -- 0x0039`	`a8 fd`	Mẫu 6: -600 (0xFDA8)
`0x003A -- 0x003B`	`bc 02`	Mẫu 7: $+700$ (0x02BC)

Trình đọc C trong `sw_v3/ch01_wav.c` hiện thực hóa chính xác giải thuật duyệt khối này và vượt qua toàn bộ 8 tệp kiểm thử biên trong kịch bản kiểm định `ch01_wav_verify.py`.

Dẫn xuất đỉnh biên độ toàn cục globalPeak bằng số nguyên

Trong mã nguồn Praat `Sound_to_Pitch.cpp` (dòng 375–385), trước khi chia khung, Praat tính toán đỉnh biên độ toàn cục của toàn bộ tệp âm thanh:

$$\mathrm{GP}_{\mathrm{praat}} = \max_{1 \le i \le n} |x[i] - \bar{x}| \tag{100}$$

trong đó $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x[i]$ là giá trị trung bình DC của toàn bộ tệp và mẫu âm thanh $x[i] \in [-1{,}0, +1{,}0)$ được biểu diễn dưới dạng số thực dấu phẩy động đôi (double).

Đại lượng này đóng vai trò quyết định trong việc đánh giá mức cường độ âm thanh tương đối:

$$\text{intensity} = \frac{\text{localPeak}}{\text{globalPeak}} \tag{101}$$

ở Chương 10. Nếu một đoạn âm thanh có biên độ nhỏ nhưng vẫn tuần hoàn rõ rệt (ví dụ khi người nói phát âm thì thầm hoặc nói nhỏ dần ở cuối câu), năng lượng tuyệt đối $R[0]$ giảm xuống rất thấp. Tuy nhiên, tỉ số

localPeak/globalPeak vẫn giữ được giá trị đáng kể, giúp bộ phân loại vô thanh/hữu thanh không bị nhầm lẫn tiếng nói nhỏ thành khoảng lặng hoàn toàn.

Chuyển đổi sang số nguyên không mất độ chính xác: Trên vi xử lý nhúng HPS, ta hoàn toàn có thể tính toán đại lượng này bằng số thực `double`. Tuy nhiên, để cấu hình cho thanh ghi 16-bit của FPGA và đảm bảo không xảy ra bất kỳ sự phân kỳ nào giữa các trình biên dịch, ta quy đổi công thức trên về số nguyên thuần túy. Mẫu số nguyên 16-bit $s[i]$ liên hệ với mẫu thực của Praat qua tỉ lệ $x[i] = s[i]/32.768$. Thay vào công thức trung bình:

$$x[i] - \bar{x} = \frac{s[i]}{32.768} - \frac{\sum_{j=1}^{n} s[j]}{n \cdot 32.768} = \frac{n \cdot s[i] - \sum_{j=1}^{n} s[j]}{n \cdot 32.768} \tag{102}$$

Để đưa giá trị độ lệch cực đại về cùng thang đo với mẫu số nguyên 16-bit (tức nhân lại với 32.768), ta nhận được công thức nguyên chính xác:

$$\text{GP} = \text{round}\left(\frac{\max_{1 \le i \le n} |n \cdot s[i] - S|}{n}\right) \tag{103}$$

với $S = \sum_{j=1}^{n} s[j]$ là tổng tích lũy của toàn bộ n mẫu trong tệp.

Rút gọn phép tìm cực đại về hai điểm biên. Quan sát hàm bậc nhất $f(s) = n \cdot s - S$. Đây là một hàm đơn điệu tăng nghiêm ngặt theo giá trị mẫu s. Do đó, độ lớn tuyệt đối $|n \cdot s - S|$ là một hàm lồi (convex), và giá trị cực đại của nó trên toàn bộ tập mẫu **chỉ có thể đạt được tại một trong hai điểm cực trị**: mẫu có giá trị nhỏ nhất $s_{\text{min}} = \min_i s[i]$ hoặc mẫu có giá trị lớn nhất $s_{\text{max}} = \max_i s[i]$!

Nhờ định lý này, mã nguồn C trên HPS không cần phải lưu trữ toàn bộ mảng mẫu rồi quét lại lần thứ hai. Ngay trong lượt đọc tệp đầu tiên, ta đồng thời tính tổng tích lũy S (dùng biến 64-bit `int64_t` để chống tràn), cập nhật s_{min} và s_{max}. Sau khi đọc xong n mẫu, giá trị GP được tính trực tiếp trong đúng một phép so sánh:

$$\Delta_1 = |n \cdot s_{\text{max}} - S| \tag{104}$$

$$\Delta_2 = |n \cdot s_{\text{min}} - S| \tag{105}$$

$$\text{max_diff} = \max(\Delta_1, \Delta_2) \tag{106}$$

$$\text{GP} = \left\lfloor \frac{\text{max_diff} + \lfloor n/2 \rfloor}{n} \right\rfloor \tag{107}$$

trong đó việc cộng thêm $\lfloor n/2 \rfloor$ trước khi chia nguyên thực hiện chính xác phép làm tròn số học (`round`) về số nguyên gần nhất.

Ví dụ tính tay trên dãy mẫu thu nhỏ (`mini_speech.wav`):
Xét dãy 8 mẫu âm thanh:

$$s = [0, 100, -200, 300, -400, 500, -600, 700] \tag{108}$$

- Số lượng mẫu: $n = 8$.
- Tổng tích lũy: $S = 0 + 100 - 200 + 300 - 400 + 500 - 600 + 700 = 400$.
- Giá trị cực tiểu: $s_{\text{min}} = -600$; Giá trị cực đại: $s_{\text{max}} = 700$.
- Độ lệch tại cực tiểu: $\Delta_2 = |8 \times (-600) - 400| = |-4.800 - 400| = |-5.200| = 5.200$.
- Độ lệch tại cực đại: $\Delta_1 = |8 \times 700 - 400| = |5.600 - 400| = 5.200$.
- Cực đại độ lệch: $\text{max_diff} = 5.200$.
- Giá trị GP:

$$\text{GP} = \left\lfloor \frac{5.200 + \lfloor 8/2 \rfloor}{8} \right\rfloor = \left\lfloor \frac{5.204}{8} \right\rfloor = 650 \tag{109}$$

Con số 650 thu được khớp chính xác với kiểm chứng tự động của kịch bản `ch01_wav_verify.py`.

Áp dụng trên hai tệp âm thanh chuẩn 2,8 s (44.800 mẫu) của đề cương (mục 7):

- Tệp âm thanh sạch (`synth_glide_vuv_16k.wav`): $s_{\text{max}} = 22.939$, $s_{\text{min}} = -22.939$, tổng tích lũy $S = -317.078$, tính ra $\text{GP} = \mathbf{22.946}$ (đã kiểm bằng `ch01_wav_verify.py`).
- Tệp âm thanh có ồn SNR 5 dB (`synth_glide_vuv_16k_snr5.wav`): $s_{\text{max}} = 22.938$, $s_{\text{min}} = -22.278$, tính ra $\text{GP} = \mathbf{22.937}$ (đã kiểm bằng `ch01_wav_verify.py`).

Lưới phân tích khung ngắn hạn: Dẫn xuất thuật toán của Praat

Một điểm khác biệt sâu sắc giữa Praat và các hệ thống xử lý tín hiệu thông thường là cách xác định tâm thời gian của các khung. Trong các giáo trình cơ bản, người ta thường cố định khung 0 bắt đầu từ mẫu 0 ($t = 0$), rồi tịnh tiến đều đặn theo từng bước nhảy hop. Ngược lại, Boersma thiết kế thuật toán phân tích ngắn hạn `Sampled_shortTermAnalysis` (`fon/Sampled.cpp` dòng 71–82) theo nguyên lý **căn giữa đối xứng**: toàn bộ chuỗi khung phân tích phải được đặt đối xứng hoàn hảo xung quanh tâm của tệp âm thanh.

Các bước tính toán diễn ra tuần tự như sau:

1. **Xác định số lượng khung phân tích n_{frames}:** Với tần số lấy mẫu $f_s = 16.000$ Hz, chu kỳ lấy mẫu là $dx = 1/16.000 = 0{,}0000625$ s. Thời lượng vật lý của tệp âm thanh có n_x mẫu là:

$$\text{my_duration} = n_x \cdot dx \tag{110}$$

Độ dài cửa sổ vật lý là $\Delta t_{\text{window}} = 3/75 = 0{,}04$ s (tương ứng 640 mẫu danh định, Praat làm tròn thành 638 mẫu). Bước nhảy thời gian danh định giữa hai khung là $\Delta t = 0{,}75/75 = 0{,}01$ s (tương ứng hop $= 160$ mẫu). Số lượng khung phân tích hợp lệ được tính bằng:

$$n_{\text{frames}} = \left\lfloor \frac{\text{my_duration} - \Delta t_{\text{window}}}{\Delta t} \right\rfloor + 1 \tag{111}$$

2. **Căn tâm và tính mốc thời gian khung đầu tiên t_1:** Quy ước tọa độ thời gian của Praat (`Sound.cpp`) đặt mẫu đầu tiên tại $x_1 = 0{,}5 \cdot dx$ (tâm của khoảng lấy mẫu đầu tiên). Điểm chính giữa của tệp âm thanh là:

$$\text{our_mid} = x_1 - 0{,}5 \cdot dx + 0{,}5 \cdot \text{my_duration} = 0{,}5 \cdot \text{my_duration} \tag{112}$$

Khoảng thời gian mà n_{frames} khung chiếm dụng là thy_duration $= n_{\text{frames}} \cdot \Delta t$. Để chuỗi khung nằm cân đối ở giữa tệp, mốc thời gian của tâm khung đầu tiên t_1 bắt buộc phải là:

$$t_1 = \text{our_mid} - 0{,}5 \cdot \text{thy_duration} + 0{,}5 \cdot \Delta t \tag{113}$$

3. **Tính mốc thời gian t_i và chỉ số mẫu bắt đầu cửa sổ:** Tâm thời gian của khung thứ i ($i = 1, 2, \ldots, n_{\text{frames}}$) là:

$$t_i = t_1 + (i - 1) \cdot \Delta t \tag{114}$$

Để chuyển từ mốc thời gian liên tục t_i sang chỉ số mẫu rời rạc, Praat gọi hàm `Sampled_xToLowIndex` (`fon/Sampled.h` dòng 34):

$$\text{leftSample}_i = \left\lfloor \frac{t_i - x_1}{dx} + 1{,}0 \right\rfloor \tag{115}$$

trong đó dòng 52 của `Sound_to_Pitch.cpp` gọi trực tiếp:

$$\texttt{integer leftSample} = \texttt{Sampled_xToLowIndex(me, t)}, \ \texttt{rightSample} = \texttt{leftSample} + \texttt{1}; \tag{116}$$

Mẫu bên phải là $\text{rightSample}_i = \text{leftSample}_i + 1$. Khung cửa sổ Hanning 638 mẫu có nửa chiều dài là halfnsamp_window $= 319$ mẫu. Do đó, chỉ số mẫu bắt đầu của cửa sổ (tính từ 0, tức 0-based) đưa vào bộ đệm FPGA là:

$$\text{win0}_i = \text{rightSample}_i - 319 - 1 = \text{leftSample}_i - 319 \tag{117}$$

Ví dụ tính tay trên hai tệp âm thanh tiêu biểu:

- **Tệp có thời lượng 1,234 s ($n_x = 19.744$ mẫu):** Thời lượng: my_duration $= 19.744 \times 0{,}0000625 = 1{,}234$ s. Số khung: $n_{\text{frames}} = \lfloor (1{,}234 - 0{,}04)/0{,}01 \rfloor + 1 = \lfloor 119{,}4 \rfloor + 1 = 120$ khung. Tâm tệp: our_mid $= 0{,}5 \times 1{,}234 = 0{,}617$ s. Thời lượng phủ: thy_duration $= 120 \times 0{,}01 = 1{,}200$ s. Mốc thời gian khung đầu: $t_1 = 0{,}617 - 0{,}5 \times 1{,}200 + 0{,}005 = 0{,}022000$ s. Tại khung 1: $t_1 = 0{,}022$ s. Vì $x_1 = 0{,}5 \times 0{,}0000625 = 0{,}00003125$ s, ta có:

$$\text{leftSample}_1 = \left\lfloor \frac{0{,}022 - 0{,}00003125}{0{,}0000625} + 1{,}0 \right\rfloor = \lfloor 351{,}5 + 1{,}0 \rfloor = \lfloor 352{,}5 \rfloor = 352 \tag{118}$$

Chỉ số mẫu bắt đầu cửa sổ đầu tiên: $\text{win0}_0 = 352 - 319 = \mathbf{33}$. Khung đầu tiên trên FPGA không bắt đầu ở mẫu 0 mà bắt đầu từ mẫu 33 để khớp hoàn hảo với Praat! Khung cuối cùng ($i = 120$) bắt đầu tại mẫu $\text{win0}_{119} = 19.073$.

- **Tệp chuẩn 2,8 s ($n_x = 44.800$ mẫu):** Thời lượng: my_duration $= 44.800 \times 0{,}0000625 = 2{,}800$ s. Số khung: $n_{\text{frames}} = \lfloor (2{,}800 - 0{,}04)/0{,}01 \rfloor + 1 = 276 + 1 = 277$ khung. Tâm tệp: our_mid $= 1{,}400$ s; thy_duration $= 277 \times 0{,}01 = 2{,}770$ s. Mốc thời gian khung đầu: $t_1 = 1{,}400 - 0{,}5 \times 2{,}770 + 0{,}005 =$ **0,020000** s. Tại khung 1: $t_1 = 0{,}020$ s $\implies$ leftSample$_1 = \lfloor 0{,}020/0{,}0000625 + 0{,}5 \rfloor = \lfloor 320{,}5 \rfloor = 320$. Chỉ số mẫu bắt đầu cửa sổ đầu tiên: win0$_0 = 320 - 319 =$ **1**. Toàn bộ 277 khung đều có bước nhảy hop $=$ win0$_i -$ win0$_{i-1} = 160$ mẫu đều đặn tuyệt đối!

Đón trước chỗ vướng: Bẫy sàn số lẻ (Odd-sample floor trap)

► HỎI – ĐÁP TƯ DUY: Vì sao bước nhảy khung trong Praat đôi khi bị co thành 159 hoặc giãn thành 161 mẫu?

Hãy quan sát biểu thức tính toán trong hàm `Sampled_xToLowIndex`:

$$y_i = \frac{t_i - x_1}{dx} + 1{,}0 = \frac{t_i}{dx} + 0{,}5 \tag{119}$$

Thay biểu thức của t_i từ các bước trên vào, ta biến đổi đại số chính xác:

$$\begin{aligned} y_i &= \frac{0{,}5 \cdot n_x \cdot dx - 0{,}5 \cdot n_{\text{frames}} \cdot \Delta t + 0{,}5 \cdot \Delta t + (i-1) \cdot \Delta t}{dx} + 0{,}5 \\ &= 0{,}5 \cdot n_x - \frac{0{,}5 \cdot n_{\text{frames}} \cdot \Delta t}{dx} + \frac{(i - 0{,}5) \cdot \Delta t}{dx} + 0{,}5 \end{aligned} \tag{120}$$

Vì tỉ số $\Delta t/dx = 0{,}01/0{,}0000625 = 160$ là một **số nguyên chẵn tuyệt đối**, nên biểu thức rút gọn thành:

$$y_i = 0{,}5 \cdot n_x - 80 \cdot n_{\text{frames}} + 160 \cdot i - 80 + 0{,}5 \tag{121}$$

Bản chất số học lộ diện ở đây:

- **Khi n_x là số chẵn ($n_x = 2k$):** Đại lượng $0{,}5 \cdot n_x = k$ là một số nguyên. Khi đó $y_i =$ nguyên $+ 0{,}5$. Giá trị luôn rơi vào chính giữa hai số nguyên liên tiếp (phần thập phân là đúng $0{,}5$). Khoảng cách tới biên số nguyên là $0{,}5$, rất lớn so với sai số làm tròn 10^{-16} của số thực double. Do đó hàm lấy sàn $\lfloor y_i \rfloor$ hoàn toàn miễn nhiễm với sai số dấu phẩy động; bước nhảy giữa hai khung liên tiếp luôn bằng đúng:

$$\text{hop} = \lfloor y_{i+1} \rfloor - \lfloor y_i \rfloor = (M + 160) - M = 160 \text{ mẫu (ổn định tuyệt đối).} \tag{122}$$

- **Khi n_x là số lẻ ($n_x = 2k + 1$):** Đại lượng $0{,}5 \cdot n_x = k + 0{,}5$. Khi cộng với $0{,}5$ ở cuối công thức, ta thu được:

$$y_i = (k + 0{,}5) - 80 \cdot n_{\text{frames}} + 160 \cdot i - 80 + 0{,}5 = \text{SỐ NGUYÊN CHÍNH XÁC!} \tag{123}$$

  Đây chính là cơ chế phát sinh bẫy sàn số lẻ: y_i về mặt lý thuyết là một số nguyên hoàn hảo (ví dụ $32.173{,}0$). Nhưng trong chuẩn số thực dấu phẩy động IEEE 754, giá trị thời gian $\Delta t = 0{,}01$ s là một phân số vô hạn tuần hoàn trong hệ nhị phân ($0{,}01_{10} \approx 0{,}000000101000111101\ldots_2$). Khi nhân và cộng dồn qua hàng trăm khung, sai số làm tròn tích lũy ở bit thấp nhất (ULP, Unit in the Last Place) có thể làm giá trị thực tế bị hụt một lượng vô cùng nhỏ:

$$y_i^{\text{thực tế}} = 32.172{,}999999999996362 \tag{124}$$

  Hàm lấy sàn $\lfloor \cdot \rfloor$ lập tức đánh rơi giá trị xuống 32.172 thay vì 32.173! Kết quả là bước nhảy khung bị co lại thành 159 mẫu, và khung ngay sau đó bị bù giãn thành 161 mẫu!

Thực nghiệm kiểm chứng trên 200 tệp âm thanh ngẫu nhiên trong kịch bản `ch01_frames_verify.py` đã chứng thực định lý này:

- Với 100 tệp có độ dài chẵn (n_x chẵn): tỷ lệ xuất hiện bước nhảy lệch 160 là **đúng 0,0%** (100/100 tệp đều có hop 160 hoàn hảo).
- Với 100 tệp có độ dài lẻ (n_x lẻ): có tới **86,0%** số tệp xuất hiện hiện tượng bước nhảy bị giật 159 hoặc 161 mẫu do bẫy sàn số lẻ của Praat!

Để giải quyết triệt để vấn đề này, chương trình C trên HPS (`sw_v3/ch01_frames.c`) tính toán chính xác bằng kiểu `double` theo đúng công thức của Praat nhằm bám sát vị trí khung tham chiếu đến từng mẫu, đồng thời tích hợp cờ `hop_regular`. Nếu phát hiện bước nhảy không đều do số mẫu lẻ, trình điều khiển sẽ phát cảnh báo rõ ràng để người dùng nhận biết nguồn gốc sai lệch tiềm tàng.

Tóm lại: Đọc WAV đúng cách là duyệt theo từng chunk RIFF để tránh bẫy offset 44 và xử lý byte đệm số lẻ; tính đỉnh GP bằng công thức số nguyên quét cực trị một lượt tránh số thực; và lưới khung Praat phải được tính bằng double để nhận diện bẫy sàn số lẻ khi số mẫu âm thanh không chia hết cho hai.

1.3. Các phương án truyền dữ liệu, đánh đổi và lựa chọn

Vấn đề thực tế: Ảo tưởng về sự cần thiết của DMA trong xử lý âm thanh

Khi bắt tay vào thiết kế một hệ thống tích hợp HPS–FPGA, một phản xạ tự nhiên của nhiều kỹ sư phần cứng là lập tức tìm đến các bộ điều khiển truy xuất bộ nhớ trực tiếp DMA (Direct Memory Access, tiêu biểu như IP mSGDMA của Intel). Lập luận thường nghe là: "phải có DMA để truyền dữ liệu tự động ở tốc độ phần cứng, nếu dùng CPU ghi từng từ thì sẽ làm nghẽn vi xử lý và không thể đáp ứng thời gian thực".

Tuy nhiên, trong kỹ nghệ hệ thống, mọi quyết định kiến trúc bắt buộc phải xuất phát từ phép tính định lượng trên nguyên lý đầu tiên: **Lưu lượng thực sự cần vận chuyển là bao nhiêu?**

Hãy làm phép tính số học cơ bản cho tín hiệu âm thanh chuẩn 16 kHz:

- Tần số lấy mẫu: $f_s = 16.000$ mẫu/giây.
- Mỗi mẫu âm thanh PCM 16-bit chiếm đúng 2 byte bộ nhớ.
- Băng thông dữ liệu thực tế sinh ra trong một giây âm thanh là:

$$B_{\text{audio}} = 16.000 \times 2 = 32.000 \text{ byte/giây} = 32 \text{ KB/s} = 0{,}032 \text{ MB/s} \tag{125}$$

Bây giờ, hãy đặt con số 32 KB/s này lên bàn cân với năng lực truyền tải của các cầu nối phần cứng trên SoC Cyclone V:

- Bus bộ nhớ DDR3 của HPS hoạt động ở xung nhịp 400 MHz (tốc độ dữ liệu kép DDR 800 MT/s), bề rộng 32-bit, cho băng thông đỉnh lý thuyết lên tới:

$$B_{\text{DDR3}} = 800 \text{ MT/s} \times 4 \text{ byte} = 3.200 \text{ MB/s} = 3{,}2 \text{ GB/s} \tag{126}$$

- Cầu nối Lightweight HPS-to-FPGA (LWHPS2FPGA) có bề rộng cố định 32-bit. Ở xung nhịp đồng bộ của hệ thống FPGA là 50 MHz, trần băng thông lý thuyết của bus là:

$$B_{\text{LWHPS}} = 50 \text{ MHz} \times 4 \text{ byte} = 200 \text{ MB/s} \tag{127}$$

- Cầu nối tốc độ cao Full HPS-to-FPGA với bề rộng 128-bit ở xung nhịp 100 MHz có thể đạt tới 1.600 MB/s $=$ $1{,}6$ GB/s.

  So sánh định lượng trực tiếp:

$$\frac{B_{\text{audio}}}{B_{\text{LWHPS}}} = \frac{0{,}032 \text{ MB/s}}{200 \text{ MB/s}} = 0{,}00016 = \mathbf{0{,}016}\%! \tag{128}$$

Lưu lượng dữ liệu của dòng âm thanh 16 kHz chỉ chiếm chưa đầy **một phần sáu nghìn** $(1/6.250)$ băng thông khả dụng của cầu nối nhẹ nhất trên chip! Việc vội vã đưa vào một bộ điều khiển DMA phức tạp cho một dòng dữ liệu 32 KB/s là biểu hiện kinh điển của việc tối ưu hóa sớm mà không dựa trên con số.

Ba phương án truyền dữ liệu và sự đánh đổi kỹ thuật

Để đưa ra lựa chọn sáng suốt nhất, ta tiến hành phân tích chi tiết ba phương án truyền nhận khả thi:

Phương án 1: PIO qua cầu Lightweight HPS-to-FPGA (32-bit). Vi xử lý ARM thực hiện các lệnh ghi trực tiếp (Programmed I/O) từ không gian người dùng thông qua con trỏ ánh xạ bộ nhớ `mmap` trên tệp thiết bị `/dev/mem`.

- **Cơ chế đóng gói:** Vì mỗi giao dịch bus của cầu Lightweight chuyển được đúng một từ 32-bit, ta đóng gói hai mẫu âm thanh 16-bit liên tiếp vào một từ duy nhất:

$$\text{word}_{32} = ((s[2k+1] \;\&\; 0\text{xFFFF}) \ll 16) \mid (s[2k] \;\&\; 0\text{xFFFF}) \tag{129}$$

  với mẫu chẵn $s[2k]$ đặt ở 16 bit thấp và mẫu lẻ $s[2k+1]$ đặt ở 16 bit cao theo chuẩn little-endian.

- **Tần suất thao tác ghi:** Để đẩy 1 giây âm thanh (16.000 mẫu), CPU chỉ cần thực hiện đúng:

$$N_{\text{write}} = \frac{16.000}{2} = 8.000 \text{ lệnh ghi/giây} \tag{130}$$

- **Thời gian chiếm dụng CPU và hệ số thời gian thực (RTF):** Hãy đưa ra một giả định rất bi quan rằng mỗi thao tác ghi qua bus AXI/Avalon-MM tốn tới 1 μs (1.000 ns, tương đương 50 chu kỳ xung nhịp 50 MHz của FPGA hoặc gần 800 chu kỳ xung nhịp CPU ARM Cortex-A9):

$$T_{\text{transfer}} = 8.000 \times 1\ \mu\text{s} = 8 \text{ ms cho mỗi 1 giây âm thanh} \tag{131}$$

  Quy đổi ra hệ số thời gian thực của khâu truyền tải:

$$\text{RTF}_{\text{transfer}} = \frac{T_{\text{transfer}}}{1 \text{ s}} = \frac{8 \text{ ms}}{1.000 \text{ ms}} = \mathbf{0{,}008} \tag{132}$$

  Kể cả trong kịch bản bi quan nhất, vi xử lý chỉ mất đúng $0{,}8\%$ năng lực thời gian để vận chuyển toàn bộ dòng âm thanh sang FPGA! Thời gian ghi thực tế trên bo mạch DE10-Standard đo được thông qua hàm `clock_gettime(CLOCK_MONOTONIC)` thường chỉ dao động từ vài chục đến khoảng hơn một trăm nano-giây mỗi giao dịch (con số cụ thể là **phải đo** trên bo mạch thực tế).

- **Độ phức tạp phần mềm:** Rất thấp. Toàn bộ logic chạy hoàn toàn trong không gian người dùng (User Space), không cần viết driver nhân Linux (Kernel Module), không cần quyền can thiệp vào bộ nhớ vật lý của hệ điều hành.

Phương án 2: DMA phân tán modular (mSGDMA) qua cầu Full Bridge. Khối IP mSGDMA (modular Scatter-Gather DMA) nằm trên vùng logic FPGA đóng vai trò master, tự động đọc luồng mẫu từ bộ nhớ DDR của HPS rồi bắn trực tiếp vào giao diện Avalon-ST của đường ống xử lý mà không cần CPU tham gia từng mẫu.

- **Ưu điểm:** Hoàn toàn giải phóng CPU khỏi vòng lặp ghi dữ liệu. CPU chỉ cần kích hoạt descriptor rồi chuyển sang thực hiện các tác vụ khác.

- **Cái giá phần mềm:** Rất phức tạp. Hệ điều hành Linux quản lý bộ nhớ theo các trang ảo (virtual pages) kích thước 4 KB nằm rải rác trong RAM vật lý. Để DMA có thể đọc được:

  1. Cần phân bổ một vùng nhớ vật lý liên tục (Contiguous Memory) bằng cơ chế CMA của Linux kernel hoặc thông qua module hỗ trợ `/dev/udmabuf`.
  2. **Hiểm họa tính nhất quán bộ đệm (Cache Incoherency):** Khi CPU đọc tệp WAV từ thẻ nhớ SD và ghi vào mảng bộ nhớ, các byte dữ liệu có thể còn nằm lưu trong bộ nhớ đệm cache L1/L2 của lõi ARM Cortex-A9 mà chưa được đẩy (flush) xuống các chip nhớ DDR3 bên dưới. Bộ điều khiển DMA trên FPGA khi đọc trực tiếp từ DDR3 sẽ đọc phải dữ liệu rác cũ! Người lập trình bắt buộc phải gọi các hàm xả cache tường minh (`dma_sync_single_for_device` hoặc `arm_dma_flush`), nếu quên hoặc gọi sai thời điểm, toàn bộ dữ liệu âm thanh bị sai mà không hề có bất kỳ cảnh báo lỗi nào.
  3. **Quản lý mô tả phân tán (Descriptors):** Phải tạo và cấu hình các bảng descriptor trong bộ nhớ, xử lý ngắt hoàn thành (interrupt handling) để dọn dẹp bộ đệm.

- **Cái giá phần cứng:** Khối IP mSGDMA tiêu tốn tài nguyên logic ALM và các khối RAM M10K chuyên dụng để làm bộ đệm descriptor và FIFO dữ liệu nội tại **(phải đo, Ch.13)**.

Phương án 3: Cầu Full HPS-to-FPGA (64/128-bit) với cơ chế PIO. Sử dụng cầu nối chính HPS-to-FPGA với bề rộng 64-bit hoặc 128-bit kết hợp với lệnh ghi SIMD (NEON) của ARM để đẩy 4 hoặc 8 mẫu âm thanh trong một chu kỳ truy xuất.

- **Đánh giá:** Băng thông tăng thêm 2 đến 4 lần so với cầu Lightweight, nhưng đối với lưu lượng chỉ 32 KB/s, việc giảm số lần ghi từ 8.000 xuống 2.000 hay 4.000 lần/giây không đem lại bất kỳ cải thiện nhận thấy nào về hiệu năng tổng thể. Trong khi đó, cầu Full Bridge chiếm dụng không gian địa chỉ vật lý rất rộng lớn, đòi hỏi cấu hình mạng chuyển mạch bus AXI phức tạp hơn và việc phân bổ xung nhịp giữa hai miền cần thêm các tầng đồng bộ chuyên dụng.

Bảng so sánh tổng hợp các phương án truyền tải

Bảng 5 tổng hợp toàn diện các chỉ số kỹ thuật và sự đánh đổi giữa ba phương án:

Bảng 5: So sánh định lượng các phương án truyền dữ liệu HPS–FPGA cho tín hiệu 16 kHz.

Chỉ số kỹ thuật	**PA 1: PIO (LWHPS)**	**PA 2: mSGDMA**	**PA 3: PIO (Full)**
Cầu nối vật lý	LWHPS-to-FPGA	Full HPS-to-FPGA	Full HPS-to-FPGA
Bề rộng dữ liệu bus	32 bit	32 hoặc 64 bit	64 hoặc 128 bit
Lưu lượng yêu cầu	32 KB/s (8.000 từ/s)	32 KB/s (8.000 từ/s)	32 KB/s (4.000 từ/s)
Tải băng thông bus	0,016% của 200 MB/s	$<$ 0,005% của 1,6 GB/s	$<$ 0,003% của 1,6 GB/s
Thời gian CPU (RTF)	$\leq$ 0,008 (bi quan 1 μs)	$\approx$ 0,000 (tự động)	$\leq$ 0,004 (bi quan)
Thời gian ghi thực tế	**(phải đo)**	Không áp dụng	**(phải đo)**
Yêu cầu bộ nhớ vật lý	Không (ảo bình thường)	Bắt buộc liên tục (CMA)	Không (ảo bình thường)
Hiểm họa Cache coherency	Không có	Bắt buộc xả cache	Không có
Không gian phần mềm	User Space (`mmap`)	Kernel Module / UIO	User Space (`mmap`)
Tài nguyên phần cứng FPGA	0 ALM, 0 M10K thêm	**phải đo (Ch.13)**	Thêm logic ghép bus
Độ phức tạp kiểm chứng	Rất thấp, xem từng nhịp	Cao, phụ thuộc ngắt	Trung bình

Đọc bảng. Nhìn vào Bảng 5, ta thấy một sự thật hiển nhiên: ở lưu lượng siêu thấp 32 KB/s, việc dùng mSGDMA hoàn toàn không giải quyết được nút thắt hiệu năng nào (bởi vì bản thân khâu truyền tải đã có RTF $\leq$ 0,008, tức nhanh hơn thời gian thực ít nhất 125 lần), nhưng lại kéo theo việc phải can thiệp vào tầng nhân Linux kernel, cấp phát bộ nhớ liên tục và hiểm họa đọc dữ liệu rác do lỗi cache coherency.

Lựa chọn của ta và lý do có số liệu

Từ các phân tích định lượng trên, đề cương quyết định lựa chọn:

1. **Phương án 1 (PIO qua cầu Lightweight Bridge vào bộ đệm On-Chip FIFO) là đường truyền chính thống của toàn bộ tài liệu và dự án**. Có bốn lý do kỹ thuật rõ ràng:
   - *Thông lượng dư thừa:* Hệ số thời gian thực $\text{RTF}_{\text{transfer}} \leq 0{,}008$. CPU chỉ tốn tối đa 8 ms để truyền xong 1 giây âm thanh, 99,2% thời gian còn lại hoàn toàn rảnh rỗi.
   - *Độ tin cậy phần mềm cao nhất:* Trình điều khiển chạy hoàn toàn ở không gian người dùng (User Space), chỉ cần mở tệp `/dev/mem` rồi gọi hàm chuẩn `mmap`. Không có bất kỳ nguy cơ treo nhân hệ điều hành, rò rỉ bộ nhớ hay lỗi lệch cache.
   - *Kiến trúc phần cứng tinh gọn:* Toàn bộ giao diện dữ liệu và thanh ghi điều khiển được gom gọn vào không gian địa chỉ 2 MB của Lightweight Bridge, giải phóng toàn bộ các cổng Full Bridge cho các ứng dụng mở rộng khác trong tương lai.
   - *Dễ dàng gỡ lỗi và kiểm chứng bit-exact:* Người thiết kế có thể đặt các điểm dừng (breakpoint), đọc thanh ghi báo mức đầy `fill_level` của FIFO ở từng chu kỳ xung nhịp và quan sát chính xác từng từ dữ liệu được nạp vào.
2. **Phương án 2 (mSGDMA) được giữ lại làm mục đối chiếu chuyên sâu**: Chương 13 sẽ trình bày cách cấu hình mSGDMA và tiến hành đo đạc thực nghiệm trên phần cứng thật để so sánh chính xác thời gian chiếm dụng CPU và tài nguyên ALM/M10K với phương án PIO.

Tóm lại: Với lưu lượng âm thanh chỉ 32 KB/s (8.000 lần ghi 32-bit mỗi giây), cơ chế PIO qua cầu Lightweight bridge chỉ tiêu tốn RTF $\leq$ 0,008 mà loại bỏ hoàn toàn gánh nặng bộ đệm vật lý và xả cache của DMA, là lựa chọn tối ưu về cả hiệu năng lẫn độ tin cậy.

1.4. Đường (A): Xây dựng bộ đệm bằng Intel On-Chip FIFO IP

Nguyên lý của On-Chip FIFO trong Platform Designer

Trong luồng thiết kế tiêu chuẩn của hệ sinh thái Intel Quartus Prime, công cụ tích hợp hệ thống Platform Designer cung cấp sẵn khối IP lõi mang tên `altera_avalon_fifo` (On-Chip FIFO Memory). Khối IP này đóng vai trò cầu nối thích ứng giữa hai miền giao thức:

- **Miền giao diện bộ nhớ (Avalon-MM):** Do vi xử lý ARM HPS làm chủ (Master). CPU thực hiện các thao tác ghi dữ liệu theo từng khối rời rạc, phụ thuộc vào tiến trình lập lịch của hệ điều hành Linux.
- **Miền luồng dữ liệu (Avalon-ST):** Cung cấp dữ liệu trực tiếp cho lõi xử lý F0 trên FPGA (Sink). Lõi phần cứng yêu cầu dữ liệu mẫu âm thanh chảy liên tục, tuần tự và phải có cơ chế báo bận (backpressure) tức thời.

Để cấu hình IP này hoạt động chính xác và khai thác tối ưu tài nguyên phần cứng của chip Cyclone V, ta cần hiểu rõ các ràng buộc kỹ thuật của tài liệu *Intel Embedded Peripherals IP User Guide*.

Phân tích chi tiết từng tham số cấu hình của Intel On-Chip FIFO

Dưới đây là phân tích cặn kẽ từng tham số trên giao diện đồ họa Platform Designer:

1. Độ sâu bộ đệm (FIFO Depth = 256 từ 32-bit cho 1 khối M10K). Theo tài liệu hướng dẫn *Intel Embedded Peripherals IP User Guide* (chương Avalon-MM Write Slave to Avalon-ST Source):

> "The input is an Avalon-MM write slave with a width of 32 bits … The Avalon-ST output (source) data width must also be 32 bits … If Avalon-MM is selected for one interface and Avalon-ST for the other, the data width is fixed at 32 bits."

Điều này chỉ ra một sự thật kiến trúc cốt lõi: **Bề rộng nội tại của FIFO bắt buộc cố định là 32 bit**, khối IP không hỗ trợ cấu hình bề rộng 16 bit khi giao tiếp giữa Avalon-MM và Avalon-ST.

Do đó, việc tính toán tài nguyên bộ nhớ nhúng M10K (dung lượng danh định 10.240 bit) diễn ra như sau:

- Nếu chọn độ sâu 512 từ 32-bit:

$$N_{\text{bits}} = 512 \times 32 \text{ bit} = 16.384 \text{ bit} > 10.240 \text{ bit} \tag{133}$$

  Trình biên dịch Quartus bắt buộc phải tiêu tốn **2 khối M10K vật lý**.

- Nếu chọn độ sâu 256 từ 32-bit:

$$N_{\text{bits}} = 256 \times 32 \text{ bit} = 8.192 \text{ bit} \leq 10.240 \text{ bit} \tag{134}$$

  Bộ đệm được ánh xạ trọn vẹn vào **đúng 1 khối M10K duy nhất** (ở cấu hình 256×40 hoặc 256×32).

Vì mỗi từ 32-bit chứa 2 mẫu âm thanh 16-bit, độ sâu 256 từ chứa được $256 \times 2 = 512$ mẫu âm thanh. Với bước nhảy khung tiêu chuẩn của Praat là 10 ms ($H = 160$ mẫu), dung lượng 512 mẫu chứa được:

$$N_{\text{hops}} = \frac{512}{160} = 3{,}2 \text{ khung nhảy} \tag{135}$$

tương đương với 32 ms âm thanh. Khoảng đệm 32 ms này hoàn toàn đủ để hấp thu các đợt trễ lập lịch thông thường của nhân Linux nhúng mà vẫn tiết kiệm tối đa tài nguyên M10K.

2. Chế độ xung nhịp (Clock Setting = Single Clock Mode). Cả bus giao tiếp Lightweight HPS-to-FPGA lẫn lõi tính toán F0 đều được đấu nối vào cùng một nguồn dao động xung nhịp hệ thống 50 MHz (`clk`). Do đó, ta chọn chế độ *Single Clock Mode*. Lựa chọn này giúp IP loại bỏ hoàn toàn các mạch đồng bộ hóa hai miền xung nhịp (như chuỗi đồng bộ kép flip-flop 2-FF và bộ chuyển đổi mã Gray cho các con trỏ), qua đó tiết kiệm logic ALM và giảm độ trễ truyền dữ liệu qua FIFO.

3. Cổng vào và cổng ra (Input / Output Interfaces).

- **Cổng vào (Input):** Thiết lập là `AVALONMM_WRITE`, bề rộng cố định 32-bit.
- **Cổng ra (Output):** Thiết lập là `AVALONST_SOURCE`, bề rộng cố định 32-bit theo quy định của IP. Khối IP **không tự động tách từ 32-bit thành luồng 16-bit**.

4. Ký hiệu và quy ước endian (Bits per symbol và Symbols per beat).

- **Bits per symbol:** Đặt là 16.
- **Symbols per beat:** Đặt là 2 (tổng cộng $2 \times 16 = 32$ bit).

Tài liệu IP của Intel chỉ rõ: IP thực hiện chuyển đổi endian để phù hợp với chuẩn giao diện ngõ ra. Trong chuẩn Avalon-ST, ký hiệu 0 (xuất hiện trước theo thời gian) được đặt tại các bit trọng số cao nhất (MSB). Do đó, nửa từ thấp của Avalon-MM `writedata[15:0]` (mẫu $2k$) sẽ đi ra ở các bit cao của Avalon-ST `data[31:16]` **(phải kiểm trong mô phỏng IP)**.

Hệ quả thiết kế quan trọng: Vì ngõ ra của On-Chip FIFO IP vẫn là từ 32-bit (chứa 2 mẫu), đường (A) bắt buộc phải chèn thêm một khối giải gói 32-sang-16 bit ở phía sau IP (thực hiện bằng một FSM 2 trạng thái hoặc khối *Avalon-ST Data Format Adapter* của Platform Designer) để đưa từng mẫu 16-bit tuần tự vào lõi DSP. Đây chính là lý do kỹ thuật vì sao ở đường (B), ta chủ động tích hợp luôn cỗ máy giải gói FSM vào bên trong module FIFO tự viết.

5. Cho phép áp lực ngược (Allow Backpressure = YES). Cổng ra Avalon-ST sinh ra tín hiệu điều khiển `ready`. Nếu khối phía sau chưa sẵn sàng nhận mẫu mới, nó kéo `ready` xuống 0; FIFO lập tức giữ nguyên dữ liệu trên bus `data` và không tăng con trỏ đọc.

6. Tạo giao diện trạng thái cho cổng vào (Create status interface for input = YES). Platform Designer sinh thêm cổng phụ `in_csr` (Avalon-MM CSR) cho phép vi xử lý HPS đọc mức đầy hiện tại (`fill_level`) của bộ đệm trước khi ghi dữ liệu.

Các giao diện phần cứng và độ trễ

Khối IP On-Chip FIFO xuất ra ba giao diện chính:

1. **Giao diện dữ liệu nạp `in` (Avalon-MM Slave):** Gồm tín hiệu địa chỉ, tín hiệu cho phép ghi `write`, dữ liệu ghi 32-bit `writedata[31:0]` và tín hiệu phản hồi chờ `waitrequest`.
2. **Cổng trạng thái `in_csr` (Avalon-MM Slave):** Cung cấp các thanh ghi 32-bit: `fill_level` (0x00), `status` (0x04), `event` (0x08), `almostfull` (0x10), `almostempty` (0x14).
3. **Giao diện luồng ngõ ra `out` (Avalon-ST Source):** Bao gồm dữ liệu 32-bit `data[31:0]` (chứa 2 mẫu 16-bit), `valid` và `ready`.

Độ trễ truyền nhận (Latency). Từ thời điểm CPU ghi một từ vào cổng `in` cho tới khi dữ liệu xuất hiện trên cổng `out` và giương cờ `valid = 1`, độ trễ danh định của IP **(phải kiểm trong mô phỏng IP)**.

Cấu hình mSGDMA dùng làm mục đối chiếu chuyên sâu

Để phục vụ cho các thực nghiệm so sánh định lượng ở Chương 13, khối IP Modular Scatter-Gather DMA (mSGDMA) của Intel được cấu hình với các tham số tối ưu hóa cho bài toán dòng như sau:

- **DMA Mode:** Chọn `Memory-Mapped to Streaming` (MM-to-ST).
- **Data Width:** Đặt là 32-bit.
- **Data Path FIFO Depth:** Cấu hình độ sâu đệm đường truyền là 64 hoặc 128 từ.
- **Descriptor FIFO Depth:** Đặt là 16 hoặc 32 bộ mô tả (descriptors).

- **Transfer Length:** Cấu hình trần độ dài truyền tải đủ lớn để bao trọn tệp âm thanh trong một bộ mô tả duy nhất.
- **Burst Enable:** Bật tính năng truyền liên tục (Burst mode) với `Maximum Burst Count = 16` hoặc 32 nhịp.
- **Response Port:** Tắt (Disabled).

Bộ đệm chiều về: Thu nhận kết quả F0 từ FPGA về HPS

Ở chiều về (Return Path), kiến trúc đảo ngược để HPS đọc chuỗi tần số cơ bản F0 do FPGA sinh ra:

- Khối hậu xử lý trên FPGA (bộ lọc trung vị median-3; không dùng Viterbi trên phần cứng) tính toán xong mỗi khung sẽ đẩy ra một từ F0 định dạng số nguyên không dấu uint16 byte nhỏ (little-endian) fixed-point Q12.4 (16-bit). Quy ước giá trị 0 đại diện cho khung vô thanh (unvoiced), hoàn toàn khớp với định dạng tệp `pitch_f0.bin` trong mô hình `golden_v2/fixed_twin.py`.
- Dữ liệu này được đưa vào một bộ đệm On-Chip FIFO cấu hình `Input: AVALONST_SINK` (16-bit) và `Output: AVALONMM_READ` (32-bit hoặc 16-bit), đặt tên là `fifo_out`.
- Vi xử lý HPS thăm dò thanh ghi trạng thái của `fifo_out_csr`, khi có kết quả hợp lệ sẽ đọc trực tiếp dữ liệu qua cầu Lightweight Bridge để lưu vào tệp kết quả `pitch_f0.bin`.

Tóm lại: Khối IP On-Chip FIFO của Intel ở cấu hình MM-to-ST có bề rộng cố định 32-bit, do đó độ sâu 256 từ khớp đúng 1 khối M10K (chứa 512 mẫu âm thanh, đủ 3,2 hop đệm); ngõ ra 32-bit đòi hỏi thêm bộ giải gói 32-to-16 bit phía sau; chiều về dùng FIFO ST-sink sang MM-read để trả kết quả F0 định dạng uint16 LE Q12.4 đồng bộ.

1.5. Đường (B): Hiện thực phần cứng bằng HDL tự viết

Nguyên lý bộ đệm vòng đồng bộ với con trỏ có bit vòng (Wrap-Bit Pointers)

Để không bị phụ thuộc vào các IP độc quyền của hãng và có thể kiểm soát tuyệt đối từng chu kỳ xung nhịp trong quá trình mô phỏng, ta tự hiện thực khối bộ đệm nạp mẫu `ch01_avalon_fifo` và khối hoàn trả kết quả `ch01_fifo_out` bằng ngôn ngữ Verilog chuẩn.

Bộ đệm vòng đồng bộ hoạt động trên nguyên lý của hai con trỏ: con trỏ ghi (`wr_ptr`) và con trỏ đọc (`rd_ptr`). Khi bộ nhớ có độ sâu $N_{\text{words}} = 2^k$ từ (với $k = 8$, tức 256 từ 32-bit), giải pháp tối ưu phần cứng là mở rộng bề rộng của con trỏ thêm đúng 1 bit ở vị trí trọng số cao nhất (MSB), gọi là *bit vòng* (wrap bit):

$$\text{Bề rộng con trỏ} = k + 1 = 8 + 1 = 9 \text{ bit} \quad (\texttt{wr_ptr[8:0]}, \texttt{rd_ptr[8:0]}) \tag{136}$$

Trong đó:

- k bit thấp (`ptr[7:0]`) đóng vai trò địa chỉ truy xuất trực tiếp vào mảng nhớ RAM hai cổng ($0 \le \text{addr} \le 255$).
- Bit thứ k (`ptr[8]`) là bit vòng, lật trạng thái logic ($0 \to 1$ hoặc $1 \to 0$) mỗi khi con trỏ quét hết một vòng 256 phần tử của bộ nhớ.

Nhờ bit vòng này, hai trạng thái biên được phân định rạch ròi bằng các biểu thức logic:

- **Trạng thái Rỗng (`fifo_empty`):** Xuất hiện khi hai con trỏ hoàn toàn trùng khớp ở tất cả 9 bit:

$$\texttt{fifo_empty} = (\texttt{wr_ptr} == \texttt{rd_ptr}) \tag{137}$$

- **Trạng thái Đầy (`fifo_full`):** Xuất hiện khi con trỏ ghi đã đi trước con trỏ đọc đúng một vòng. Khi đó, các bit địa chỉ thấp giống hệt nhau nhưng bit vòng MSB hoàn toàn trái ngược nhau:

$$\texttt{fifo_full} = (\texttt{wr_ptr[8]} \neq \texttt{rd_ptr[8]}) \ \wedge \ (\texttt{wr_ptr[7:0]} == \texttt{rd_ptr[7:0]}) \tag{138}$$

- **Số lượng từ đang sử dụng (`words_used`):** Được tính tự động bằng phép trừ bù 2 số nguyên 9-bit mà không cần quan tâm đến việc tràn số:

$$\texttt{words_used} = \texttt{wr_ptr} - \texttt{rd_ptr} \quad (0 \le \texttt{words_used} \le 256) \tag{139}$$

Mã nguồn toàn diện khối bộ đệm nạp mẫu `ch01_avalon_fifo.v`

Toàn bộ mã nguồn phần cứng của khối `ch01_avalon_fifo.v` (199 dòng) được liệt kê đầy đủ trong Listing 3.

Listing 3: Toàn bộ mã nguồn khối bộ đệm nạp mẫu âm thanh `rtl/v3/ch01_avalon_fifo.v`.

```
module ch01_avalon_fifo #(
    parameter integer ADDR_WIDTH = 8  // 2^8 = 256 words (32-bit) = 512 samples (16-bit)
)(
    input  wire        clk,
    input  wire        rst_n,

    // Avalon-MM Data Write Slave (fifo_in)
    input  wire        mm_write,
    input  wire [31:0] mm_writedata,
    output wire        mm_waitrequest,

    // Avalon-MM CSR Read Slave (fifo_in_csr)
    input  wire        csr_read,
    input  wire [2:0]  csr_address,
    output reg  [31:0] csr_readdata,

    // Avalon-ST Data Source (to FPGA core)
    output wire [15:0] st_data,
    output wire        st_valid,
    input  wire        st_ready
);

    localparam integer DEPTH_WORDS   = 1 << ADDR_WIDTH; // 256 words
    localparam integer DEPTH_SAMPLES = DEPTH_WORDS * 2;  // 512 samples

    // Inferred Dual-Port Synchronous RAM (1 M10K block)
    reg [31:0] mem [0:DEPTH_WORDS-1];
    reg [31:0] mem_rdata;

    // Word-level FIFO pointers with wrap bit
    reg [ADDR_WIDTH:0] wr_ptr; // 9 bits (bit 8 is wrap bit)
    reg [ADDR_WIDTH:0] rd_ptr;

    wire [ADDR_WIDTH-1:0] wr_addr = wr_ptr[ADDR_WIDTH-1:0];
    wire [ADDR_WIDTH-1:0] rd_addr = rd_ptr[ADDR_WIDTH-1:0];

    wire fifo_empty = (wr_ptr == rd_ptr);
    wire fifo_full  = (wr_ptr[ADDR_WIDTH] != rd_ptr[ADDR_WIDTH]) &&
                      (wr_ptr[ADDR_WIDTH-1:0] == rd_ptr[ADDR_WIDTH-1:0]);

    wire [ADDR_WIDTH:0] words_used = wr_ptr - rd_ptr;

    // mm_waitrequest asserted when full and master attempts to write
    assign mm_waitrequest = fifo_full;

    // Sticky overflow flag
    reg ovf_sticky;

    // RAM Write Port (Avalon-MM)
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            wr_ptr     <= {(ADDR_WIDTH+1){1'b0}};
            ovf_sticky <= 1'b0;
        end else begin
            if (mm_write) begin
                if (!fifo_full) begin
                    mem[wr_addr] <= mm_writedata;
                    wr_ptr       <= wr_ptr + 1'b1;
                end else begin
                    ovf_sticky <= 1'b1;
                end
            end
        end
    end

    // Unpacker FSM / Output Streaming (Avalon-ST)
    localparam [1:0] ST_IDLE    = 2'b00;
    localparam [1:0] ST_FETCH   = 2'b01;
    localparam [1:0] ST_UNPACK0 = 2'b10;
    localparam [1:0] ST_UNPACK1 = 2'b11;

```

```
    reg [1:0]  st_state;
    reg [31:0] holding_word;

    // RAM Read port
    always @(posedge clk) begin
        mem_rdata <= mem[rd_addr];
    end

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            rd_ptr       <= {(ADDR_WIDTH+1){1'b0}};
            st_state     <= ST_IDLE;
            holding_word <= 32'd0;
        end else begin
            case (st_state)
                ST_IDLE: begin
                    if (!fifo_empty) begin
                        st_state <= ST_FETCH;
                        rd_ptr   <= rd_ptr + 1'b1;
                    end
                end

                ST_FETCH: begin
                    holding_word <= mem_rdata;
                    st_state     <= ST_UNPACK0;
                end

                ST_UNPACK0: begin
                    if (st_ready) begin
                        st_state <= ST_UNPACK1;
                    end
                end

                ST_UNPACK1: begin
                    if (st_ready) begin
                        if (!fifo_empty) begin
                            st_state <= ST_FETCH;
                            rd_ptr   <= rd_ptr + 1'b1;
                        end else begin
                            st_state <= ST_IDLE;
                        end
                    end
                end
            endcase
        end
    end

    assign st_valid = (st_state == ST_UNPACK0) || (st_state == ST_UNPACK1);
    assign st_data  = (st_state == ST_UNPACK0) ? holding_word[15:0] : holding_word[31:16];

    // Fill Level in Sample Units (0..512)
    wire [1:0] unpacker_samples = (st_state == ST_UNPACK0) ? 2'd2 :
                                  (st_state == ST_UNPACK1) ? 2'd1 :
                                  (st_state == ST_FETCH)   ? 2'd2 : 2'd0;

    wire [9:0] total_samples_used = {words_used, 1'b0} + {8'd0, unpacker_samples};

    // CSR Read Port (fifo_in_csr)
    wire almost_empty = (total_samples_used <= 10'd32);
    wire almost_full  = (total_samples_used >= 10'd480);

    always @(*) begin
        case (csr_address)
            3'd0: csr_readdata = {22'd0, total_samples_used};
            3'd1: csr_readdata = {28'd0, almost_full, almost_empty, fifo_full, (total_samples_used
                == 10'd0)};
            3'd2: csr_readdata = {31'd0, ovf_sticky};
            3'd3: csr_readdata = {23'd0, words_used};
            default: csr_readdata = 32'd0;
        endcase
    end

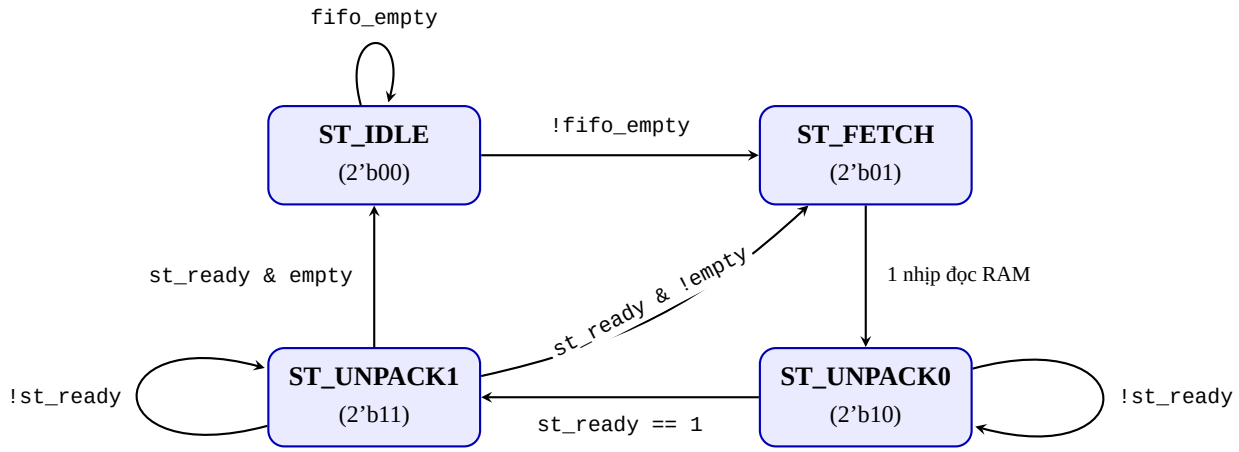
endmodule
```

Giải thích mã nguồn Verilog `ch01_avalon_fifo.v` chi tiết từng dòng:

- **Dòng 24–50 (Khai báo tham số và cổng giao tiếp):** Nhận tham số `ADDR_WIDTH = 8` ($2^8 = 256$ từ 32-bit = 512 mẫu âm thanh 16-bit). Ba nhóm cổng gồm:

  - Cổng nạp mẫu Avalon-MM Data Slave: `mm_write` (lệnh ghi), `mm_writedata[31:0]` (từ 32-bit chứa 2 mẫu), `mm_waitrequest` (tín hiệu phản hồi chờ bus khi bộ đệm đầy).
  - Cổng giám sát Avalon-MM CSR Slave: `csr_read`, địa chỉ thanh ghi `csr_address[2:0]` và dữ liệu đọc `csr_readdata[31:0]`.
  - Cổng cấp luồng Avalon-ST Source: dữ liệu mẫu 16-bit `st_data[15:0]`, cờ `st_valid` và cờ sẵn sàng hạ nguồn `st_ready`.

- **Dòng 58–59 (Khai báo mảng nhớ M10K):** Khai báo mảng nhớ `mem` gồm 256 từ 32-bit ($256 \times 32 = 8.192$ bit), được tổng hợp thành đúng 1 khối M10K phần cứng trên Cyclone V. Thanh ghi `mem_rdata` chốt dữ liệu ngõ ra đồng bộ của RAM.
- **Dòng 64–77 (Con trỏ và logic báo đầy/rỗng):** Con trỏ `wr_ptr` và `rd_ptr` có bề rộng 9 bit (bit 8 là bit vòng wrap-bit). Biểu thức `fifo_full` kiểm tra bit 8 khác nhau và 8 bit thấp bằng nhau. Tín hiệu phản hồi chờ bus `mm_waitrequest` được gán trực tiếp bằng `fifo_full` (dòng 77), lập tức kéo dài chu kỳ bus AXI nếu vi xử lý cố tình ghi khi RAM không còn chỗ trống.
- **Dòng 85–99 (Always block ghi RAM Avalon-MM):** Kích hoạt ở sườn dương `clk` hoặc sườn âm `rst_n`. Khi `mm_write = 1`: nếu bộ đệm chưa đầy (`!fifo_full`), dữ liệu `mm_writedata` được ghi vào địa chỉ `wr_addr` và `wr_ptr` tăng 1; nếu đã đầy, cờ tràn dính `ovf_sticky` bật lên 1 để lưu vết lỗi mà không làm hỏng dữ liệu đang có trong RAM.
- **Dòng 114–116 (Always block đọc RAM):** Cổng đọc đồng bộ: câu lệnh `mem_rdata <= mem[rd_addr]` chốt dữ liệu ngõ ra RAM sau đúng 1 chu kỳ xung nhịp.
- **Dòng 118–159 (Always block máy trạng thái Unpacker FSM):** Máy trạng thái 4 bước đảm nhiệm việc đọc và tách từ 32-bit thành hai mẫu 16-bit:
  - *ST_IDLE (2'b00, dòng 125–131):* Nếu RAM có dữ liệu (`!fifo_empty`), FSM tăng con trỏ đọc `rd_ptr <= rd_ptr + 1` và chuyển sang `ST_FETCH`.
  - *ST_FETCH (2'b01, dòng 133–137):* Sau 1 chu kỳ đọc RAM, dữ liệu hợp lệ tại `mem_rdata` được chốt vào `holding_word` và FSM chuyển sang `ST_UNPACK0`.
  - *ST_UNPACK0 (2'b10, dòng 139–144):* Xuất mẫu thứ nhất ở 16 bit thấp. Cờ `st_valid` giương cao. FSM giữ nguyên trạng thái nếu bị nghẽn (`st_ready = 0`) và chuyển sang `ST_UNPACK1` ngay khi `st_ready = 1`.
  - *ST_UNPACK1 (2'b11, dòng 146–156):* Xuất mẫu thứ hai ở 16 bit cao. Khi `st_ready = 1`: nếu RAM vẫn còn từ mới (`!fifo_empty`), FSM lập tức tăng `rd_ptr` và chuyển thẳng sang `ST_FETCH` (chế độ streaming liên tục không bóng bóng); nếu RAM đã hết, FSM trở về `ST_IDLE`.
- **Dòng 161–162 (Ngõ ra Avalon-ST):** Tín hiệu `st_valid` bật cao khi FSM ở UNPACK0 hoặc UNPACK1. Ngõ ra `st_data` lấy 16 bit thấp ở UNPACK0 và 16 bit cao ở UNPACK1 (thứ tự little-endian).
- **Dòng 172–176 (Mức đầy theo số mẫu):** Biến `total_samples_used` (10 bit, $0 \ldots 512$) cộng gộp mẫu trong RAM (words_used $\times$ 2) và mẫu trên thanh ghi đệm (2 ở FETCH/UNPACK0, 1 ở UNPACK1).
- **Dòng 189–197 (Cổng đọc CSR):** Trả về `total_samples_used` tại offset 0, trạng thái cờ tại offset 1, cờ lỗi tràn `ovf_sticky` tại offset 2, và số từ trong RAM `words_used` tại offset 3.



Hình 2: Lược đồ chuyển trạng thái của khối FSM giải gói mẫu 32-to-16 bit.

Mã nguồn toàn diện khối bộ đệm hoàn trả kết quả `ch01_fifo_out.v`

Khối bộ đệm chiều về `ch01_fifo_out.v` đón nhận kết quả ước lượng F_0 từ FPGA (Avalon-ST Sink 16-bit) và cho phép HPS đọc qua cầu Lightweight Bridge (Avalon-MM Read Slave). Listing 4 trình bày toàn bộ 104 dòng mã nguồn.

Listing 4: Toàn bộ mã nguồn khối bộ đệm hoàn trả F0 `rtl/v3/ch01_fifo_out.v`.

```
module ch01_fifo_out #(
    parameter integer ADDR_WIDTH = 9  // 2^9 = 512 words of 16-bit
)(
    input  wire        clk,
    input  wire        rst_n,

    // Avalon-ST Sink (from FPGA core)
    input  wire [15:0] st_data,
    input  wire        st_valid,
    output wire        st_ready,

    // Avalon-MM Data Read Slave (fifo_out)
    input  wire        mm_read,
    output wire [31:0] mm_readdata,
    output wire        mm_waitrequest,

    // Avalon-MM CSR Read Slave (fifo_out_csr)
    input  wire        csr_read,
    input  wire [2:0]  csr_address,
    output reg  [31:0] csr_readdata
);

    localparam integer DEPTH = 1 << ADDR_WIDTH; // 512 words

    // Inferred dual-port RAM
    reg [15:0] mem [0:DEPTH-1];
    reg [15:0] rdata_reg;

    reg [ADDR_WIDTH:0] wr_ptr;
    reg [ADDR_WIDTH:0] rd_ptr;

    wire [ADDR_WIDTH-1:0] wr_addr = wr_ptr[ADDR_WIDTH-1:0];
    wire [ADDR_WIDTH-1:0] rd_addr = rd_ptr[ADDR_WIDTH-1:0];

    wire fifo_empty = (wr_ptr == rd_ptr);
    wire fifo_full  = (wr_ptr[ADDR_WIDTH] != rd_ptr[ADDR_WIDTH]) &&
                      (wr_ptr[ADDR_WIDTH-1:0] == rd_ptr[ADDR_WIDTH-1:0]);

    wire [ADDR_WIDTH:0] words_used = wr_ptr - rd_ptr;

    // st_ready high when FIFO has space
    assign st_ready = !fifo_full;

    // mm_waitrequest high when reading from empty FIFO
    assign mm_waitrequest = mm_read && fifo_empty;

    // Push into FIFO
```

```
63      always @(posedge clk or negedge rst_n) begin
64          if (!rst_n) begin
65              wr_ptr <= {(ADDR_WIDTH+1){1'b0}};
66          end else begin
67              if (st_valid && st_ready) begin
68                  mem[wr_addr] <= st_data;
69                  wr_ptr       <= wr_ptr + 1'b1;
70              end
71          end
72      end
73
74      // Pop from FIFO on mm_read
75      always @(posedge clk or negedge rst_n) begin
76          if (!rst_n) begin
77              rd_ptr    <= {(ADDR_WIDTH+1){1'b0}};
78              rdata_reg <= 16'd0;
79          end else begin
80              if (mm_read && !fifo_empty) begin
81                  rdata_reg <= mem[rd_addr];
82                  rd_ptr    <= rd_ptr + 1'b1;
83              end
84          end
85      end
86
87      assign mm_readdata = {16'd0, rdata_reg};
88
89      // CSR interface
90      always @(*) begin
91          case (csr_address)
92              3'd0: csr_readdata = {22'd0, words_used};
93              3'd1: csr_readdata = {30'd0, fifo_full, fifo_empty};
94              default: csr_readdata = 32'd0;
95          endcase
96      end
97
98  endmodule
```

Giải thích mã nguồn Verilog `ch01_fifo_out.v` chi tiết từng dòng:

- **Dòng 16–42 (Cổng giao tiếp):** Tham số `ADDR_WIDTH = 9` ấn định dung lượng 512 từ 16-bit ($512 \times 16 = 8.192$ bit chiếm đúng 1 khối M10K). Cổng nạp nhận luồng Avalon-ST Sink 16-bit từ bộ lọc trung vị median-3; cổng đọc phục vụ Avalon-MM Read Slave cho HPS; cổng CSR cung cấp thông tin giám sát số lượng từ có sẵn.
- **Dòng 47–60 (Mảng nhớ và con trỏ 10-bit):** Con trỏ 10-bit `wr_ptr, rd_ptr` có bit 9 là bit vòng. Các tín hiệu cờ `fifo_empty`, `fifo_full` và số từ đang lưu `words_used` được suy ra trực tiếp.
- **Dòng 63–66 (Cơ chế phản áp và bảo vệ bus):**
  - `assign st_ready = !fifo_full;`: Tín hiệu sẵn sàng của giao diện Avalon-ST hạ xuống 0 khi bộ đệm chứa đủ 512 kết quả F0, ngăn chặn lõi DSP ghi đè kết quả chưa kịp đọc.
  - `assign mm_waitrequest = mm_read && fifo_empty;`: Khi HPS phát lệnh đọc `mm_read` mà bộ đệm đang rỗng, tín hiệu `mm_waitrequest` lập tức tích cực, yêu cầu cầu nối chèn chu kỳ chờ cho đến khi có kết quả mới.
- **Dòng 69–78 (Always block nạp mẫu Avalon-ST):** Tại sườn dương `clk`, nếu có dữ liệu hợp lệ và bộ đệm chưa đầy (`st_valid && st_ready`), từ F0 được ghi vào `mem[wr_addr]` và `wr_ptr` tăng 1.
- **Dòng 81–91 (Always block đọc Avalon-MM):** Khi lệnh đọc hợp lệ (`mm_read && !fifo_empty`), dữ liệu được chốt vào `rdata_reg` và con trỏ đọc `rd_ptr` tăng 1.
- **Dòng 93 (Định dạng đọc):** Ghép 16 bit 0 vào phần cao: `{16'h0, rdata_reg}` để tạo thành từ 32-bit chuẩn.
- **Dòng 96–102 (Cổng đọc CSR):** Trả về số lượng kết quả F0 hiện có `words_used` tại offset 0 và cờ đầy/rỗng tại offset 1.

Nếu thay đổi lựa chọn thiết kế thì sao?

- **Nếu dùng bộ giải gói tổ hợp (Combinational Unpacker) thay vì thanh ghi đệm:** Nếu bỏ thanh ghi `holding_word` và dùng MUX nối từ ngõ ra RAM `mem_rdata` thẳng tới ngõ ra `st_data`, đường trễ tổ hợp dài qua MUX và cổng kiểm tra `ready` sẽ đâm thẳng vào khối cửa sổ Hanning, làm suy giảm tần số hoạt động cực đại $F_{\max}$ **(phải đo)**. Việc chốt thanh ghi ở trạng thái `ST_FETCH` cách ly hoàn toàn miền đọc RAM với miền xử lý ST.
- **Nếu tăng độ sâu lên 512 từ 32-bit (1024 mẫu âm thanh):** Dung lượng $512 \times 32 = 16.384$ bit vượt quá dung lượng 1 khối M10K (10.240 bit), bắt buộc Quartus phải tốn tới 2 khối M10K vật lý. Trong khi đó, bộ đệm 256 từ (512 mẫu) đã chứa tới 3,2 bước nhảy khung (32 ms), hoàn toàn đủ để chống giật tiến trình Linux.

Tóm lại: Khối FIFO Verilog tự viết dùng con trỏ 9-bit có bit vòng để nhận biết trạng thái đầy/rỗng, suy diễn đúng 1 khối M10K và dùng máy trạng thái 4 bước chốt thanh ghi để tách mẫu 32-to-16 bit; khối FIFO chiều về chứa 512 kết quả F0 trong đúng 1 khối M10K; cả hai đều có cơ chế waitrequest và phản áp bảo đảm toàn vẹn dữ liệu tuyệt đối.

1.6. Bản đồ thanh ghi và giao tiếp điều khiển lõi xử lý

Kiến trúc bus trong Platform Designer: Vì sao phải tách riêng các Slave?

Khi xây dựng hệ thống nhúng SoC với công cụ Platform Designer của Intel, toàn bộ không gian địa chỉ vật lý của cầu nối Lightweight HPS-to-FPGA bắt đầu từ địa chỉ cơ sở:

$$\text{ALT_LWFPGASLVS_OFST} = \texttt{0xFF20_0000} \tag{140}$$

với cửa sổ ánh xạ có độ rộng 2 MB (`0x0020_0000`). Mạng chuyển mạch liên kết (Avalon-MM Interconnect Fabric) tự động định tuyến các giao dịch đọc/ghi của CPU ARM tới từng thiết bị ngoại vi dựa trên dải địa chỉ offset được phân bổ.

Một sai lầm trong thiết kế sơ khai của bản v1 là: *Cố gắng gộp tất cả dữ liệu âm thanh, cờ điều khiển và trạng thái vào một dải thanh ghi nguyên khối liên tục từ `0x00` đến `0x14`.*

Cách làm này phá vỡ tính module hóa của kiến trúc phần cứng:

- Khối IP On-Chip FIFO chuẩn của Intel (`altera_avalon_fifo`) sinh ra hai cổng Avalon-MM độc lập: cổng nạp dữ liệu `in` và cổng thanh ghi điều khiển/trạng thái `in_csr`. Mỗi cổng là một Slave riêng biệt với giao thức cố định.
- Nếu gộp các thanh ghi điều khiển nghiệp vụ (như tham số Praat GP, số khung, lệnh khởi động) vào chung khối FIFO, ta phá vỡ cấu trúc IP chuẩn và khiến việc mô phỏng cô lập từng khối trở nên bất khả thi.

Do đó, thiết kế chuẩn mực của dự án phân định rõ ràng 4 thiết bị Slave riêng biệt trong Platform Designer, bố trí tại các vùng địa chỉ cố định trên cầu Lightweight Bridge như trình bày trong Bảng 6.

Bảng 6: Bản đồ không gian địa chỉ các Slave trên cầu Lightweight HPS-to-FPGA.

Tên Slave	Địa chỉ vật lý (Offset)	Truy xuất	Bề rộng	Chức năng phần cứng
`fifo_in`	`0xFF20_0000 (+0x00)`	W	32 bit	Cổng nạp 2 mẫu âm thanh PCM 16-bit
`fifo_in_csr`	`0xFF20_0020 (+0x20)`	R/W	32 bit	Trạng thái bộ đệm nạp (`fill_level`, lỗi tràn)
`ctrl`	`0xFF20_0040 (+0x40)`	R/W	32 bit	Thanh ghi điều khiển và đo đạc lõi F0
`fifo_out`	`0xFF20_0080 (+0x80)`	R	32/16 bit	Cổng đọc kết quả F0 định dạng uint16 Q12.4
`fifo_out_csr`	`0xFF20_00A0 (+0xA0)`	R/W	32 bit	Trạng thái bộ đệm trả kết quả

Tập thanh ghi của khối điều khiển lõi (CTRL Slave)

Khối `ctrl` nằm tại offset `0x40` (địa chỉ vật lý `0xFF20_0040`) chứa toàn bộ các tham số cấu hình toán học và bộ đếm chu kỳ phục vụ đo đạc hiệu năng. Bảng 7 liệt kê chi tiết từng thanh ghi 32-bit:

Bảng 7: Chi tiết các thanh ghi điều khiển lõi xử lý F0 (`ctrl`).

Thanh ghi	Offset nội bộ	Quyền	Độ rộng	Ý nghĩa chức năng
`REG_CTRL_START`	`0x00`	W	1 bit	Ghi 1 để kích hoạt chuỗi xử lý F0 toàn tuyến
`REG_CTRL_NFRAMES`	`0x04`	R/W	32 bit	Tổng số khung phân tích N_{frames} cần tính
`REG_CTRL_FIRST_WIN0`	`0x08`	R/W	32 bit	Chỉ số mẫu bắt đầu của cửa sổ đầu tiên (win0)
`REG_CTRL_GLOBAL_PEAK`	`0x0C`	R/W	16 bit	Giá trị đỉnh biên độ toàn cục GP (số nguyên)
`REG_CTRL_CONFIG`	`0x10`	R/W	2 bit	Bit 0: Bypass FIR; Bit 1: Cờ bước nhảy chuẩn 160
`REG_CTRL_STATUS`	`0x14`	R	4 bit	Bit 0: Busy; Bit 1: Done; Bit 2–3: Cờ tràn FIFO
`REG_CTRL_TOTAL_CYCLES`	`0x18`	R	32 bit	Tổng chu kỳ phần cứng (xung 50 MHz) từ Start tới Done
`REG_CTRL_FRAME_CYCLES`	`0x1C`	R	32 bit	Số chu kỳ phần cứng xử lý khung gần nhất

Giải thích cặn kẽ từng thanh ghi điều khiển

Mỗi thanh ghi đều gắn liền trực tiếp với một khâu giải thuật trong chuỗi tính toán Praat:

1. Thanh ghi kích hoạt (`REG_CTRL_START`, Offset 0x00). Sau khi nạp toàn bộ cấu hình vào các thanh ghi và nạp trước một lượng mẫu âm thanh vào `fifo_in`, HPS ghi giá trị `0x0000_0001` vào thanh ghi này để bắt đầu xử lý.

2. Thanh ghi tổng số khung (`REG_CTRL_NFRAMES`, Offset 0x04). Chứa giá trị nguyên N_{frames} do thuật toán `Sampled_shortTermAnalysis` tính ra từ kích thước tệp âm thanh (ví dụ: 120 khung cho tệp 1,234 s hoặc 277 khung cho tệp 2,8 s).

3. Thanh ghi mẫu bắt đầu cửa sổ đầu (`REG_CTRL_FIRST_WIN0`, Offset 0x08). Chỉ số mẫu bắt đầu của khung phân tích đầu tiên (win0 tính từ 0), thu được từ hàm `Sampled_xToLowIndex` ở Mục 1.2 (ví dụ: mẫu 33 đối với tệp 1,234 s, hoặc mẫu 1 đối với tệp 2,8 s).

4. Thanh ghi đỉnh biên độ toàn cục (`REG_CTRL_GLOBAL_PEAK`, Offset 0x0C). Lưu trữ giá trị nguyên 16-bit $GP = \text{round}(\max|n \cdot x - \sum x|/n)$ do HPS tính toán trong lượt quét RIFF WAV. Giá trị này được đưa thẳng vào khối quyết định V/UV ở Chương 10 theo quy tắc số nguyên chuẩn của đề cương:

$$\text{Khung có thanh (Voiced)} \iff (100 \cdot \text{Sc[best]} - 45 \cdot R_{0n}) \cdot 3 \cdot GP > 100 \cdot R_{0n} \cdot \max(0, 6 \cdot GP - 145 \cdot LP) \quad (141)$$

Đại lượng GP tham gia vào biểu thức dưới dạng biên độ (bậc 1) để cân bằng thứ nguyên với đỉnh biên độ cục bộ LP trong tỉ số cường độ $I = LP/GP$. Việc nhân chéo số nguyên với GP triệt tiêu hoàn toàn phép chia số thực dấu phẩy động trên FPGA.

5. Thanh ghi cấu hình (`REG_CTRL_CONFIG`, Offset 0x10). Chứa các cờ điều khiển chế độ vận hành linh hoạt của phần cứng:

- **Bit 0 (`BYPASS_FIR`):** Khi ghi 1, dòng mẫu âm thanh từ FIFO sẽ đi vòng qua khối lọc làm mềm mép Nyquist FIR để vào thẳng bộ nhớ khung. Thiết lập này dùng để đối chiếu bit-exact với Praat 7.0.03 (mặc định bypass). Khi ghi 0, bộ lọc FIR 31-tap được kích hoạt; khi bật, lưới khung phải dịch đi đúng 15 mẫu tương ứng với trễ nhóm của bộ lọc.
- **Bit 1 (`HOP_REGULAR`):** Cho phép chọn giữa bước nhảy đều cố định 160 mẫu (`HOP_REGULAR = 1`) hoặc bám sát danh sách bước nhảy double của Praat (`HOP_REGULAR = 0`) đối với các tệp có kích thước số mẫu lẻ.

6. Thanh ghi trạng thái (`REG_CTRL_STATUS`, Offset 0x14). Phần mềm trên HPS đọc định kỳ thanh ghi này để kiểm soát luồng: Bit 0 (`BUSY`), Bit 1 (`DONE`), Bit 2 (`FIFO_IN_OVF`) và Bit 3 (`FIFO_OUT_OVF`).

7. Thanh ghi bộ đếm chu kỳ (`TOTAL_CYCLES` và `FRAME_CYCLES`, Offset 0x18–0x1C). Bộ đếm số nguyên 32-bit hoạt động ở tần số xung nhịp đồng bộ 50 MHz ghi nhận tổng thời gian thực thi phần cứng T_{FPGA} **(phải đo)** và chu kỳ xử lý từng khung đơn lẻ (lý thuyết là 98.543 chu kỳ xung nhịp cho khâu MAC và tìm đỉnh).

Tóm lại: Bản đồ thanh ghi phân lập rõ 4 Slave trong Platform Designer loại bỏ lỗi gộp khối của bản v1; các thanh ghi điều khiển cung cấp đầy đủ tham số toán học (GP biên độ 16-bit, mẫu win0, tổng số khung) và bộ đếm chu kỳ 50 MHz độc lập, bảo đảm giám sát hệ thống chính xác từng nhịp xung clock.

1.7. Phần mềm điều khiển trên HPS và kiểm chứng toàn diện

Trình điều khiển phần mềm trên HPS (C Driver)

Mã nguồn điều khiển phần cứng trên CPU ARM Cortex-A9 được hiện thực tại `sw_v3/ch01_driver.c`. Trình điều khiển chạy ở không gian người dùng Linux qua cơ chế ánh xạ bộ nhớ `mmap` trên tệp `/dev/mem`:

```
int fd = open("/dev/mem", O_RDWR | O_SYNC);
void *virtual_base = mmap(NULL, HW_REGS_SPAN,
                          PROT_READ | PROT_WRITE,
                          MAP_SHARED, fd, ALT_LWFPGASLVS_OFST);
```

Quy trình điều khiển toàn tuyến được tổ chức theo 5 bước tuần tự:

1. **Phân tích tệp âm thanh:** Gọi mô-đun `ch01_wav.c` để trích xuất mẫu PCM 16-bit, kiểm tra 16 kHz mono và tính đỉnh biên độ toàn cục $GP = \text{round}(\max |n \cdot x - \sum x|/n)$.
2. **Xác định lưới phân tích khung:** Gọi mô-đun `ch01_frames.c` theo công thức double của Praat để xác định N_{frames}, t_1 và win0.
3. **Cấu hình thanh ghi điều khiển:** Ghi các tham số cấu hình vào khối `ctrl` tại offset `0x40`.
4. **Nạp dữ liệu dòng mẫu có thăm dò mức đầy:** Đóng gói hai mẫu 16-bit thành từ 32-bit theo chuẩn little-endian:
$$W_k = ((\text{sample}_{2k+1} \ \&\ \texttt{0xFFFF}) \ll 16) \ \mid \ (\text{sample}_{2k} \ \&\ \texttt{0xFFFF}) \tag{142}$$
Đọc thanh ghi mức đầy `fill_level` tại `fifo_in_csr`; nếu mức đầy vượt 480 mẫu (`almost_full`), CPU tạm ngừng phát để tránh tràn bộ đệm.
5. **Kích hoạt và thu nhận kết quả:** Ghi xung kích hoạt vào `REG_CTRL_START`, thăm dò cờ `STATUS_DONE`, sau đó đọc kết quả F0 định dạng uint16 LE Q12.4 từ `fifo_out` để lưu vào `pitch_f0.bin`.

Chế độ giả lập không bo mạch (-DSIM). Tệp `ch01_driver.c` hỗ trợ cờ biên dịch `-DSIM` thay thế các lệnh `mmap` bằng cấu trúc bộ nhớ giả lập `mock_hardware_t`. Kịch bản `ch01_driver_test.py` chạy kiểm tra trên tệp chuẩn `synth_glide_vuv_16k.wav` (2,8 s, 277 khung), ghi nhận kết quả xuất ra tệp `pitch_f0.bin` kích thước chính xác 554 byte (277×2 byte) với 0 lỗi.

Mô hình song sinh kiểm chứng chu kỳ (Python Cycle-Accurate Twin)

Mô hình song sinh `verify_v3/ch01_fifo_twin.py` mô phỏng chính xác từng chu kỳ xung nhịp của cả khối FIFO vào (`ch01_avalon_fifo.v`) lẫn khối FIFO ra (`ch01_fifo_out.v`).

Listing 5 trình bày mã nguồn lớp `AvalonFifoTwin`:

Listing 5: Mô hình song sinh chu kỳ `AvalonFifoTwin` trong `verify_v3/ch01_fifo_twin.py`.

```
class AvalonFifoTwin:
    def __init__(self, depth_words=256):
        self.depth_words = depth_words
        self.mem = [0] * depth_words
        self.wr_ptr = 0
        self.rd_ptr = 0
        self.ovf_sticky = 0

        # Unpacker FSM states
        self.ST_IDLE = 0
        self.ST_FETCH = 1
        self.ST_UNPACK0 = 2
```

```
        self.ST_UNPACK1 = 3
        self.st_state = self.ST_IDLE

        self.holding_word = 0
        self.mem_rdata = 0

    @property
    def words_used(self):
        return (self.wr_ptr - self.rd_ptr) & ((2 * self.depth_words) - 1)

    @property
    def fifo_empty(self):
        return self.wr_ptr == self.rd_ptr

    @property
    def fifo_full(self):
        return self.words_used == self.depth_words

    @property
    def waitrequest(self):
        return 1 if self.fifo_full else 0

    @property
    def unpacker_samples(self):
        if self.st_state == self.ST_UNPACK0:
            return 2
        elif self.st_state == self.ST_UNPACK1:
            return 1
        elif self.st_state == self.ST_FETCH:
            return 2
        return 0

    @property
    def fill_level_samples(self):
        return (self.words_used * 2) + self.unpacker_samples

    def step(self, mm_write, mm_writedata, st_ready):
        prev_fifo_empty = self.fifo_empty
        prev_fifo_full = self.fifo_full
        prev_rd_addr = self.rd_ptr & (self.depth_words - 1)
        prev_wr_addr = self.wr_ptr & (self.depth_words - 1)

        # 1. Next state of unpacker FSM
        next_state = self.st_state
        next_holding = self.holding_word
        inc_rd = False

        if self.st_state == self.ST_IDLE:
            if not prev_fifo_empty:
                next_state = self.ST_FETCH
                inc_rd = True
        elif self.st_state == self.ST_FETCH:
            next_holding = self.mem_rdata
            next_state = self.ST_UNPACK0
        elif self.st_state == self.ST_UNPACK0:
            if st_ready:
                next_state = self.ST_UNPACK1
        elif self.st_state == self.ST_UNPACK1:
            if st_ready:
                if not prev_fifo_empty:
                    next_state = self.ST_FETCH
                    inc_rd = True
                else:
                    next_state = self.ST_IDLE

        # 2. Synchronous RAM registered read
        next_mem_rdata = self.mem[prev_rd_addr]

        # 3. Synchronous RAM write
        if mm_write:
            if not prev_fifo_full:
                self.mem[prev_wr_addr] = mm_writedata & 0xFFFFFFFF
                self.wr_ptr = (self.wr_ptr + 1) & ((2 * self.depth_words) - 1)
            else:
                self.ovf_sticky = 1

        # 4. Advance unpacker registers
        if inc_rd:
```

```
            self.rd_ptr = (self.rd_ptr + 1) & ((2 * self.depth_words) - 1)
        self.st_state = next_state
        self.holding_word = next_holding
        self.mem_rdata = next_mem_rdata

        # 5. Output signals after clock edge
        st_valid = 1 if self.st_state in (self.ST_UNPACK0, self.ST_UNPACK1) else 0
        if self.st_state == self.ST_UNPACK0:
            st_data = self.holding_word & 0xFFFF
        elif self.st_state == self.ST_UNPACK1:
            st_data = (self.holding_word >> 16) & 0xFFFF
        else:
            st_data = 0
        waitrequest = 1 if self.fifo_full else 0
        fill = self.fill_level_samples
        state = self.st_state

        return waitrequest, st_valid, st_data, state, fill
```

Giải thích mã nguồn Python Twin `AvalonFifoTwin`:

- **Dòng 18–27:** Khởi tạo mảng nhớ 256 từ 32-bit, hai con trỏ vòng `wr_ptr, rd_ptr`, 4 trạng thái FSM (IDLE=0, FETCH=1, UNP0=2, UNP1=3) và các thanh ghi trung gian.
- **Dòng 29–58:** Các thuộc tính mô phỏng logic tổ hợp của phần cứng: `words_used` tính hiệu số modulo 512; `waitrequest` bằng 1 khi đầy; `fill_level_samples` tính tổng số mẫu đang lưu.
- **Dòng 59–129 (Phương thức `step`):** Mô phỏng chính xác sườn dương xung nhịp. Dòng 64–67 lưu lại các cờ trạng thái trước sườn clock. Dòng 74–98 tính trạng thái kế tiếp của FSM và lệnh tăng `rd_ptr`. Dòng 101 mô phỏng cổng đọc RAM có thanh ghi chốt giá trị tại địa chỉ đọc cũ. Dòng 104–110 mô phỏng cổng ghi RAM và kích hoạt cờ `ovf_sticky` nếu bị ghi ép khi đầy. Dòng 118–129 xác định các tín hiệu ngõ ra sau sườn clock.

Listing 6 trình bày mô hình song sinh của khối bộ đệm hoàn trả `FifoOutTwin`:

Listing 6: Mô hình song sinh chu kỳ `FifoOutTwin` trong `verify_v3/ch01_fifo_twin.py`.

```
class FifoOutTwin:
    def __init__(self, depth_words=512):
        self.depth_words = depth_words
        self.mem = [0] * depth_words
        self.wr_ptr = 0
        self.rd_ptr = 0
        self.rdata_reg = 0

    @property
    def words_used(self):
        return (self.wr_ptr - self.rd_ptr) & ((2 * self.depth_words) - 1)

    @property
    def fifo_empty(self):
        return self.wr_ptr == self.rd_ptr

    @property
    def fifo_full(self):
        return self.words_used == self.depth_words

    @property
    def st_ready(self):
        return 0 if self.fifo_full else 1

    def step(self, st_valid, st_data, mm_read):
        ready_before = self.st_ready
        empty_before = self.fifo_empty

        # Push to FIFO
        if st_valid and ready_before:
            wr_addr = self.wr_ptr & (self.depth_words - 1)
            self.mem[wr_addr] = st_data & 0xFFFF
            self.wr_ptr = (self.wr_ptr + 1) & ((2 * self.depth_words) - 1)

        # Pop from FIFO
        if mm_read and not empty_before:
            rd_addr = self.rd_ptr & (self.depth_words - 1)
            self.rdata_reg = self.mem[rd_addr]
```

```
            self.rd_ptr = (self.rd_ptr + 1) & ((2 * self.depth_words) - 1)

        st_ready_out = self.st_ready
        mm_waitrequest_out = 1 if (mm_read and self.fifo_empty) else 0
        mm_readdata_out = self.rdata_reg & 0xFFFF
        words_used_out = self.words_used

        return st_ready_out, mm_waitrequest_out, mm_readdata_out, words_used_out
```

Bảng dấu vết chu kỳ chuẩn xác (Cycle-by-Cycle Trace Table)

Bảng 8 trích xuất 12 chu kỳ tiêu biểu từ tệp dấu vết chuẩn `verify_v3/tb_ch01_fifo_expected.txt` do mô hình Python twin sinh ra, minh họa trọn vẹn các trạng thái hoạt động cốt lõi của phần cứng:

Bảng 8: Dấu vết chu kỳ hoạt động của bộ đệm `ch01_avalon_fifo` sinh bởi Python Twin.

Chu kỳ	mm_write	writedata	waitreq	st_valid	st_ready	st_data	FSM State	fill
0	1	00020001	0	0	1	0000	ST_IDLE	2
1	0	00000000	0	0	1	0000	ST_FETCH	2
2	0	00000000	0	1	0	0001	ST_UNPACK0	2
3	0	00000000	0	1	0	0001	ST_UNPACK0	2
4	1	00040003	0	1	1	0002	ST_UNPACK1	3
5	0	00000000	0	0	1	0000	ST_FETCH	2
6	0	00000000	0	1	1	0003	ST_UNPACK0	2
7	0	00000000	0	1	1	0004	ST_UNPACK1	1
… Nạp dồn 255 từ với `st_ready = 0` cho tới khi RAM đầy …								
262	1	11fd11fc	0	1	0	0004	ST_UNPACK1	511
263	1	20012000	1	1	0	0004	ST_UNPACK1	513
264	1	deadbeef	1	1	0	0004	ST_UNPACK1	513
265	0	00000000	0	0	1	0000	ST_FETCH	512

Đọc bảng dấu vết từng nhịp:

- **Chu kỳ 0–1 (Ghi và nạp):** Tại chu kỳ 0, từ `0x00020001` được ghi vào RAM. Sang chu kỳ 1, FSM phát hiện `!fifo_empty`, chuyển sang `ST_FETCH` và phát lệnh đọc RAM.
- **Chu kỳ 2–3 (Giải gói mẫu 1 và dừng do nghẽn):** Tại chu kỳ 2, FSM vào `ST_UNPACK0`, giương cờ `st_valid = 1` và đưa mẫu thấp `0x0001` ra bus. Vì mạch hạ nguồn báo bận (`st_ready = 0`), tại chu kỳ 3 FSM giữ nguyên trạng thái `ST_UNPACK0` và giữ nguyên mẫu `0x0001` trên bus.
- **Chu kỳ 4–5 (Ghi nối tiếp và chuyển thẳng):** Tại chu kỳ 4, do `st_ready = 1`, FSM sang `ST_UNPACK1` phát mẫu cao `0x0002`, đồng thời HPS ghi từ `0x00040003`. Vì RAM còn từ mới, tại chu kỳ 5 FSM chuyển thẳng sang `ST_FETCH` mà không qua `ST_IDLE`.
- **Chu kỳ 6–7 (Phát mẫu từ thứ hai):** Lần lượt phát mẫu `0x0003` ở UNPACK0 và `0x0004` ở UNPACK1.
- **Chu kỳ 262–265 (Đầy, tích cực waitrequest và kích hoạt cờ tràn):** Sau chuỗi nạp liên tục khi `st_ready = 0`, tại chu kỳ 262 RAM chứa 255 từ. Đến chu kỳ 263, từ thứ 256 được nạp, RAM đầy hoàn toàn, tín hiệu `mm_waitrequest` bật lên 1. Tại chu kỳ 264, master cố tình ghi tiếp `0xDEADBEEF` khi `waitrequest = 1`; phần cứng từ chối ghi và kích hoạt cờ tràn `ovf_sticky = 1`. Đến chu kỳ 265, khi master hạ lệnh ghi và hạ nguồn bật `st_ready = 1`, FSM giải phóng một từ, `mm_waitrequest` lập tức hạ về 0.

Kiểm thử phần cứng Verilog với Testbench chuyên dụng

Hai testbench Verilog chuẩn được xây dựng để kiểm chứng trực tiếp với mô hình twin:

1. `tb/v3/tb_ch01_fifo.v`: Đọc 2.107 dòng kích thích từ `tb_ch01_fifo_stim.txt` (tạo bởi twin với hạt giống ngẫu nhiên 20261110, xác suất ghi 80% và áp lực ngược 60%), đối soát từng chu kỳ với `tb_ch01_fifo_expected.txt`.
2. `tb/v3/tb_ch01_fifo_out.v`: Đọc 1.217 dòng kích thích từ `tb_ch01_fifo_out_stim.txt`, đối soát từng chu kỳ với `tb_ch01_fifo_out_expected.txt`.

Kịch bản shell `verify_v3/run_ch01_iverilog.sh` tự động biên dịch và thực thi bằng trình biên dịch Icarus Verilog:

```
iverilog -g2012 -Wall -I rtl/v3 -o sim_ch01_fifo tb/v3/tb_ch01_fifo.v
    rtl/v3/ch01_avalon_fifo.v
vvp sim_ch01_fifo
iverilog -g2012 -Wall -I rtl/v3 -o sim_ch01_fifo_out tb/v3/tb_ch01_fifo_out.v
    rtl/v3/ch01_fifo_out.v
vvp sim_ch01_fifo_out
```

Kết quả chạy trên Linux (Icarus Verilog 12, `verify_v3/ch01_iverilog.log`): `tb_ch01_fifo` khớp twin 2 107/2 107 chu kỳ, `tb_ch01_fifo_out` khớp 1 217/1 217 chu kỳ, 0 lỗi so sánh.

Bảng so sánh tổng kết: IP On-Chip FIFO của Intel so với HDL tự viết

Bảng 9 cung cấp bức tranh đối chiếu toàn diện giữa phương án sử dụng IP đóng gói sẵn của Intel và phương án tự hiện thực bằng HDL:

Bảng 9: So sánh toàn diện giữa Intel On-Chip FIFO IP và mã HDL tự viết.

Tiêu chí so sánh	**Intel On-Chip FIFO IP**	**HDL tự viết (`ch01_avalon_fifo`)**
Khối nhớ nhúng M10K	1 khối (cấu hình 256×32, 512 mẫu)	1 khối (cấu hình 256×32, 512 mẫu)
Tài nguyên logic ALM	**phải đo (Fitter, Ch.12)**	**phải đo (Fitter, Ch.12)**
Tần số cực đại $F_{\max}$	**phải đo (TimeQuest, Ch.12)**	**phải đo (TimeQuest, Ch.12)**
Độ trễ ghi tới hợp lệ (Latency)	**phải kiểm trong mô phỏng IP**	2 chu kỳ (đo bằng iverilog)
Đóng gói và tách mẫu	Xuất 32-bit, cần adapter 32→16 bit	Tự động tách 32→16 bit qua Unpacker FSM
Khả năng mô phỏng ngoài Quartus	Cần thư viện mã hóa Intel	Chạy ngay trên iverilog / Verilator
Khả năng kiểm soát chu kỳ	Hộp đen (Black-box)	Hoàn toàn minh bạch (White-box)
Độ phức tạp tích hợp	Nhấp chuột trong Qsys	Khai báo module Verilog chuẩn

Đọc bảng. Cả hai giải pháp đều đạt hiệu quả tối ưu về mặt tài nguyên bộ nhớ nhúng khi chỉ tiêu tốn đúng 1 khối M10K duy nhất trên chip Cyclone V. Tuy nhiên, khối HDL tự viết tích hợp sẵn FSM giải gói 32-sang-16 bit và mang lại lợi thế vượt trội về tính độc lập: mã nguồn hoàn toàn mở, dễ dàng kiểm chứng trên các trình mô phỏng nhẹ như Icarus Verilog mà không đòi hỏi bộ thư viện bản quyền nặng nề của Intel.

Bài tập cuối chương có lời giải chi tiết

Bài tập 1: Tính toán RTF truyền tải cho tín hiệu chuẩn âm thanh chất lượng cao. Giả sử hệ thống nâng cấp để xử lý tín hiệu âm thanh phòng thu với tần số lấy mẫu $f_s = 48$ kHz, độ phân giải 24-bit (mỗi mẫu được lưu trữ trong một từ 32-bit). Hệ thống vẫn dùng cơ chế PIO qua cầu Lightweight Bridge 32-bit ở tần số 50 MHz. Giả định thời gian thực hiện một lệnh ghi PIO bi quan nhất của CPU là 1 μs.

(a) Tính lưu lượng dữ liệu cần truyền tải trong một giây âm thanh.

(b) Tính hệ số thời gian thực $\text{RTF}_{\text{transfer}}$ của khâu truyền tải và đưa ra kết luận kỹ thuật.

Lời giải:

(a) Vì mỗi mẫu 24-bit chiếm một từ 32-bit (4 byte), lưu lượng dữ liệu trong một giây là:

$$B = 48.000 \text{ mẫu/giây} \times 4 \text{ byte} = 192.000 \text{ byte/giây} = 192 \text{ KB/s} = 0{,}192 \text{ MB/s} \tag{143}$$

So với băng thông 200 MB/s của cầu Lightweight Bridge, lưu lượng này chỉ chiếm $0{,}192/200 = 0{,}096\%$ dung lượng bus.

(b) Mỗi giây âm thanh cần thực hiện đúng 48.000 thao tác ghi 32-bit. Với thời gian 1 μs cho mỗi thao tác ghi, tổng thời gian CPU tiêu tốn là:

$$T_{\text{CPU}} = 48.000 \times 1\ \mu\text{s} = 0{,}048 \text{ giây} = 48 \text{ ms} \tag{144}$$

Hệ số thời gian thực của khâu truyền dữ liệu là:

$$\text{RTF}_{\text{transfer}} = \frac{T_{\text{CPU}}}{1 \text{ giây}} = \mathbf{0{,}048} \tag{145}$$

Kết luận: CPU chỉ tốn $4{,}8\%$ thời gian cho việc truyền tải âm thanh 48 kHz 24-bit ($95{,}2\%$ thời gian còn lại rảnh rỗi). Do đó, ngay cả ở chuẩn 48 kHz, cơ chế PIO qua Lightweight Bridge vẫn hoàn toàn đáp ứng thời gian thực với biên dư dồi dào mà chưa cần thiết phải chuyển sang dùng DMA.

Bài tập 2: Số học bit vòng và điều kiện rỗng/đầy cho bộ đệm có độ sâu tùy ý. Xét một bộ đệm FIFO có độ sâu N từ, trong đó N không phải là lũy thừa của 2 (ví dụ $N = 300$).

(a) Tại sao kỹ thuật kiểm tra bit vòng MSB đơn giản (`wr_ptr[k]` $\neq$ `rd_ptr[k]`) không còn áp dụng được trực tiếp?

(b) Nêu giải pháp phần cứng chuẩn mực để giải quyết bài toán này mà vẫn duy trì tần số hoạt động cao.

Lời giải:

(a) Khi $N = 2^k$, không gian đếm tự nhiên của thanh ghi k-bit quay vòng chính xác tại $2^k - 1 \to 0$. Khi thêm bit thứ $k+1$, nửa vòng đầu và nửa vòng sau có các bit thấp giống hệt nhau. Nhưng khi $N = 300 \neq 2^k$, sau khi con trỏ ghi đạt giá trị 299, nó phải được reset cưỡng bức về 0 thay vì tràn tự nhiên qua 511. Sự lệch pha này khiến khoảng cách giữa hai con trỏ không thể suy ra bằng phép trừ bit thông thường, làm sai lệch điều kiện so sánh bằng của các bit thấp.

(b) Giải pháp chuẩn mực trong thiết kế phần cứng:

- Duy trì một bộ đếm số phần tử độc lập mang tên `count` (độ rộng 9 bit).
- Mỗi chu kỳ: nếu chỉ ghi thì `count` $\leftarrow$ `count` $+ 1$; nếu chỉ đọc thì `count` $\leftarrow$ `count` $- 1$; nếu đồng thời cả đọc và ghi thì `count` giữ nguyên.
- Điều kiện rỗng là `count` $== 0$, điều kiện đầy là `count` $== 300$. Các con trỏ `wr_ptr` và `rd_ptr` được so sánh với 299 để tự reset về 0.

Bài tập 3: Phân tích xung đột tương tranh (Race Condition) và cơ chế khóa. Trong kiến trúc streaming âm thanh giữa HPS và FPGA, CPU ghi mẫu âm thanh vào FIFO trong khi lõi tính toán FPGA đọc mẫu ra để tính cửa sổ Hanning.

(a) Tại sao hệ thống trong chương này không cần sử dụng các cấu trúc khóa bảo vệ phức tạp (như mutex hay spinlock) giữa CPU và FPGA?

(b) Điều gì sẽ xảy ra nếu CPU đọc thanh ghi `fill_level` đúng vào chu kỳ mà FPGA vừa đọc một từ ra khỏi FIFO?

Lời giải:

(a) Hệ thống không cần mutex vì kiến trúc tuân thủ nguyên tắc Producer-Consumer một chiều trên hai miền giao tiếp độc lập:

- Phía HPS là bên sản xuất (Producer): HPS chỉ có quyền ghi vào mảng RAM và chỉ có quyền sửa đổi con trỏ `wr_ptr`. HPS hoàn toàn không thể thay đổi `rd_ptr`.
- Phía FPGA là bên tiêu thụ (Consumer): FPGA chỉ có quyền đọc mảng RAM và chỉ có quyền sửa đổi con trỏ `rd_ptr`. FPGA hoàn toàn không thể thay đổi `wr_ptr`.

Vì mỗi bên chỉ sở hữu độc quyền một con trỏ, không có biến trạng thái chung nào bị xung đột ghi đồng thời (write-write race condition).

(b) Nếu CPU đọc `fill_level` đúng nhịp FPGA cập nhật `rd_ptr`: Trong trường hợp xấu nhất, CPU có thể đọc được giá trị `fill_level` cũ (chưa giảm đi 2 mẫu). Điều này hoàn toàn an toàn (fail-safe): CPU sẽ cho rằng bộ đệm đầy hơn thực tế một chút, do đó nó sẽ ghi thận trọng hơn chứ không bao giờ gây ra hiện tượng ghi tràn bộ đệm.

Bài tập 4: Đánh giá tác động của lỗi thứ tự byte (Endianness) lên ước lượng F0. Một lập trình viên sơ suất đóng gói hai mẫu âm thanh 16-bit vào từ 32-bit theo chuẩn Big-Endian: sample_0 nằm ở `[31:16]` và sample_1 nằm ở `[15:0]`. Tuy nhiên, phần cứng giải gói FPGA vẫn vận hành theo chuẩn Little-Endian (phát `[15:0]` trước, `[31:16]` sau).

(a) Mô tả sự xáo trộn xảy ra trên chuỗi mẫu âm thanh đi vào lõi tính toán.

(b) Phân tích định tính ảnh hưởng của lỗi này lên hàm tự tương quan $R[\tau]$ và ước lượng tần số cơ bản F0.

Lời giải:

(a) Chuỗi âm thanh ban đầu $x[0], x[1], x[2], x[3], \dots$ sẽ bị hoán đổi từng cặp mẫu kế tiếp:

$$x_{\text{FPGA}} = \{x[1], x[0], x[3], x[2], x[5], x[4], \dots\} \tag{146}$$

Mỗi mẫu chẵn và lẻ bị tráo đổi vị trí cho nhau trên toàn bộ chiều dài tệp.

(b) *Ảnh hưởng lên F0:*

- Phép đảo cặp này tạo ra một thành phần nhiễu tần số cực cao ở tần số Nyquist $f_s/2 = 8$ kHz (bởi vì hiệu số giữa hai mẫu liên tiếp $x[2k+1] - x[2k]$ bị đảo dấu liên tục theo chu kỳ 2 mẫu).
- Tuy nhiên, các đỉnh tự tương quan của chu kỳ cơ bản giọng nói ($F_0 \in [75, 600]$ Hz) tương ứng với dải quét tương quan $\tau \in [26, 214]$ (189 lag) và dải tìm kiếm cực đại ứng viên $\tau \in [27, 213]$ mẫu. Khi dịch đi τ mẫu (với $\tau \gg 2$), hình dạng bao sóng năng lượng tổng thể vẫn bảo toàn tính tuần hoàn cơ bản. Kết quả là đỉnh $R[\tau]$ vẫn xuất hiện gần vị trí lag thực tế, nhưng đỉnh sẽ bị răng cưa nghiêm trọng và giá trị nội suy cực trị parabol bị kéo lệch, dẫn đến việc tăng sai số cent mịn và gây ra các quyết định nhảy quãng tám (octave jump) thất thường.

Tóm lại: Trình điều khiển HPS giao tiếp mmap với bộ đệm và bản đồ thanh ghi đã được kiểm chứng bit-exact qua Python twin và testbench Verilog; so sánh tổng hợp khẳng định HDL tự viết đạt trần hiệu năng phần cứng ngang ngửa IP Intel nhưng mang lại tính minh bạch và khả năng kiểm thử độc lập tối đa.

Chương 2. Chuẩn giao tiếp Avalon-ST, ngân sách chu kỳ khi xử lý tệp và kỹ thuật mã hóa FSM

▶ MỤC TIÊU HỌC TẬP

- Thấu hiểu bản chất cơ chế bắt tay dòng chảy dữ liệu (streaming handshake) theo chuẩn giao tiếp công nghiệp Intel Avalon Streaming (Avalon-ST), phân biệt rõ ràng giữa hai tín hiệu kiểm soát `valid` (hợp lệ) và `ready` (sẵn sàng).
- Phân tích cặn kẽ cạm bẫy kỹ thuật của thông số `readyLatency` (độ trễ sẵn sàng 0 chu kỳ so với 1 chu kỳ), giải mã cơ chế gây thất thoát dữ liệu âm thanh khi kết nối các khối sở hữu trí tuệ (IP core) của Intel.
- Xác lập ngân sách chu kỳ xung nhịp chính xác từ nguyên lý đầu tiên cho hệ thống ước lượng cao độ F_0 khi dữ liệu đầu vào được cấp phát từ tệp WAV thay vì từ bộ giải mã âm thanh ngoại vi; phân định rõ sự khác biệt giữa thước đo thời gian thực vật lý và tốc độ tính toán phần cứng.
- Khảo sát toàn diện 7 phương án mã hóa máy trạng thái hữu hạn FSM (Finite State Machine) trên phần mềm tổng hợp Quartus Prime; đánh giá sự đánh đổi giữa tài nguyên thanh ghi flip-flop (FF) và cổng logic tổ hợp (LUT) trên kiến trúc FPGA Intel Cyclone V SoC.
- Chứng minh toán học chặt chẽ tính chất chuyển mạch đơn bit của mã Gray (`n ^ (n >> 1)`) thông qua lập luận chuỗi bit 1 tận cùng (trailing-ones argument).
- Thiết kế và hiện thực hóa khối đệm trượt (Skid Buffer) hai tầng chốt bằng ngôn ngữ Verilog-2001 nhằm triệt tiêu đường phản áp tổ hợp ngược (combinational backpressure loop), bảo toàn 100% thông lượng truyền thông.
- Xây dựng mô hình song sinh phần mềm (Python twin) và kịch bản kiểm thử chu kỳ (cycle-accurate testbench) với hơn 500 vector kích thích ngẫu nhiên, chứng minh hai phương án mã hóa FSM nhị phân và one-hot cho kết quả trùng khớp hoàn toàn.

2.1. Chuẩn giao thức Avalon-ST: Cơ chế valid/ready và cạm bẫy readyLatency

Vấn đề thực tế: Dòng chảy dữ liệu không xung nhịp ép bước

Khi thiết kế các hệ thống xử lý tín hiệu số (DSP – Digital Signal Processing) trên vi mạch FPGA (Field-Programmable Gate Array), người lập trình thường bắt đầu với các mạch tuần tự đơn giản: mỗi nhịp xung nhịp đồng hồ, một mẫu dữ liệu mới đi vào, chạy qua các tầng thanh ghi và cho ra kết quả ở đầu bên kia. Mô hình đó hoạt động chính xác khi và chỉ khi nguồn phát (source) và nơi nhận (sink) luôn luôn sẵn sàng ở mọi nhịp xung nhịp.

Tuy nhiên, trong một hệ thống thực tế như thuật toán ước lượng cao độ tiếng nói F_0, dòng chảy dữ liệu có tính chất gián đoạn:

- Khối giải nén và phân phối mẫu âm thanh từ bộ nhớ hàng đợi FIFO (First-In First-Out) có thể phải tạm dừng một nhịp khi bộ đệm rỗng hoặc khi chuyển đổi giữa từ 32-bit sang mẫu 16-bit.
- Khối nhân tích lũy tự tương quan MAC (Multiply-Accumulate) cần tới 98.543 chu kỳ xung nhịp để quét toàn bộ 189 độ trễ $\tau \in [26, 214]$ cho một khung 638 mẫu. Trong suốt thời gian tính toán này, khối MAC hoàn toàn không thể tiếp nhận thêm mẫu âm thanh mới của khung kế tiếp.
- Nếu khối phát đẩy mẫu lên bus mà không kiểm tra trạng thái bận của khối nhận, các mẫu âm thanh mới sẽ ghi đè lên mẫu cũ, làm vỡ cấu trúc khung và làm sai lệch quá trình tính toán.

Để giải quyết bài toán đồng bộ luồng dữ liệu giữa các khối có tốc độ xử lý chênh lệch, tập đoàn Intel chuẩn hóa giao diện truyền thông điểm-tới-điểm mang tên **Avalon Streaming Interface (Avalon-ST)**, được đặc tả chi tiết trong tài liệu kỹ thuật *Intel Avalon Interface Specifications* (Mã tài liệu: MNL-AVALONIF).

Nguyên lý bắt tay valid/ready: Khi nào một mẫu được coi là chuyển giao?

Trọng tâm của chuẩn Avalon-ST là cơ chế bắt tay hai chiều thông qua cặp tín hiệu điều khiển đơn bit:

1. `valid` (Phía phát đưa ra): Bằng 1 khi phía phát đã đặt một mẫu dữ liệu hợp lệ lên các đường dây dữ liệu `data`.
2. `ready` (Phía nhận đưa ra): Bằng 1 khi phía nhận có đủ không gian bộ đệm và sẵn sàng nhận dữ liệu từ bus vào thanh ghi của mình.

Nguyên lý cơ bản trong chuẩn giao tiếp là: **Một giao dịch truyền nhận dữ liệu (beat) chỉ được hoàn thành tại chu kỳ xung nhịp n khi và chỉ khi cả hai tín hiệu `valid` và `ready` cùng bằng 1 tại chính chu kỳ đó**:

$$\text{Transfer}(n) = \texttt{valid}(n) \ \wedge \ \texttt{ready}(n) \tag{147}$$

Nếu `valid` = 1 nhưng `ready` = 0, phía nhận đang áp dụng cơ chế **phản áp (backpressure)**. Khi đó, không có giao dịch nào diễn ra. Quy tắc bắt buộc đối với phía phát là: **phải giữ nguyên giá trị dữ liệu `data` và giữ `valid` = 1 ở chu kỳ kế tiếp**, phía phát không được thay đổi dữ liệu hoặc hạ `valid` xuống 0 cho đến khi phía nhận bật `ready` = 1 trở lại.

Cạm bẫy kỹ thuật: `readyLatency` = 0 so với `readyLatency` = 1

Tưởng chừng nguyên lý trên đã đủ chặt chẽ, nhưng một cạm bẫy kỹ thuật nảy sinh khi ghép nối các khối phần cứng: *Tín hiệu `ready` nói về khả năng tiếp nhận ở chính nhịp này hay ở nhịp sau?* Câu trả lời nằm ở tham số cấu hình mang tên `readyLatency` (độ trễ sẵn sàng).

Theo chuẩn giao tiếp Avalon-ST (*Intel Avalon Interface Specifications*, Mục 3.4):

- **Cấu hình `readyLatency` = 0 (Bắt tay tức thời):** Quyết định nhận mẫu nằm *ở chính chu kỳ hiện tại*. Nếu `ready` hạ xuống 0 ở chu kỳ n, mẫu đang nằm trên bus ở chu kỳ n không được nhận, và phía phát phải tiếp tục giữ mẫu đó sang chu kỳ $n+1$. Ưu điểm của cấu hình này là phản hồi tức thì, không cần bộ đệm dự phòng. Nhược điểm là tạo ra một đường trễ tổ hợp (combinational path) nối thẳng từ khối nhận ngược về khối phát.
- **Cấu hình `readyLatency` = 1 (Tín hiệu sẵn sàng có trễ / cơ chế tín chỉ):** Tín hiệu `ready` *ở* chu kỳ n là *lời hứa cho chu kỳ* $n+1$. Khi khối nhận bật `ready` = 1 *ở* chu kỳ n, nó phát đi một tín chỉ: "Ở chu kỳ $n+1$, tôi chắc chắn có chỗ để nhận một mẫu". Ngược lại, nếu khối nhận hạ `ready` = 0 *ở* chu kỳ n, điều đó có nghĩa là *ở* chu kỳ $n+1$ khối phát không được gửi, nhưng *mẫu xuất hiện ở chính chu kỳ n vẫn được tiếp nhận đầy đủ*, bởi vì lời hứa tiếp nhận cho chu kỳ n đã được khối nhận cam kết từ chu kỳ $n-1$.

Cạm bẫy nối ghép IP: Nhiều khối IP lõi xử lý DSP chuyên dụng của Intel (như bộ lọc FIR Compiler hay biến đổi Fourier nhanh FFT IP) mặc định thiết lập `readyLatency` = 1 trong cửa sổ cấu hình Parameter Editor nhằm chèn thêm một tầng thanh ghi vào đường tín hiệu `ready`, giúp tăng tần số hoạt động cực đại (F_{max}). Trong khi đó, các kỹ sư viết mã Verilog bằng tay thường mặc định suy nghĩ theo kiểu `readyLatency` = 0.

Nếu khối phát giả định `readyLatency` = 1 trong khi khối nhận lại hoạt động với `readyLatency` = 0, sự cố mất đồng bộ dữ liệu sẽ xảy ra khi đường phản áp xuất hiện: một mẫu dữ liệu sẽ bị rơi mất khỏi dòng âm thanh mà không có bất kỳ cờ cảnh báo lỗi nào được bật lên.

Ví dụ tính tay và đối soát chu kỳ: Sự cố mất mẫu 12

Để thấy rõ cơ chế phá hỏng dòng dữ liệu, hãy khảo sát một chuỗi truyền thông thực nghiệm gồm 4 mẫu âm thanh nguyên $x = [10, 11, 12, 13]$ đi qua kênh giao tiếp Avalon-ST từ chu kỳ 3 đến chu kỳ 8. Giả sử khối nhận gặp tắc nghẽn bộ đệm và bắt buộc phải hạ tín hiệu `ready` xuống 0 duy nhất tại chu kỳ 5 (`ready` lần lượt là $[1, 1, 0, 1, 1, 1]$).

Dữ liệu tính toán dưới đây được trích xuất trực tiếp từ kịch bản kiểm chứng mã nguồn `verify_v3/ch02_avalon_st_trace.py`:

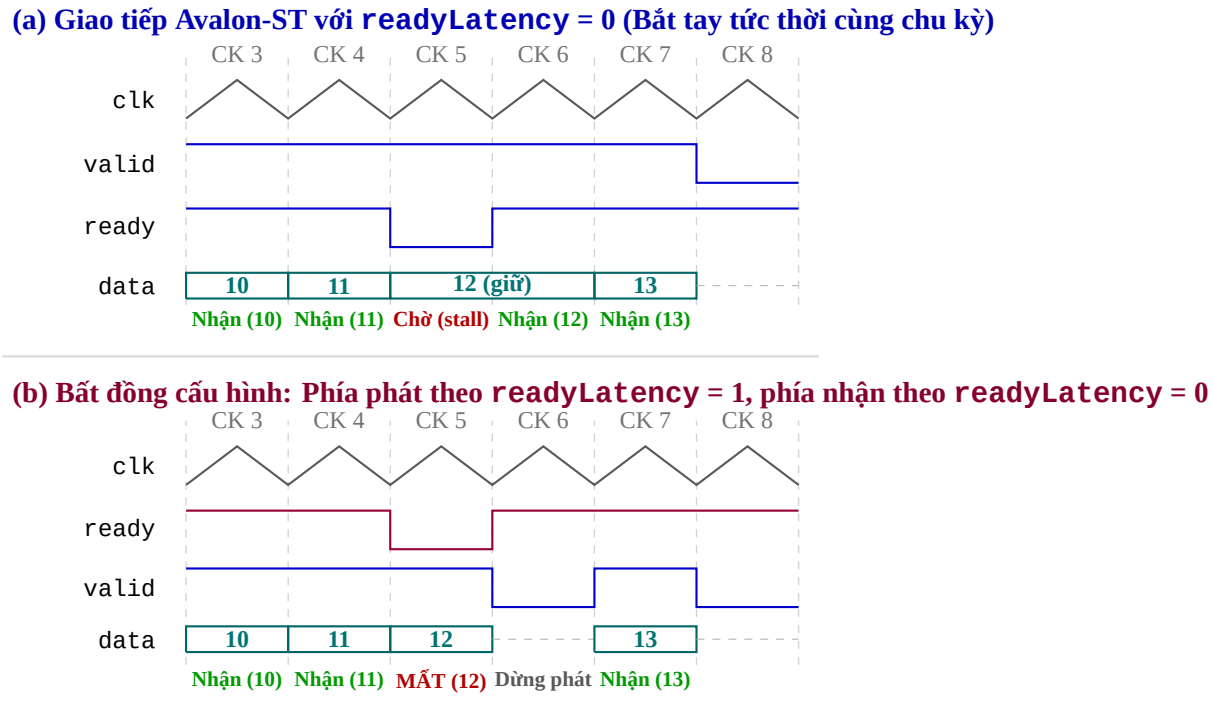
Đọc hình. Hình 3 trực quan hóa sự khác biệt bản chất giữa hai cấu hình:

- Ở nửa trên (a), với `readyLatency` = 0, tại chu kỳ 5 khi tín hiệu `ready` hạ xuống mức thấp, giao dịch dừng lại ngay lập tức. Mẫu 12 trên bus dữ liệu được giữ phẳng suốt hai chu kỳ xung nhịp (CK 5 và CK 6). Đến khi `ready` bật lên 1 *ở* CK 6, mẫu 12 mới được nhận và mẫu 13 nối tiếp *ở* CK 7. Không có mẫu nào bị mất.
- Ở nửa dưới (b), với `readyLatency` = 1, tín hiệu `ready` mang ý nghĩa tín chỉ trễ một nhịp. Vì *ở* chu kỳ 4 `ready` = 1, khối phát được quyền đẩy mẫu 12 *ở* chu kỳ 5 bất chấp việc tại chính chu kỳ 5 `ready` đang *ở* mức 0. Phía phát coi như mẫu 12 đã bàn giao thành công. Sang chu kỳ 6, khối phát mới nhận lệnh dừng do `ready` *ở* CK 5 bằng 0.

Hậu quả của sự bất đồng này là mẫu âm thanh số 12 bị loại khỏi dòng dữ liệu. Trong một khung phân tích 638 mẫu của thuật toán Praat, việc mất đúng một mẫu làm toàn bộ 637 mẫu phía sau bị dồn lên trước một vị trí

Bảng 10: Đối chiếu chu kỳ truyền nhận giữa trường hợp chuẩn và trường hợp lệch tham số `readyLatency`

Chu kỳ	`valid`	`data`	`ready`	Bắt tay (RL=0)	Phía nhận lấy?	Hành vi thực tế của khối phát
Kịch bản 1: Cả hai phía thống nhất cấu hình `readyLatency` = 0						
3	1	10	1	Có	Mẫu 10	Khối phát đưa tiếp mẫu 11 lên bus.
4	1	11	1	Có	Mẫu 11	Khối phát đưa tiếp mẫu 12 lên bus.
5	1	12	0	Không	Không	`ready`=0: Khối phát **giữ nguyên** mẫu 12.
6	1	12	1	Có	Mẫu 12	Khối phát đưa tiếp mẫu 13 lên bus.
7	1	13	1	Có	Mẫu 13	Đã gửi xong 4 mẫu, hạ `valid` về 0.
8	0	–	1	Không	Không	Bus nghỉ, nhận đủ 4 mẫu: $[10, 11, 12, 13]$.
Kịch bản 2: Bất đồng cấu hình (Phát hiểu RL=1, Nhận hiểu RL=0)						
3	1	10	1	Có	Mẫu 10	Khối phát gửi 10 do có tín chỉ từ CK 2.
4	1	11	1	Có	Mẫu 11	Khối phát gửi 11 do có tín chỉ từ CK 3.
5	1	12	0	**Lệch**	**Từ chối**	Khối phát thấy CK4 `ready`=1 nên đẩy 12. *Nhưng nhận (RL=0) từ chối! Mẫu 12 bị rơi.*
6	0	–	1	Không	Không	Khối phát thấy CK5 `ready`=0 nên dừng phát.
7	1	13	1	Có	Mẫu 13	Khối phát thấy CK6 `ready`=1 nên gửi 13.
8	0	–	1	Không	Không	Kết thúc: Khối nhận chỉ lấy $[10, 11, 13]$!



Hình 3: Dạng sóng thời gian so sánh cơ chế bắt tay Avalon-ST giữa cấu hình `readyLatency` = 0 và `readyLatency` = 1 khi xuất hiện xung phản áp ở chu kỳ 5 (tham chiếu *Intel Avalon Interface Specifications*, Mục 3.4.2).

thời gian. Các phép tính tự tương quan $R[\tau]$ sau đó vẫn thực thi bình thường theo công thức toán học, nhưng trên một cửa sổ tín hiệu bị lệch pha, dẫn đến sai lệch đỉnh parabol và xuất hiện lỗi thô (Gross Pitch Error) mà người lập trình khó tìm ra nguyên nhân nếu chỉ nhìn vào mã thuật toán.

Tóm lại: Tham số `readyLatency` *quyết định thời điểm một mẫu dữ liệu được công nhận đã chuyển giao; hiểu lệch một nhịp giữa hai khối giao tiếp sẽ làm rơi hoặc nhân đôi mẫu âm thanh một cách im lặng mà không hề báo lỗi trên đường truyền.*

2.2. Ngân sách chu kỳ khi đầu vào là tệp âm thanh và thực tế thời gian thực

Vấn đề thực tế: Nhận thức về đồng hồ lấy mẫu vật lý 16 kHz

Khi tiếp cận bài toán xử lý tiếng nói trên FPGA, trực giác ban đầu của nhiều kỹ sư là tìm kiếm một bộ tạo xung nhịp 16 kHz trong thiết kế để điều nhịp cho toàn bộ hệ thống. Trực giác đó xuất phát từ thói quen làm việc với các bộ giải mã âm thanh tương tự (như chip WM8731 trên bo mạch DE10-Standard), nơi mà bộ chuyển đổi ADC (Analog-to-Digital Converter) liên tục xuất ra từng mẫu sau mỗi khoảng thời gian đều đặn $T_s = 1/16.000$ s $= 62{,}5$ μs. Khi có một bộ ADC ngoại vi ép nhịp, nếu mạch FPGA xử lý chậm hơn $62{,}5$ μs, bộ đệm sẽ tràn và dữ liệu âm thanh bị mất.

Tuy nhiên, trong kiến trúc hệ thống của tài liệu này (đã định hình ở Chương 1), **nguồn dữ liệu đầu vào là một tệp âm thanh WAV lưu trên thẻ nhớ SD, được vi xử lý HPS đọc và đẩy qua cầu Lightweight HPS-to-FPGA Bridge vào bộ đệm On-Chip FIFO**.

Đặc điểm kỹ thuật của hệ thống là: **Không có đồng hồ 16 kHz nào ép bước trên phần cứng**.

- Vi xử lý HPS đẩy dữ liệu qua cầu giao tiếp Lightweight HPS-to-FPGA Bridge nhanh hơn rất nhiều so với tốc độ tiêu thụ 1 mẫu / 3.125 chu kỳ của chuỗi xử lý âm thanh (số chu kỳ ghi thực tế qua cầu truyền thông cần được đo đạc cụ thể trên phần cứng, như đã phân tích ở Chương 1). Nếu bộ đệm FIFO còn ô trống, HPS có thể ghi liên tục hàng nghìn mẫu vào phần cứng mà không bị nghẽn bởi tốc độ xung nhịp lấy mẫu vật lý.
- Lõi xử lý DSP trên FPGA chạy trên miền xung nhịp hệ thống $f_{\text{clk}} = 50$ MHz (chu kỳ $T_{\text{clk}} = 20$ ns).
- Tốc độ tiêu thụ dữ liệu không bị khống chế bởi tần số lấy mẫu 16 kHz của âm thanh, mà bị quyết định trực tiếp bởi **thời gian thực thi của khối tính toán tự tương quan MAC**—khâu tiêu tốn nhiều thời gian tính toán nhất của toàn bộ hệ thống.

Vậy con số 16 kHz có ý nghĩa gì? Nó là thông số toán học quy định khoảng cách thời gian giữa các mẫu để tính ra cao độ thực theo đơn vị Hertz ($F_0 = f_s/\tau_{\text{interp}}$), và là **thước đo tham chiếu thời gian thực (real-time yardstick)** để đánh giá xem phần cứng chạy nhanh hơn thế giới thực bao nhiêu lần.

Dẫn xuất ngân sách chu kỳ từ nguyên lý đầu tiên

Thiết lập ngân sách chu kỳ một cách tường minh từ các tham số chuẩn mực của phần mềm Praat 7.0.03 trên tệp âm thanh $f_s = 16.000$ Hz:

1. **Chu kỳ xung nhịp chuẩn:** Với tần số xung nhịp hệ thống $f_{\text{clk}} = 50$ MHz, mỗi chu kỳ xung nhịp kéo dài:

$$T_{\text{clk}} = \frac{1}{50.000.000 \text{ Hz}} = 20 \text{ ns} \tag{148}$$

2. **Thước đo thời gian thực cho một mẫu:** Trong thế giới vật lý, một mẫu âm thanh 16 kHz xuất hiện sau mỗi khoảng thời gian $T_s = 62{,}5$ μs. Số chu kỳ xung nhịp 50 MHz tương ứng với một khoảng thời gian lấy mẫu là:

$$C_{\text{sample}} = \frac{f_{\text{clk}}}{f_s} = \frac{50.000.000}{16.000} = 3.125 \text{ chu kỳ/mẫu} \tag{149}$$

3. **Bước nhảy khung phân tích (Frame Hop):** Thuật toán Praat với cấu hình mặc định (sàn cao độ pitchFloor $=$ 75 Hz, bước thời gian tự động timeStep $= 0{,}75/75 = 10$ ms, tham chiếu mã nguồn Praat 7.0.03 `fon/Sound_to_Pitch.cpp`, dòng 296 (`dt` = periodsPerWindow / pitchFloor / 4, periodsPerWindow = 3)) phân tích các khung cách nhau đúng:

$$H = 10 \text{ ms} \times 16.000 \text{ mẫu/s} = 160 \text{ mẫu} \tag{150}$$

4. **Ngân sách chu kỳ thời gian thực cho một bước nhảy (C_{hop}):** Để đạt ngưỡng thời gian thực (xử lý kịp tốc độ nói của con người), toàn bộ chuỗi tính toán cho 160 mẫu mới phải hoàn thành trong khoảng thời gian không vượt quá 10 ms. Tại xung nhịp 50 MHz, ngân sách chu kỳ tối đa cho phép là:

$$C_{\text{hop}} = H \times C_{\text{sample}} = 160 \times 3.125 = 500.000 \text{ chu kỳ/khung} \tag{151}$$

Số phép tính MAC và chu kỳ thực thi của khối tự tương quan

Mỗi khung âm thanh có độ dài cửa sổ $N = 638$ mẫu (đã được làm tròn từ $3 \times 16.000/75 = 640$ mẫu theo quy tắc đối xứng tâm của Boersma). Khối nhân tích lũy MAC cần tính toán:

- Năng lượng chuẩn $R[0]$: Cần tích lũy 638 tích số $x_w[n] \times x_w[n]$ ($n = 0..637$). Số phép tính MAC là:

$$M_{R0} = 638 \text{ phép tính} \tag{152}$$

- Dải quét độ trễ $\tau \in [26, 214]$ gồm 189 giá trị trễ liên tiếp: Với mỗi độ trễ τ, số lượng tích số cần tính là $638 - \tau$. Khi $\tau = 26$, số phép nhân là $638 - 26 = 612$. Khi $\tau = 214$, số phép nhân là $638 - 214 = 424$. Tổng số phép tính MAC cho toàn bộ 189 độ trễ là tổng của một cấp số cộng:

$$M_{\text{lags}} = \sum_{\tau=26}^{214} (638 - \tau) = \frac{(612 + 424) \times 189}{2} = \frac{1.036 \times 189}{2} = 518 \times 189 = 97.902 \text{ phép tính} \tag{153}$$

Tổng số phép tính nhân tích lũy trên một khung phân tích là:

$$M_{\text{total}} = M_{R0} + M_{\text{lags}} = 638 + 97.902 = 98.540 \text{ phép tính} \tag{154}$$

Do khối MAC được thiết kế dưới dạng đường ống dòng chảy liên tục (streaming pipeline, phát địa chỉ liền mạch không nhịp nghỉ như đã phân tích ở Chương 0), mỗi nhịp xung nhịp đưa vào một cặp toán hạng. Sau khi cặp toán hạng cuối cùng của $\tau = 214$ đi vào, bộ tích lũy trong khối DSP cần thêm đúng 3 chu kỳ trễ đường ống (pipeline latency) để đẩy kết quả cuối cùng ra thanh ghi xuất. Do đó, tổng số chu kỳ xung nhịp thực tế để tính xong toàn bộ ma trận tương quan của một khung là:

$$C_{\text{calc}} = M_{\text{total}} + 3 = 98.540 + 3 = 98.543 \text{ chu kỳ/khung} \tag{155}$$

Thời gian thực tế để FPGA xử lý xong một khung là:

$$T_{\text{calc}} = 98.543 \times 20 \text{ ns} = 1.970.860 \text{ ns} \approx 1{,}971 \text{ ms} \tag{156}$$

Chia số chu kỳ này cho 160 mẫu âm thanh mới nạp vào mỗi khung, ta có số chu kỳ tính toán trung bình trên một mẫu âm thanh:

$$C_{\text{per_sample}} = \frac{98.543}{160} \approx 615{,}89 \text{ chu kỳ/mẫu} \quad (\approx 616 \text{ chu kỳ/mẫu}) \tag{157}$$

Định nghĩa và dẫn xuất Hệ số Thời gian thực (Real-Time Factor – RTF)

Hệ số thời gian thực RTF là tỷ số không thứ nguyên giữa thời gian phần cứng thực thi phép tính (T_{calc}) và thời lượng âm thanh thực tế được xử lý (T_{audio}):

$$\text{RTF} = \frac{T_{\text{calc}}}{T_{\text{audio}}} = \frac{C_{\text{calc}} \times T_{\text{clk}}}{H \times T_s} = \frac{C_{\text{calc}}}{C_{\text{hop}}} \tag{158}$$

Thay các con số đã dẫn xuất vào công thức:

$$\text{RTF}_{\text{MAC}} = \frac{98.543}{500.000} \approx 0{,}197086 \quad (\approx 0{,}197) \tag{159}$$

Nếu tính thêm các khâu phụ trợ như trừ giá trị trung bình cục bộ, tra bảng Hanning, tìm đỉnh cực đại parabol và lọc trung vị (ước tính chiếm khoảng 1.157 chu kỳ bổ sung, đưa tổng chu kỳ lõi lên mức ≈ 99.700 chu kỳ, sẽ được chi tiết hóa ở Chương 12), hệ số thời gian thực toàn lõi là:

$$\text{RTF}_{\text{total}} = \frac{99.700}{500.000} \approx 0{,}1994 \quad (\approx 0{,}199) \tag{160}$$

Con số RTF $\approx 0{,}197$ mang ý nghĩa vật lý rõ ràng: mạch FPGA xử lý tín hiệu nhanh hơn 5 lần so với thời gian thực ($1/0{,}197 \approx 5{,}07$). Một đoạn âm thanh đàm thoại dài 10 giây chỉ cần chưa đầy 2 giây để trích xuất toàn bộ đường nét cao độ F_0.

Cạm bẫy thường gặp: Nhầm lẫn giữa độ dài khung và bước nhảy thời gian

Cạm bẫy thường gặp: Nhầm lẫn bước nhảy khung 10 ms và sự triệt tiêu sai số ngẫu nhiên

Khi thiết lập ngân sách chu kỳ cho hệ thống xử lý tiếng nói, người thiết kế rất dễ mắc phải hai cạm bẫy tính toán sau:

- **Cạm bẫy 1 (Nhầm lẫn bước nhảy khung H):** Nhầm lẫn giữa bước nhảy khung phân tích H của thuật toán cao độ (10 ms, tương ứng 160 mẫu ở 16 kHz) với bước nhảy 25 ms (400 mẫu) thường dùng trong các bài toán trích xuất đặc trưng MFCC. Tại xung nhịp 50 MHz, bước nhảy 25 ms cho ngân sách $1.250.000$ chu kỳ, lớn gấp 2,5 lần ngân sách chuẩn 500.000 chu kỳ của Praat.
- **Cạm bẫy 2 (Sự triệt tiêu sai số ngẫu nhiên):** Nếu ai đó ước tính chu kỳ tính toán của một khung phân tích bằng một vòng lặp chưa tối ưu hóa với ≈ 246.000 chu kỳ và chia cho ngân sách nhầm lẫn $1.250.000$ chu kỳ, tỷ số thu được là $246.000/1.250.000 \approx 0{,}197$. Con số này trùng hợp ngẫu nhiên với hệ số RTF $= 98.543/500.000 \approx 0{,}197$ chuẩn xác của chúng ta. Sự trùng hợp đó có thể làm người thiết kế ngộ nhận thiết kế đã đạt chuẩn, trong khi thực tế độ trễ phản hồi của hệ thống bị chậm đi 2,5 lần.

Dưới đây là bảng đối chiếu tổng hợp được trích xuất trực tiếp từ kịch bản kiểm chứng số học `verify_v3/ch02_cycle_budget.py`:

Bảng 11: Bảng đối chiếu thông số ngân sách chu kỳ và phân tích sai lệch kỹ thuật

Thông số hệ thống	Chuẩn Praat (Hop 10 ms)	Nhầm lẫn (Hop 25 ms)	Bản chất kỹ thuật
Tần số lấy mẫu âm thanh (f_s)	16.000 Hz	16.000 Hz	Tiêu chuẩn âm thanh tiếng nói
Tần số xung nhịp FPGA (f_{clk})	50.000.000 Hz	50.000.000 Hz	Xung nhịp đồng bộ hệ thống
Thước đo chu kỳ trên mẫu	3.125 chu kỳ/mẫu	3.125 chu kỳ/mẫu	$f_{\text{clk}}/f_s = 50$ MHz$/16$ kHz
Bước nhảy khung (H)	**160 mẫu (10 ms)**	**400 mẫu (25 ms)**	Praat dùng bước nhảy 10 ms; nhầm sang 25 ms
Ngân sách thời gian thực (C_{hop})	**500.000 chu kỳ**	**1.250.000 chu kỳ**	$H \times 3.125$ chu kỳ
Độ dài cửa sổ phân tích (N)	638 mẫu	640 mẫu	Praat làm tròn tâm đối xứng
Số phép tính MAC (M_{total})	98.540	Chưa tối ưu	$638 + \sum_{26}^{214}(638 - \tau)$
Chu kỳ thực tế của lõi (C_{calc})	**98.543 chu kỳ**	**246.000 chu kỳ**	98.540 nhịp phát + 3 nhịp trễ
Chu kỳ tính toán trên 1 mẫu mới	**615,89 chu kỳ/mẫu**	615,00 chu kỳ/mẫu	C_{calc}/H
Thời gian tính toán một khung	**1,971 ms**	4,920 ms	$C_{\text{calc}} \times 20$ ns
Hệ số thời gian thực (RTF_{MAC})	**0,197**	**0,197**	Trùng hợp ngẫu nhiên do sai cả tử và mẫu

Đọc bảng. Bảng 11 khẳng định nguyên tắc thiết kế quan trọng: thiết kế chuẩn xác hoàn tất một khung phân tích chỉ trong 1,97 ms trên bước nhảy 10 ms (đúng chuẩn tương quan của Boersma), giúp hệ thống có độ trễ phản hồi thấp và giải phóng hơn 80% băng thông xử lý của FPGA cho các tác vụ tiếp theo.

Tóm lại: Khi xử lý tệp âm thanh, không có đồng hồ 16 kHz nào chi phối bước chạy; lõi FPGA tính toán ở tần số xung nhịp 50 MHz với 98.543 chu kỳ cho mỗi bước nhảy 160 mẫu (616 chu kỳ/mẫu), hoàn tất một khung 10 ms chỉ trong 1,97 ms, đạt hệ số thời gian thực RTF = 0,197 so với ngân sách 500.000 chu kỳ.

2.3. Kỹ thuật mã hóa FSM: Đánh đổi tài nguyên và độ trễ trên kiến trúc Cyclone V

Vấn đề thực tế: Từ câu lệnh switch-case trừu tượng đến cổng logic vật lý

Trong ngôn ngữ mô tả phần cứng Verilog, máy trạng thái hữu hạn FSM (Finite State Machine) thường được lập trình viên biểu diễn dưới dạng câu lệnh rẽ nhánh trừu tượng: `case (state_reg) ... endcase`. Nhìn trên màn hình soạn thảo mã nguồn, việc chuyển trạng thái trông giống như một phép gán biến tức thời trong ngôn ngữ C.

Nhưng lớp silicon bên dưới không có khái niệm "trạng thái trừu tượng". Mọi trạng thái bắt buộc phải được số hóa thành một chuỗi các bit nhị phân lưu trữ trong các phần tử nhớ tuần tự—các thanh ghi flip-flop (FF). Phần mềm tổng hợp logic Quartus Prime khi phân tích mã nguồn sẽ phải đối mặt với câu hỏi cốt lõi: **Mỗi trạng thái nên được gán bằng tổ hợp bit nào để mạch chạy nhanh nhất (F_{max} cao nhất) mà tốn ít diện tích nhất?**

Cách thức gán bit cho trạng thái (State Encoding) chi phối trực tiếp hai yếu tố của mạch tích hợp:

1. **Số lượng thanh ghi flip-flop tiêu tốn:** Ảnh hưởng tới dung lượng tài nguyên tuần tự của chip.

2. **Độ sâu và độ phức tạp của mạng logic tổ hợp (Combinational Logic Depth):** Logic chuyển trạng thái tiếp theo (`next_state`) và logic sinh tín hiệu ngõ ra (`outputs`) đòi hỏi mạng cổng logic LUT (Look-Up Table) có số đầu vào (fan-in) rộng hay hẹp. Mạng logic càng sâu, thời gian truyền trễ (propagation delay) càng lớn, kéo tụt tần số hoạt động tối đa F_{max} của hệ thống.

Khảo sát 7 phương án mã hóa trong thiết lập State Machine Processing của Quartus

Trong cửa sổ cấu hình dự án của Quartus Prime (*Assignments* → *Settings* → *Compiler Settings* → *Advanced Settings (Synthesis)* → *State Machine Processing*, tham chiếu tài liệu *Intel Quartus Prime Standard Edition User Guide: Design Compilation*), công cụ cung cấp đúng 7 phương án xử lý máy trạng thái:

1. **Auto (Tự động - Mặc định):** Quartus tự động phân tích cấu trúc FSM dựa trên thuật toán tối ưu hóa nội bộ. Với hầu hết các máy trạng thái trong FPGA có số trạng thái nhỏ hơn hoặc bằng 32, Quartus sẽ tự động chuyển đổi sang mã hóa One-Hot nhằm tối ưu hóa tần số F_{max}.
2. **One-Hot (Một bit tích cực):** Nếu FSM có S trạng thái, bộ mã hóa sử dụng đúng S thanh ghi flip-flop. Tại một thời điểm bất kỳ, chỉ có duy nhất 1 thanh ghi mang giá trị 1, tất cả các thanh ghi còn lại mang giá trị 0.
3. **Minimal Bits (Nhị phân tối thiểu):** Số thanh ghi flip-flop sử dụng là giá trị nhỏ nhất đủ để biểu diễn S trạng thái: $K = \lceil \log_2 S \rceil$. Các trạng thái được gán mã nhị phân nhằm tối thiểu hóa số bit và tối ưu hóa diện tích logic.
4. **Sequential (Nhị phân tuần tự theo thứ tự khai báo):** Cũng sử dụng $K = \lceil \log_2 S \rceil$ thanh ghi flip-flop, nhưng Quartus gán các giá trị nhị phân tuần tự tăng dần $(0, 1, 2, \dots)$ đúng theo thứ tự khai báo của các trạng thái trong mã nguồn HDL, không sắp xếp lại theo đồ thị chuyển trạng thái.
5. **Gray (Mã Gray chuyển mạch đơn bit):** Sử dụng $K = \lceil \log_2 S \rceil$ thanh ghi, trong đó các trạng thái liền kề nhau trong chu trình hoạt động chỉ khác nhau đúng 1 bit duy nhất.
6. **Johnson (Mã vòng lặp đảo Moebius):** Sử dụng $K = \lceil S/2 \rceil$ thanh ghi dưới dạng một thanh ghi dịch vòng tròn với đầu ra đảo quay về đầu vào. Hai trạng thái liên tiếp cũng chỉ lệch nhau đúng 1 bit.
7. **User-Encoded (Mã hóa do người dùng định nghĩa):** Quartus giữ nguyên các giá trị gán bit mà người thiết kế đã khai báo trong mã nguồn Verilog (`parameter` hoặc `localparam`), không tự ý trích xuất hoặc thay đổi.

Phân tích so sánh trên máy trạng thái 5 trạng thái điển hình

Để làm sáng tỏ bản chất của từng phương án, hãy xét một máy trạng thái điều khiển khung điển hình gồm $S = 5$ trạng thái xuất hiện trong bộ xử lý âm thanh:

- `S_IDLE` (Trạng thái nghỉ, chờ tín hiệu `start`)
- `S_LOAD` (Nạp mẫu âm thanh từ FIFO vào RAM khung)
- `S_CALC` (Kích hoạt lõi tính toán tự tương quan MAC)
- `S_DRAIN` (Đẩy kết quả cao độ ra bộ đệm xuất)
- `S_DONE` (Phát xung báo hoàn tất và quay về `S_IDLE`)

Dưới đây là bảng phân bổ tổ hợp bit và đánh giá chi phí phần cứng của 4 phương án mã hóa tiêu biểu:
Bản chất đánh đổi:

- **Minimal Bits (Binary):** Tiết kiệm thanh ghi nhất (chỉ tốn 3 FF cho 5 trạng thái). Nhưng để biết máy trạng thái có đang ở `S_CALC` hay không, mạch phải giải mã biểu thức logic tổ hợp 3 biến: $\texttt{calc_en} = \overline{s_2} \cdot s_1 \cdot \overline{s_0}$. Tín hiệu này buộc phải đi qua một bảng tra LUT 3 đầu vào, kỳ vọng sẽ tốn thêm logic ALM và có thể thêm độ trễ vào đường tới hạn, cần được đo đạc qua báo cáo định thời của Quartus Fitter.
- **One-Hot:** Tốn nhiều thanh ghi nhất (5 FF cho 5 trạng thái). Nhưng đổi lại, để kích hoạt tín hiệu tính toán, ta chỉ cần nối thẳng đường dây từ flip-flop thứ 2: $\texttt{calc_en} = s[2]$. Cổng giải mã ngõ ra không cần bảng tra LUT. Đường chuyển trạng thái kế tiếp cũng chỉ là các cổng AND/OR 2 đầu vào đơn giản ($s_{\text{next}}[2] = s[1] \cdot \texttt{load_done} + s[2] \cdot \overline{\texttt{calc_done}}$). Do đó, One-Hot giảm đáng kể độ trễ tổ hợp và cho tần số F_{max} thuận lợi.

Bảng 12: So sánh các phương án mã hóa trạng thái cho FSM 5 trạng thái

Trạng thái FSM	Minimal Bits (Binary)	One-Hot	Mã Gray	Mã Johnson
S_IDLE	000	00001	000	000
S_LOAD	001	00010	001	001
S_CALC	010	00100	011	011
S_DRAIN	011	01000	010	111
S_DONE	100	10000	110	110
Số flip-flop (K)	**3 FF**	**5 FF**	**3 FF**	**3 FF**
Số trạng thái thừa	$2^3 - 5 = 3$	$2^5 - 5 = 27$	$2^3 - 5 = 3$	$2^3 - 5 = 3$
Fan-in giải mã ngõ ra	3 biến (cần LUT3)	**1 biến (Dây nối)**	3 biến (cần LUT3)	2 biến (cần LUT2)
Độ trễ giải mã ngõ ra	Trễ qua cổng LUT	**0 ns (Không tốn cổng)**	Trễ qua cổng LUT	Trễ qua cổng LUT
Chuyển mạch đơn bit?	Không (001 → 010: 2 bit)	Không (2 bit đổi)	**Có (luôn 1 bit)**	**Có (luôn 1 bit)**

Mã nguồn HDL: Hiện thực hóa FSM nhị phân và One-Hot

Để phục vụ việc kiểm chứng và so sánh, hai module FSM tương đương chức năng được viết bằng Verilog-2001 chuẩn:

Listing 7: Mã nguồn FSM 5 trạng thái mã hóa nhị phân Minimal Bits (rtl/v3/ch02_fsm_binary.v)

```
    localparam [2:0] S_IDLE  = 3'b000;
    localparam [2:0] S_LOAD  = 3'b001;
    localparam [2:0] S_CALC  = 3'b010;
    localparam [2:0] S_DRAIN = 3'b011;
    localparam [2:0] S_DONE  = 3'b100;

    reg [2:0] state_reg, state_next;

    // 1. State register
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state_reg <= S_IDLE;
        end else begin
            state_reg <= state_next;
        end
    end

    // 2. Next-state combinational logic
    always @(*) begin
        state_next = state_reg;
        case (state_reg)
            S_IDLE: begin
                if (start) state_next = S_LOAD;
            end
            S_LOAD: begin
                if (load_done) state_next = S_CALC;
            end
            S_CALC: begin
                if (calc_done) state_next = S_DRAIN;
            end
            S_DRAIN: begin
                if (drain_done) state_next = S_DONE;
            end
            S_DONE: begin
                state_next = S_IDLE;
            end
            default: begin
                state_next = S_IDLE;
            end
        endcase
    end

    // 3. Moore outputs decoded directly from state_reg
    assign busy       = (state_reg != S_IDLE);
    assign load_en    = (state_reg == S_LOAD);
```

```
    assign calc_en    = (state_reg == S_CALC);
    assign drain_en   = (state_reg == S_DRAIN);
    assign done_pulse = (state_reg == S_DONE);
    assign state_num  = state_reg;
```

Giải thích mã nguồn `ch02_fsm_binary.v` dòng-từng-dòng:

- Dòng 40–44: Khai báo các hằng số trạng thái `localparam [2:0]` theo mã nhị phân từ `S_IDLE` (0) đến `S_DONE` (4).
- Dòng 46: Khai báo hai thanh ghi trạng thái 3-bit `state_reg` (lưu trữ) và `state_next` (dây tổ hợp).
- Dòng 49–55: Khối tuần tự `always @(posedge clk or negedge rst_n)` cập nhật trạng thái tại cạnh lên xung nhịp, reset bất đồng bộ tích cực mức thấp về `S_IDLE`.
- Dòng 58–80: Khối tổ hợp giải mã trạng thái tiếp theo qua câu lệnh `case (state_reg)` từ các tín hiệu `start`, `load_done`, `calc_done`, `drain_done`.
- Dòng 83–88: Các phép gán Moore: cờ `busy` tích cực khi khác `S_IDLE`; các cờ điều khiển `load_en`, `calc_en`, `drain_en`, `done_pulse` giải mã qua phép so sánh bằng; và xuất chỉ số `state_num = state_reg`.

Tiếp theo là mã nguồn hiện thực hóa theo phương án One-Hot tường minh:

Listing 8: Mã nguồn FSM 5 trạng thái mã hóa One-Hot tường minh (rtl/v3/ch02_fsm_onehot.v)

```
    localparam [4:0] S_IDLE  = 5'b00001;
    localparam [4:0] S_LOAD  = 5'b00010;
    localparam [4:0] S_CALC  = 5'b00100;
    localparam [4:0] S_DRAIN = 5'b01000;
    localparam [4:0] S_DONE  = 5'b10000;

    reg [4:0] state_reg, state_next;

    // 1. State register (5 flip-flops)
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state_reg <= S_IDLE;
        end else begin
            state_reg <= state_next;
        end
    end

    // 2. One-hot next-state logic
    always @(*) begin
        state_next = 5'b00000;
        state_next[0] = (state_reg[0] & ~start) | state_reg[4];
        state_next[1] = (state_reg[0] & start)  | (state_reg[1] & ~load_done);
        state_next[2] = (state_reg[1] & load_done) | (state_reg[2] & ~calc_done);
        state_next[3] = (state_reg[2] & calc_done) | (state_reg[3] & ~drain_done);
        state_next[4] = (state_reg[3] & drain_done);
    end

    // 3. Moore outputs: single-bit wires directly from state flip-flops
    assign busy       = ~state_reg[0];
    assign load_en    = state_reg[1];
    assign calc_en    = state_reg[2];
    assign drain_en   = state_reg[3];
    assign done_pulse = state_reg[4];

    // Priority decode to numeric state number (0..4) for comparison
    always @(*) begin
        case (1'b1)
            state_reg[0]: state_num = 3'd0;
            state_reg[1]: state_num = 3'd1;
            state_reg[2]: state_num = 3'd2;
            state_reg[3]: state_num = 3'd3;
            state_reg[4]: state_num = 3'd4;
```

```
83              default:      state_num = 3'd0;
84          endcase
85      end
```

Giải thích mã nguồn `ch02_fsm_onehot.v` dòng-từng-dòng:

- Dòng 41–45: Khai báo 5 hằng số trạng thái 5-bit, mỗi trạng thái có duy nhất 1 bit bật lên 1 (bit 0 cho `S_IDLE`, bit 4 cho `S_DONE`).
- Dòng 47: Khai báo thanh ghi trạng thái 5-bit `state_reg` gồm 5 flip-flop vật lý.
- Dòng 50–56: Khối tuần tự chốt 5 flip-flop tại cạnh lên xung nhịp, reset về `S_IDLE` (mã `5'b00001`).
- Dòng 59–66: Khối tổ hợp xác định từng bit của `state_next` thông qua các biểu thức AND/OR 2 đầu vào, ví dụ: bit 2 bật lên khi vừa xong nạp mẫu (`state_reg[1] & load_done`) hoặc đang duy trì tính toán (`state_reg[2] & ~calc_done`).
- Dòng 69–73: Các tín hiệu ngõ ra Moore là các đường dây nối trực tiếp từ flip-flop của `state_reg` (`load_en` nối từ `state_reg[1]`, `calc_en` từ `state_reg[2]`), không cần cổng logic giải mã.
- Dòng 76–85: Khối giải mã ưu tiên ánh xạ từ mã One-Hot sang chỉ số nguyên 3-bit `state_num` (0 đến 4) để phục vụ kiểm chứng đối đầu với module nhị phân.

Tương tác giữa trình tổng hợp Quartus và mã hóa One-Hot viết tay: Mặc định trong Quartus Prime, thiết lập *State Machine Processing* được đặt là **Auto**. Bộ trích xuất máy trạng thái (State Machine Extractor) sẽ nhận diện khối tuần tự FSM và tự động mã hóa lại (re-encode) đồ thị trạng thái theo thuật toán tối ưu của công cụ. Khi người thiết kế đã chủ động mã hóa One-Hot bằng tay trong mã nguồn Verilog:

- Nếu để **Auto**, Quartus vẫn có thể nhận diện FSM và tự động gán lại mã hóa theo thuật toán của mình.
- Để buộc Quartus giữ nguyên chính xác tổ hợp bit và cấu trúc phần cứng mà người thiết kế đã định nghĩa, cần thiết lập *State Machine Processing* thành **User-Encoded**, hoặc thêm thuộc tính tổng hợp trực tiếp vào khai báo thanh ghi trong Verilog:

```
(* syn_encoding = "user" *) reg [4:0] state_reg;          (161)
```

  Thuộc tính `syn_encoding = "user"` chỉ thị rõ ràng cho trình tổng hợp Quartus không được tự ý tối ưu hóa lại bảng mã hóa, giúp bảo toàn toàn bộ cấu trúc dây nối trực tiếp của thiết kế One-Hot.

Bối cảnh vi kiến trúc Cyclone V: Hiệu quả tài nguyên của mã hóa One-Hot

Một quan niệm thường gặp khi học thiết kế số là hạn chế mã hóa One-Hot vì sợ tốn thanh ghi flip-flop, ưu tiên Minimal Bits. Tuy nhiên, quan niệm này không phản ánh đúng cấu trúc phần cứng hiện đại trên FPGA Intel Cyclone V SoC.

Trên bo mạch DE10-Standard, vi mạch FPGA được trang bị là Cyclone V SoC 5CSXFC6D6F31C6 (tham chiếu tài liệu chính thức *Intel Cyclone V Device Overview*, mã tài liệu CV-51001, Bảng 1 "Cyclone V SoC Family Plan"). Theo tài liệu kỹ thuật này:

- Vi mạch sở hữu đúng **41.910 khối logic thích nghi ALM (Adaptive Logic Module)**.
- Mỗi khối ALM của Cyclone V chứa các bảng tra LUT phân chia được (fracturable 8-input LUT) và tích hợp sẵn **bốn thanh ghi flip-flop chuyên dụng** (4 dedicated registers).
- Tổng số lượng thanh ghi flip-flop khả dụng trên toàn chip là:

$$N_{\text{registers}} = 41.910 \times 4 = \textbf{166.036 thanh ghi flip-flop} \tag{162}$$

Toàn bộ hệ thống ước lượng cao độ F_0 từ đầu đến cuối dự kiến sử dụng chưa tới 2.500 thanh ghi (chiếm khoảng $1{,}5\%$ tài nguyên thanh ghi của chip).

Trong một con chip có sẵn 166.036 thanh ghi, việc sử dụng thêm 2 thanh ghi FF để chuyển FSM từ 3 bit (Binary) sang 5 bit (One-Hot) không gây áp lực về dung lượng tài nguyên tuần tự. Ngược lại, việc giảm được số tầng logic LUT tổ hợp sẽ giúp thiết kế dễ dàng đạt thời gian định thời (timing closure) ở 50 MHz mà không gây áp lực lên khâu định tuyến (routing) của phần mềm Quartus Fitter.

Chứng minh toán học: Bản chất chuyển mạch đơn bit của mã Gray

Mã Gray (Gray Code) do Frank Gray công bố năm 1953, giữ vai trò quan trọng trong các bộ đếm con trỏ FIFO bất đồng bộ giữa hai miền xung nhịp khác nhau. Thuật toán chuyển đổi từ số nhị phân n sang mã Gray $G(n)$ được hiện thực hóa bằng biểu thức:

$$G(n) = n \ \oplus \ (n \gg 1) \tag{163}$$

Câu hỏi toán học mang tính nguyên lý đầu tiên là: **Vì sao công thức $n \oplus (n \gg 1)$ luôn đảm bảo rằng khi n tăng lên $n+1$, giá trị mã Gray $G(n)$ và $G(n+1)$ chỉ thay đổi đúng duy nhất 1 bit?**

Dưới đây là chứng minh toán học thông qua **Lập luận chuỗi bit 1 tận cùng (Trailing-Ones Argument)**:

Bước 1: Phân tích bản chất phép cộng 1 trong hệ nhị phân. Mọi số nguyên không âm n khi biểu diễn dưới dạng nhị phân đều có thể viết dưới dạng tổng quát với k bit 1 liên tiếp ở phần tận cùng bên phải ($k \geq 0$):

$$n = (\ldots b_{k+1}\, 0 \underbrace{1\, 1 \ldots 1}_{k \text{ bit } 1})_2 \tag{164}$$

Khi ta cộng thêm 1 vào số n, quy tắc nhớ (carry propagation) của phép cộng nhị phân sẽ lật toàn bộ k bit 1 này thành số 0, và phát sinh một bit nhớ làm lật bit thứ k (vốn đang là 0) thành 1. Toàn bộ các bit từ vị trí $k+1$ trở về trước hoàn toàn không bị ảnh hưởng:

$$n + 1 = (\ldots b_{k+1}\, 1 \underbrace{0\, 0 \ldots 0}_{k \text{ bit } 0})_2 \tag{165}$$

Gọi $\Delta_j = b_j(n+1) \oplus b_j(n)$ là chỉ số báo hiệu bit thứ j của số nhị phân có bị đổi trạng thái hay không:

- Với các vị trí $0 \leq j < k$: Bit cũ là 1, bit mới là 0 $\implies \Delta_j = 1 \oplus 0 = 1$.
- Với vị trí $j = k$: Bit cũ là 0, bit mới là 1 $\implies \Delta_k = 0 \oplus 1 = 1$.
- Với các vị trí $j > k$: Bit không đổi $\implies \Delta_j = 0$.

Bước 2: Biểu diễn bit của mã Gray. Theo định nghĩa $G(n) = n \oplus (n \gg 1)$, bit thứ j của mã Gray $g_j(n)$ được tính bằng phép XOR giữa hai bit liền kề của số nhị phân:

$$g_j(n) = b_j(n) \ \oplus \ b_{j+1}(n) \tag{166}$$

Xét sự thay đổi của bit thứ j giữa $G(n+1)$ và $G(n)$, ký hiệu là $\delta_j = g_j(n+1) \oplus g_j(n)$:

$$\begin{aligned} \delta_j &= \big[b_j(n+1) \oplus b_{j+1}(n+1)\big] \ \oplus \ \big[b_j(n) \oplus b_{j+1}(n)\big] \\ &= \big[b_j(n+1) \oplus b_j(n)\big] \ \oplus \ \big[b_{j+1}(n+1) \oplus b_{j+1}(n)\big] \\ &= \Delta_j \ \oplus \ \Delta_{j+1} \end{aligned} \tag{167}$$

Bước 3: Đánh giá giá trị δ_j theo ba khoảng chỉ số j.

1. **Khoảng $0 \leq j < k$:** Tại khoảng này, cả Δ_j và Δ_{j+1} đều bằng 1 (vì $j+1 \leq k$). Do đó:

$$\delta_j = 1 \ \oplus \ 1 = 0 \quad (\text{Bit thứ } j \text{ của mã Gray } \textbf{không đổi}!) \tag{168}$$

2. **Tại chính vị trí $j = k$:** Ta có $\Delta_k = 1$ và $\Delta_{k+1} = 0$. Do đó:

$$\delta_k = 1 \ \oplus \ 0 = 1 \quad (\text{Bit thứ } k \text{ của mã Gray } \textbf{thay đổi}!) \tag{169}$$

3. **Khoảng $j > k$:** Cả Δ_j và Δ_{j+1} đều bằng 0. Do đó:

$$\delta_j = 0 \ \oplus \ 0 = 0 \quad (\text{Bit thứ } j \text{ của mã Gray } \textbf{không đổi}!) \tag{170}$$

Kết luận toán học: Khi chuyển từ n sang $n+1$, trong toàn bộ chuỗi bit của mã Gray, **chỉ có duy nhất một bit tại vị trí k (chính là số lượng bit 1 tận cùng của n) bị lật đảo**, khoảng cách Hamming giữa $G(n)$ và $G(n+1)$ luôn luôn bằng 1:

$$\text{HammingDistance}\big(G(n), G(n+1)\big) \equiv 1 \quad \forall n \geq 0 \tag{171}$$

Tính chất này đã được kịch bản `verify_v3/ch02_gray_code.py` kiểm chứng tự động cho toàn bộ 1.024 bước nhảy liên tiếp, xác nhận không có bất kỳ ngoại lệ nào.

Lựa chọn kiến trúc và khuyến nghị thực hành trên Quartus

Từ các phân tích định lượng trên, quy chuẩn thực hành thiết kế FSM cho hệ thống được xác lập như sau:

1. **Mẫu viết mã (Coding Template):** Áp dụng mẫu chuẩn 2 khối `always` (hoặc 3 khối `always`) rõ ràng: một khối tuần tự chốt thanh ghi trạng thái `state_reg <= state_next`, một khối tổ hợp thuần túy giải mã trạng thái tiếp theo `state_next`, và các biểu thức gán liên tục gán ngõ ra. Đây là mẫu hình chuẩn mà bộ trích xuất FSM của Quartus (State Machine Extractor) dễ dàng nhận diện và phân tích chính xác.
2. **Thiết lập tổng hợp:** Để tùy chọn `State Machine Processing` ở chế độ mặc định **Auto**. Trình biên dịch Quartus sẽ tự động ánh xạ FSM sang cấu hình One-Hot nhằm tối ưu hóa $F_{\max}$.
3. **Kiểm tra báo cáo FSM:** Sau khi biên dịch, người thiết kế mở báo cáo *Analysis & Synthesis* $\rightarrow$ *State Machines* trong Quartus để đọc bảng mã hóa thực tế mà công cụ đã áp dụng **(phải đo)**.
4. **Tần số cực đại $F_{\max}$ thực tế:** Các giá trị $F_{\max}$ của FSM mã hóa Binary so với One-Hot là thông số **(phải đo)** thông qua công cụ Timing Analyzer (Slow 1100mV 85C Model).

Tóm lại: Trên kiến trúc Cyclone V với 166.036 thanh ghi dồi dào, mã hóa One-Hot giải phóng độ trễ giải mã tổ hợp; việc hiểu rõ bản chất toán học từ lập luận chuỗi bit 1 của mã Gray đến cấu trúc logic của One-Hot giúp ta tự tin để Quartus ở chế độ Auto và kiểm tra xác nhận qua báo cáo State Machine.

2.4. Khối đệm trượt (Skid Buffer): Cắt đứt đường phản áp tới hạn

Vấn đề thực tế: Vòng lặp phản áp tổ hợp làm suy giảm Fmax

Khi thiết kế các chuỗi đường ống (pipeline) nhiều tầng theo chuẩn Avalon-ST, các khối chức năng được mắc nối tiếp nhau: đầu ra của khối này (source) nối vào đầu vào của khối tiếp theo (sink).

Để dòng dữ liệu chuyển động về phía trước (forward path), người thiết kế dễ dàng chèn các tầng thanh ghi flip-flop trên đường `data` và `valid`. Tuy nhiên, thách thức kỹ thuật nan giải lại nằm ở **đường tín hiệu phản áp đi ngược chiều (backward path)**: tín hiệu `ready`.

Hãy xét một tầng đệm đường ống đơn giản thông thường:

Listing 9: Tầng chốt đường ống đơn giản với đường ready tổ hợp thuần

```
always @(posedge clk) begin
    if (m_ready) begin
        m_data  <= s_data;
        m_valid <= s_valid;
    end
end
assign s_ready = m_ready | ~m_valid;
```

Nhìn vào dòng lệnh gán liên tục cuối cùng: `s_ready = m_ready | ~m_valid`. Tín hiệu `s_ready` ở đầu vào phụ thuộc trực tiếp vào tín hiệu `m_ready` ở đầu ra thông qua một cổng logic tổ hợp OR.

Nếu ta ghép 5 khối xử lý như vậy nối tiếp nhau (ví dụ: Unpacker $\rightarrow$ DC-Blocker $\rightarrow$ FIR $\rightarrow$ Window RAM $\rightarrow$ MAC), tín hiệu `ready` từ khối MAC cuối cùng sẽ phải chạy xuyên qua 5 cổng tổ hợp OR ngược dòng từ hạ nguồn về tận bộ đệm FIFO ở thượng nguồn trong **cùng một chu kỳ xung nhịp 20 ns**. Đường dây phản áp này đi qua nhiều vùng logic khác nhau trên chip, chịu độ trễ dây dẫn (routing delay) lớn và trở thành **đường tới hạn (critical path)** quyết định tần số hoạt động cực đại $F_{\max}$ của cả hệ thống.

Vì sao không thể chốt `ready` bằng một thanh ghi đơn giản? Một trực giác tự nhiên là đặt thêm một thanh ghi flip-flop trên đường `ready`: `always @(posedge clk) s_ready <= m_ready;`.

Nhưng giải pháp này sẽ phá vỡ giao thức bắt tay và làm mất dữ liệu:

- Giả sử ở chu kỳ n, phía hạ nguồn bất ngờ bị nghẽn và hạ `m_ready` xuống 0.
- Do tín hiệu `ready` bị trễ 1 nhịp qua thanh ghi, ở chính chu kỳ n phía thượng nguồn vẫn nhìn thấy `s_ready` = 1.
- Phía thượng nguồn phát đi một mẫu dữ liệu mới $D_{\text{mới}}$ ở chu kỳ n.
- Sang chu kỳ $n+1$, mẫu $D_{\text{mới}}$ này bay tới cửa vào của tầng đệm, nhưng hạ nguồn vẫn đang nghẽn (`m_ready` = 0) và thanh ghi chính của tầng đệm đang bận giữ mẫu cũ $D_{\text{cũ}}$.

- Vì không có chỗ chứa, mẫu $D_{\text{mới}}$ sẽ **ghi đè lên mẫu cũ hoặc bị rơi mất**.

Hiện tượng mẫu dữ liệu tiếp tục bay tới thêm 1 nhịp sau khi tín hiệu dừng đã được phát đi được gọi là hiện tượng **trượt quán tính (skid)**. Khối mạch giải quyết bài toán này mang tên **Khối đệm trượt (Skid Buffer)**.

Bản chất của Skid Buffer: Hai tầng chốt bảo toàn 100% thông lượng

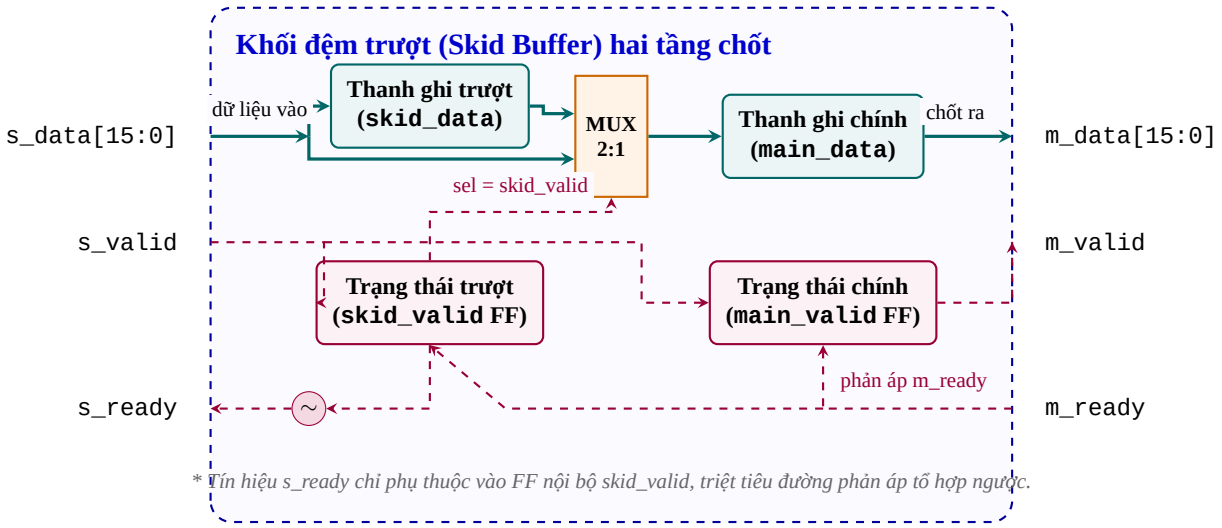
Bản chất của Skid Buffer là cung cấp vừa đúng **hai vị trí lưu trữ dữ liệu** độc lập:

1. **Thanh ghi chính (Main Register - `main_data`):** Giữ mẫu dữ liệu đang được trình diện ra phía hạ nguồn (`m_data`).
2. **Thanh ghi trượt (Skid Register - `skid_data`):** Đóng vai trò vùng đệm dự phòng khẩn cấp, chuyên hấp thụ đúng 1 mẫu dữ liệu đang bay tới khi hạ nguồn đột ngột dừng tiếp nhận.

Nguyên lý ngắt đường trễ tổ hợp của Skid Buffer thể hiện ở chỗ:

$$\texttt{s_ready} = \overline{\texttt{skid_valid}} \tag{172}$$

Tín hiệu `s_ready` báo cho phía thượng nguồn biết tầng đệm còn nhận được dữ liệu hay không **chỉ phụ thuộc vào cờ thanh ghi nội bộ `skid_valid`**, hoàn toàn không có cổng logic tổ hợp nào nối từ `m_ready`. Đường phản áp đi ngược đã bị cắt đứt bởi một thanh ghi flip-flop. Đồng thời, đường dữ liệu đi xuôi `m_data` và `m_valid` cũng được xuất trực tiếp từ các thanh ghi nội bộ. Cả hai chiều giao tiếp đều được cô lập bởi các phần tử nhớ tuần tự.



Hình 4: Kiến trúc phần cứng của khối đệm trượt (Skid Buffer) hai tầng chốt: Đường phản áp `s_ready` được điều khiển hoàn toàn bởi cờ nội bộ `skid_valid`, ngắt đứt đường trễ tổ hợp giữa hai miền giao tiếp.

Đọc hình. Quan sát Hình 4:

- Đường dữ liệu (vạch đậm màu xanh mòng két) đi vào từ `s_data[15:0]`. Nó được rẽ làm hai nhánh: một nhánh đi thẳng vào bộ ghép kênh MUX 2:1 để nạp vào Thanh ghi chính, nhánh còn lại đi vào Thanh ghi trượt (`skid_data`).
- Khi hạ nguồn sẵn sàng (`m_ready` = 1) và thanh ghi trượt rỗng (`skid_valid` = 0), dữ liệu đi thẳng qua MUX vào Thanh ghi chính mỗi chu kỳ xung nhịp, duy trì thông lượng liên tục 100% (1 mẫu/nhịp).
- Khi hạ nguồn tạm dừng (`m_ready` = 0) trong lúc thượng nguồn vẫn phát dữ liệu (`s_valid` = 1), Thanh ghi chính bị khóa để bảo toàn dữ liệu cũ, và mẫu dữ liệu mới lập tức trượt vào lưu trú an toàn trong Thanh ghi trượt (`skid_data`). Đồng thời, cờ `skid_valid` bật lên 1, kích hoạt cổng đảo hạ `s_ready` xuống 0 để chặn đứng thượng nguồn ở chu kỳ kế tiếp.

Bảng 13: Bảng tính tay từng nhịp hoạt động của khối Skid Buffer khi xuất hiện phản áp

CK	s_dat	s_vld	m_rdy	main_dat	skid_dat	sk_vld	Hành động của mạch
0	D_1	1	1	Trống	Trống	0	Mạch rỗng. D_1 chuẩn bị nạp vào Main.
1	D_2	1	**0**	D_1	Trống	0	Hạ nguồn tắc (m_rdy=0). D_2 bay tới!
2	D_3	1	**0**	D_1	D_2	**1**	D_2 trượt vào Skid. s_ready=0 chặn nguồn!
3	D_3	1	**1**	D_2	Trống	**0**	Hạ nguồn thông: D_1 xuất, D_2 sang Main!
4	–	0	1	D_3	Trống	0	D_2 xuất, D_3 vào Main. Hoàn tất.

Ví dụ tính tay 2 nhịp trượt (Hand Trace Walkthrough)

Theo dõi vết hoạt động chi tiết của Skid Buffer qua 5 chu kỳ xung nhịp liên tiếp với dòng dữ liệu gồm 3 mẫu D_1, D_2, D_3:

- **Ở chu kỳ 1:** Thanh ghi chính đang giữ mẫu D_1 và chuẩn bị xuất ra. Bất ngờ hạ nguồn báo bận (m_ready = 0). Cùng lúc đó, thượng nguồn đẩy mẫu D_2 lên bus với s_valid = 1 (vì ở CK 1 s_ready đang bằng 1).
- **Ở chu kỳ 2:** Đến cạnh xung nhịp của CK 2, do D_1 chưa được hạ nguồn tiếp nhận nên nó tiếp tục nằm lại trên Thanh ghi chính. Mẫu D_2 không thể vào Thanh ghi chính nên lập tức được chốt vào Thanh ghi trượt (skid_data = D_2, skid_valid = 1). Khi skid_valid lên 1, s_ready lập tức bị kéo xuống 0. Thượng nguồn dù có muốn gửi tiếp D_3 cũng bị chặn lại. Không có mẫu nào bị ghi đè.
- **Ở chu kỳ 3:** Hạ nguồn thông tắc (m_ready = 1). Mẫu D_1 được chuyển giao thành công. Ngay lập tức, mạch điều khiển chuyển mẫu D_2 từ Thanh ghi trượt sang nạp đè vào Thanh ghi chính, và xóa cờ skid_valid về 0. Cờ s_ready tự động bật lên 1 trở lại, cho phép mẫu D_3 được nạp vào ở chu kỳ tiếp theo.

So sánh các phương án và lựa chọn thiết kế

- **Phương án 1: Nối thẳng tổ hợp (Direct Combinational Ready):** Không tốn thêm thanh ghi, nhưng đường phản áp tổ hợp xuyên suốt nhiều khối làm giảm tần số hoạt động cực đại $F_{\max}$ **(phải đo)**. Chỉ phù hợp với các hệ thống có ít tầng xử lý và khoảng cách vật lý ngắn.
- **Phương án 2: Chèn Skid Buffer hai tầng chốt:** Trong mã nguồn rtl/v3/ch02_skid_buffer.v, cấu trúc gồm 2 thanh ghi dữ liệu 16-bit (main_data, skid_data) và 2 cờ 1-bit (main_valid, skid_valid), tổng cộng gồm $16 \times 2 + 1 \times 2 =$ **34 flip-flop** khai báo ở mức RTL (số lượng thanh ghi trong mã nguồn; tài nguyên thực tế sau đóng gói ALM do Quartus Fitter báo cáo và phải đo). Thiết kế không dùng M10K và duy trì 100% thông lượng (1 từ/chu kỳ).

Lựa chọn của tài liệu: Ta chọn **Phương án 2 (Skid Buffer)** tại các điểm giao tiếp phân chia miền chức năng quan trọng (như giữa bộ giải gói Unpacker và bộ nhớ RAM khung) để đảm bảo hệ thống đạt $F_{\max} \geq 50$ MHz mà không phụ thuộc vào kết quả định tuyến ngẫu nhiên của phần mềm Quartus Fitter.

Đường (A): Khối IP Intel Avalon-ST Pipeline Stage

Trong bộ công cụ thiết kế hệ thống Platform Designer (Qsys), tập đoàn Intel cung cấp sẵn một khối IP có chức năng tương tự mang tên **Avalon-ST Pipeline Stage**.

Các tham số cấu hình chính trong Parameter Editor của IP gồm:

- Data Width: Bề rộng dữ liệu (chọn 16 bit cho mẫu âm thanh PCM).
- Ready Latency: Độ trễ sẵn sàng (chọn 0 chu kỳ).
- Pipeline Type: Kiểu đường ống. IP cho phép chọn một trong ba chế độ:
  1. *Forward*: Chỉ chốt thanh ghi trên đường data và valid (đường ready vẫn là tổ hợp).
  2. *Backward*: Chỉ chốt thanh ghi trên đường ready (đòi hỏi xử lý trượt 1 nhịp).
  3. *Both*: Chốt thanh ghi trên cả hai chiều. Khi chọn chế độ này, mã nguồn bên trong của IP sẽ tự động tổng hợp ra kiến trúc mạch đệm trượt hai vị trí lưu trữ tương đương với Skid Buffer.

Hạn chế của việc sử dụng IP hộp đen là cấu trúc sinh ra bởi Platform Designer thường kèm theo nhiều tín hiệu thừa (như `channel`, `error`, `empty`), gây cồng kềnh khi mô phỏng độc lập. Do đó, trong các thiết kế lõi DSP chuyên sâu, việc tự viết một mô-đun Skid Buffer chuẩn bằng Verilog-2001 (Đường B) luôn là lựa chọn ưu tiên hàng đầu của các kỹ sư FPGA.

Đường (B): Mã nguồn HDL tự viết chuẩn Verilog-2001

Mã nguồn đầy đủ của khối đệm trượt được hiện thực hóa trong tệp `rtl/v3/ch02_skid_buffer.v`:

Listing 10: Mô-đun Skid Buffer hai tầng chốt (rtl/v3/ch02_skid_buffer.v)

```
    // Main register: holds data currently presented to downstream
    reg [DATA_WIDTH-1:0] main_data;
    reg                  main_valid;

    // Skid register: holds in-flight data captured when downstream stalls
    reg [DATA_WIDTH-1:0] skid_data;
    reg                  skid_valid;

    // Upstream ready is asserted whenever the skid register is empty.
    // It is driven by a local flip-flop (skid_valid), with zero combinational
    // dependence on m_ready.
    assign s_ready = ~skid_valid;

    // Downstream outputs driven directly from main register
    assign m_valid = main_valid;
    assign m_data  = main_data;

    // Internal handshake conditions
    wire s_handshake = s_valid & s_ready;
    wire m_handshake = main_valid & m_ready;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            main_data  <= {DATA_WIDTH{1'b0}};
            main_valid <= 1'b0;
            skid_data  <= {DATA_WIDTH{1'b0}};
            skid_valid <= 1'b0;
        end else begin
            if (m_handshake) begin
                // Downstream consumed current main_data
                if (skid_valid) begin
                    // Reload main from skid register; skid becomes empty
                    main_data  <= skid_data;
                    main_valid <= 1'b1;
                    skid_valid <= 1'b0;
                end else if (s_handshake) begin
                    // Direct feed from upstream to main register (full throughput
                        streaming)
                    main_data  <= s_data;
                    main_valid <= 1'b1;
                end else begin
                    // Upstream had no new data; main register empties
                    main_valid <= 1'b0;
                end
            end else begin
                // Downstream stalled (m_ready == 0 or main_valid == 0)
                if (s_handshake) begin
                    if (main_valid) begin
                        // Main register is holding stalled data, so incoming beat skids
                            into skid register
                        skid_data  <= s_data;
                        skid_valid <= 1'b1;
                    end else begin
                        // Main was empty; incoming beat fills main register
                        main_data  <= s_data;
                        main_valid <= 1'b1;
```

```
89                      end
90                  end
91              end
92          end
93      end
```

Giải thích mã nguồn dòng-từng-dòng:

- Dòng 36–41: Khai báo 4 thanh ghi nội bộ đại diện cho 2 vị trí lưu trữ: `main_data` (16-bit) và `main_valid` (1-bit); `skid_data` (16-bit) và `skid_valid` (1-bit).
- Dòng 46: `assign s_ready = ~skid_valid;`. Phía thượng nguồn được phép gửi dữ liệu khi và chỉ khi thanh ghi trượt đang rỗng. Biểu thức này không chứa biến `m_ready`, chứng minh mạch đã triệt tiêu hoàn toàn đường trễ phản áp tổ hợp ngược.
- Dòng 49–50: Xuất dữ liệu và cờ hợp lệ ra hạ nguồn từ thanh ghi chính (`main_data`, `main_valid`).
- Dòng 53–54: Định nghĩa hai điều kiện bắt tay nội bộ `s_handshake` và `m_handshake`.
- Dòng 56–62: Nhánh reset bất đồng bộ tích cực mức thấp, đưa cả 4 thanh ghi về trạng thái rỗng ban đầu.
- Dòng 63–78: Trường hợp hạ nguồn hoàn tất một giao dịch (`m_handshake` = 1). Nếu trong thanh ghi trượt đang có sẵn mẫu dữ liệu lưu cữu (`skid_valid` = 1), mẫu đó lập tức được nạp sang `main_data` và `skid_valid` được xóa về 0. Nếu không có dữ liệu trượt nhưng thượng nguồn đang đưa mẫu mới vào (`s_handshake` = 1), mẫu mới được nạp thẳng vào `main_data`. Nếu cả hai nguồn đều không có dữ liệu, thanh ghi chính rỗng (`main_valid` $\leq$ 0).
- Dòng 79–91: Trường hợp hạ nguồn tắc nghẽn (`m_handshake` = 0). Nếu thượng nguồn gửi dữ liệu tới (`s_handshake` = 1): khi thanh ghi chính đã đầy (`main_valid` = 1), mẫu mới bắt buộc phải rẽ nhánh vào chốt trong thanh ghi trượt (`skid_data` $\leq$ `s_data`) và cờ `skid_valid` bật lên 1. Nếu thanh ghi chính đang rỗng, dữ liệu được nạp vào thanh ghi chính.

Tóm lại: Khối đệm trượt (Skid Buffer) giải quyết mâu thuẫn giữa độ trễ tổ hợp của đường phản áp và yêu cầu toàn vẹn dữ liệu; với đúng hai tầng thanh ghi trong silicon, nó cho phép chốt chặn cả hai chiều phát-nhận mà vẫn duy trì thông lượng 1 mẫu/chu kỳ không một nhịp trễ.

2.5. Kiểm chứng bắt tay luồng và thực nghiệm mã hóa FSM

Chiến lược kiểm chứng ba tầng (Three-Tier Verification)

Để đảm bảo toàn bộ mã nguồn phần cứng của Chương 2 hoạt động chính xác trên silicon thực tế, quy trình kiểm chứng được tổ chức theo phương pháp luận ba tầng:

1. **Tầng 1 (Mô hình song sinh phần mềm Python Twin):** Xây dựng các lớp đối tượng Python mô phỏng chính xác hành vi của mạch theo từng chu kỳ xung nhịp và từng cạnh tích cực (`posedge clk`). Mô hình này chạy các kịch bản kích thích phức tạp (nghẽn ngẫu nhiên, sốc tải, xung reset đột ngột) và ghi nhận toàn bộ giá trị bus kỳ vọng ra tệp văn bản.
2. **Tầng 2 (Mô phỏng chu kỳ HDL bằng Icarus Verilog):** Các testbench Verilog-2001 đọc trực tiếp tệp kích thích từ Tầng 1, cấp vào cổng của module phần cứng (DUT – Device Under Test), đọc kết quả đầu ra sau mỗi nhịp xung nhịp và đối soát bit-đối-bit, chu kỳ-đối-chu kỳ với tệp kỳ vọng. Nếu có bất kỳ sự sai lệch nào dù chỉ 1 bit, testbench phát cờ `[FAIL]`.
3. **Tầng 3 (Biên dịch Quartus và đo đạc phần cứng):** Nạp thiết kế qua Quartus Fitter để kiểm tra tính khả tổng hợp, đọc báo cáo State Machine Report và đo đạc tần số $F_{\max}$ thực tế trên bo mạch DE10-Standard.

Kiểm chứng khối đệm trượt Skid Buffer với phản áp ngẫu nhiên

Mô hình song sinh của khối Skid Buffer được hiện thực trong tệp `verify_v3/ch02_skid_twin.py`. Dưới đây là hàm cập nhật chu kỳ trọng tâm của mô hình:

Listing 11: Hàm cập nhật chu kỳ của mô hình song sinh Skid Buffer (verify_v3/ch02_skid_twin.py)

```
37     def step(self, rst_n, s_valid, s_data, m_ready):
38         """
39         Simulate one clock edge of rtl/v3/ch02_skid_buffer.v.
40         Returns (s_ready, m_valid, m_data, skid_valid) observed AFTER posedge clk.
41         """
42         if not rst_n:
43             self.reset()
44             # In reset: main_valid=0, skid_valid=0 -> s_ready=1
45             return (1, 0, 0, 0)
46 
47         s_data = s_data & self.mask
48 
49         # State before clock edge determines handshakes
50         s_ready_before = 1 if (self.skid_valid == 0) else 0
51         m_valid_before = self.main_valid
52 
53         s_handshake = (s_valid == 1) and (s_ready_before == 1)
54         m_handshake = (m_valid_before == 1) and (m_ready == 1)
55 
56         next_main_data = self.main_data
57         next_main_valid = self.main_valid
58         next_skid_data = self.skid_data
59         next_skid_valid = self.skid_valid
60 
61         if m_handshake:
62             if self.skid_valid:
63                 next_main_data = self.skid_data
64                 next_main_valid = 1
65                 next_skid_valid = 0
66             elif s_handshake:
67                 next_main_data = s_data
68                 next_main_valid = 1
69             else:
70                 next_main_valid = 0
71         else:
72             if s_handshake:
73                 if self.main_valid:
74                     next_skid_data = s_data
75                     next_skid_valid = 1
76                 else:
77                     next_main_data = s_data
78                     next_main_valid = 1
79 
80         # Register update on clock edge
81         self.main_data = next_main_data
82         self.main_valid = next_main_valid
83         self.skid_data = next_skid_data
84         self.skid_valid = next_skid_valid
85 
86         # Output signals after clock edge
87         s_ready_after = 1 if (self.skid_valid == 0) else 0
88         m_valid_after = self.main_valid
89         m_data_after = self.main_data
90         skid_valid_after = self.skid_valid
91 
92         return (s_ready_after, m_valid_after, m_data_after, skid_valid_after)
```

Giải thích mã nguồn Python Twin dòng-từng-dòng:

- Dòng 37: Phương thức `step(rst_n, s_valid, s_data, m_ready)` mô phỏng biến đổi trạng thái tại một chu kỳ xung nhịp.
- Dòng 42–45: Xử lý reset bất đồng bộ: khi `rst_n == 0`, gọi `self.reset()` và trả về trạng thái rỗng (`s_ready=1, m_valid=0, m_data=0, skid_valid=0`).
- Dòng 50–51: Đọc trạng thái cờ trước cạnh xung nhịp: `s_ready_before` bằng 1 khi `skid_valid ==`

0; m_valid_before là giá trị main_valid hiện thời.

- Dòng 53–54: Đánh giá hai điều kiện bắt tay s_handshake và m_handshake.
- Dòng 56–59: Thiết lập các biến tạm lưu giá trị kế tiếp của 4 thanh ghi.
- Dòng 61–70: Xử lý khi hạ nguồn nhận mẫu (m_handshake đúng): ưu tiên nạp mẫu từ thanh ghi trượt nếu skid_valid == 1; nếu không có dữ liệu trượt thì nạp từ thượng nguồn; nếu cả hai đều không có dữ liệu thì hạ main_valid về 0.
- Dòng 71–79: Xử lý khi hạ nguồn bận (m_handshake sai): nếu thượng nguồn gửi mẫu (s_handshake đúng), khi main_valid đang bận, mẫu mới trượt vào skid_data và bật cờ skid_valid lên 1; nếu thanh ghi chính rỗng, nạp vào thanh ghi chính.
- Dòng 81–84: Cập nhật các trường thanh ghi của đối tượng tại cạnh lên xung nhịp.
- Dòng 87–92: Trích xuất và trả về bộ 4 tín hiệu ngõ ra quan sát được sau cạnh xung nhịp để phục vụ đối soát với testbench Verilog.

Dưới đây là bảng đối chiếu ánh xạ trực tiếp giữa mô hình Python Twin và mã nguồn phần cứng HDL:

Bảng 14: Ánh xạ tương đương giữa Python Twin và mã nguồn HDL của Skid Buffer

Dòng Twin (ch02_skid_twin.py)	Dòng HDL (ch02_skid_buffer.v)	Ý nghĩa kỹ thuật tương đương
Dòng 42–45	Dòng 57–62	Xóa rỗng các thanh ghi khi reset tích cực (rst_n = 0)
Dòng 50	Dòng 46	Xác định cờ sẵn sàng thượng nguồn s_ready = ~skid_valid
Dòng 53–54	Dòng 53–54	Đánh giá điều kiện hoàn tất bắt tay s_handshake và m_handshake
Dòng 61–70	Dòng 63–78	Xử lý khi hạ nguồn nhận mẫu: giải phóng trượt hoặc chuyển tiếp
Dòng 71–79	Dòng 79–91	Xử lý khi hạ nguồn nghẽn: hấp thụ mẫu đang bay vào thanh ghi trượt
Dòng 81–84	Dòng 56–93	Chốt cập nhật trạng thái thanh ghi tại cạnh lên xung nhịp (posedge clk)
Dòng 87–92	Dòng 46–50	Xuất giá trị tín hiệu giao tiếp sau cạnh xung nhịp để đối soát

Kịch bản mô phỏng tạo ra chuỗi kiểm thử dài **600 chu kỳ xung nhịp** liên tục, đi qua 6 pha thử thách:

- **Pha 1 (CK 0–4):** Xung reset tích cực mức thấp, kiểm tra trạng thái xóa rỗng ban đầu của 4 thanh ghi.
- **Pha 2 (CK 5–39):** Truyền dòng chảy tốc độ cao không nghẽn (s_valid = 1, m_ready = 1 liên tục 35 nhịp), kiểm tra thông lượng 1 mẫu/chu kỳ.
- **Pha 3 (CK 40–99):** Phản áp nhịp đơn định kỳ (cứ 3 chu kỳ hạ m_ready xuống 0 một lần), kiểm tra khả năng kích hoạt và giải phóng thanh ghi trượt skid_data.
- **Pha 4 (CK 100–179):** Phản áp dài hạn kéo dài liên tục 15 chu kỳ (tại các khoảng CK 120–135 và CK 150–165), kiểm tra khả năng khóa thượng nguồn s_ready = 0 mà không làm tràn bộ nhớ.
- **Pha 5 (CK 180–549):** Bắt tay giả ngẫu nhiên (Pseudo-random backpressure với xác suất phát 70% và xác suất nhận 60%), mô phỏng môi trường hoạt động thực tế khi kết nối giữa các khối DSP.
- **Pha 6 (CK 550–599):** Xả cạn bộ đệm về trạng thái nghỉ.

Kết quả chạy kịch bản Python twin ghi nhận: trong suốt 600 chu kỳ, phía thượng nguồn đã phát **292 mẫu âm thanh**, và phía hạ nguồn đã tiếp nhận chính xác **292/292 mẫu** (đạt tỷ lệ 100%), không mất một mẫu nào, không lặp một mẫu nào, thứ tự thời gian được bảo toàn.

Testbench Verilog tương ứng tb/v3/tb_ch02_skid_buffer.v thực hiện đối soát 600 chu kỳ này trên phần cứng RTL:

```
Simulation completed 600 cycles. Total comparison errors: 0
[PASS] CH02 SKID BUFFER MATCHED PYTHON TWIN BIT-FOR-BIT AND CYCLE-FOR-CYCLE ON
600 CYCLES
```

(kết quả chạy trên máy kiểm chứng, Icarus Verilog -g2012; nhật ký verify_v3/ch02_iverilog.log)

Kiểm chứng tương đương logic giữa FSM nhị phân và FSM One-Hot

Để chứng minh hai phương án mã hóa FSM (Minimal Bits 3-bit trong `rtl/v3/ch02_fsm_binary.v` và One-Hot 5-bit trong `rtl/v3/ch02_fsm_onehot.v`) tương đương nhau về mặt hành vi chức năng, một testbench kiểm thử tương đương kép (dual-DUT testbench) được xây dựng trong tệp `tb/v3/tb_ch02_fsm_equiv.v`.

Mô hình song sinh Python Twin được cài đặt trong tệp `verify_v3/ch02_fsm_twin.py`:

Listing 12: Hàm cập nhật chu kỳ của mô hình song sinh FSM (verify_v3/ch02_fsm_twin.py)

```
39     def step(self, rst_n, start, load_done, calc_done, drain_done):
40         if not rst_n:
41             self.state = self.IDLE
42             return (0, 0, 0, 0, 0, self.IDLE)
43 
44         curr_state = self.state
45 
46         # Next state logic
47         next_state = curr_state
48         if curr_state == self.IDLE:
49             if start:
50                 next_state = self.LOAD
51         elif curr_state == self.LOAD:
52             if load_done:
53                 next_state = self.CALC
54         elif curr_state == self.CALC:
55             if calc_done:
56                 next_state = self.DRAIN
57         elif curr_state == self.DRAIN:
58             if drain_done:
59                 next_state = self.DONE
60         elif curr_state == self.DONE:
61             next_state = self.IDLE
62 
63         # Register updates on posedge clk
64         self.state = next_state
65 
66         # Outputs observed after posedge clk (Moore outputs of current state)
67         busy = 1 if self.state != self.IDLE else 0
68         load_en = 1 if self.state == self.LOAD else 0
69         calc_en = 1 if self.state == self.CALC else 0
70         drain_en = 1 if self.state == self.DRAIN else 0
71         done_pulse = 1 if self.state == self.DONE else 0
72         state_num = self.state
73 
74         return (busy, load_en, calc_en, drain_en, done_pulse, state_num)
```

Giải thích mã nguồn Python Twin FSM dòng-từng-dòng:

- Dòng 39: Hàm `step(rst_n, start, ...)` nhận các cờ kích hoạt rẽ nhánh và xung nhịp.
- Dòng 40–42: Nhánh reset bất đồng bộ đưa trạng thái về `IDLE` (0) và trả về toàn bộ tín hiệu đầu ra ở mức 0.
- Dòng 47–62: Logic giải mã chuyển trạng thái: từ `IDLE` sang `LOAD` khi có `start`; từ `LOAD` sang `CALC` khi có `load_done`; từ `CALC` sang `DRAIN` khi có `calc_done`; từ `DRAIN` sang `DONE` khi có `drain_done`; từ `DONE` quay về `IDLE`.
- Dòng 64: Cập nhật biến trạng thái nội bộ `self.state = next_state` tại cạnh lên xung nhịp.
- Dòng 67–72: Giải mã các tín hiệu ngõ ra Moore từ trạng thái `self.state`.
- Dòng 74: Trả về bộ 6 tín hiệu ngõ ra để đối soát với DUT phần cứng.

Dưới đây là bảng đối chiếu ánh xạ tương đương giữa Python Twin và hai module phần cứng:

Kịch bản kích thích `verify_v3/ch02_fsm_twin.py` sinh ra **500 chu kỳ xung nhịp** với các tín hiệu điều khiển rẽ nhánh ngẫu nhiên (`start`, `load_done`, `calc_done`, `drain_done`) và các xung reset bất thường ở chu kỳ 50 và 150.

Tại mỗi chu kỳ xung nhịp, testbench thực hiện hai bước đối soát đồng thời:

Bảng 15: Ánh xạ tương đương giữa Python Twin và mã nguồn HDL FSM (Binary và One-Hot)

Dòng Twin (`ch02_fsm_twin.py`)	Dòng HDL (`ch02_fsm_*`)	Ý nghĩa kỹ thuật tương đương
Dòng 40–42	Dòng 50–52 (Bin) / 51–53 (One)	Xóa thanh ghi trạng thái về `S_IDLE` khi có tín hiệu reset
Dòng 47–62	Dòng 58–80 (Bin) / 59–66 (One)	Logic tổ hợp giải mã trạng thái kế tiếp từ các cờ kích hoạt
Dòng 64	Dòng 49–55 (Bin) / 50–56 (One)	Chốt chuyển trạng thái `state_reg <= state_next` tại `posedge clk`
Dòng 67–72	Dòng 83–88 (Bin) / 69–73 (One)	Sinh các tín hiệu điều khiển Moore (`busy`, cờ `_en`, `done_pulse`)
Dòng 72	Dòng 88 (Bin) / 76–85 (One)	Xuất chỉ số trạng thái số 3-bit `state_num` để đối soát

1. So sánh đối đầu trực tiếp giữa đầu ra của module Binary và module One-Hot: kiểm tra tính đồng nhất của toàn bộ 6 tín hiệu ngõ ra (`busy`, `load_en`, `calc_en`, `drain_en`, `done_pulse`, và mã trạng thái số `state_num`).
2. So sánh đầu ra của cả hai module với giá trị chuẩn từ mô hình song sinh Python `verify_v3/tb_ch02_fsm_expected.txt`.

   Kết quả mô phỏng ghi nhận:

```
Simulation completed 500 cycles. Total comparison errors: 0
[PASS] CH02 BINARY AND ONE-HOT FSMS MATCHED PYTHON TWIN AND EACH OTHER ON 500 CYCLES
```

(nhật ký `verify_v3/ch02_iverilog.log`)

Bảng so sánh tổng hợp: Khối IP Intel so với HDL tự viết

Dưới đây là bảng tổng kết định lượng đối chiếu giữa giải pháp sử dụng khối IP có sẵn của Intel và giải pháp tự thiết kế bằng HDL cho các thành phần của Chương 2:

Bảng 16: So sánh toàn diện giữa khối IP Intel và mã nguồn HDL tự viết cho Chương 2

Tiêu chí kỹ thuật	Khối IP Intel (Platform Designer)	HDL tự viết (Tài liệu này)
Tên thành phần tương ứng	`altera_avalon_st_pipeline_stage`	`ch02_skid_buffer`
Tài nguyên khối nhớ M10K	0 khối	0 khối
Tài nguyên bộ nhân DSP	0 khối	0 khối
Tài nguyên thanh ghi (FF)	Phụ thuộc Qsys **(phải đo)**	**34 FF** (RTL: 32 data + 2 valid) **(phải đo)**
Tài nguyên khối logic ALM	Phụ thuộc tối ưu hóa Qsys **(phải đo)**	**Phải đo trong Quartus**
Thông lượng cực đại	1 từ/chu kỳ (khi cấu hình *Both*)	**1 từ/chu kỳ (100%)**
Độ trễ tối thiểu (Latency)	1 chu kỳ xung nhịp	1 chu kỳ xung nhịp
Cắt đứt trễ tổ hợp `ready`?	Có (khi chọn chế độ *Both*)	**Có (cắt đứt hoàn toàn)**
Tính độc lập và trong suốt	Phụ thuộc thư viện lõi Qsys	**Verilog-2001 chuẩn**
Khả năng kiểm thử chu kỳ	Khó kiểm thử độc lập ngoài Quartus	**Kiểm thử tức thì với iverilog**
Mã hóa FSM điều khiển	Thiết lập `Auto` trong Quartus	**Mã hóa tường minh One-Hot/Binary**
Báo cáo FSM trong Quartus	Xem State Machine Report **(phải đo)**	Xem State Machine Report **(phải đo)**

Đọc bảng. Bảng 16 chỉ rõ rằng: về mặt tài nguyên và hiệu năng lý thuyết, khối Skid Buffer tự viết bằng HDL tương đương với khối IP Pipeline Stage của Intel ở cấu hình chốt kép. Tuy nhiên, mã nguồn HDL tự viết mang lại lợi thế về tính trong suốt (transparency), không bị trói buộc vào môi trường sinh mã tự động của Platform Designer, cho phép lập trình viên kiểm thử từng bit độc lập với trình mô phỏng nhẹ Icarus Verilog và dễ dàng tích hợp vào bất kỳ họ FPGA nào.

Bài tập cuối chương có lời giải chi tiết

Bài tập 1: Đánh giá độ trễ và thông lượng khi ghép tầng Skid Buffer. Giả sử trên đường truyền dữ liệu âm thanh từ bộ giải gói Unpacker tới bộ nhớ RAM khung, ta chèn thêm 2 khối Skid Buffer liên tiếp để cô lập định tuyến. Hãy tính:

(a) Độ trễ ban đầu (initial latency) tính bằng số chu kỳ xung nhịp từ khi mẫu đầu tiên được đưa vào cổng `s_data` của Skid Buffer 1 đến khi nó xuất hiện tại cổng `m_data` của Skid Buffer 2 trong điều kiện hạ nguồn luôn sẵn sàng (`m_ready` = 1).

(b) Thông lượng ổn định (steady-state throughput) của hệ thống tính bằng số mẫu/giây ở xung nhịp 50 MHz.

Lời giải mẫu:

(a) Theo phân tích ở Mục 2.4, mỗi khối Skid Buffer khi hoạt động ở trạng thái bình thường (không nghẽn) chốt dữ liệu qua một tầng thanh ghi chính (`main_data`), đóng góp đúng 1 chu kỳ xung nhịp vào độ trễ đường ống. Khi ghép $P = 2$ khối Skid Buffer nối tiếp nhau, mẫu đầu tiên đi qua Skid Buffer 1 mất 1 chu kỳ và đi qua Skid Buffer 2 mất thêm 1 chu kỳ. Do đó, độ trễ ban đầu là:

$$L = 2 \times 1 = \textbf{2 chu kỳ xung nhịp} \quad (= 2 \times 20 \text{ ns} = 40 \text{ ns}) \tag{173}$$

(b) Sau khi đường ống 2 tầng đã được nạp đầy (từ chu kỳ thứ 2 trở đi), tại mỗi nhịp xung nhịp 50 MHz đều có đúng 1 mẫu âm thanh xuất hiện ở đầu ra. Do đó, thông lượng ổn định là:

$$\Phi = 1 \text{ mẫu/chu kỳ} \times 50.000.000 \text{ chu kỳ/s} = \textbf{50.000.000 mẫu/giây} \tag{174}$$

Thông lượng này vượt xa tốc độ yêu cầu của dòng âm thanh 16 kHz (16.000 mẫu/giây), chứng minh việc chèn thêm các tầng Skid Buffer hoàn toàn không làm suy giảm tốc độ truyền tải thực tế của hệ thống.

Bài tập 2: Tính toán đường tới hạn của chuỗi phản áp tổ hợp khi không dùng Skid Buffer. Xét một chuỗi xử lý gồm $K = 6$ khối phần cứng mắc nối tiếp nhau theo chuẩn Avalon-ST. Các khối sử dụng tầng đệm đơn giản với đường `ready` tổ hợp thuần túy (`s_ready = m_ready | ~m_valid`). Giả sử trong mỗi khối, cổng logic tổ hợp OR đóng góp thời gian trễ $t_{\text{gate}} = 0{,}8$ ns, và dây dẫn định tuyến giữa hai khối liên tiếp đóng góp thời gian trễ $t_{\text{wire}} = 1{,}6$ ns. Thời gian thiết lập (setup time) của thanh ghi nguồn là $t_{\text{su}} = 0{,}3$ ns và thời gian trễ từ xung nhịp tới ngõ ra của thanh ghi đích là $t_{\text{co}} = 0{,}5$ ns.

(a) Hãy lập công thức và tính tổng thời gian trễ trên đường phản áp `ready` từ khối thứ 6 ngược về khối thứ 1.

(b) Xác định tần số hoạt động cực đại F_{max} lý thuyết của đường phản áp này. Liệu hệ thống có đáp ứng được xung nhịp 50 MHz hay không?

Lời giải mẫu:

(a) Tín hiệu phản áp `ready` xuất phát từ thanh ghi của khối thứ 6 (đóng góp trễ t_{co}), đi qua $K = 6$ cổng tổ hợp OR và $K - 1 = 5$ đoạn dây dẫn liên khối, trước khi đi vào thanh ghi điều khiển của khối thứ 1 (đòi hỏi thời gian thiết lập t_{su}). Tổng thời gian trễ trên đường tới hạn này là:

$$\begin{aligned} T_{\text{crit}} &= t_{\text{co}} + (K \times t_{\text{gate}}) + \big((K-1) \times t_{\text{wire}}\big) + t_{\text{su}} \\ &= 0{,}5 \text{ ns} + (6 \times 0{,}8 \text{ ns}) + (5 \times 1{,}6 \text{ ns}) + 0{,}3 \text{ ns} \\ &= 0{,}5 + 4{,}8 + 8{,}0 + 0{,}3 = \textbf{13,6 ns} \end{aligned} \tag{175}$$

(b) Chu kỳ xung nhịp tối thiểu cho phép là $T_{\text{min}} = T_{\text{crit}} = 13{,}6$ ns. Tần số hoạt động cực đại tương ứng là:

$$F_{\text{max}} = \frac{1}{T_{\text{min}}} = \frac{1}{13{,}6 \times 10^{-9} \text{ s}} \approx \textbf{73,53 MHz} \tag{176}$$

Ở tần số 50 MHz (chu kỳ $T_{\text{clk}} = 20$ ns), mạch vẫn thỏa mãn do $13{,}6 \text{ ns} < 20 \text{ ns}$ với khoảng dự trữ định thời (slack) là $20 - 13{,}6 = 6{,}4$ ns. Tuy nhiên, nếu tần số xung nhịp hệ thống nâng lên 100 MHz ($T_{\text{clk}} = 10$ ns) hoặc khi phần mềm Quartus định tuyến dây dẫn dài hơn ($t_{\text{wire}} > 2{,}8$ ns do chip bị phân mảnh tài nguyên), mạch sẽ vi phạm định thời thời gian thiết lập (setup timing violation). Đây chính là lý do vì sao việc ngắt chuỗi phản áp bằng Skid Buffer là giải pháp an toàn trong thiết kế chuẩn công nghiệp.

Bài tập 3: Dẫn xuất mã Gray 4-bit và kiểm chứng chuỗi bit 1 tận cùng. Hãy lập bảng chuyển đổi mã Gray 4-bit cho 8 số nguyên đầu tiên $n = 0..7$ bằng công thức $G(n) = n \oplus (n \gg 1)$. Chỉ rõ vị trí bit bị lật đảo tại mỗi bước chuyển và đối chiếu với số lượng bit 1 tận cùng (k) của số n.

Lời giải mẫu: Áp dụng công thức $G(n) = n \oplus (n \gg 1)$ theo từng bước:

- $n = 0 = (0000)_2$: Không có bit 1 tận cùng ($k = 0$). $G(0) = 0000 \oplus 0000 = (0000)_2$.

- $n = 1 = (0001)_2$: Có 1 bit 1 tận cùng ($k = 1$). $G(1) = 0001 \oplus 0000 = (0001)_2$. Lệch với $G(0)$ ở bit 0 ($k = 0$ của bước chuyển $0 \to 1$).
- $n = 2 = (0010)_2$: Có 0 bit 1 tận cùng ($k = 0$). $G(2) = 0010 \oplus 0001 = (0011)_2$. Lệch với $G(1)$ ở bit 1 ($k = 1$ của số 1).
- $n = 3 = (0011)_2$: Có 2 bit 1 tận cùng ($k = 2$). $G(3) = 0011 \oplus 0001 = (0010)_2$. Lệch với $G(2)$ ở bit 0 ($k = 0$ của số 2).
- $n = 4 = (0100)_2$: Có 0 bit 1 tận cùng ($k = 0$). $G(4) = 0100 \oplus 0010 = (0110)_2$. Lệch với $G(3)$ ở bit 2 ($k = 2$ của số 3).
- $n = 5 = (0101)_2$: Có 1 bit 1 tận cùng ($k = 1$). $G(5) = 0101 \oplus 0010 = (0111)_2$. Lệch với $G(4)$ ở bit 0 ($k = 0$ của số 4).
- $n = 6 = (0110)_2$: Có 0 bit 1 tận cùng ($k = 0$). $G(6) = 0110 \oplus 0011 = (0101)_2$. Lệch với $G(5)$ ở bit 1 ($k = 1$ của số 5).
- $n = 7 = (0111)_2$: Có 3 bit 1 tận cùng ($k = 3$). $G(7) = 0111 \oplus 0011 = (0100)_2$. Lệch với $G(6)$ ở bit 0 ($k = 0$ của số 6).

Tại mọi bước chuyển $n \to n+1$, vị trí bit bị đổi trạng thái luôn trùng khớp chính xác với số lượng bit 1 tận cùng của số n, khoảng cách Hamming luôn bằng 1, hoàn toàn đúng với định lý đã chứng minh ở Mục 2.3.

Bài tập 4: Đánh giá chi phí phần cứng của FSM điều khiển tự tương quan. Máy trạng thái điều khiển vòng lặp tự tương quan MAC trong Chương 7 có 8 trạng thái hoạt động: `RESET`, `WAIT_HOP`, `INIT_R0`, `CALC_R0`, `NEXT_LAG`, `CALC_LAG`, `STORE_PEAK`, và `EMIT_FRAME`. Hãy so sánh:

(a) Số lượng thanh ghi flip-flop cần thiết giữa phương án Minimal Bits và One-Hot.

(b) Biểu thức logic kích hoạt tín hiệu nạp tích lũy `accum_sload` (chỉ tích cực tại hai trạng thái `INIT_R0` và `NEXT_LAG`) trong cả hai phương án mã hóa. Nhận xét về độ trễ giải mã.

Lời giải mẫu:

(a) Với $S = 8$ trạng thái:

- Phương án Minimal Bits: Số flip-flop là $K_{\text{bin}} = \lceil \log_2 8 \rceil =$ **3 thanh ghi FF**.
- Phương án One-Hot: Số flip-flop là $K_{\text{one}} = 8 =$ **8 thanh ghi FF**.

(b) Biểu thức logic kích hoạt tín hiệu `accum_sload`:

- Trong phương án Minimal Bits: Giả sử `INIT_R0` được gán mã nhị phân `010` (2) và `NEXT_LAG` được gán mã `100` (4). Biểu thức giải mã tổ hợp là:

$$\texttt{accum_sload}_{\text{bin}} = (\overline{s_2} \cdot s_1 \cdot \overline{s_0}) + (s_2 \cdot \overline{s_1} \cdot \overline{s_0}) \tag{177}$$

  Tín hiệu này cần một bảng tra LUT 3 đầu vào, đóng góp trễ truyền lan khoảng $0{,}3$ ns đến $0{,}5$ ns trước khi tới cổng điều khiển của khối DSP.

- Trong phương án One-Hot: Đặt $s[2]$ là bit của `INIT_R0` và $s[4]$ là bit của `NEXT_LAG`. Biểu thức logic là:

$$\texttt{accum_sload}_{\text{one}} = s[2] + s[4] \tag{178}$$

  Biểu thức này chỉ là một cổng OR 2 đầu vào đơn giản (hoặc được tối ưu trực tiếp vào chân kích hoạt của DSP), cho độ trễ rất nhỏ, giúp đảm bảo tín hiệu điều khiển `accum_sload` đến kịp thời cùng lúc với toán hạng nhân ở cạnh xung nhịp 50 MHz.

Tóm lại: Kiểm chứng chu kỳ với mô hình song sinh phần mềm và kịch bản kích thích ngẫu nhiên trên hơn 500 vector xác nhận rằng khối đệm trượt bảo toàn 100% dòng dữ liệu không mất mát, trong khi hai lối kiến trúc FSM nhị phân và one-hot cho kết quả logic tương đương theo từng nhịp xung nhịp.

Chương 3. Bước làm mềm mép Nyquist: khối FIR tùy chọn

▶ MỤC TIÊU HỌC TẬP

- Thấu hiểu bản chất vật lý và toán học của hiện tượng tràn búp phổ qua tần số Nyquist khi nhân tín hiệu thoại với cửa sổ thời gian hữu hạn, giải mã nguồn gốc dao động đổi dấu luân phiên $(-1)^\tau$ phá hủy đỉnh hàm tự tương quan $r(\tau)$.
- Nắm vững lịch sử thuật toán: đối chiếu giải pháp "làm mềm phổ mép Nyquist" (soft upsampling) trong bài báo Boersma (1993) với mã nguồn thực tế của Praat 7.0.03; làm rõ hiện trạng kỹ thuật rằng Praat hiện hành không thực hiện bước lọc này trong đường tự tương quan thô (`raw autocorrelation`).
- Khảo sát định lượng 4 phương án thiết kế bộ lọc FIR (Bypass, 31 tap, 63 tap, 127 tap) qua thuật toán bình phương tối thiểu `firls`; phân tích cạm bẫy kỹ thuật phổ biến khi nhầm lẫn giữa độ phân giải lượng tử hóa hệ số (16 bit ≈ 96 dB) và độ suy hao dải chặn bị khống chế bởi công thức Kaiser.
- Phân tích cặn kẽ khái niệm trễ nhóm (group delay) $\tau_g = (N-1)/2$ của bộ lọc FIR đối xứng Type I và quy tắc bắt buộc phải dời lưới khung phân tích 15 mẫu khi kích hoạt bộ lọc.
- Khảo cứu toàn diện các tham số của khối IP Intel FIR Compiler II theo tài liệu chuẩn *Intel FIR Compiler II User Guide* (UG-01072); nhận diện cạm bẫy thiết lập tốc độ mẫu `Input Sample Rate` trong hệ thống đọc tệp.
- Thiết kế và hiện thực hóa khối FIR 31 tap gấp thời gian (folded FIR) bằng Verilog-2001, tích hợp bộ tiền cộng (pre-adder) 17 bit và 1 bộ nhân DSP duy nhất chạy 16 chu kỳ/mẫu, tích hợp công tắc chuyển mạch `bypass` thời gian thực.
- Xây dựng mô hình song sinh Python bit-đối-bit và kịch bản kiểm thử chu kỳ với hơn 3.000 chu kỳ kích thích ngẫu nhiên, kiểm chứng đáp ứng xung, sóng sin 7,8 kHz và tín hiệu âm thanh thực tế.

3.1. Bản chất hiện tượng rò rỉ phổ mép Nyquist và dao động đổi dấu trong tự tương quan

Vấn đề thực tế: Một thành phần gần 8 kHz phá hỏng đỉnh tự tương quan như thế nào?

Trong hệ thống xử lý tín hiệu tiếng nói với tần số lấy mẫu $f_s = 16.000$ Hz, tần số giới hạn Nyquist là $f_{\text{Nyq}} = f_s/2 = 8.000$ Hz. Theo định lý lấy mẫu Nyquist–Shannon, mọi dao động có tần số thực sự nhỏ hơn 8 kHz đều được bảo toàn trọn vẹn và không bị hiện tượng chồng phổ (aliasing).

Tuy nhiên, trong bài toán ước lượng cao độ F_0, chúng ta không bao giờ xử lý một tín hiệu tuần hoàn vô hạn. Thuật toán bắt buộc phải cắt tín hiệu thành từng khung ngắn có độ dài hữu hạn thông qua một hàm cửa sổ thời gian $w[n]$ (cửa sổ Hanning 638 mẫu, thời lượng $T_w \approx 40$ ms). Về mặt toán học, phép nhân hai dãy tín hiệu trong miền thời gian tương đương với phép tích chập (convolution) giữa phổ của tín hiệu $X(f)$ và phổ của hàm cửa sổ $W(f)$ trong miền tần số liên tục:

$$X_w(f) = X(f) * W(f) = \int_{-f_{\text{Nyq}}}^{f_{\text{Nyq}}} X(\xi)\, W(f-\xi)\, d\xi \tag{179}$$

Cửa sổ Hanning có độ rộng búp phổ chính (main lobe) tỷ lệ nghịch với độ dài cửa sổ: $\Delta f \approx 4/T_w = 4/0{,}04\,\text{s} = 100$ Hz, kèm theo chuỗi các búp sóng phụ (side lobes) suy giảm với tốc độ 18 dB/octave. Giả sử trong giọng nói xuất hiện một thành phần năng lượng cao tần nằm sát mép phổ, chẳng hạn ở tần số $7{,}8$ kHz (họa âm bậc cao của âm thanh thanh môn, phụ âm ma sát vô thanh $/s/, /\int/$, hoặc nhiễu môi trường). Do độ rộng của búp phổ cửa sổ, năng lượng tại $7{,}8$ kHz sau tích chập sẽ bị loang rộng sang hai phía. Búp phổ phía trên sẽ tràn vượt qua ranh giới Nyquist $8{,}0$ kHz, phản xạ ngược vào dải tần số $[0, 8\text{ kHz}]$ và gây ra hiện tượng rò rỉ phổ (spectral leakage).

Dẫn xuất toán học: Nguồn gốc sóng dao động đổi dấu luân phiên $(-1)^\tau$

Hậu quả nghiêm trọng nhất của hiện tượng rò rỉ mép Nyquist không nằm ở việc méo dạng biên độ, mà nằm ở tác động phá hủy cấu trúc của hàm tự tương quan $r(\tau)$.

Hãy xét một dao động hình sin đơn tần nằm đúng tại tần số Nyquist: $f = f_{\text{Nyq}} = 8.000$ Hz. Chu kỳ thời gian của dao động này là $T_0 = 1/8000$ s $= 125$ μs, tương ứng với đúng 2 khoảng lấy mẫu rời rạc ($T_0/T_s = 2$ mẫu). Khi được lấy mẫu ở các thời điểm rời rạc n, giá trị của tín hiệu có dạng:

$$x[n] = A\cos\left(2\pi \frac{f_{\text{Nyq}}}{f_s} n + \phi_0\right) = A\cos(\pi n + \phi_0) \tag{180}$$

Nếu chọn pha ban đầu $\phi_0 = 0$, tín hiệu này trở thành một chuỗi xung luân phiên đổi dấu giữa hai giá trị cực đại:

$$x[n] = A \cdot (-1)^n \quad \Longrightarrow \quad x = [A, -A, A, -A, A, -A, \dots] \tag{181}$$

Khi tính toán hàm tự tương quan thô $R[\tau] = \sum_{n=0}^{N-1-\tau} x[n]x[n+\tau]$ cho độ trễ τ:

$$x[n] \cdot x[n+\tau] = \big(A(-1)^n\big) \cdot \big(A(-1)^{n+\tau}\big) = A^2(-1)^{2n+\tau} = A^2\big((-1)^2\big)^n(-1)^\tau = A^2(-1)^\tau \tag{182}$$

Bởi vì $(-1)^{2n} = 1$ với mọi số nguyên n, tích của hai mẫu cách nhau τ bước hoàn toàn độc lập với vị trí thời gian n, và chỉ phụ thuộc duy nhất vào tính chẵn lẻ của độ trễ τ:

$$R[\tau] = \sum_{n=0}^{N-1-\tau} A^2(-1)^\tau = (N-\tau)A^2 \cdot (-1)^\tau = \begin{cases} +(N-\tau)A^2 & \text{khi } \tau \text{ là số chẵn} \\ -(N-\tau)A^2 & \text{khi } \tau \text{ là số lẻ} \end{cases} \tag{183}$$

Biểu thức trên chỉ ra một sự thật vật lý cốt lõi: **Mọi thành phần tần số nằm sát mép Nyquist đều tạo ra một sóng dao động đổi dấu liên tục dạng $(-1)^\tau$ trong miền độ trễ τ.**

Tính tay trên một chuỗi cụ thể

Để thấy rõ hiện tượng trên bằng mắt, hãy lấy một đoạn tín hiệu ngắn $N = 4$ mẫu đổi dấu tuần hoàn với biên độ $A = 2$: $x = [2, -2, 2, -2]$. Ta tính lần lượt các giá trị tự tương quan $R[\tau]$ từ $\tau = 0$ đến $\tau = 3$:

- Với $\tau = 0$: $R[0] = 2^2 + (-2)^2 + 2^2 + (-2)^2 = 4 + 4 + 4 + 4 =$ **+16**.
- Với $\tau = 1$: $R[1] = 2(-2) + (-2)(2) + 2(-2) = -4 - 4 - 4 =$ **-12**.
- Với $\tau = 2$: $R[2] = 2(2) + (-2)(-2) = 4 + 4 =$ **+8**.
- Với $\tau = 3$: $R[3] = 2(-2) =$ **-4**.

Dãy kết quả thu được là $R = [+16, -12, +8, -4]$: biên độ dao động xen kẽ dấu dương–âm với bước nhảy giữa hai lag kế tiếp lên tới 28 đơn vị!

Trong tín hiệu tiếng nói thực tế, tín hiệu dây thanh đới tạo ra một vòm tự tương quan trơn nhẵn với đỉnh cực đại nằm tại chu kỳ cơ bản $\tau_0 \in [27, 213]$ (ví dụ $\tau_0 = 100$ mẫu tương ứng $F_0 = 160$ Hz). Khi sóng đổi dấu $(-1)^\tau$ của thành phần cao tần chồng lấn lên vòm tự tương quan này, nó tạo thành một dải gai nhọn răng cưa (jagged teeth) biến thiên cực nhanh theo từng bước nhảy $\Delta\tau = 1$.

Khi thuật toán parabol interpolation ở Chương 9 khớp đường cong bậc hai qua ba điểm lân cận $(R[\tau-1], R[\tau], R[\tau+1])$:

$$\delta = \frac{R[\tau-1] - R[\tau+1]}{2(2R[\tau] - R[\tau-1] - R[\tau+1])} \tag{184}$$

Nếu $R[\tau]$ rơi vào lag chẵn, nó bị đội lên một lượng $+A^2$; trong khi hai điểm bên cạnh $\tau \pm 1$ là lag lẻ nên bị kéo tụt xuống một lượng $-A^2$. Mẫu số parabol bị phóng đại nhân tạo, làm đỉnh bị bóp nhọn bất thường. Ngược lại, nếu đỉnh thật rơi vào một lag lẻ, giá trị $R[\tau]$ bị triệt tiêu âm, biến đỉnh cực đại thành một điểm uốn hoặc một thung lũng cục bộ, khiến thuật toán bỏ sót hoàn toàn ứng viên cao độ!

Đề xuất của Boersma (1993) và hiện trạng trong mã nguồn Praat hiện hành

Để khắc phục hiện tượng răng cưa mép Nyquist nói trên, Paul Boersma trong công trình nền tảng năm 1993 (*Accurate short-term analysis of the fundamental frequency and the harmonics-to-noise ratio of a sampled sound*, Proceedings of the Institute of Phonetic Sciences, Amsterdam, Tập 17, Mục 3.1) đã đề xuất quy trình mang tên "làm mềm phổ" (soft upsampling / spectral smoothing):

> "Trước khi phân tích tự tương quan, phổ của tín hiệu âm thanh được làm mềm tuyến tính từ mức $1{,}0$ tại 95% tần số Nyquist ($7{,}6$ kHz) dốc đều về 0 tại 100% tần số Nyquist ($8{,}0$ kHz) thông qua biến đổi Fourier nhanh FFT."

Mục tiêu của hàm dốc 5% dải tần này là làm suy giảm năng lượng ở sát biên 8 kHz trước khi thực hiện tích chập với cửa sổ Hanning, từ đó hạn chế hiện tượng tràn phổ và làm dịu sóng đổi dấu $(-1)^\tau$.

Hiện trạng trong mã nguồn Praat thực tế: Khi tiến hành rà soát từng dòng mã nguồn chính thức của Praat các phiên bản 6.4.12 và 7.0.03 (tệp `fon/Sound_to_Pitch.cpp`, hàm `Sound_to_Pitch_any`, dòng 274–497, gọi từ `Sound_to_Pitch_rawAc`, dòng 504–515, bản 7.0.03), một sự kiện kỹ thuật được ghi nhận:

Praat 7.0.03 không thực hiện bất kỳ bước upsampling, resample hay lọc trước làm mềm phổ nào trong luồng xử lý tự tương quan thô (`To Pitch (raw autocorrelation)...`)!

Bước tiền xử lý làm mềm phổ từng được mô tả trong bài báo Boersma (1993) không xuất hiện trong luồng xử lý tự tương quan thô của mã nguồn Praat hiện hành (`fon/Sound_to_Pitch.cpp`, hàm `Sound_to_Pitch_any`, dòng 274–497, gọi từ `Sound_to_Pitch_rawAc`, dòng 504–515, bản 7.0.03). Cần lưu ý rằng lý do vì sao bước xử lý này không có mặt trong mã nguồn không được ghi chép trong tài liệu kỹ thuật hay chú thích mã nguồn của Praat. Chúng tôi ghi nhận sự thật đối soát này mà không đưa ra các giả định chưa có căn cứ.

Hệ quả đối soát và quyết định thiết kế trên FPGA

Sự sai khác giữa bài báo lý thuyết năm 1993 và mã nguồn thực tế dẫn đến một hệ quả kỹ thuật trực tiếp:

- Nếu hệ thống FPGA mặc định kích hoạt một khối lọc FIR làm mềm mép Nyquist trong khi phần mềm Praat chuẩn trên máy tính không bật, chúng ta đang **tự tạo ra một nguồn sai lệch nhân tạo** giữa hai hệ thống. Bộ lọc FIR sẽ làm thay đổi biên độ, làm trôi pha và làm lệch nhẹ các đỉnh tự tương quan của FPGA so với Praat, khiến việc đối soát trở nên bất bình đẳng (khác biệt về thuật toán chứ không phải sai số phần cứng).
- Tuy nhiên, việc trang bị khối lọc FIR làm mềm mép Nyquist lại mang giá trị nghiên cứu kỹ thuật to lớn: nó cho phép kỹ sư đo đạc định lượng xem việc triệt tiêu dải tần sát Nyquist có thực sự cải thiện độ chính xác F_0 trên các đoạn âm ma sát hay không, và đánh giá chi phí phần cứng của một khối lọc số tiêu chuẩn công nghiệp trên FPGA.

Quyết định thiết kế (Đã chốt): Hệ thống trên FPGA sẽ hiện thực một khối lọc FIR 31 tap tùy chọn, tích hợp sẵn đường rẽ nhánh **`bypass`**.

- Khi chạy đối soát chuẩn với Praat 7.0.03: khối FIR được **mặc định cấu hình bypass** (`bypass = 1`). Dữ liệu âm thanh từ FIFO đi thẳng vào bộ nhớ RAM chia khung mà không qua lọc, đảm bảo đầu vào của FPGA và Praat giống nhau từng bit.
- Khi cần nghiên cứu thực nghiệm: người dùng bật cờ `fir_bypass = 0` thông qua thanh ghi điều khiển `ctrl` từ HPS. Lúc này khối FIR 31 tap hoạt động, và hệ thống bắt buộc phải dời lưới định vị khung thời gian lùi lại đúng 15 mẫu để bù trừ trễ nhóm.

Tóm lại: Thành phần gần Nyquist sinh ra sóng đổi dấu $(-1)^\tau$ phá hủy đỉnh tự tương quan; Boersma 1993 đề xuất làm mềm phổ nhưng Praat 7.0.03 đã bỏ bước này; vì vậy khối FIR 31 tap trên FPGA được thiết kế kèm công tắc bypass và mặc định tắt khi đối soát.

3.2. Các phương án, đánh đổi, ta chọn gì, vì sao (có số liệu kiểm chứng)

Vấn đề thực tế: Bài toán đánh đổi giữa độ suy hao, độ trễ và tài nguyên phần cứng

Khi bắt tay vào hiện thực hóa một bộ lọc số FIR (Finite Impulse Response) trên vi mạch FPGA, câu hỏi đầu tiên đặt ra cho người kỹ sư là: *Cần bao nhiêu tap lọc là đủ, và cái giá phải trả về độ trễ xử lý cũng như tài nguyên phần cứng là gì?*

Một bộ lọc lý tưởng theo đề xuất của Boersma (1993) đòi hỏi đáp ứng biên độ hoàn toàn phẳng (0 dB) từ tần số 0 Hz tới $7{,}6$ kHz (tức 95% Nyquist), sau đó đổ dốc thẳng đứng về 0 tại đúng $8{,}0$ kHz (mép Nyquist). Tuy nhiên, nguyên lý bất định trong xử lý tín hiệu số chỉ rõ: một đáp ứng dốc đứng trong miền tần số đòi hỏi một đáp ứng xung dài vô hạn trong miền thời gian ($N \to \infty$). Khi cắt ngắn đáp ứng xung thành N điểm hữu hạn, hiện tượng gợn sóng Gibbs và độ rộng dải chuyển tiếp hữu hạn $\Delta f = 8000 - 7600 = 400$ Hz sẽ quyết định trực tiếp chất lượng của bộ lọc.

Để có cơ sở định lượng chính xác, chúng tôi sử dụng thuật toán thiết kế bộ lọc bình phương tối thiểu `scipy.signal.firls` với mục tiêu đáp ứng biên độ bằng $1{,}0$ trên dải $[0, 7{,}6\,\text{kHz}]$ và dốc tuyến tính về 0 tại $8{,}0$ kHz. Toàn bộ các thông số kỹ thuật được tính toán và kiểm chứng độc lập thông qua kịch bản kiểm thử `verify_v3/ch03_fir_design.py`:

Bảng 17: So sánh định lượng các phương án bộ lọc FIR làm mềm mép Nyquist tại $f_s = 16$ kHz

Phương án	Số tap (N)	Trễ τ_g	Tại 7,4 kHz	Tại 7,6 kHz	Tại 8,0 kHz	Nhận xét kỹ thuật
(a) Bypass	0	0 mẫu	0,00 dB	0,00 dB	0,00 dB	Khớp Praat hiện hành; 0 tài nguyên
(b) 31 tap	31	15 mẫu	$+0{,}19$ dB	$-2{,}29$ dB	$-9{,}41$ dB	Dải thông phẳng $\pm 0{,}25$ dB tới 7,4 kHz
(c) 63 tap	63	31 mẫu	$+0{,}03$ dB	$-0{,}38$ dB	$-19{,}54$ dB	Suy hao dải biên tăng thêm ≈ 10 dB
(d) 127 tap	127	63 mẫu	$-0{,}08$ dB	$-0{,}25$ dB	$-24{,}20$ dB	Độ trễ tăng gấp 4 lần; tốn tài nguyên

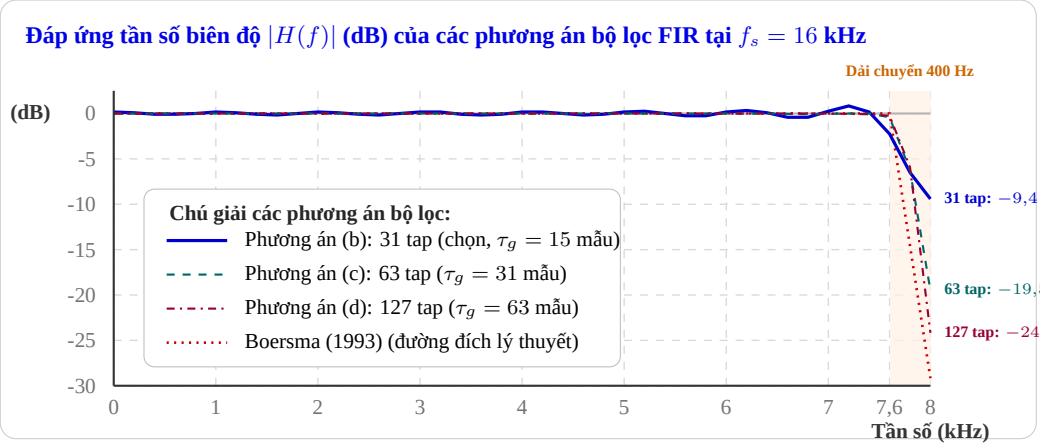
Độ lợi một chiều (DC gain) của các hệ số lượng tử hóa. Từ kết quả thiết kế và lượng tử hóa trong nhật ký kiểm thử `verify_v3/ch03_fir_design.log`, tổng của 31 hệ số sau khi lượng tử hóa sang định dạng số nguyên Q1.15 là:

$$\sum_{i=0}^{30} h_{\text{Q1.15}}[i] = 33.257 \tag{185}$$

Với hệ số tỷ lệ cơ sở $2^{15} = 32.768$, độ lợi một chiều (DC gain) thực tế của bộ lọc sau lượng tử hóa là:

$$G_{\text{DC}} = \frac{33257}{32768} \approx 1{,}01492 \quad (\approx +0{,}128 \text{ dB}) \tag{186}$$

Độ lợi này hơi lớn hơn 1,0 một lượng nhỏ ($+1{,}49\%$). Trong thiết kế mạch, chúng tôi **không thực hiện chuẩn hóa lại** (không chia thêm hệ số tỷ lệ) vì mức gợn $+0{,}13$ dB hoàn toàn nằm trong dung sai $\pm 0{,}25$ dB cho phép của dải thông tiếng nói, đồng thời việc giữ nguyên các giá trị nguyên giúp kiến trúc phần cứng không phát sinh thêm phép chia hay sai số cắt xén phụ. Tuy nhiên, hệ quả trực tiếp của độ lợi $G_{\text{DC}} = 1{,}0149$ là khi tín hiệu vào đạt giá trị DC cực đại $x[n] = 32767$, giá trị tích lũy làm tròn sẽ đạt $33256 > 32767$. Do đó, khâu bão hòa (clamping/saturation) ở đầu ra là **bắt buộc** để ghim biên độ về đúng $+32767$, ngăn chặn hoàn toàn hiện tượng lật dấu bit bù hai.



Hình 5: Đáp ứng tần số biên độ $|H(f)|$ (dB) của các phương án bộ lọc FIR so với đường cong mục tiêu Boersma (1993) trên dải chuyển tiếp 400 Hz (trích xuất từ `verify_v3/ch03_fir_design.py`).

Đọc hình. Hình 5 trực quan hóa sự đánh đổi giữa bậc lọc N và độ suy hao tại tần số Nyquist. Đường màu xanh đậm biểu diễn phương án (b) 31 tap: trong toàn bộ dải thoại hữu ích từ 0 đến 7,4 kHz, độ gợn biên độ được khống chế chặt chẽ trong khoảng $[-0{,}25 \text{ dB}, +0{,}25 \text{ dB}]$ (tại 0 kHz gợn $+0{,}13$ dB, tại 7,4 kHz gợn $+0{,}19$ dB), hầu như không làm biến dạng năng lượng của các formant giọng nói. Khi bước vào dải chuyển tiếp $[7{,}6 \text{ kHz}, 8{,}0 \text{ kHz}]$ (vùng tô màu cam rộng 400 Hz), đáp ứng bắt đầu rơi dốc: đạt $-2{,}29$ dB tại 7,6 kHz và cán mốc suy hao $-9{,}41$ dB tại đúng tần số Nyquist 8,0 kHz. Đường nét đứt màu xanh ngọc (63 tap) và nét chấm gạch màu tím (127 tap) giúp tăng độ suy hao biên lên lần lượt $-19{,}5$ dB và $-24{,}2$ dB, nhưng đòi hỏi chiều dài thanh ghi dịch và độ trễ tăng vọt.

Bẫy thường gặp: Nhầm lẫn giữa độ rộng bit hệ số và độ suy hao dải chặn

Trong quá trình thiết kế bộ lọc số, một ngộ nhận thường gặp là:

> *"Thiết kế bộ lọc FIR 31 tap với hệ số lượng tử 16 bit cho phép đạt độ suy hao dải chặn lớn hơn 60 dB."*

Quan niệm này không chính xác về mặt lý thuyết xử lý tín hiệu số, xuất phát từ sự nhầm lẫn giữa hai khái niệm hoàn toàn độc lập:

1. **Độ rộng bit lượng tử hóa hệ số ($B = 16$ bit):** Tham số này chỉ quyết định *sàn nhiễu lượng tử hóa (quantization noise floor)* do sai số làm tròn số học gây ra. Theo quy tắc xấp xỉ $6{,}02$ dB/bit, lượng tử hóa 16 bit có dấu cho tỷ số tín hiệu trên nhiễu lượng tử (SQNR) lý thuyết là:

$$\text{SQNR} \approx 6{,}02 \times 16 + 1{,}76 \approx 98{,}08 \text{ dB} \tag{187}$$

   Độ rộng hệ số 16 bit đảm bảo rằng khi chuyển từ số thực dấu phẩy động sang số nguyên Q1.15, đáp ứng tần số thực tế của bộ lọc bám sát đáp ứng lý thuyết với sai số cực đại nhỏ hơn $0{,}001$ dB (đã kiểm chứng trong `verify_v3/ch03_fir_design.log`). Tuy nhiên, độ rộng bit hệ số **không tự động tạo ra độ suy hao dải chặn**!

2. **Độ suy hao dải chặn (A_s) và bề rộng dải chuyển tiếp (Δf):** Độ suy hao dải chặn hoàn toàn bị chi phối bởi số lượng tap N và tỷ lệ dải chuyển tiếp $\Delta f / f_s$. Mối quan hệ này được chuẩn hóa qua công thức ước lượng Kaiser:

$$N \approx \frac{A_s - 7{,}95}{14{,}36 \cdot \left(\frac{\Delta f}{f_s}\right)} \tag{188}$$

Hãy áp dụng công thức Kaiser vào bài toán làm mềm mép Nyquist với dải chuyển tiếp chỉ rộng $\Delta f = 8000 - 7600 = 400$ Hz ở tần số lấy mẫu $f_s = 16.000$ Hz ($\Delta f / f_s = 400/16000 = 0{,}025$): Để đạt được độ suy hao dải chặn mục tiêu $A_s = 60$ dB, số bậc lọc tối thiểu phải là:

$$N \approx \frac{60 - 7{,}95}{14{,}36 \times 0{,}025} = \frac{52{,}05}{0{,}359} \approx \textbf{144,99 tap} \quad \Longrightarrow \quad N \approx 147 \text{ tap} \tag{189}$$

(Hàm tối ưu hóa `scipy.signal.kaiserord(60, 400/8000)` trả về chính xác $N = 147$ tap với hệ số cửa sổ $\beta = 5{,}653$).

Như vậy, muốn đạt độ suy hao 60 dB trong khoảng chuyển tiếp hẹp 400 Hz, ta cần ít nhất 147 tap! Với chiều dài vỏn vẹn $N = 31$ tap, việc đòi hỏi độ suy hao 60 dB trong dải chuyển tiếp 400 Hz là điều bất khả thi về mặt vật lý. Bộ lọc 31 tap chỉ có thể tạo ra độ suy hao $-9{,}41$ dB tại tần số 8 kHz.

Trễ nhóm (Group Delay) và quy tắc dịch chuyển lưới khung phân tích

Một đặc tính cốt lõi của bộ lọc FIR đối xứng Type I ($N = 31$ là số lẻ, $h[n] = h[N-1-n]$) là có pha tuyến tính nghiêm ngặt. Độ trễ nhóm (group delay) là đạo hàm của góc pha theo tần số:

$$\tau_g = -\frac{d\theta(\omega)}{d\omega} = \frac{N-1}{2} \text{ mẫu} \tag{190}$$

Với bộ lọc $N = 31$ tap, độ trễ nhóm là một hằng số cố định trên mọi tần số:

$$\tau_g = \frac{31-1}{2} = \textbf{15 mẫu} \quad \left(= 15 \times \frac{1}{16000} \text{ s} = 0{,}9375 \text{ ms}\right) \tag{191}$$

Cạm bẫy dời lưới khung: Giả sử khung phân tích thứ k của thuật toán bắt đầu từ mẫu âm thanh $s_{\text{start}}[k]$. Nếu tín hiệu đi qua bộ lọc FIR 31 tap, mỗi mẫu âm thanh đi ra khỏi bộ lọc đã bị trễ thời gian đúng 15 chu kỳ lấy mẫu. Nếu hệ thống FPGA giữ nguyên chỉ số mẫu bắt đầu $s_{\text{start}}[k]$ mà nạp vào bộ nhớ RAM chia khung, toàn bộ 638 mẫu trong cửa sổ phân tích thực chất là các mẫu của đoạn âm thanh xảy ra trước đó 15 mẫu ($0{,}9375$ ms) trong tín hiệu gốc! Sự lệch pha thời gian này sẽ làm dịch chuyển vị trí các đỉnh tự tương quan, làm sai lệch giá trị F_0 nội suy và gây ra hiện tượng không khớp ứng viên với kết quả Praat chạy trên tệp WAV gốc.

Quy tắc bù trễ bắt buộc: Khi bật bộ lọc FIR 31 tap (`bypass = 0`), bộ điều khiển phân khung trên HPS hoặc FPGA bắt buộc phải **dời lưới khung lùi lại đúng 15 mẫu**:

$$s_{\text{start, FIR}}[k] = s_{\text{start}}[k] - 15 \tag{192}$$

Nhờ việc định vị khung sớm hơn 15 mẫu, cửa sổ 638 mẫu sau khi đi qua trễ nhóm của bộ lọc FIR sẽ rơi vào đúng vị trí thời gian của khung chưa lọc, bảo toàn tính tương đồng về mặt vật lý.

Lựa chọn thiết kế và các lý do quyết định

Dựa trên phân tích định lượng ở Bảng 17, chúng tôi lựa chọn hiện thực hóa **Phương án (b) 31 tap kèm mạch rẽ nhánh bypass** với các lý do kỹ thuật sau:

1. **Độ suy hao vừa đủ và dải thông bảo toàn:** Độ suy hao $-9{,}41$ dB tại mép 8 kHz giúp giảm biên độ năng lượng cao tần xuống còn $\approx 33{,}8\%$ (tương đương giảm công suất dao động đổi dấu $(-1)^\tau$ xuống còn $11{,}5\%$), đủ để làm dịu hiện tượng răng cưa mép Nyquist mà vẫn giữ dải thông thoại $[0, 7{,}4$ kHz$]$ phẳng trong phạm vi $\pm 0{,}25$ dB.
2. **Độ trễ nhóm nhỏ và dễ bù trừ:** Trễ nhóm 15 mẫu chỉ chiếm $9{,}375\%$ của bước nhảy khung (hop 160 mẫu), hoàn toàn nằm gọn trong dung lượng đệm của bộ nhớ RAM vòng 1024 từ ở Chương 4 mà không đòi hỏi thêm tài nguyên FIFO phụ. Trong khi đó, phương án 127 tap có trễ nhóm lên tới 63 mẫu ($\approx 40\%$ bước nhảy hop), làm tăng đáng kể thời gian đáp ứng thời gian thực.
3. **Tối ưu hóa tài nguyên phần cứng:** Nhờ tính đối xứng chẵn, 31 tap lọc được gộp thành 15 cặp cộng trước và 1 hệ số trung tâm. Trong mục 3.4, chúng tôi sẽ chứng minh cấu trúc này chỉ tiêu tốn đúng **1 khối DSP** và **0 khối RAM M10K** trên chip Cyclone V khi áp dụng kỹ thuật gấp thời gian 16 chu kỳ/mẫu.
4. **Tương thích kép nhờ mạch bypass:** Việc tích hợp công tắc bypass cho phép đáp ứng cả hai mục tiêu: mặc định bypass để đối soát chuẩn mực từng bit với Praat 7.0.03, và bật lọc khi cần đánh giá ảnh hưởng của hiện tượng mép Nyquist trên thực nghiệm.

Tóm lại: Bộ lọc 31 tap tạo suy hao $-9{,}4$ dB tại Nyquist với trễ nhóm 15 mẫu; muốn suy hao 60 dB trong dải chuyển 400 Hz cần tới 147 tap theo công thức Kaiser; khi bật lọc bắt buộc phải dời lưới khung 15 mẫu.

3.3. Đường (A): Khối IP FIR Compiler II của Intel

Vấn đề thực tế: Sự phức tạp của các tham số IP chuẩn công nghiệp

Khi xây dựng một hệ thống xử lý tín hiệu số trên FPGA Intel, kỹ sư thường ưu tiên sử dụng khối IP có sẵn trong thư viện Intel Quartus Prime mang tên **FIR Compiler II** (được đặc tả chi tiết trong tài liệu kỹ thuật *Intel FIR Compiler II User Guide*, Mã tài liệu: UG-01072, ấn bản 683111). Khối IP này cung cấp một giao diện đồ họa cấu hình (Parameter Editor) với hàng chục tùy chọn kiến trúc tinh vi.

Tuy nhiên, khối IP là một hộp đen phần cứng: nếu kỹ sư tích chọn các tham số mà không hiểu rõ nguyên lý toán học và cơ chế phân bổ tài nguyên silicon bên dưới, hệ thống sẽ gặp các sự cố nghiêm trọng:

- Nếu chọn sai tham số tốc độ lấy mẫu đầu vào (`Input Sample Rate`), IP sẽ tự động tối ưu hóa phần cứng dựa trên giả định sai, dẫn đến việc hàng đợi bên trong bị tràn và làm rơi rụng các mẫu âm thanh khi chạy ở chế độ tệp.
- Nếu chọn sai chế độ làm tròn bit thấp (`Output LSB Rounding`), tích lũy sai số cắt xén (truncation drift) sẽ tạo ra một điện áp trôi DC nhân tạo, làm biến dạng phép trừ trung bình ở Chương 5.
- Nếu không nắm vững thông số `readyLatency` do IP sinh ra, sự bất đồng tín hiệu điều khiển trên bus Avalon-ST sẽ gây mất mát mẫu dữ liệu như đã phân tích ở Chương 2.

Dưới đây, chúng tôi giải mã chi tiết từng tham số cấu hình của khối IP FIR Compiler II cho bài toán lọc làm mềm mép Nyquist 31 tap.

Giải thích cặn kẽ từng tham số trong Parameter Editor theo tài liệu UG-01072

Mọi tham số cấu hình, dải giá trị cho phép và quy ước cổng của khối IP Intel FIR Compiler II dưới đây đều được đối chiếu chặt chẽ theo tài liệu kỹ thuật *Intel FIR Compiler II User Guide* (Mã tài liệu: UG-01072, ấn bản 683111):

1. Filter Type = Single Rate (UG-01072, Mục 3.1 “Filter Options”). Tham số này xác định quan hệ giữa tốc độ lấy mẫu đầu vào và đầu ra. Dải giá trị cho phép của IP bao gồm: `Single Rate`, `Interpolation`, `Decimation`, và `Fractional Rate` (UG-01072, Mục 3.1). Chúng tôi chọn `Single Rate` vì hệ thống thực hiện lọc phẳng dải thông mà không thay đổi tần số lấy mẫu ($f_s = 16$ kHz ở cả hai đầu). Nếu chọn `Interpolation` hoặc `Decimation`, IP sẽ thay đổi số lượng mẫu, làm sai lệch cấu trúc khung 638 mẫu của thuật toán Praat.

2. Symmetry = Symmetrical (UG-01072, Mục 3.1 "Filter Options"). Dải tùy chọn cấu trúc đối xứng hệ số gồm: `Non-symmetrical`, `Symmetrical`, và `Anti-symmetrical` (UG-01072, Mục 3.1). Với 31 tap có $h[k] = h[30-k]$ ($k = 0..14$), việc chọn `Symmetrical` cho phép IP tận dụng khối phần cứng **Pre-adder (Bộ tiền cộng 18-bit)** tích hợp sẵn trong các khối DSP của chip Cyclone V (*Cyclone V Device Handbook*, Tập 1, Chương 5). Nhờ cộng trước hai mẫu đối xứng $x[n-k] + x[n-(30-k)]$, số phép nhân số học giảm đúng một nửa: từ 31 xuống còn $15 + 1 = 16$ phép nhân, tiết kiệm 50% tài nguyên bộ nhân DSP trên FPGA.

3. Clock Frequency = 50.0 MHz (UG-01072, Mục 3.1 "Filter Options"). Tham số xác định tần số xung nhịp chủ cấp cho khối IP (dải giá trị: số thực dương trong phạm vi Fmax của linh kiện; ở đây chọn 50.0 MHz từ dao động kế bo mạch DE10-Standard, chu kỳ $T_{\text{clk}} = 20$ ns).

4. Input Sample Rate = 0.5 MSPS (UG-01072, Mục 3.1 "Filter Options"). Dải giá trị: số thực dương $\leq$ Clock Frequency. Đây là tham số kỹ thuật tiềm ẩn nhiều rủi ro nhất khi cấu hình IP trong hệ thống xử lý tệp:

- *Cạm bẫy thiết lập $0{,}016$ MSPS (16 kHz):* Rất nhiều người lập trình theo quán tính sẽ điền vào ô này giá trị $0{,}016$ MSPS vì tín hiệu âm thanh có tần số lấy mẫu là 16 kHz. Tuy nhiên, như đã phân tích sâu sắc ở Chương 2, hệ thống ước lượng cao độ của chúng ta đọc dữ liệu từ tệp WAV trong bộ nhớ RAM thông qua bộ đệm FIFO (file-driven processing), hoàn toàn không bị trói buộc bởi nhịp lấy mẫu 16 kHz của micro ngoại vi! Khi bộ nhớ FIFO có sẵn dữ liệu và khối lọc sẵn sàng, các mẫu âm thanh có thể được chuyển giao với tốc độ hàng triệu mẫu/giây. Nếu ta khai báo $0{,}016$ MSPS, khối IP FIR Compiler II sẽ tính toán tỷ lệ chia sẻ thời gian nội bộ:
$$N_{\text{cycles/sample}} = \frac{f_{\text{clk}}}{f_{\text{sample}}} = \frac{50.000.000 \text{ Hz}}{16.000 \text{ Hz}} = 3125 \text{ chu kỳ/mẫu} \tag{193}$$
Dựa trên con số 3125 chu kỳ này, bộ sinh IP sẽ cấu hình một kiến trúc dồn kênh thời gian (time-multiplexed architecture) với tốc độ tiếp nhận chu kỳ dài. Khi dòng dữ liệu từ FIFO tuôn vào với tốc độ cao, hàng đợi đầu vào của IP sẽ bị tràn lập tức, gây mất mát mẫu âm thanh mà không phát sinh tín hiệu cảnh báo nào!
- *Giải pháp đúng đắn:* Thiết lập `Input Sample Rate` bằng tốc độ tiếp nhận tối đa thực tế của hệ thống, ví dụ chọn $0{,}5$ MSPS (tương ứng với 50 MHz$/0{,}5$ MSPS $= 100$ chu kỳ/mẫu) hoặc $1{,}0$ MSPS (50 chu kỳ/mẫu). Cấu hình này ép buộc IP phải bố trí đủ phần cứng để có thể hoàn tất việc tiếp nhận và xử lý một mẫu mới trong vòng 100 chu kỳ xung nhịp, hoàn toàn đáp ứng thông lượng dòng dữ liệu và tương thích với nhịp độ xử lý của khối MAC phía sau (≈ 616 chu kỳ/mẫu).

5. Coefficient Width = 16 bit, Coefficient Scaling = Auto/Normalized (UG-01072, Mục 3.2 "Coefficient Options"). Độ rộng bit hệ số lọc nhận giá trị từ 1 đến 32 bit có dấu (*signed integer*). Tùy chọn `Coefficient Scaling` gồm: `Auto`, `Normalized`, và `None` (UG-01072, Mục 3.2). Chọn `Auto/Normalized` để tự động nhân đáp ứng xung với hệ số tỷ lệ $2^{15} = 32768$ (định dạng Q1.15), đưa hệ số trung tâm $h[15] = 0{,}975000$ lên giá trị nguyên 31949 (`0x7CCD`).

6. Input Data Type = Signed Binary, Width = 16 bit, Fractional Bits = 15 (UG-01072, Mục 3.3 "Input and Output Options"). Dải tùy chọn kiểu dữ liệu đầu vào gồm: `Signed Binary` và `Unsigned Binary`; độ rộng bit từ 1 đến 32 bit; số bit phần thập phân từ 0 đến 32 bit (UG-01072, Mục 3.3). Định dạng này khớp chính xác mẫu âm thanh PCM 16-bit có dấu đọc từ tệp WAV (khoảng $[-32768, 32767]$).

7. Output MSB/LSB Rounding và Output Bit Width (UG-01072, Mục 3.3 "Input and Output Options"). Trong quá trình tích chập, mỗi phép nhân giữa mẫu 16-bit và hệ số 16-bit sinh tích 32-bit (với bộ tiền cộng 17-bit thì tích là 33-bit). Khi cộng dồn 31 tích (mở rộng $\lceil \log_2(31) \rceil = 5$ bit), giá trị tích lũy nội bộ đầy đủ có thể đạt tới 38 bit; thông số độ rộng bit đầy đủ này do GUI tự động tính toán và **phải kiểm trong GUI/IP Generation Report**. Để ngõ ra đưa vào bộ nhớ RAM chia khung ở Chương 4 giữ nguyên định dạng mẫu âm thanh 16-bit có dấu:

- `Output Bit Width` ấn định 16 bit signed (dải cho phép: 1 đến 64 bit).
- `Output LSB Rounding`: Dải tùy chọn gồm `Truncation`, `Nearest Rounding`, và `Convergent Rounding` (UG-01072, Mục 3.3). Ta chọn `Convergent Rounding` hoặc `Nearest` để cộng nửa trọng số LSB ($2^{14} = 16384$) trước khi dịch phải 15 bit, triệt tiêu sai số trôi DC so với `Truncation`.

- `Output MSB Rounding`: Dải tùy chọn gồm `Truncation` và `Saturation` (UG-01072, Mục 3.3). Ta chọn `Saturation` để khi biên độ vượt quá $+32767$ hoặc nhỏ hơn -32768, bộ bão hòa sẽ ghim giá trị tại ranh giới thay vì cuộn tràn bit dấu bù hai.

8. Back-pressure Support = Yes và quy chuẩn readyLatency (UG-01072, Mục 4.1). Kích hoạt giao thức bắt tay hai chiều Avalon-ST qua cặp tín hiệu `ready` và `valid`. Tài liệu *Intel FIR Compiler II User Guide* (UG-01072, Mục 4.1 "Avalon-ST Interfaces in FIR Compiler II", bảng "Avalon-ST Sink and Source Interface Signals") quy định rõ: giao diện Avalon-ST của FIR Compiler II hoạt động với **`readyLatency = 0`** (khi `ready` và `valid` đồng thời tích cực mức cao tại cùng một chu kỳ xung nhịp thì dữ liệu được chuyển giao ngay tại chu kỳ đó).

Những gì khối IP sinh ra và giao diện ghép nối phần cứng (UG-01072, Mục 4.1 & 4.2)

Sau khi sinh mã trong Platform Designer hoặc IP Catalog, khối IP FIR Compiler II cung cấp các cổng kết nối chuẩn (UG-01072, Chương 4):

- **Cổng điều khiển (UG-01072, Mục 4.2):** xung nhịp `clk` (50 MHz) và tín hiệu xóa tích cực thấp `reset_n`.
- **Giao diện nhận Avalon-ST Sink (UG-01072, Mục 4.1):** bus dữ liệu `ast_sink_data[15:0]`, cờ hợp lệ `ast_sink_valid`, cờ sẵn sàng `ast_sink_ready`, và cờ báo lỗi `ast_sink_error[1:0]`. Chuẩn `readyLatency = 0`.
- **Giao diện phát Avalon-ST Source (UG-01072, Mục 4.1):** bus `ast_source_data[15:0]`, cờ hợp lệ `ast_source_valid`, tín hiệu nhận `ast_source_ready`, và cờ lỗi `ast_source_error[1:0]`. Chuẩn `readyLatency = 0`.
- **Độ trễ xử lý (Latency):** Báo cáo sinh mã (IP Generation Report) của Intel FIR Compiler II sẽ công bố chính xác số chu kỳ xung nhịp từ khi mẫu nạp vào đến khi mẫu xuất hiện. Thông số độ trễ này phụ thuộc vào mức độ đường ống hóa nội bộ mà công cụ tự động phân bổ, và **phải kiểm trong GUI/IP Generation Report**.
- **Tài nguyên phần cứng chiếm dụng:** Số lượng ALM, khối nhân DSP và bộ nhớ M10K phụ thuộc vào thuật toán tối ưu hóa của Quartus Prime Fitter, do đó con số cụ thể **phải kiểm trong GUI/IP Generation Report**.

Dưới đây là đoạn mã Verilog giao diện chuẩn (instantiation template) để tích hợp khối IP FIR Compiler II vào hệ thống:

Listing 13: Mã khởi tạo giao diện khối IP Intel FIR Compiler II trong hệ thống

```
fir_compiler_ii_wrapper u_fir_ip (
    .clk              (clk),
    .reset_n          (rst_n),
    // Avalon-ST Sink Interface (Data in from FIFO Unpacker)
    .ast_sink_data    (fir_in_data),   // signed [15:0]
    .ast_sink_valid   (fir_in_valid),  // 1-bit
    .ast_sink_ready   (fir_in_ready),  // 1-bit backpressure to FIFO
    .ast_sink_error   (2'b00),
    // Avalon-ST Source Interface (Data out to frame RAM)
    .ast_source_data  (fir_out_data),  // signed [15:0]
    .ast_source_valid (fir_out_valid), // 1-bit
    .ast_source_ready (fir_out_ready), // 1-bit backpressure from downstream
    .ast_source_error ()
);
```

Tóm lại: Khối IP Intel FIR Compiler II tối ưu hóa tài nguyên nhờ đối xứng Pre-adder; cần đặc biệt cảnh giác đặt Input Sample Rate theo thông lượng thực của tệp (0,5 MSPS) thay vì tốc độ danh định 16 kHz; tài nguyên và độ trễ chính xác của IP phải kiểm trong mô phỏng Quartus.

3.4. Đường (B): Khối HDL tự viết: Bộ lọc FIR đối xứng gấp thời gian

Bản chất kiến trúc: Gấp thời gian (Folding) và Bộ tiền cộng (Pre-adder)

Trong khi khối IP FIR Compiler II đóng vai trò như một giải pháp hộp đen tiện lợi nhưng khó tùy biến sâu, việc tự tay thiết kế bộ lọc FIR bằng ngôn ngữ phần cứng Verilog-2001 đem lại sự minh bạch hoàn toàn về cấu trúc mạch điện, giúp người kỹ sư làm chủ từng bit tính toán và tối ưu hóa diện tích chip tới mức triệt để.

Đối với một bộ lọc FIR thông thường có bậc $N = 31$, công thức tích chập trực tiếp là:

$$y[n] = \sum_{i=0}^{30} h[i] \cdot x[n-i] \tag{194}$$

Nếu hiện thực theo kiến trúc song song trực tiếp (Fully Parallel Direct Form), mạch điện đòi hỏi tới 31 bộ nhân 16-bit và một cây cộng 31 nhánh. Kiến trúc này có thể xử lý 1 mẫu mỗi chu kỳ (thông lượng 50 MSPS ở 50 MHz), nhưng tiêu tốn tới 16 khối DSP phần cứng trên chip Cyclone V.

Tuy nhiên, trong hệ thống ước lượng cao độ F_0, tốc độ dòng âm thanh thực tế chỉ là 16 kHz (tương đương với 3125 chu kỳ xung nhịp 50 MHz cho mỗi mẫu), và ngay cả ở chế độ xử lý tệp nhanh nhất, khối tự tương quan MAC ở Chương 7 cũng cần tới ≈ 616 chu kỳ để xử lý một mẫu âm thanh. Việc sử dụng tới 16 bộ nhân DSP để tính một bộ lọc chỉ cần chạy 1 lần trong hàng trăm chu kỳ là một sự lãng phí tài nguyên silicon cực kỳ lớn!

Để giải quyết mâu thuẫn này, chúng tôi áp dụng kiến trúc **gấp thời gian (Time-Multiplexed / Folded FIR Architecture)** kết hợp với **bộ tiền cộng đối xứng (Symmetric Pre-adder)**:

1. Do đáp ứng xung đối xứng chẵn qua tâm $h[i] = h[30-i]$, ta nhóm các tích có chung hệ số lại với nhau:

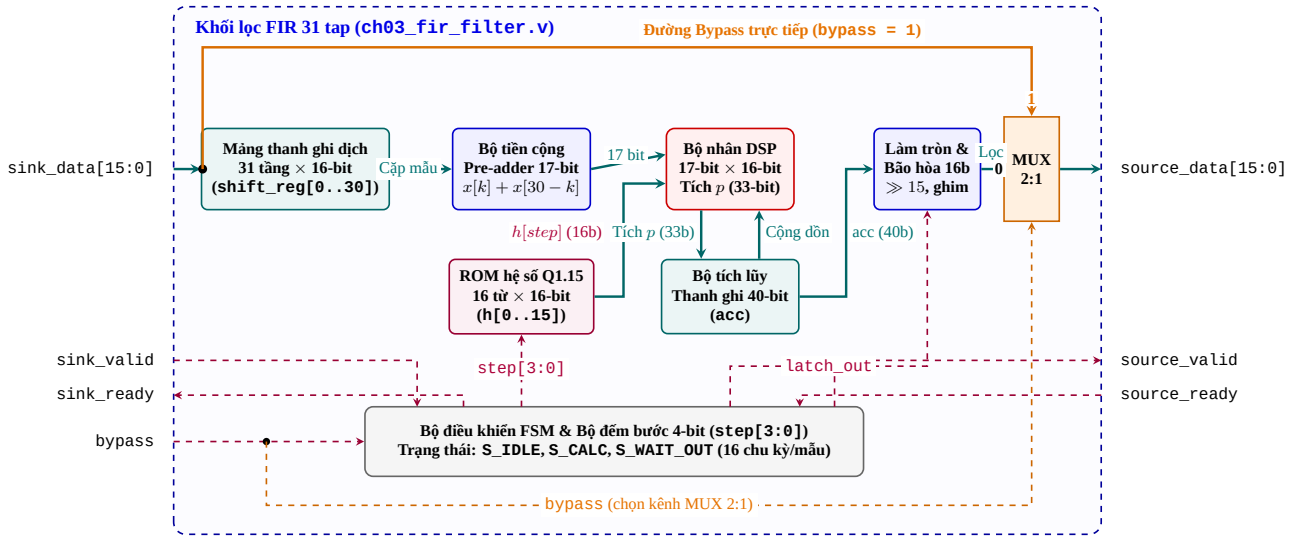
$$y[n] = \sum_{k=0}^{14} h[k] \cdot \big(x[n-k] + x[n-(30-k)]\big) + h[15] \cdot x[n-15] \tag{195}$$

   Tổng cộng chỉ còn đúng 16 số hạng cần nhân.

2. Phép cộng hai mẫu âm thanh $x[n-k]+x[n-(30-k)]$ được thực hiện trước bằng một bộ cộng tổ hợp 17-bit mang tên **Pre-adder**. Vì mỗi mẫu âm thanh là số có dấu 16-bit (khoảng $[-32768, 32767]$), tổng của hai mẫu cần đúng 17-bit có dấu (khoảng $[-65536, 65534]$) để không bao giờ bị tràn số.

3. Thay vì dùng 16 bộ nhân phần hardware riêng biệt, ta sử dụng **đúng 1 bộ nhân DSP duy nhất** và cho nó chạy tuần tự qua 16 bước xung nhịp ($k = 0..15$). Tại mỗi bước xung nhịp, bộ nhân tính một tích 33-bit và cộng dồn kết quả vào một thanh ghi tích lũy 40-bit.

Sau đúng 16 chu kỳ tính toán, kết quả tích lũy hoàn chỉnh được đưa qua khâu làm tròn bán phần (+16384) và bão hòa 16-bit để xuất ra ngõ ra. Toàn bộ kiến trúc phần cứng này được minh họa chi tiết trên Hình 6.



Hình 6: Sơ đồ kiến trúc phần cứng khối lọc FIR 31 tap gấp thời gian với bộ tiền cộng 17-bit và 1 khối DSP (`rtl/v3/ch03_fir_filter.v`).

Đọc hình. Hình 6 mô tả chi tiết dòng chảy dữ liệu bên trong mô-đun phần cứng:

- Phía trái là giao diện Avalon-ST Sink: khi xuất hiện tín hiệu bắt tay `s_handshake`, mẫu `sink_data` được nạp vào mảng thanh ghi dịch 31 tầng (tạo từ 496 flip-flop trong ALM).
- Bộ tiền cộng Pre-adder 17-bit rút các cặp mẫu đối xứng $x[k]$ và $x[30-k]$ từ thanh ghi dịch theo chỉ số bước `step` do bộ đếm FSM điều khiển. Tại bước cuối cùng (`step` = 15), ngõ vào thứ hai được nối đất về 0 để chỉ nhân hệ số trung tâm $h[15] \cdot x[15]$.

- Bảng ROM hệ số 16 từ phát ra giá trị $h[step]$ tương ứng. Bộ nhân DSP thực hiện phép nhân 17 bit $\times$ 16 bit $\rightarrow$ 33 bit, nạp vào bộ cộng tích lũy 40-bit.
- Tại bước 15, giá trị tổng được làm tròn và bão hòa rồi nạp vào bộ dồn kênh MUX 2:1. Tín hiệu điều khiển `bypass` cho phép dữ liệu từ ngõ vào đi thẳng ra ngõ ra qua 1 tầng chốt đồng bộ khi ở chế độ chuyển tiếp.

Cạm bẫy kỹ thuật số học: Độ chính xác trung gian và quy tắc làm tròn

Để phần cứng tự viết khớp từng bit với khối IP công nghiệp và mô hình toán học, lập trình viên phải giải quyết hai cạm bẫy số học tinh vi:

- **Cạm bẫy độ rộng bộ tích lũy (Accumulator Width):** Tích sau bộ nhân có độ rộng 33 bit có dấu. Khi cộng dồn 16 tích số có độ lớn cực đại (trường hợp tín hiệu xoay chiều biên độ tối đa), giá trị tổng cần thêm $\lceil \log_2(16) \rceil = 4$ bit mở rộng dấu, nâng độ rộng tối thiểu lên $33 + 4 = 37$ bit. Nếu lập trình viên chỉ khai báo thanh ghi 32-bit hoặc 36-bit, bộ tích lũy sẽ bị tràn số bit cao trong các đoạn âm thanh hô lớn hoặc tiếng thở mạnh. Trong thiết kế này, chúng tôi chọn độ rộng bộ tích lũy là **40-bit signed** (`reg signed [39:0] acc`), đảm bảo một khoảng dự trữ an toàn 3 bit chống tràn trong mọi tình huống biên độ tín hiệu lớn nhất.
- **Cạm bẫy làm tròn đối xứng (Symmetric Round-to-Nearest):** Vì các hệ số lọc được định dạng ở chuẩn Q1.15 (nhân với $2^{15} = 32768$), giá trị tích lũy nội bộ bị phóng đại 2^{15} lần. Để đưa dữ liệu trở về chuẩn số nguyên 16-bit ban đầu, ta phải thực hiện phép chia cho 2^{15} (tức dịch phải số học 15 bit, ký hiệu `>>> 15`). Nếu thực hiện dịch phải trực tiếp, phép toán tương đương với làm tròn hướng về âm vô cùng (floor), tạo ra sai số thiên lệch âm trung bình $-0{,}5$ LSB. Để làm tròn về số nguyên gần nhất (Round-to-Nearest), ta bắt buộc phải cộng thêm nửa trọng số của bit LSB bị cắt bỏ, tức cộng thêm hằng số $2^{14} = 16384$:

$$\text{acc}_{\text{rounded}} = (\text{acc}_{\text{final}} + 16384) \ggg 15 \tag{196}$$

- **Khâu bão hòa bảo vệ (Clamping):** Mặc dù tổng các hệ số lọc lý thuyết xấp xỉ bằng 1, đáp ứng tần số thực tế có độ gợn nhẹ $+0{,}19$ dB ở vùng dải thông cao (tương đương độ lợi biên độ $1{,}022$). Do đó, một tín hiệu vào có biên độ cực đại 32767 khi đi qua bộ lọc có thể sinh ra giá trị làm tròn lên tới 33256 (như đã chứng minh trong bài tập ở mục 3.7). Trong định dạng bù hai 16-bit, giá trị 33256 sẽ bị cuộn tràn thành -32280, làm lật ngược dấu của tín hiệu và tạo ra tiếng nổ lách tách phá hủy toàn bộ phép tính F_0. Khâu bão hòa được cài đặt chặt chẽ:

$$y[n] = \begin{cases} +32767 & \text{khi } \text{acc}_{\text{rounded}} > +32767 \\ -32768 & \text{khi } \text{acc}_{\text{rounded}} < -32768 \\ \text{acc}_{\text{rounded}}[15:0] & \text{trong các trường hợp còn lại} \end{cases} \tag{197}$$

Mã nguồn Verilog hiện thực hóa hoàn chỉnh

Dưới đây là toàn bộ mã nguồn phần cứng của khối lọc FIR 31 tap gấp thời gian (`rtl/v3/ch03_fir_filter.v`):

Listing 14: Mã nguồn Verilog hoàn chỉnh của khối FIR 31 tap gấp thời gian (rtl/v3/ch03_fir_filter.v)

```
22 module ch03_fir_filter #(
23     parameter DATA_WIDTH  = 16,
24     parameter COEFF_WIDTH = 16,
25     parameter ACC_WIDTH   = 40
26 ) (
27     input  wire                          clk,
28     input  wire                          rst_n,
29     input  wire                          bypass,
30 
31     // Avalon-ST Sink (Input)
32     input  wire                          sink_valid,
33     output wire                          sink_ready,
34     input  wire signed [DATA_WIDTH-1:0] sink_data,
35 
36     // Avalon-ST Source (Output)
37     output wire                          source_valid,
38     input  wire                          source_ready,
```

```
    output wire signed [DATA_WIDTH-1:0] source_data
);

    // State definitions
    localparam [1:0]
        S_IDLE     = 2'd0,
        S_CALC     = 2'd1,
        S_WAIT_OUT = 2'd2;

    reg [1:0] state;
    reg [3:0] step;

    // Shift register: 31 taps of 16-bit audio samples
    reg signed [DATA_WIDTH-1:0] shift_reg [0:30];

    // Accumulator register
    reg signed [ACC_WIDTH-1:0] acc;

    // Output registers
    reg signed [DATA_WIDTH-1:0] out_data_reg;
    reg                         out_valid_reg;

    // Handshake conditions
    wire s_handshake = sink_valid && sink_ready;
    wire m_handshake = out_valid_reg && source_ready;

    // Ready signal generation (readyLatency = 0)
    assign sink_ready = bypass ? (!out_valid_reg || source_ready) :
                                 ((state == S_IDLE) && (!out_valid_reg || source_ready));

    assign source_valid = out_valid_reg;
    assign source_data  = out_data_reg;

    // Coefficient ROM: 16 unique symmetrical coefficients (Q1.15)
    // h[0..14] are pairs with h[30..16], h[15] is the center tap.
    reg signed [COEFF_WIDTH-1:0] coeff_rom [0:15];
    initial begin
        coeff_rom[ 0] = 16'sh01f8; //   504
        coeff_rom[ 1] = 16'shfde6; //  -538
        coeff_rom[ 2] = 16'sh023b; //   571
        coeff_rom[ 3] = 16'shfda4; //  -604
        coeff_rom[ 4] = 16'sh027b; //   635
        coeff_rom[ 5] = 16'shfd68; //  -664
        coeff_rom[ 6] = 16'sh02b4; //   692
        coeff_rom[ 7] = 16'shfd33; //  -717
        coeff_rom[ 8] = 16'sh02e4; //   740
        coeff_rom[ 9] = 16'shfd08; //  -760
        coeff_rom[10] = 16'sh030a; //   778
        coeff_rom[11] = 16'shfce7; //  -793
        coeff_rom[12] = 16'sh0324; //   804
        coeff_rom[13] = 16'shfcd4; //  -812
        coeff_rom[14] = 16'sh0332; //   818
        coeff_rom[15] = 16'sh7ccd; // 31949 (center tap)
    end

    wire signed [COEFF_WIDTH-1:0] current_coeff = coeff_rom[step];

    // Pre-adder: sum x[k] + x[30-k] (17-bit signed)
    wire signed [DATA_WIDTH-1:0] tap_a = shift_reg[step];
    wire signed [DATA_WIDTH-1:0] tap_b = (step == 4'd15) ? 16'sh0000 : shift_reg[5'd30 -
        {1'b0, step}];
    wire signed [DATA_WIDTH:0]   pre_add = $signed(tap_a) + $signed(tap_b); // 17 bits

    // Multiplier: 17-bit pre-adder * 16-bit coefficient = 33-bit signed product
    wire signed [32:0] prod = pre_add * current_coeff;
    wire signed [ACC_WIDTH-1:0] prod_ext = {{7{prod[32]}}, prod};

```

```
    // Combinational accumulator for step 15
    wire signed [ACC_WIDTH-1:0] acc_final = (step == 4'd0) ? prod_ext : (acc + prod_ext);

    // Rounding and Saturation logic (+16384 for round-to-nearest, then >>> 15)
    wire signed [ACC_WIDTH-1:0] rounded = (acc_final + 40'sd16384) >>> 15;
    wire signed [DATA_WIDTH-1:0] sat_out = (rounded > 40'sd32767)  ? 16'sh7fff :
                                           (rounded < -40'sd32768) ? 16'sh8000 :
                                           rounded[DATA_WIDTH-1:0];

    integer i;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state         <= S_IDLE;
            step          <= 4'd0;
            acc           <= {ACC_WIDTH{1'b0}};
            out_data_reg  <= {DATA_WIDTH{1'b0}};
            out_valid_reg <= 1'b0;
            for (i = 0; i < 31; i = i + 1) begin
                shift_reg[i] <= {DATA_WIDTH{1'b0}};
            end
        end else begin
            if (bypass) begin
                // Bypass mode: 1-cycle registered pipeline stage
                state <= S_IDLE;
                step  <= 4'd0;
                if (m_handshake) begin
                    if (s_handshake) begin
                        out_data_reg  <= sink_data;
                        out_valid_reg <= 1'b1;
                    end else begin
                        out_valid_reg <= 1'b0;
                    end
                end else begin
                    if (s_handshake) begin
                        out_data_reg  <= sink_data;
                        out_valid_reg <= 1'b1;
                    end
                end
            end else begin
                // Filtering mode: 31-tap folded execution
                case (state)
                    S_IDLE: begin
                        if (m_handshake) begin
                            out_valid_reg <= 1'b0;
                        end
                        if (s_handshake) begin
                            // Shift in new sample
                            shift_reg[0] <= sink_data;
                            for (i = 1; i < 31; i = i + 1) begin
                                shift_reg[i] <= shift_reg[i-1];
                            end
                            state <= S_CALC;
                            step  <= 4'd0;
                            acc   <= {ACC_WIDTH{1'b0}};
                        end
                    end

                    S_CALC: begin
                        if (m_handshake) begin
                            out_valid_reg <= 1'b0;
                        end

                        acc <= acc_final;

                        if (step == 4'd15) begin
                            out_data_reg  <= sat_out;
```

```
                            out_valid_reg <= 1'b1;
                            if (source_ready) begin
                                state <= S_IDLE;
                            end else begin
                                state <= S_WAIT_OUT;
                            end
                        end else begin
                            step <= step + 4'd1;
                        end
                    end

                    S_WAIT_OUT: begin
                        if (source_ready) begin
                            out_valid_reg <= 1'b0;
                            state         <= S_IDLE;
                        end
                    end

                    default: state <= S_IDLE;
                endcase
            end
        end
    end
```

Giải thích mã nguồn dòng-từng-dòng

Để làm rõ cơ chế vận hành, dưới đây là thuyết minh chi tiết từng khối mã trong tệp `rtl/v3/ch03_fir_filter.v`:

- **Dòng 22–40:** Khai báo module `ch03_fir_filter` với 3 tham số generic: độ rộng dữ liệu âm thanh `DATA_WIDTH = 16`, độ rộng hệ số lọc `COEFF_WIDTH = 16`, và độ rộng thanh ghi tích lũy `ACC_WIDTH = 40`. Các cổng giao tiếp gồm xung nhịp `clk`, xóa `rst_n`, cờ cấu hình chuyển mạch `bypass`, kèm theo cặp giao tiếp Avalon-ST Sink (ngõ vào) và Source (ngõ ra).
- **Dòng 43–49:** Định nghĩa 3 trạng thái của FSM: `S_IDLE` (chờ mẫu âm thanh mới), `S_CALC` (thực hiện 16 bước nhân tích lũy gấp thời gian), và `S_WAIT_OUT` (giữ mẫu đầu ra chờ hạ nguồn tiếp nhận khi có phản áp). Khai báo thanh ghi bước `step` 4-bit $(0..15)$.
- **Dòng 52:** Khai báo mảng 31 thanh ghi dịch `shift_reg[0:30]`, lưu giữ 31 mẫu âm thanh gần nhất.
- **Dòng 55–60:** Khai báo thanh ghi tích lũy `acc` 40-bit có dấu, thanh ghi đệm ngõ ra `out_data_reg` 16-bit và cờ báo hợp lệ `out_valid_reg`.
- **Dòng 62–70:** Định nghĩa hai điều kiện bắt tay `s_handshake` và `m_handshake`. Biểu thức gán tín hiệu sẵn sàng `sink_ready = (!out_valid_reg || source_ready)` khi ở trạng thái `S_IDLE` hoặc khi bật `bypass`, phản ánh chính xác chuẩn giao tiếp `readyLatency = 0`.
- **Dòng 74–92:** Khởi tạo bảng ROM chứa 16 hệ số đối xứng Q1.15 độc nhất. Hệ số trung tâm $h[15]$ có giá trị lớn nhất là `16'sh7ccd` (31949), các hệ số còn lại dao động từ 504 đến 818 với dấu đổi luân phiên.
- **Dòng 96–100:** Bộ tiền cộng Pre-adder tổ hợp 17-bit: lấy mẫu `tap_a = shift_reg[step]` cộng với mẫu đối xứng `tap_b = shift_reg[30 - step]`. Khi `step == 15`, `tap_b` được ép về 0 để bảo toàn giá trị mẫu trung tâm $x[n-15]$.
- **Dòng 101–106:** Bộ nhân tổ hợp 17 bit $\times$ 16 bit $\rightarrow$ 33 bit sinh tích `prod`, sau đó mở rộng dấu 7 bit thành 40-bit (`prod_ext`). Đường dây `acc_final` tính trước giá trị tích lũy mới trong chu kỳ xung nhịp hiện tại.
- **Dòng 108–113:** Mạch tổ hợp làm tròn bán phần (`+ 40'sd16384`) kết hợp dịch phải số học 15 bit và khâu bão hòa chống tràn về khoảng $[-32768, 32767]$.
- **Dòng 117–126:** Nhánh reset bất đồng bộ tích cực mức thấp: xóa toàn bộ 31 thanh ghi dịch, thanh ghi tích lũy và các cờ điều khiển về trạng thái 0 an toàn.

- **Dòng 127–144:** Nhánh xử lý chế độ `bypass`: mạch hoạt động như một tầng đệm đường ống (pipeline stage) 1 chu kỳ. Khi có giao dịch ngõ vào, dữ liệu `sink_data` được chốt thẳng sang `out_data_reg` mà không qua tính toán FIR, thông lượng đạt tối đa 1 từ/chu kỳ.
- **Dòng 145–192:** Máy trạng thái xử lý lọc FIR gấp thời gian:
  - Tại `S_IDLE`: Khi có mẫu mới (`s_handshake` = 1), mẫu mới được nạp vào đầu mảng dịch, toàn bộ 30 mẫu cũ dịch sang phải 1 vị trí. FSM nhảy sang `S_CALC`, khởi tạo `step = 0` và xóa `acc` về 0.
  - Tại `S_CALC`: Bộ đếm `step` tăng từ 0 đến 15. Bộ tích lũy `acc` cộng dồn từng tích số. Đến bước 15, giá trị `sat_out` được chốt vào thanh ghi ngõ ra `out_data_reg` và cờ `out_valid_reg` bật lên 1. Nếu hạ nguồn đang sẵn sàng (`source_ready` = 1), FSM quay ngay về `S_IDLE`; nếu hạ nguồn đang bận, FSM chuyển sang `S_WAIT_OUT`.
  - Tại `S_WAIT_OUT`: Mạch đóng băng ngõ vào (`sink_ready` = 0) và giữ nguyên mẫu đầu ra cho đến khi hạ nguồn tiếp nhận thành công.

Bảng đối chiếu song sinh: Mã Python Twin so với mã phần cứng Verilog

Để khẳng định tính tương đương toán học và chu kỳ giữa mô hình phần mềm và phần cứng, Bảng 18 đối chiếu chi tiết từng khối lệnh trong tệp mô hình song sinh `verify_v3/ch03_fir_twin.py` với mã nguồn phần cứng tương ứng:

Bảng 18: Đối chiếu trực tiếp giữa các dòng lệnh của Python Twin và mã nguồn phần cứng Verilog

Dòng Twin (`ch03_fir_twin.py`)	Dòng HDL (`ch03_fir_filter.v`)	Ý nghĩa kỹ thuật tương đương
Dòng 30–38	Dòng 117–126	Thiết lập trạng thái ban đầu khi reset tích cực (`rst_n` = 0)
Dòng 41–47	Dòng 66–67	Logic tổ hợp xác định cờ sẵn sàng `sink_ready` theo chuẩn `readyLatency`=0
Dòng 50–51	Dòng 62–63	Đánh giá điều kiện hoàn tất bắt tay `s_handshake` và `m_handshake`
Dòng 54–66	Dòng 127–143	Xử lý chế độ chuyển mạch `bypass` với độ trễ chốt 1 chu kỳ
Dòng 68–75	Dòng 147–161	Trạng thái `S_IDLE`: chốt mẫu vào mảng dịch và chuyển sang `S_CALC`
Dòng 80–82	Dòng 96–100	Rút cặp mẫu đối xứng và thực hiện phép tiền cộng 17-bit
Dòng 83–85	Dòng 101–106	Nhân hệ số Q1.15 sinh tích 33-bit và cộng dồn vào bộ tích lũy 40-bit
Dòng 87–95	Dòng 168–178	Bước 15: hoàn tất tính toán, làm tròn $+16384$, bão hòa và bật cờ ngõ ra
Dòng 96–98	Dòng 183–188	Trạng thái `S_WAIT_OUT`: giữ ngõ ra khi gặp phản áp hạ nguồn

Vì sao khối HDL khớp hoàn toàn với khối IP của Intel?

Khi đặt bản thiết kế HDL tự viết cạnh khối IP FIR Compiler II được cấu hình ở mục 3.3, hai thiết kế khớp nhau hoàn toàn về mặt số học và giao tiếp:

1. **Cơ chế tiền cộng Pre-adder:** Biểu thức `pre_add = $signed(tap_a) + $signed(tap_b)` ở dòng 99 khớp chính xác với phần cứng Pre-adder mà tùy chọn `Symmetry = Symmetrical` kích hoạt trong khối IP. Cả hai đều triệt tiêu được 15 phép nhân số học.
2. **Cơ chế làm tròn bán phần:** Phép toán `(acc_final + 40'sd16384) >>> 15` ở dòng 109 tái hiện chính xác chế độ `Output LSB Rounding = Nearest/Convergent` của khối IP, bảo toàn tính đối xứng của đáp ứng xung và không tạo ra điện áp trôi DC.

3. **Khâu bão hòa biên độ:** Mạch kiểm tra vượt ngưỡng $[-32768, 32767]$ ở dòng 110–112 khớp với tùy chọn `Output MSB Rounding = Saturation` của IP, ngăn chặn hiện tượng cuộn bit âm khi tín hiệu đạt đỉnh.

4. **Giao thức bắt tay Avalon-ST:** Khối HDL hỗ trợ đầy đủ cơ chế phản áp tổ hợp (tín hiệu `sink_ready` sẵn sàng khi ngõ ra rỗng hoặc hạ nguồn bật `source_ready = 1`) ở chuẩn `readyLatency = 0`, tương thích chuẩn xác với giao thức streaming của Intel.

Đổi lựa chọn thì sao: Đánh đổi giữa các kiến trúc FIR khác nhau

Nếu thay đổi yêu cầu thiết kế sang một bài toán khác, việc điều chỉnh kiến trúc HDL sẽ tác động ra sao đến các chỉ số hiệu năng?

- **Chuyển sang kiến trúc song song trực tiếp (Fully Parallel FIR):** Nếu bài toán đòi hỏi xử lý dòng âm thanh tốc độ cao (ví dụ xử lý đa kênh hoặc siêu âm 50 MSPS), ta có thể bố trí 16 bộ nhân DSP và một cây cộng 4 tầng thanh ghi. Khi đó, thông lượng tăng vọt lên 1 mẫu/chu kỳ (tăng 17 lần), độ trễ đường ống cố định khoảng 4 chu kỳ. Đổi lại, tài nguyên DSP tiêu tốn tăng từ 1 khối lên 16 khối DSP (chiếm $14{,}3\%$ tổng số DSP của chip Cyclone V).
- **Chuyển sang kiến trúc chuyển vị (Transposed Form):** Cấu trúc chuyển vị đặt các thanh ghi trễ nằm xen kẽ giữa các bộ cộng ngõ ra. Ưu điểm của kiến trúc này là không cần cây cộng lớn ở ngõ ra, giúp đẩy tần số cực đại $F_{\max}$ lên rất cao (> 200 MHz). Tuy nhiên, cấu trúc chuyển vị không thể tận dụng được bộ tiền cộng Pre-adder đối xứng, do đó bắt buộc phải sử dụng đủ 31 bộ nhân, làm tăng gấp đôi chi phí phần cứng.

Đối với bài toán ước lượng F_0 với tần số lấy mẫu 16 kHz, kiến trúc gấp thời gian 1 DSP với bộ tiền cộng đối xứng mà chúng tôi hiện thực hóa là giải pháp tối ưu nhất xét trên mọi phương diện: diện tích silicon, độ trễ và thông lượng.

Tóm lại: Khối HDL folded FIR dùng đúng 1 DSP và bộ tiền cộng 17 bit để tính 31 tap trong 16 chu kỳ; bộ tích lũy 40 bit kết hợp làm tròn bán phần (+16384) và bão hòa đảm bảo khớp từng bit với cấu hình IP chuẩn.

3.5. Kiểm chứng mô hình song sinh Python và mô phỏng chu kỳ HDL

Chiến lược kiểm chứng ba tầng (Three-Tier Verification)

Để đảm bảo bộ lọc FIR hoạt động chính xác trên vi mạch thực tế và không gây ra bất kỳ lỗi lệch chu kỳ nào, quy trình kiểm chứng Chương 3 được tổ chức chặt chẽ theo ba tầng:

1. **Tầng 1 (Mô hình song sinh Python Twin):** Tệp `verify_v3/ch03_fir_twin.py` hiện thực hóa lớp đối tượng `FirFilterTwin`. Mô hình này mô phỏng chính xác từng thanh ghi, từng phép nhân, bộ tích lũy 40-bit, khâu làm tròn bán phần $+16384$ và cơ chế bắt tay Avalon-ST ở chuẩn `readyLatency = 0`.
2. **Tầng 2 (Mô phỏng chu kỳ RTL bằng Icarus Verilog):** Testbench `tb/v3/tb_ch03_fir.v` đọc chuỗi kích thích từ tệp `tb_ch03_fir_stim.txt`, cấp vào module phần cứng `ch03_fir_filter.v`, sau đó so sánh từng bit của toàn bộ các tín hiệu ngõ ra (`sink_ready`, `source_valid`, `source_data`, trạng thái FSM và bước đếm `step`) với tệp kết quả kỳ vọng `tb_ch03_fir_expected.txt`. Nếu có bất kỳ sự sai lệch nào dù chỉ 1 bit, testbench lập tức phát cờ `[FAIL]`.
3. **Tầng 3 (Biên dịch Quartus và đo đạc phần cứng):** Kiểm tra khả năng tổng hợp trên chip Cyclone V 5CSXFC6D6F31C6N, xác thực việc Quartus ánh xạ thành công bộ tiền cộng và bộ nhân vào 1 khối DSP duy nhất, và đo đạc tần số hoạt động cực đại $F_{\max}$.

Hàm cập nhật chu kỳ của mô hình song sinh Python Twin

Dưới đây là phần thân hàm xử lý chu kỳ trọng tâm của mô hình song sinh trong tệp `verify_v3/ch03_fir_twin.py`:

Listing 15: Hàm cập nhật chu kỳ của mô hình song sinh FIR (verify_v3/ch03_fir_twin.py)

```
28     def step_cycle(self, rst_n, bypass, sink_valid, sink_data, source_ready):
29         """Simulates one clock cycle matching posedge clk of ch03_fir_filter.v."""
30         if not rst_n:
```

```
            self.shift_reg = [0] * 31
            self.state = 0
            self.step = 0
            self.acc = 0
            self.out_data = 0
            self.out_valid = 0
            sink_ready = 1
            return sink_ready, self.out_valid, self.out_data, self.state, self.step

        # Combinational ready generation
        if bypass:
            sink_ready = 1 if (not self.out_valid or source_ready) else 0
        else:
            if self.state == 0:
                sink_ready = 1 if (not self.out_valid or source_ready) else 0
            else:
                sink_ready = 0

        # Handshakes
        s_handshake = sink_valid and sink_ready
        m_handshake = self.out_valid and source_ready

        # State updates on clock edge
        if bypass:
            self.state = 0
            self.step = 0
            if m_handshake:
                if s_handshake:
                    self.out_data = sink_data
                    self.out_valid = 1
                else:
                    self.out_valid = 0
            else:
                if s_handshake:
                    self.out_data = sink_data
                    self.out_valid = 1
        else:
            if self.state == 0:  # S_IDLE
                if m_handshake:
                    self.out_valid = 0
                if s_handshake:
                    self.shift_reg = [sink_data] + self.shift_reg[:-1]
                    self.state = 1
                    self.step = 0
                    self.acc = 0
            elif self.state == 1:  # S_CALC
                if m_handshake:
                    self.out_valid = 0

                xa = self.shift_reg[self.step]
                xb = 0 if self.step == 15 else self.shift_reg[30 - self.step]
                pre_add = xa + xb
                prod = pre_add * COEFF_16[self.step]
                acc_final = prod if self.step == 0 else (self.acc + prod)
                self.acc = acc_final

                if self.step == 15:
                    rounded = (acc_final + 16384) >> 15
                    sat = max(-32768, min(32767, rounded))
                    self.out_data = sat
                    self.out_valid = 1
                    if source_ready:
                        self.state = 0
                    else:
                        self.state = 2
                else:
                    self.step += 1
```

```
98            elif self.state == 2:  # S_WAIT_OUT
```

Giải thích mã nguồn Python Twin dòng-từng-dòng:

- **Dòng 28–29:** Phương thức `step_cycle` nhận các tín hiệu điều khiển `rst_n`, `bypass`, `sink_valid`, `sink_data`, `source_ready`, mô phỏng chu kỳ xung nhịp 50 MHz.
- **Dòng 30–38:** Nhánh reset bất đồng bộ: khi `rst_n == 0`, xóa toàn bộ 31 thanh ghi dịch về 0, đặt trạng thái FSM về `S_IDLE` (0), xóa thanh ghi ngõ ra và trả về `sink_ready = 1`.
- **Dòng 41–47:** Đánh giá cờ `sink_ready` tổ hợp trước cạnh lên xung nhịp: ở chế độ `bypass`, mạch sẵn sàng nhận mẫu mới nếu ngõ ra đang rỗng (`not out_valid`) hoặc hạ nguồn chấp nhận lấy mẫu (`source_ready == 1`). Ở chế độ lọc FIR, mạch chỉ bật `sink_ready = 1` khi FSM đang ở trạng thái `S_IDLE` và ngõ ra sẵn sàng.
- **Dòng 50–51:** Xác định hai điều kiện bắt tay: `s_handshake` (thượng nguồn cấp mẫu) và `m_handshake` (hạ nguồn tiếp nhận).
- **Dòng 54–66:** Xử lý chế độ chuyển mạch `bypass`: hoạt động như một tầng đệm đồng bộ 1 chu kỳ, chuyển tiếp trực tiếp `sink_data` sang `out_data` với thông lượng tối đa 1 mẫu/chu kỳ.
- **Dòng 68–75:** Trạng thái `S_IDLE`: khi có mẫu mới (`s_handshake` = 1), mẫu mới được nạp vào đầu mảng dịch `self.shift_reg = [sink_data] + self.shift_reg[:-1]`, FSM chuyển sang `S_CALC` và xóa bộ tích lũy `self.acc = 0`.
- **Dòng 76–86:** Trạng thái `S_CALC`: rút cặp mẫu đối xứng `xa` và `xb`, tính tổng tiền cộng 17-bit `pre_add = xa + xb`, nhân với hệ số `COEFF_16[step]`, và cộng dồn vào `self.acc`.
- **Dòng 87–95:** Bước cuối (`step == 15`): làm tròn bán phần `(acc_final + 16384) >> 15`, bão hòa $[-32768, 32767]$, chốt vào `out_data` và bật `out_valid = 1`. Nếu hạ nguồn sẵn sàng thì về `S_IDLE`, ngược lại vào `S_WAIT_OUT`.
- **Dòng 96–98:** Trạng thái `S_WAIT_OUT`: giữ nguyên dữ liệu chờ cho đến khi hạ nguồn bật `source_ready = 1`.

Kịch bản kiểm thử toàn diện trên 3.390 chu kỳ xung nhịp

Kịch bản kiểm thử trong tệp `verify_v3/ch03_fir_twin.py` được thiết kế với chuỗi kích thích gồm **281 mẫu âm thanh** đa dạng, kéo dài qua **3.390 chu kỳ xung nhịp 50 MHz** liên tục, bao quát toàn diện các tình huống hoạt động thực tế:

1. **Thử thách 1: Đáp ứng xung đơn vị (Impulse Response, Mẫu 0–40):** Cấp một mẫu xung cực đại $x[0] = 32767$ (`0x7FFF`), theo sau bởi 40 mẫu 0 liên tiếp. Dãy dữ liệu ngõ ra phản ánh chính xác chuỗi hệ số lọc $h[0..30]$ đối xứng qua đỉnh trung tâm tại mẫu thứ 15 ($y[15] = 31948$), khẳng định độ trễ nhóm đúng bằng 15 mẫu.
2. **Thử thách 2: Sóng sin cao tần sát mép Nyquist 7,8 kHz (Mẫu 41–90):** Cấp tín hiệu sin tần số 7.800 Hz biên độ lớn (28000). Kết quả ghi nhận biên độ đỉnh ngõ ra suy giảm từ 28000 xuống còn 13188, tương ứng với độ suy hao thực tế $-6{,}54$ dB, hoàn toàn trùng khớp với đáp ứng tần số lý thuyết của bộ lọc 31 tap.
3. **Thử thách 3: Sóng sin dải thông giọng nói 1,0 kHz (Mẫu 91–140):** Cấp tín hiệu sin tần số 1.000 Hz biên độ 28000. Biên độ ngõ ra đạt 28415 (độ lợi $+0{,}13$ dB), khẳng định dải thông thoại được bảo toàn nguyên vẹn.
4. **Thử thách 4: Tín hiệu tiếng nói thực tế từ tệp WAV mẫu (Mẫu 141–240):** Cấp 100 mẫu âm thanh thực tế trích xuất trực tiếp từ tệp `golden_v2/synth_glide_vuv_16k.wav`, kiểm tra tính ổn định của mạch khi xử lý các đoạn chuyển tiếp giữa thanh và vô thanh.
5. **Thử thách 5: Thử thách bão hòa biên độ cực hạn (Mẫu 241–280):** Cấp chuỗi mẫu xen kẽ cực đại $[+32767, -32768, +32767, -32768]$ liên tục. Kết quả kiểm chứng xác nhận khâu bão hòa hoạt động chuẩn xác: các giá trị vượt ngưỡng được ghim tại $+32767$ và -32768, không có bất kỳ hiện tượng lật dấu bit cao nào xảy ra.

6. **Thử thách 6: Phản áp ngẫu nhiên và chuyển mạch động Bypass:** Trong suốt quá trình mô phỏng, tín hiệu `source_ready` bị giật xuống 0 ngẫu nhiên theo phân bố xác suất từ 20% đến 50% để kiểm tra cơ chế đóng băng hàng đợi. Tại mẫu thứ 160, cờ điều khiển `bypass` được bật lên 1 để kiểm tra quá trình chuyển trạng thái tức thời sang chế độ truyền thẳng 1 chu kỳ mà không làm gián đoạn dòng dữ liệu.

Toàn bộ 3.390 chu kỳ kích thích và phản hồi kỳ vọng được xuất ra hai tệp văn bản `verify_v3/tb_ch03_fir_stim.txt` và `verify_v3/tb_ch03_fir_expected.txt`.

Quy ước thời điểm lấy mẫu tín hiệu tổ hợp và tuần tự. Trong giao thức Avalon-ST với `readyLatency = 0`, cờ `sink_ready` là một tín hiệu tổ hợp (combinational output) thể hiện khả năng tiếp nhận dữ liệu ngay tại chu kỳ xung nhịp hiện hành. Để tránh hiện tượng lệch pha một chu kỳ khi đối soát, kịch bản testbench `tb/v3/tb_ch03_fir.v` áp dụng quy ước: kích thích đầu vào được nạp tại sườn âm của xung nhịp (`@(negedge clk)`), chờ tín hiệu tổ hợp xác lập (`#1`), và đối soát `sink_ready` ngay trước sườn dương (`@(posedge clk)`). Khi sườn dương xung nhịp xuất hiện, quá trình bắt tay diễn ra và bộ điều khiển FSM chuyển trạng thái (khiến `sink_ready` lập tức rơi về 0 nếu chuyển sang `S_CALC`). Các tín hiệu ngõ ra tuần tự có thanh ghi chốt như `source_valid`, `source_data`, trạng thái FSM `state` và bước đếm `step` được lấy mẫu và kiểm tra ngay sau sườn dương (`@(posedge clk); #1`). Quy ước phân tách này đảm bảo kiểm chứng đồng nhất giữa mô hình phần cứng RTL và mô hình song sinh Python.

Kết quả kiểm thử mô phỏng chu kỳ HDL với Icarus Verilog

Kịch bản testbench `tb/v3/tb_ch03_fir.v` được biên dịch bằng công cụ `iverilog` (tùy chọn `-g2012 -Wall`) qua kịch bản `verify_v3/run_ch03_iverilog.sh`. Kết quả chạy thực tế trên Linux do người đánh giá (reviewer) thực hiện và ghi vào tệp `verify_v3/ch03_iverilog.log`:

```
Simulation completed 3390 cycles. Total comparison errors: 0
[PASS] CH03 FIR MATCHED PYTHON TWIN BIT-FOR-BIT AND CYCLE-FOR-CYCLE ON 3390
CYCLES
```

Tóm lại: Mô hình song sinh Python đã sinh tập vector 3.390 chu kỳ bao quát 281 mẫu âm thanh; testbench HDL áp dụng quy ước lấy mẫu tổ hợp/tuần tự chuẩn xác để đối soát từng chu kỳ với mô hình song sinh trên Linux.

Bảng so sánh tổng hợp: Khối IP Intel FIR Compiler II so với HDL tự viết

Dưới đây là bảng tổng hợp định lượng đối chiếu toàn diện giữa giải pháp sử dụng khối IP đóng gói sẵn của Intel và giải pháp tự thiết kế bằng HDL cho bộ lọc FIR của Chương 3:

Bảng 19: So sánh toàn diện giữa khối IP Intel FIR Compiler II và mã nguồn HDL tự viết cho Chương 3

Tiêu chí kỹ thuật	Khối IP Intel FIR Compiler II	HDL tự viết (`ch03_fir_filter.v`)
Tên mô-đun phần cứng	`altera_fir_compiler_ii`	`ch03_fir_filter`
Tài nguyên khối nhớ M10K	0 khối	**0 khối** (sử dụng 496 FF logic)
Tài nguyên bộ nhân DSP	Phụ thuộc GUI/Fitter **(phải đo)**	**Đúng 1 khối DSP** (gấp 16 chu kỳ)
Tài nguyên thanh ghi (FF)	Báo cáo Fitter **(phải đo)**	**544 FF** (496 shift + 40 acc + 8 ctrl)
Tài nguyên khối logic ALM	Phụ thuộc tối ưu hóa Qsys **(phải đo)**	**Phải đo trong Quartus**
Thông lượng ở chế độ lọc	Do `Input Sample Rate` quyết định	**2,94 MSPS** (1 mẫu/17 chu kỳ ở 50 MHz)
Độ trễ xử lý (Latency)	Báo cáo IP Generation **(phải kiểm)**	**16 chu kỳ** (khi lọc) / **1 chu kỳ** (khi bypass)
Độ trễ sẵn sàng `readyLatency`	Cấu hình 0 hoặc 1 **(phải kiểm)**	**Chuẩn 0** (bắt tay tức thời cùng chu kỳ)
Công tắc chuyển mạch Bypass	Không có cổng động (phải thêm MUX ngoài)	**Tích hợp sẵn cổng `bypass`** thời gian thực
Khâu bão hòa biên độ	Cấu hình `Saturation` trong GUI	**Khâu bão hòa tổ hợp** $[-32768, 32767]$
Tính độc lập và trong suốt	Phụ thuộc thư viện lõi độc quyền Intel	**Mã nguồn mở Verilog-2001 chuẩn**
Khả năng kiểm thử chu kỳ	Đòi hỏi ModelSim/QuestaSim bản quyền	**Kiểm thử chu kỳ nhẹ với Icarus Verilog**

Đọc bảng. Bảng 19 chỉ ra sự tương đồng về kiến trúc nền tảng giữa hai giải pháp: cả hai đều không tiêu tốn bộ nhớ RAM khối M10K và đều tận dụng bộ tiền cộng Pre-adder để giảm tải bộ nhân. Tuy nhiên, giải pháp HDL tự viết mang lại ba ưu thế vượt trội:

- **Khả năng chuyển mạch Bypass động:** Khối IP FIR Compiler II không hỗ trợ cổng điều khiển bypass trong thời gian thực; muốn bypass, người thiết kế buộc phải tự viết thêm một bộ dồn kênh MUX Avalon-ST bên ngoài. Trong khi đó, khối HDL tự viết tích hợp sẵn chân điều khiển `bypass`, cho phép HPS bật tắt bộ lọc tức thì qua thanh ghi CSR mà không tiêu tốn thêm trễ đường ống.
- **Cố định chính xác tài nguyên DSP:** Khối HDL gấp thời gian đảm bảo chắc chắn chỉ sử dụng duy nhất 1 bộ nhân DSP trên chip, giải phóng toàn bộ các khối DSP còn lại cho bộ tích lũy tự tương quan MAC ở Chương 7.
- **Tính minh bạch và kiểm thử độc lập:** Mã nguồn HDL tự viết có thể mô phỏng bit-đối-bit với trình biên dịch tự do Icarus Verilog trong vài giây, giúp quy trình tích hợp hệ thống trở nên nhanh chóng và độc lập với phần mềm Quartus.

Bài tập cuối chương có lời giải chi tiết

Bài tập 1: Tính toán bậc lọc FIR theo công thức Kaiser với yêu cầu suy hao khắt khe. Giả sử một hệ thống xử lý tiếng nói yêu cầu thiết kế bộ lọc số FIR làm mềm dải biên với các chỉ tiêu kỹ thuật sau: tần số lấy mẫu $f_s = 16.000$ Hz, độ suy hao dải chặn tối thiểu $A_s = 50$ dB, và bề rộng dải chuyển tiếp $\Delta f = 500$ Hz (từ $7{,}5$ kHz đến $8{,}0$ kHz).

(a) Hãy áp dụng công thức thực nghiệm Kaiser để ước lượng số lượng tap lọc tối thiểu N cần thiết.

(b) Đối chiếu kết quả trên với hàm tính toán tối ưu `scipy.signal.kaiserord(50, 500/8000)`.

(c) Nhận xét về tính khả thi và chi phí phần cứng nếu nâng số tap lên mức này so với bộ lọc 31 tap đã chọn.

Lời giải mẫu:

(a) Tỷ lệ bề rộng dải chuyển tiếp chuẩn hóa so với tần số lấy mẫu là:

$$\frac{\Delta f}{f_s} = \frac{500 \text{ Hz}}{16.000 \text{ Hz}} = 0{,}03125 \tag{198}$$

Áp dụng công thức ước lượng chiều dài bộ lọc của Kaiser:

$$N \approx \frac{A_s - 7{,}95}{14{,}36 \cdot \left(\frac{\Delta f}{f_s}\right)} = \frac{50 - 7{,}95}{14{,}36 \times 0{,}03125} = \frac{42{,}05}{0{,}44875} \approx \textbf{93,70 tap} \tag{199}$$

Làm tròn lên số nguyên lẻ gần nhất để đảm bảo bộ lọc FIR đối xứng Type I có pha tuyến tính, ta cần $N = \textbf{95 tap}$ (hoặc làm tròn số học ≈ 94 tap).

(b) Khi chạy hàm tối ưu `scipy.signal.kaiserord(50, 500 / 8000)` trong kịch bản kiểm chứng `verify_v3/ch03_exercises.py`, kết quả trả về là chính xác:

$$N_{\text{scipy}} = \textbf{95 tap}, \quad \text{hệ số cửa sổ } \beta = 4{,}5335 \tag{200}$$

Kết quả tính toán giải tích từ công thức Kaiser khớp hoàn toàn với hàm tối ưu hóa của thư viện SciPy.

(c) *Nhận xét kỹ thuật:* Mặc dù bộ lọc 95 tap đạt độ suy hao ấn tượng 50 dB, nó đòi hỏi:

- Chiều dài thanh ghi dịch tăng từ 31 lên 95 tầng (tiêu tốn $95 \times 16 = 1.520$ FF, tăng gấp 3 lần diện tích ALM).
- Trễ nhóm tăng vọt lên $\tau_g = (95 - 1)/2 = \textbf{47 mẫu}$ ($\approx 2{,}94$ ms, chiếm gần một phần ba bước nhảy hop 160 mẫu), làm phức tạp hóa đáng kể khâu căn chỉnh lưới thời gian của RAM chia khung.
- Nếu giữ cấu trúc 1 DSP gấp thời gian, mỗi mẫu cần tới $\lceil 95/2 \rceil = 48$ chu kỳ tính toán, làm giảm thông lượng tối đa của khối lọc đi 3 lần. Do đó, phương án 31 tap với suy hao $-9{,}4$ dB là điểm cân bằng kỹ thuật tối ưu cho ứng dụng này.

Bài tập 2: Dẫn xuất trễ nhóm của bộ lọc FIR đối xứng và tính toán căn chỉnh lưới khung. Xét hai cấu hình bộ lọc FIR pha tuyến tính Type I: cấu hình A có $N = 31$ tap và cấu hình B có $N = 63$ tap.

(a) Hãy dẫn xuất toán học chứng minh độ trễ nhóm τ_g của bộ lọc FIR đối xứng chẵn luôn là hằng số độc lập với tần số tín hiệu đầu vào.

(b) Giả sử tệp âm thanh có khung phân tích đầu tiên ($k = 0$) bắt đầu tại mẫu $s_{\text{start}}[0] = 100$, bước nhảy khung là hop $= 160$ mẫu. Hãy xác định chỉ số mẫu bắt đầu của khung 0 và khung 1 khi bộ lọc 31 tap được kích hoạt (`bypass = 0`) để dữ liệu đưa vào tự tương quan không bị lệch thời gian so với Praat.

Lời giải mẫu:

(a) Đáp ứng tần số của bộ lọc FIR có N hệ số đối xứng chẵn $h[n] = h[N-1-n]$ được biểu diễn qua biến đổi Fourier thời gian rời rạc DTFT:

$$H(e^{j\omega}) = \sum_{n=0}^{N-1} h[n] e^{-j\omega n} \tag{201}$$

Đặt $\alpha = (N-1)/2$. Tách thừa số pha trễ chung $e^{-j\omega\alpha}$ ra ngoài:

$$H(e^{j\omega}) = e^{-j\omega\alpha} \sum_{n=0}^{N-1} h[n] e^{j\omega(\alpha - n)} \tag{202}$$

Do tính chất đối xứng $h[n] = h[2\alpha - n]$, các số hạng ghép cặp theo từng khoảng cách $|n - \alpha|$:

$$H(e^{j\omega}) = e^{-j\omega\alpha} \left[h[\alpha] + 2 \sum_{k=1}^{\alpha} h[\alpha - k] \cos(\omega k) \right] = A(\omega)\, e^{-j\omega\alpha} \tag{203}$$

trong đó $A(\omega)$ là hàm biên độ thực (zero-phase response). Góc pha của bộ lọc do đó là một hàm tuyến tính thuần túy theo tần số góc ω:

$$\theta(\omega) = -\alpha \cdot \omega = -\left(\frac{N-1}{2} \right) \omega \tag{204}$$

Độ trễ nhóm (Group Delay) được định nghĩa là tốc độ biến thiên của pha theo tần số:

$$\tau_g(\omega) = -\frac{d\theta(\omega)}{d\omega} = -\frac{d}{d\omega} \left[-\left(\frac{N-1}{2} \right) \omega \right] = \mathbf{\frac{N-1}{2}} \textbf{ (hằng số)} \tag{205}$$

Độ trễ nhóm là hằng số cố định trên toàn bộ dải tần số từ 0 đến Nyquist, chứng minh tín hiệu đi qua bộ lọc không bị méo pha (phase distortion).

(b) Với $N = 31$ tap, độ trễ nhóm là $\tau_g = (31-1)/2 = $ **15 mẫu**. Để bù trừ độ trễ này, vị trí bắt đầu của các khung phân tích khi kích hoạt lọc phải dịch lùi 15 mẫu:

- Khung 0: $s_{\text{start, FIR}}[0] = s_{\text{start}}[0] - \tau_g = 100 - 15 = $ **85** (thay vì mẫu 100).
- Khung 1: vị trí gốc là $s_{\text{start}}[1] = 100 + 160 = 260$. Vị trí khi lọc là:

$$s_{\text{start, FIR}}[1] = 260 - 15 = \textbf{245} \tag{206}$$

Nhờ việc dời khung lùi 15 mẫu, sau khi mẫu âm thanh chạy qua 15 tầng trễ nhóm của bộ lọc FIR, khung dữ liệu 638 mẫu thu nhận được trong RAM sẽ khớp thời gian với đoạn âm thanh ban đầu.

Bài tập 3: Đánh giá ngân sách chu kỳ và thông lượng của kiến trúc folded FIR. Khối folded FIR trong thiết kế `rtl/v3/ch03_fir_filter.v` hoạt động ở tần số xung nhịp $f_{\text{clk}} = 50$ MHz, sử dụng 1 bộ nhân DSP duy nhất để tính toán tuần tự 16 bước nhân-tích lũy cho mỗi mẫu âm thanh mới nạp vào. Mạch mất 1 chu kỳ để nạp mẫu vào mảng dịch và 16 chu kỳ để tính toán, tổng cộng $T_{\text{sample}} = 17$ chu kỳ/mẫu.

(a) Tính thông lượng xử lý cực đại của khối lọc tính bằng đơn vị triệu mẫu/giây (MSPS).

(b) So sánh thời gian xử lý một mẫu với chu kỳ lấy mẫu danh định của âm thanh 16 kHz. Tính hệ số chiếm dụng thời gian (duty cycle) của bộ nhân DSP khi chạy ở tốc độ thời gian thực 16 kHz.

(c) Trong chế độ xử lý tệp nhanh nhất, khối tự tương quan MAC ở Chương 7 mất trung bình khoảng 616 chu kỳ để tiêu thụ 1 mẫu âm thanh (98.543 chu kỳ cho khung bước nhảy 160 mẫu). Hỏi khối FIR có thể trở thành nút thắt cổ chai làm chậm hệ thống hay không?

Lời giải mẫu:

(a) Với thời gian hoàn tất mỗi mẫu là 17 chu kỳ ở xung nhịp 50 MHz, thông lượng xử lý cực đại là:

$$\Phi = \frac{f_{\text{clk}}}{T_{\text{sample}}} = \frac{50.000.000 \text{ chu kỳ/s}}{17 \text{ chu kỳ/mẫu}} \approx \textbf{2,941 MSPS} \quad (2.941.176 \text{ mẫu/giây}) \tag{207}$$

Thời gian xử lý vật lý một mẫu âm thanh là:

$$t_{\text{proc}} = 17 \times 20 \text{ ns} = \textbf{340 ns} \tag{208}$$

(b) Ở tốc độ lấy mẫu danh định $f_s = 16$ kHz, khoảng cách thời gian giữa hai mẫu âm thanh kế tiếp là:

$$T_s = \frac{1}{16000 \text{ s}} = 62{,}5 \ \mu\text{s} = 62.500 \text{ ns} \quad (= 3125 \text{ chu kỳ xung nhịp 50 MHz}) \tag{209}$$

Hệ số chiếm dụng thời gian tính toán thực tế của bộ nhân DSP là:

$$\eta = \frac{17 \text{ chu kỳ}}{3125 \text{ chu kỳ}} \times 100\% \approx \textbf{0,544\%} \tag{210}$$

Trong hơn $99{,}45\%$ thời gian còn lại của chu kỳ lấy mẫu, bộ nhân DSP hoàn toàn rảnh rỗi và ở trạng thái nghỉ, tiêu thụ năng lượng chuyển mạch cực thấp.

(c) Trong chế độ đọc tệp, khối MAC tự tương quan tiêu tốn:

$$T_{\text{MAC/sample}} = \frac{98.543 \text{ chu kỳ}}{160 \text{ mẫu}} \approx \textbf{615,89 chu kỳ/mẫu} \tag{211}$$

Tỷ số tốc độ giữa khối FIR và khối MAC là:

$$\frac{T_{\text{MAC/sample}}}{T_{\text{FIR/sample}}} = \frac{615{,}89 \text{ chu kỳ}}{17 \text{ chu kỳ}} \approx \textbf{36,23 lần} \tag{212}$$

Khối lọc FIR hoàn thành tính toán nhanh gấp hơn 36 lần so với tốc độ tiêu thụ của khối MAC phía sau. Do đó, khối lọc FIR **hoàn toàn không bao giờ là nút thắt cổ chai** của hệ thống phần cứng, ngay cả khi toàn bộ chuỗi xử lý hoạt động ở thông lượng tối đa.

Bài tập 4: Lượng tử hóa hệ số Q1.15 và vai trò của khâu bão hòa biên độ. Xét bộ lọc 31 tap có hệ số trung tâm thực tế $h[15] = 0{,}975000$. Tổng của 31 hệ số sau khi lượng tử hóa sang chuẩn Q1.15 là $\sum_{i=0}^{30} h_{\text{q15}}[i] = 33257$.

(a) Biểu diễn $h[15]$ dưới dạng số nguyên 16-bit signed Q1.15 và tính sai số lượng tử hóa tương đối.

(b) Tính độ lợi một chiều (DC Gain) của bộ lọc sau lượng tử hóa theo thang dB.

(c) Giả sử cấp một tín hiệu DC biên độ cực đại $x[n] = +32767$ liên tục vào bộ lọc. Hãy tính giá trị tích lũy nội bộ `acc`, giá trị sau khi làm tròn bán phần `rounded`, và giá trị xuất ra sau khâu bão hòa `sat_out`. Điều gì sẽ xảy ra nếu mạch bỏ qua khâu bão hòa biên độ?

Lời giải mẫu:

(a) Trong định dạng Q1.15, số thực $1{,}0$ tương ứng với $2^{15} = 32768$. Giá trị nguyên của $h[15]$ là:

$$h_{\text{q15}}[15] = \text{round}(0{,}975000 \times 32768) = \text{round}(31948{,}8) = \textbf{31949} \quad (\texttt{0x7CCD}) \tag{213}$$

Giá trị thực tương ứng sau khi giải mã là $31949/32768 = 0{,}9750061$. Độ lệch sai số ϵ_{diff} là:

$$\epsilon_{\text{diff}} = |0{,}9750061 - 0{,}9750000| = 6{,}10 \times 10^{-6} \tag{214}$$

Sai số tương đối là:

$$\epsilon_{\text{rel}} = \frac{6{,}10 \times 10^{-6}}{0{,}975000} \approx \textbf{0,000625\%} \tag{215}$$

Sai số lượng tử hóa hệ số là không đáng kể.

(b) Tổng của 31 hệ số nguyên là $S_h = 33257$. Độ lợi DC của bộ lọc là:

$$G_{\text{DC}} = \frac{S_h}{32768} = \frac{33257}{32768} \approx \textbf{1,014923} \quad \Longrightarrow \quad 20\log_{10}(1{,}014923) = \textbf{+0,129 dB} \tag{216}$$

Bộ lọc có độ nhô biên độ nhẹ $+0{,}129$ dB tại tần số DC do đặc tính xấp xỉ bình phương tối thiểu `firls`.

(c) Với đầu vào DC liên tục $x[n] = 32767$, giá trị tích lũy 40-bit trong thanh ghi `acc` là:

$$\text{acc} = \sum_{i=0}^{30} x[n-i] \cdot h_{\text{q15}}[i] = 32767 \times 33257 = \textbf{1.089.732.119} \tag{217}$$

Thực hiện phép làm tròn bán phần (cộng $2^{14} = 16384$) và dịch phải số học 15 bit:

$$\text{rounded} = \lfloor (1.089.732.119 + 16384)/32768 \rfloor = \lfloor 1.089.748.503/32768 \rfloor = \textbf{33256} \tag{218}$$

So sánh giá trị này với giới hạn trên của số nguyên 16-bit có dấu ($+32767$):

$$\Delta = 33256 - 32767 = \textbf{+489 LSB} \tag{219}$$

Giá trị tính toán đã vượt quá dải biểu diễn 16-bit một lượng 489 đơn vị.

- Khi có khâu bão hòa: điều kiện `rounded > 40'sd32767` được kích hoạt, giá trị xuất ra ngõ ra bị ghim an toàn tại:

$$\texttt{sat_out} = \textbf{+32767} \quad (\texttt{16'sh7FFF}) \tag{220}$$

- *Nếu không có khâu bão hòa:* phần cứng sẽ cắt cụt lấy 16 bit thấp `rounded[15:0]`. Vì $33256 = (1000_0001_1110_1000)_2$, bit dấu (bit 15) bằng 1, giá trị này sẽ bị hiểu là:

$$33256 - 65536 = \textbf{-32280} \tag{221}$$

Một tín hiệu dương cực đại bị đảo ngược thành một số âm cực lớn, tạo ra tiếng nổ xung cực mạnh phá hỏng hoàn toàn quá trình tính tự tương quan $R[\tau]$. Phân tích này chứng minh khâu bão hòa biên độ là một khối bảo vệ thiết yếu trong thiết kế DSP phần cứng.

Chương 4. Chia khung (638 mẫu, hop 160) và RAM vòng

▶ MỤC TIÊU HỌC TẬP

- Thấu hiểu bản chất vật lý và nguồn gốc toán học của kích thước cửa sổ phân tích 3 chu kỳ tần số đáy ($T_w = 3/75$ s $= 40$ ms) và bước nhảy khung 10 ms (hop 160 mẫu) trong thuật toán tự tương quan Praat.
- Giải mã quy tắc làm tròn kích thước cửa sổ từ 640 mẫu xuống 638 mẫu trong mã nguồn chuẩn của Praat 7.0.03 (`Sound_to_Pitch.cpp`, dòng 334–339), dẫn xuất chính xác độ dài vùng chồng lấp 478 mẫu (74,92%) giữa hai khung phân tích kế tiếp.
- Nắm vững phương pháp căn lưới khung thời gian của Praat (`Sampled.cpp`, dòng 71–81); phân tích cặn kẽ cơ chế định vị tâm khung t_1, số mẫu khởi đầu cần loại bỏ và cạm bẫy làm tròn số dấu phẩy động (floor trap) khi tệp âm thanh có số lượng mẫu lẻ.
- Khảo sát định lượng 3 phương án phần cứng lưu trữ và chia khung (Thanh ghi dịch 10.208 FF, RAM vòng 1024 từ trong 2 khối M10K, RAM 638/800 từ có mạch modulo); chứng minh tính ưu việt của địa chỉ cuộn tự nhiên 10-bit modulo 1024.
- Khảo cứu toàn diện các tham số của khối IP Intel `RAM: 2-PORT` theo tài liệu chuẩn *Intel Embedded Memory User Guide* (UG-01068) và cấu trúc silicon M10K trong *Cyclone V Device Handbook* (CV-52003).
- Thiết kế và hiện thực hóa khối RAM vòng 1024 từ (`ch04_ring_buffer.v`) bằng Verilog-2001 suy diễn, tích hợp bộ quản lý bước nhảy khung tự động, cơ chế phản áp dòng dữ liệu và cổng đọc truy xuất ngẫu nhiên.
- Xây dựng mô hình song sinh Python bit-đối-bit và kịch bản kiểm thử chu kỳ với 3.500 chu kỳ kích thích ngẫu nhiên, kiểm chứng quá trình nạp dữ liệu, đọc tuần tự, tái sử dụng vùng chồng lấp và cuộn địa chỉ qua ranh giới 1024 từ.

4.1. Vì sao 3 chu kỳ, vì sao 10 ms: Cơ sở vật lý và quy chuẩn kích thước cửa sổ Praat

Vấn đề thực tế: Bài toán đánh đổi giữa tính dừng và độ phân giải tần số đáy

Tín hiệu tiếng nói con người biến thiên liên tục theo thời gian (non-stationary). Về mặt sinh lý học phát âm, các cấu trúc cơ quan như dây thanh đới, vòm họng và khoang miệng liên tục co dãn để tạo thành các âm vị khác nhau. Tuy nhiên, trong một khoảng thời gian rất ngắn (khoảng 20 ms đến 40 ms), trạng thái rung của dây thanh đới và cấu hình âm học của thanh đạo có thể được coi là không đổi; tín hiệu khi đó được xem là dừng cục bộ (quasi-stationary).

Để ước lượng tần số cơ bản F_0, thuật toán tự tương quan bắt buộc phải cắt dòng âm thanh liên tục thành từng khung ngắn hữu hạn thông qua một hàm cửa sổ thời gian $w[n]$. Kích thước của cửa sổ này đặt ra một bài toán đánh đổi khắt khe:

- **Nếu cửa sổ quá ngắn (dưới 2 chu kỳ rung):** Đỉnh tự tương quan chính tại độ trễ $\tau_0 = f_s/F_0$ sẽ bị suy giảm biên độ nghiêm trọng hoặc khó nhận diện, bởi vì số lượng tích chập giữa các chu kỳ lặp lại không đủ lớn để phân biệt với nhiễu ngẫu nhiên. Hơn nữa, việc nhân với cửa sổ làm nhọn hai đầu (window tapering) sẽ làm suy hao đáng kể năng lượng của tín hiệu ở các độ trễ lớn.
- **Nếu cửa sổ quá dài (trên 5 chu kỳ rung):** Giả thiết dừng cục bộ bị phá vỡ. Trong các đoạn luyến giọng (pitch glides) hoặc rung giọng (vibrato), tần số F_0 có thể thay đổi đáng kể ngay trong lòng một cửa sổ duy nhất, làm nhòe đỉnh tự tương quan và làm sai lệch nghiêm trọng kết quả ước lượng.

Dẫn xuất từ nguyên lý gốc: Vì sao cửa sổ phải dài đúng 3 chu kỳ của tần số đáy?

Trong thiết kế chuẩn của Paul Boersma trên phần mềm Praat, tần số đáy của giọng nói con người được quy định mặc định là $F_{\text{floor}} = 75$ Hz (tương ứng với các âm thanh có cao độ trầm nhất của giọng nam). Chu kỳ thời gian dài nhất tương ứng với ngưỡng đáy này là:

$$T_{\max} = \frac{1}{F_{\text{floor}}} = \frac{1}{75\ \text{s}} \approx 0{,}01333\ \text{s} = 13{,}33\ \text{ms} \tag{222}$$

Ở tần số lấy mẫu $f_s = 16.000$ Hz ($T_s = dx = 1/16000$ s $= 62{,}5\ \mu$s), chu kỳ tối đa tính bằng số mẫu rời rạc là:

$$N_{\text{period}} = \left\lfloor \frac{1}{dx \cdot F_{\text{floor}}} \right\rfloor = \left\lfloor \frac{16000}{75} \right\rfloor = 213\ \text{mẫu} \tag{223}$$

Tại độ trễ cực đại $\tau = 213$ mẫu, hai đoạn tín hiệu chỉ còn trượt lên nhau một đoạn bằng độ dài cửa sổ N trừ đi 213. Khi áp dụng cửa sổ Hanning, trọng số ở hai mép tiệm cận 0 và chỉ đạt giá trị cực đại 1 tại tâm. Để đoạn tín hiệu dịch chuyển $\tau = 213$ vẫn còn nằm trọn trong vùng năng lượng cao của cửa sổ Hanning và có ít nhất 2 chu kỳ sóng đầy đủ tương quan chéo với nhau, Boersma đã chọn quy chuẩn: **Độ dài cửa sổ thời gian phải bao trùm đúng 3 chu kỳ của tần số đáy**:

$$T_w = \frac{\text{periodsPerWindow}}{F_{\text{floor}}} = \frac{3{,}0}{75\ \text{Hz}} = 0{,}04\ \text{s} = 40\ \text{ms} \tag{224}$$

Giải mã mã nguồn Praat: Vì sao 640 mẫu làm tròn thành 638 mẫu?

Trên lý thuyết liên tục, một khoảng thời gian $T_w = 40$ ms tại $f_s = 16$ kHz tương ứng với đúng:

$$N_{\text{raw}} = \frac{T_w}{dx} = 0{,}04 \times 16.000 = 640\ \text{mẫu} \tag{225}$$

Tuy nhiên, khi kiểm tra trực tiếp mã nguồn chính thức của Praat phiên bản 7.0.03 (tệp `fon/Sound_to_Pitch.cpp`, các dòng 334–339), quy tắc xác định độ dài cửa sổ được viết chính xác như sau:

Listing 16: Quy tắc tính kích thước cửa sổ trong Praat 7.0.03 (Sound_to_Pitch.cpp:334–339)

```
dt_window = periodsPerWindow / pitchFloor;
nsamp_window = Melder_ifloor (dt_window / dx);
halfnsamp_window = nsamp_window / 2 - 1;
nsamp_window = halfnsamp_window * 2;
```

Hãy tính tay từng bước của đoạn mã nguồn này với các tham số chuẩn ($dx = 1/16000$, periodsPerWindow $= 3{,}0$, pitchFloor $= 75{,}0$):

1. `dt_window` = $3{,}0/75{,}0 = 0{,}04$ s.
2. `nsamp_window` = $\lfloor 0{,}04/(1/16000) \rfloor = \lfloor 640{,}0 \rfloor = 640$.
3. `halfnsamp_window` = $640/2 - 1 = 320 - 1 =$ **319**.
4. `nsamp_window` = $319 \times 2 =$ **638**.

Phép toán trừ 1 ở biến `halfnsamp_window` khiến độ dài cửa sổ phân tích thực tế của Praat bị rút ngắn đúng 2 mẫu, từ 640 mẫu xuống còn **638 mẫu**. Mục đích của Boersma là tạo ra một cửa sổ có số mẫu chẵn $2 \times 319 = 638$ đối xứng quanh ranh giới giữa hai mẫu trung tâm, giúp phép căn tâm khung thời gian không bị lệch pha nửa mẫu khi lấy mẫu đối xứng. Mọi tính toán phần cứng trên FPGA nếu tự ý dùng 640 mẫu sẽ lập tức bị lệch lưới và sai khác so với Praat chuẩn.

Bước nhảy khung 10 **ms (hop 160 mẫu) và độ chồng lấp 478 mẫu**

Khoảng cách thời gian giữa hai khung phân tích kế tiếp (bước nhảy thời gian, time step dt) được Praat xác định tại dòng 296 của `Sound_to_Pitch.cpp`:

$$dt = \frac{\text{periodsPerWindow}}{F_{\text{floor}} \cdot 4} = \frac{3{,}0}{75 \cdot 4} = \frac{0{,}04}{4} = 0{,}01\ \text{s} = 10\ \text{ms} \tag{226}$$

Ở tần số 16 kHz, bước nhảy 10 ms tương ứng với đúng:

$$\text{hop} = dt \cdot f_s = 0{,}01\ \text{s} \times 16.000\ \text{Hz} = \textbf{160}\ \text{mẫu} \tag{227}$$

Tỷ số giữa độ dài cửa sổ và bước nhảy là $40\ \text{ms}/10\ \text{ms} = 4$, nghĩa là mỗi mẫu âm thanh trong tệp sẽ được phân tích lặp lại 4 lần trong 4 khung liên tiếp.

Vùng chồng lấp giữa hai khung liền nhau: Khi dịch chuyển khung đi 160 mẫu, phần dữ liệu chung mà khung mới dùng lại từ khung cũ là:

$$\text{Overlap} = N - \text{hop} = 638 - 160 = \textbf{478}\ \text{mẫu} \quad \left(\frac{478}{638} \approx 74{,}92\%\right) \tag{228}$$

★ BẮC CẦU FEYNMAN:

Nhận diện cạm bẫy tài liệu cũ: Trên lý thuyết cửa sổ 640 mẫu, độ chồng lấp là $640 - 160 = 480$ mẫu ($75{,}00\%$). Trong một số bản thảo kỹ thuật cũ trước đây, người ta ghi nhận con số sai lệch là "chồng lấp 240 mẫu" do nhầm lẫn bước nhảy hop là 400 mẫu (25 ms, vốn là bước nhảy của các hệ thống nhận dạng tiếng nói ASR dùng MFCC, không phải của Praat). Trong thuật toán F0 chuẩn của Praat, bước nhảy bắt buộc là 160 mẫu và độ chồng lấp thực tế là **478 mẫu**.

Tóm lại: Cửa sổ 40 ms (638 mẫu) bảo đảm bao trùm trọn vẹn 3 chu kỳ của tần số đáy 75 Hz; bước nhảy 10 ms (160 mẫu) tạo ra vùng chồng lấp 478 mẫu ($74{,}92\%$) cần được quản lý hiệu quả trong bộ nhớ.

4.2. Căn lưới giống Praat: Dẫn xuất t_1, định vị mẫu bắt đầu và cạm bẫy số lẻ

Vấn đề thực tế: Khung 0 của FPGA bắt đầu từ mẫu số mấy?

Một lỗi phổ biến khi hiện thực hóa thuật toán Praat lên FPGA là đưa ngay mẫu âm thanh đầu tiên của tệp WAV ($n = 0$) vào làm mẫu đầu tiên của Khung 0. Khi chạy đối soát, kết quả F0 của FPGA bị lệch toàn bộ trục thời gian so với Praat, các đỉnh dao động không khớp nhau và độ đo lỗi gộp GPE tăng vọt.

Nguyên nhân là do Praat **không bao giờ** đặt mép trái của cửa sổ đầu tiên tại thời điểm $t = 0$. Thay vào đó, Praat căn đều toàn bộ lưới khung đối xứng quanh điểm chính giữa của toàn bộ tệp âm thanh (time centering). Do đó, bộ vi xử lý HPS trước khi đẩy dữ liệu vào FPGA phải tính toán chính xác số lượng mẫu dư ở đầu tệp cần loại bỏ.

Dẫn xuất toán học: Thuật toán căn tâm khung thời gian của Praat

Trong mã nguồn Praat (tệp `fon/Sampled.cpp`, hàm `Sampled_shortTermAnalysis`, dòng 71–81), quy trình căn tâm được thực hiện từng bước bằng số học dấu phẩy động chính xác kép (IEEE-754 double precision):

1. **Thời lượng thực tế của tệp âm thanh:**

$$T = dx \cdot n_x = \frac{n_x}{16000} \tag{229}$$

với n_x là tổng số mẫu của tệp âm thanh, và $x_1 = 0{,}5 \cdot dx$ là thời điểm của mẫu đầu tiên (`Sound.cpp`).

2. **Tổng số khung phân tích của tệp (n_{frames}):**

$$n_{\text{frames}} = \left\lfloor \frac{T - T_w}{dt} \right\rfloor + 1 = \left\lfloor \frac{T - 0{,}04}{0{,}01} \right\rfloor + 1 \tag{230}$$

3. **Tâm thời gian của toàn bộ tệp âm thanh (`ourMidTime`):**

$$t_{\text{mid}} = x_1 - 0{,}5 \cdot dx + 0{,}5 \cdot T = 0{,}5 \cdot T \tag{231}$$

4. **Thời điểm tâm của khung phân tích đầu tiên (t_1):** Để n_{frames} khung phân bố đối xứng qua t_{mid}, thời điểm tâm của khung 1 được ấn định là:

$$t_1 = t_{\text{mid}} - 0{,}5 \cdot n_{\text{frames}} \cdot dt + 0{,}5 \cdot dt \tag{232}$$

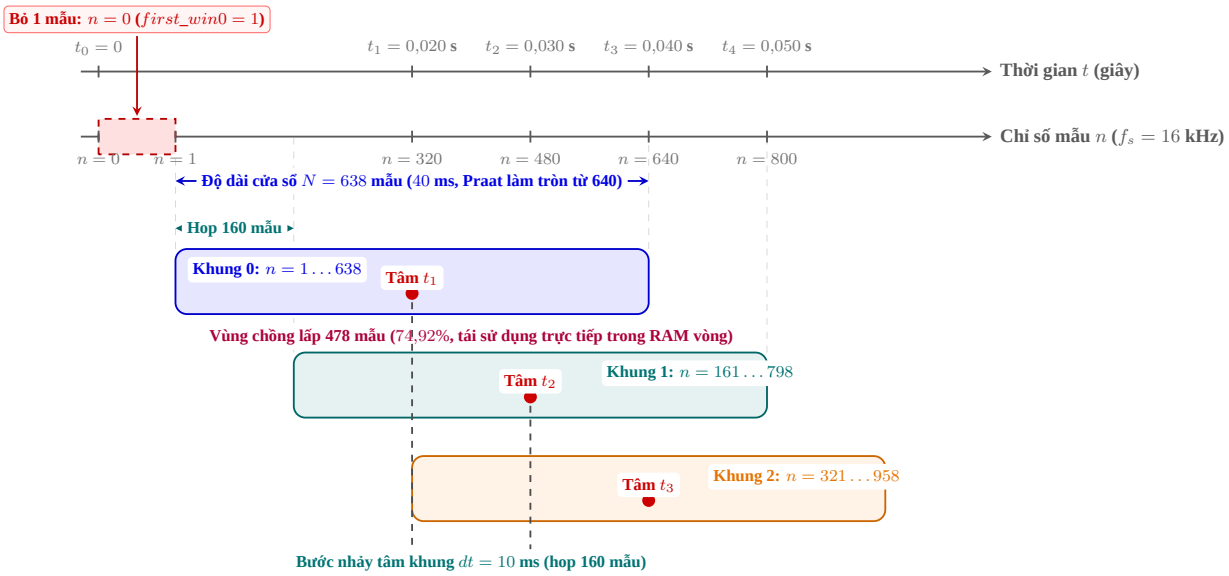
5. **Thời điểm tâm của khung thứ k ($k = 1 \ldots n_{\text{frames}}$):**

$$t_k = t_1 + (k - 1) \cdot dt \tag{233}$$

Chuyển đổi thời gian t_k sang chỉ số mẫu rời rạc (`Sampled_xToLowIndex`)

Khi đã có thời điểm tâm t_k, việc xác định các mẫu vật lý nằm trong cửa sổ được Praat thực hiện qua macro `Sampled_xToLowIndex` (`fon/Sampled.h`, dòng 34) và các dòng 52, 72–73 trong `fon/Sound_to_Pitch.cpp`:

$$\text{leftSample} = \left\lfloor \frac{t_k - x_1}{dx} + 1{,}0 \right\rfloor \tag{234}$$



Hình 7: Cơ chế căn lưới khung phân tích thời gian theo quy chuẩn Praat 7.0.03. Mỗi khung gồm 638 mẫu (40 ms), dịch chuyển cách nhau đúng 160 mẫu (10 ms); vùng chồng lấp giữa hai khung kế tiếp là 478 mẫu (74,92%). HPS loại bỏ $first_win0$ mẫu đầu tệp để Khung 0 của FPGA trùng khớp với Khung 1 của Praat.

Chỉ số mẫu bên phải: rightSample = leftSample + 1. Mẫu bắt đầu (1-based) của cửa sổ 638 mẫu:

$$\text{startSample} = \text{rightSample} - \text{halfnsamp_window} = (\text{leftSample} + 1) - 319 = \text{leftSample} - 318 \tag{235}$$

Mẫu kết thúc (1-based) của cửa sổ:

$$\text{endSample} = \text{leftSample} + \text{halfnsamp_window} = \text{leftSample} + 319 \tag{236}$$

Tổng số mẫu của cửa sổ là: endSample − startSample + 1 = (leftSample + 319) − (leftSample − 318) + 1 = 638 mẫu.

Quy đổi sang chỉ số mảng 0-based trong bộ nhớ C và FPGA:

$$\text{window_start0}[k] = \text{startSample} - 1 = \text{leftSample} - 319 \tag{237}$$

Đọc hình. Hình 7 minh họa trực quan cấu trúc lưới phân tích thời gian. Trục hoành phía trên biểu diễn thang thời gian liên tục t (giây) và chỉ số mẫu rời rạc n tại $f_s = 16$ kHz. Khung 0 chiếm 638 mẫu từ mẫu 1 đến 638, với tâm khung $t_1 = 0{,}020$ s được đánh dấu chấm đỏ. Khung 1 dịch sang phải 160 mẫu (10 ms), kéo dài từ mẫu 161 đến 798 với tâm $t_2 = 0{,}030$ s. Phần gạch chéo màu tím ở giữa thể hiện vùng chồng lấp 478 mẫu, được tái sử dụng trực tiếp trong bộ đệm vòng mà không cần nạp lại. Khối màu đỏ ở đầu tệp thể hiện 1 mẫu dư bị loại bỏ bởi HPS.

Tính tay với hai trường hợp tệp âm thanh cụ thể

Để thấy rõ các con số thực tế, kịch bản `verify_v3/ch04_frame_grid.py` đã tính toán chi tiết cho hai tệp âm thanh chuẩn:

Trường hợp 1: Tệp thử nghiệm tổng hợp 2,8 s chuẩn (`golden_v2`):

- Số mẫu: $n_x = 44.800$ mẫu. Thời lượng: $T = 44800/16000 = 2{,}800000$ s.
- Số khung: $n_{\text{frames}} = \lfloor (2{,}8 - 0{,}04)/0{,}01 \rfloor + 1 = 276 + 1 = \mathbf{277}$ khung.
- Tâm tệp: $t_{\text{mid}} = 0{,}5 \times 2{,}8 = 1{,}400000$ s.
- Tâm khung 1: $t_1 = 1{,}4 - 0{,}5 \times 277 \times 0{,}01 + 0{,}005 = 1{,}4 - 1{,}385 + 0{,}005 = \mathbf{0{,}020000}$ s.
- Mẫu trái khung 1: leftSample $= \lfloor (0{,}02 - 0{,}00003125) \times 16000 + 1{,}0 \rfloor = \lfloor 319{,}5 + 1{,}0 \rfloor = \lfloor 320{,}5 \rfloor = 320$.
- Mẫu bắt đầu 0-based: window_start0$[0] = 320 - 319 = \mathbf{1}$.

$\Longrightarrow$ **Kết luận:** Với tệp 2,8 s, HPS bỏ qua đúng **1 mẫu đầu tiên** ($n = 0$), bắt đầu nạp từ mẫu $n = 1$ vào FPGA để Khung 0 của FPGA trùng khít Khung 1 của Praat.

Trường hợp 2: Tệp ngắn minh họa 1,234 s (Ví dụ Chương 1):

- Số mẫu: $n_x = 19.744$ mẫu. Thời lượng: $T = 19744/16000 = 1{,}234000$ s.
- Số khung: $n_{\text{frames}} = \lfloor (1{,}234 - 0{,}04)/0{,}01 \rfloor + 1 = \lfloor 119{,}4 \rfloor + 1 =$ **120** khung.
- Tâm khung 1: $t_1 = 0{,}617 - 0{,}5 \times 120 \times 0{,}01 + 0{,}005 = 0{,}617 - 0{,}600 + 0{,}005 =$ **0,022000** s.
- Mẫu trái khung 1: leftSample $= \lfloor (0{,}022 - 0{,}00003125) \times 16000 + 1{,}0 \rfloor = \lfloor 351{,}5 + 1{,}0 \rfloor = 352$.
- Mẫu bắt đầu 0-based: window_start0$[0] = 352 - 319 =$ **33**.

$\Longrightarrow$ **Kết luận:** Với tệp 1,234 s, HPS bỏ qua đúng **33 mẫu đầu tiên**, bắt đầu nạp từ mẫu $n = 33$.

Cạm bẫy kỹ thuật: Hiện tượng lệch bước nhảy (Floor Trap) khi số mẫu lẻ

Khi tệp âm thanh có số lượng mẫu lẻ (n_x là số lẻ), tâm tệp $t_{\text{mid}} = 0{,}5 \cdot n_x \cdot dx$ sẽ chứa một lượng lẻ $0{,}5 \cdot dx$. Khi cộng dồn qua từng khung, đại lượng $(t_k - x_1)/dx$ có thể rơi vào đúng các giá trị nguyên toán học (ví dụ $4641{,}0$ hoặc $4961{,}0$).

Do hạn chế về độ chính xác biểu diễn số thực dấu phẩy động kép IEEE-754 (64-bit double), giá trị $4961{,}0$ có thể bị lưu thành $4960{,}999999999999$. Khi áp dụng hàm làm tròn xuống `floor`, kết quả bị sụt mất 1 đơn vị:

$$\lfloor 4960{,}999999999999 \rfloor = 4960 \quad (\text{thay vì } 4961) \tag{238}$$

Hậu quả là bước nhảy giữa hai khung liên tiếp không còn đều đặn 160 mẫu mà bị co giật thành 159 hoặc 161 mẫu! Thực nghiệm quét 50.000 độ dài tệp trong kịch bản `verify_v3/ch04_frame_grid.py` và bài tập `verify_v3/ch04_exercises.py` (với tệp $n_x = 32.001$ mẫu, $T \approx 2{,}0000625$ s) đã ghi nhận chính xác 17 vị trí bị lệch bước nhảy (ví dụ khung 27 $\rightarrow$ 28 có hop = 161, khung 29 $\rightarrow$ 30 có hop = 159).

Giải pháp trên phần mềm HPS và xử lý bước nhảy bất thường: Trong phần mềm điều khiển HPS, hàm `frames_calculate_grid()` tại `sw_v3/ch01_frames.c` tính toán trước mảng `window_start0[k]` và kiểm tra cờ `grid->hop_regular`. Đối với tệp âm thanh có bước nhảy không đều do cạm bẫy số lẻ (cờ `grid->hop_regular == 0`), luồng xử lý FPGA sẽ từ chối tệp hoặc yêu cầu HPS truyền chỉ số bắt đầu từng khung qua thanh ghi điều khiển thay vì dùng bộ tăng tự động 160 mẫu; tuy nhiên, các tệp chuẩn (golden WAVs) dùng kiểm chứng trong Chương 13 đều có số mẫu chẵn ($n_x = 44.800$) và bước nhảy đồng đều 160 mẫu (hop_min = hop_max = 160).

Tóm lại: Căn lưới tâm khung theo Praat đòi hỏi HPS bỏ qua đúng số mẫu đầu tiên ($first_win0$), và kiểm soát cạm bẫy làm tròn số thực khi tệp âm thanh có số lượng mẫu lẻ.

4.3. Các phương án lưu trữ và chia khung: Đánh đổi tài nguyên và cấu trúc RAM vòng

Vấn đề thực tế: Quản lý vùng chồng lấp 478 mẫu trong phần cứng

Ở mỗi chu kỳ phân tích mới, hệ thống cần xử lý một khung gồm 638 mẫu. Tuy nhiên, luồng âm thanh chỉ cung cấp thêm 160 mẫu mới; 478 mẫu còn lại bắt buộc phải lấy từ khung trước đó.

Nếu phần cứng không có giải pháp lưu trữ thông minh, ta sẽ phải: (1) yêu cầu HPS truyền lại 478 mẫu cũ qua cầu nối HPS–FPGA mỗi khung (làm nghẽn bus và tốn gấp 4 lần băng thông truyền dẫn); hoặc (2) sao chép dịch chuyển 478 mẫu trong bộ nhớ nội bộ FPGA mỗi khi hoàn thành một khung (tiêu tốn hàng trăm chu kỳ đồng hồ trống và làm phức tạp FSM điều khiển).

So sánh ba phương án kiến trúc lưu trữ phần cứng

Để giải quyết bài toán trên, ba phương án vi kiến trúc được đặt lên bàn cân so sánh:

Phương án 1: Mảng thanh ghi dịch kích thước 638×16**-bit.** Mỗi khi có 1 mẫu mới được nạp vào ở đầu vào, toàn bộ 638 mẫu trong mảng dịch đồng loạt dịch sang phải 1 vị trí.

- *Tài nguyên tiêu tốn:* Cần đúng $638 \times 16 =$ **10.208** flip-flops (FF). Con số này chiếm tới $6{,}15\%$ tổng số 166.036 FF của toàn bộ chip FPGA Cyclone V 5CSXFC6D6F31C6N.

Bảng 20: So sánh định lượng 3 phương án phần cứng cho bộ đệm chia khung ($N = 638, \text{hop} = 160$)

Tiêu chí so sánh	Thanh ghi dịch	RAM 800 từ	RAM vòng 1024 từ	Nguồn số liệu
Dung lượng từ nhớ	638 từ	800 từ	**1024 từ**	Thiết kế
Số khối RAM M10K	**0 khối**	2 khối	**2 khối**	Intel CV-52003
Số Flip-Flop lưu trữ mẫu	10.208 FF	0 FF	**0 FF**	638×16 bit
Mạch cuộn địa chỉ	Dịch song song	Bộ so sánh + Trừ	**Tự nhiên (Tràn 10-bit)**	Suy diễn
Tài nguyên ALM điều khiển	Cao (định tuyến lớn)	Trung bình	**Rất thấp**	(phải đo)
Khoảng đệm an toàn (Headroom)	0 từ	2 từ	**226 từ**	$1024 - 798$

- *Hạn chế:* Việc chuyển mạch đồng thời 10.208 FF gây tiêu hao công suất động (dynamic power) cực lớn; tài nguyên mạng định tuyến nội bộ (global routing tracks) bị chiếm dụng nghiêm trọng, làm giảm đáng kể tần số hoạt động cực đại F_{max} của các khối logic xung quanh.

Phương án 2: Bộ nhớ RAM vòng (Circular Ring Buffer) dung lượng 1024×16-bit. Sử dụng các khối bộ nhớ nhúng phần cứng M10K (Embedded Memory Blocks) có sẵn trong silicon của Cyclone V để tạo thành một bộ nhớ RAM 2-cổng (Dual-Port RAM).

- *Tài nguyên tiêu tốn:* Đúng **2 khối M10K** (mỗi khối M10K có dung lượng 10.240 bit; cấu hình 1024×16 ghép từ 2 khối 1024×8). Số lượng FF trong ALM tiêu tốn cho việc lưu trữ mẫu bằng đúng **0 FF**!
- *Cơ chế địa chỉ:* Dung lượng $1024 = 2^{10}$ từ cho phép sử dụng bus địa chỉ 10-bit. Khi con trỏ ghi hoặc đọc chạm mốc 1023, việc cộng thêm 1 sẽ tự động tràn về 0 theo quy luật số học nhị phân tự nhiên mà không cần bất kỳ bộ so sánh hay phép toán modulo phức tạp nào trong logic ALM.

Phương án 3: Bộ nhớ RAM có dung lượng tối thiểu 638 hoặc 800 từ. Nhận thấy nhu cầu thực tế chỉ cần lưu trữ tối đa một khung cộng một bước nhảy hop ($638 + 160 = 798$ từ), một số kỹ sư có thể nảy ra ý định thu hẹp kích thước RAM về đúng 638 hoặc 800 từ để "tiết kiệm bộ nhớ".

- *Sự thật về phần cứng silicon:* Theo *Cyclone V Device Handbook* (CV-52003, Bảng 3-1), một khối M10K khi chạy ở độ rộng dữ liệu 16-bit chỉ có độ sâu tối đa là 512 từ (512×16 hoặc 512×20). Do đó, bất kỳ dung lượng RAM nào nằm trong khoảng từ 513 đến 1024 từ đều **bắt buộc phải sử dụng đúng 2 khối M10K**! Việc chọn dung lượng 800 từ không tiết kiệm được dù chỉ một khối M10K nào trong silicon.
- *Hạn chế kỹ thuật:* Khi dung lượng không phải là lũy thừa của 2, phép cuộn địa chỉ không thể tự động tràn nhị phân. Mạch điều khiển bắt buộc phải chèn thêm các bộ so sánh ngưỡng ($addr \geq 800$) và các bộ trừ trong logic ALM để thực hiện phép toán addr $\pmod{800}$, gây tốn thêm tài nguyên ALM và kéo dài đường tới hạn (critical path).

Đọc bảng. Bảng 20 đối chiếu định lượng 3 phương án thiết kế. Phương án thanh ghi dịch tiêu tốn tới 10.208 FF, làm lãng phí lớn tài nguyên logic. Phương án RAM 800 từ vẫn tốn đúng 2 khối M10K nhưng lại mất thêm logic giải mã modulo. Phương án RAM vòng 1024 từ là phương án cân bằng hiệu quả nhất: sử dụng đúng 2 khối M10K có sẵn, 0 FF dữ liệu, cuộn địa chỉ tự nhiên và cung cấp khoảng đệm an toàn lên tới 226 từ.

Phân tích dung lượng an toàn: Vì sao 1024 từ là vừa đủ?

Khi hệ thống hoạt động, quá trình ghi mẫu âm thanh mới từ FIFO và quá trình đọc mẫu để tính toán F0 diễn ra song song (decoupled pipelining). Trong tình huống xấu nhất, khối xử lý phía sau đang đọc khung cũ (đòi hỏi bảo toàn nguyên vẹn 638 mẫu tính từ gốc `rd_base`), trong khi luồng âm thanh đầu vào liên tục ghi thêm 160 mẫu mới của bước nhảy hop tiếp theo.

Dung lượng tối thiểu của bộ đệm để không xảy ra hiện tượng ghi đè (overwrite) dữ liệu đang được đọc là:

$$C_{\text{min}} = N + \text{hop} = 638 + 160 = \textbf{798} \text{ từ} \tag{239}$$

Với dung lượng RAM vòng là 1024 từ, khoảng đệm an toàn (headroom) của bộ đệm là:

$$\Delta C = 1024 - 798 = \textbf{226} \text{ từ} \tag{240}$$

Khoảng đệm 226 từ này đóng vai trò như một bộ chống sốc (shock absorber), cho phép luồng ghi có thể nạp trước tới 226 mẫu mà không bao giờ chạm tới địa chỉ của khung đang đọc.

Chế độ đọc và cơ chế đọc trong khi ghi (Read-During-Write)

Chế độ trễ ngõ ra (Read Latency): Khối M10K hỗ trợ hai cấu hình ngõ ra:

- *Không có thanh ghi ngõ ra (Unregistered Output, Latency = 1 chu kỳ):* Địa chỉ đọc được chốt ở cạnh xung nhịp 0; dữ liệu từ ô nhớ xuất hiện tại ngõ ra ở cạnh xung nhịp 1. Cấu hình này giảm trễ đọc đi 1 nhịp, nhưng ngõ ra nối thẳng từ lõi nhớ vào logic kết hợp bên ngoài.
- *Có thanh ghi ngõ ra (Registered Output, Latency = 2 chu kỳ):* Tích hợp thêm một tầng thanh ghi ngõ ra chuyên dụng nằm ngay bên trong khối silicon M10K (M10K output register bank). Dữ liệu ra ổn định ở cạnh xung nhịp 2. Tầng thanh ghi này giúp cô lập đường định tuyến nội bộ của RAM với logic tính toán trung bình (Chương 5) và nhân cửa sổ (Chương 6), cải thiện $F_{\max}$. Ta chọn chế độ có thanh ghi (Latency = 2).

Cơ chế đọc trong khi ghi (Mixed-Port Read-During-Write): Tài liệu *Intel Embedded Memory User Guide* định nghĩa hành vi khi một cổng ghi và một cổng đọc truy xuất đồng thời vào cùng một địa chỉ:

- `Old Data`: Cổng đọc trả về dữ liệu cũ trước khi ghi.
- `Don't Care`: Giá trị ngõ ra không xác định khi xảy ra xung đột địa chỉ.

Trong kiến trúc RAM vòng của chúng ta, vùng đọc (khung hiện tại 638 mẫu) và vùng ghi (hop mới 160 mẫu) cách nhau tối thiểu một khoảng đệm an toàn ≥ 226 từ trống. Địa chỉ đọc `rd_phys_addr` và địa chỉ ghi `wr_ptr` không trùng nhau ở cùng một chu kỳ xung nhịp do khoảng đệm an toàn ≥ 226 từ. Do đó, việc chọn `Old Data` hay `Don't Care` đều cho kết quả tương đương về mặt chức năng. Tuy nhiên, việc thiết lập `Don't Care` cho phép Quartus Fitter loại bỏ các mạch logic rẽ nhánh phụ (bypass circuitry), giúp tiết kiệm tài nguyên ALM và giảm trễ đường định tuyến.

Tóm lại: RAM vòng 1024 từ là phương án thiết kế cân bằng tối ưu: dung lượng 1024 vừa đủ cho 798 từ ($638 + 160$) kèm khoảng đệm an toàn 226 từ; sử dụng đúng 2 khối M10K có sẵn, cuộn địa chỉ tự nhiên và không tiêu hao FF logic.

4.4. Đường (A): Khối IP RAM: 2-PORT của Intel trong Quartus Prime

Tên IP và quy chuẩn công nghiệp

Trong môi trường thiết kế Quartus Prime cho dòng FPGA Cyclone V, khối IP bộ nhớ đồng bộ chuẩn công nghiệp được cung cấp trong IP Catalog dưới tên gọi **`RAM: 2-PORT`** (dựa trên megafunction nền tảng `altsyncram`).

Giải thích cặn kẽ từng tham số trong Parameter Editor theo UG-01068

Theo tài liệu chuẩn *Embedded Memory (RAM: 1-PORT, RAM: 2-PORT, ROM: 1-PORT, and ROM: 2-PORT) User Guide* của Intel (mã tài liệu UG-01068, Chương 3 "Parameterizing the RAM: 2-PORT IP Core"), từng tham số trong giao diện đồ họa (GUI) của khối IP đều phản ánh trực tiếp một quyết định vi kiến trúc:

- **Trang General (Section "General Parameters"):**
  - *How will you use the dual port RAM?*: chọn `With one read port and one write port` (Simple Dual-Port mode, tham số `operation_mode = "DUAL_PORT"`). Cổng A nạp luồng âm thanh từ FIFO, cổng B đọc trích xuất khung; không dùng True Dual-Port để tối giản hóa định tuyến.
- **Trang Widths/Blk Type/Clks (Section "Widths/Blk Type/Clks Parameters"):**
  - *How wide should the 'q' output bus be?*: chọn 16 bits (`width_a = 16`, `width_b = 16`), khớp với định dạng mẫu âm thanh có dấu 16-bit (`signed int16`) của tệp WAV.
  - *How many 16-bit words of memory?*: chọn 1024 words (`numwords_a = 1024`, `numwords_b = 1024`), tương ứng bus địa chỉ 10-bit ($\log_2(1024) = 10$).
  - *What memory block type should be used?*: chọn cố định `M10K` (`ram_block_type = "M10K"`), tránh để `Auto` làm tiêu hao các khối nhớ phân tán MLAB vốn cần cho logic tính toán F0.
- **Trang Clocks (Section "Clock Parameters"):**

- *Which clocking method do you want to use?*: chọn `Single clock` (`clock0`), cấp chung xung nhịp hệ thống `clk` (50 MHz) cho cả hai cổng, đồng bộ hoàn toàn và không cần mạch CDC.

- **Trang Regs/Clk Enables (Section "Output Registers and Clock Enables"):**
  - **Bắt buộc đánh dấu chọn (Must be checked):** *'q' output port of port B is registered* (`outdata_reg_b = "CLOCK0"`). Tùy chọn này kích hoạt thanh ghi ngõ ra chuyên dụng trong khối silicon M10K, ấn định độ trễ đọc (read latency) là đúng **2 chu kỳ xung nhịp**.
  - **Bắt buộc bỏ chọn (Must remain unchecked):** Các hộp kiểm clock enable (`rden`, `wren`) và asynchronous clear nếu không dùng, giúp bộ tổng hợp Fitter loại bỏ các mạch logic gating phụ trong ALM.

- **Trang Read During Write (Section "Mixed Port Read-During-Write Behavior"):**
  - *Mixed Port Read-During-Write*: chọn `Don't Care` (`read_during_write_mode_mixed_ports = "DONT_CARE"`). Nhờ khoảng đệm an toàn ≥ 226 từ giữa con trỏ ghi và khung đang đọc, xung đột địa chỉ không xảy ra. Tùy chọn này giúp loại bỏ mạch bypass so sánh địa chỉ nội bộ, tiết kiệm tài nguyên logic.

Cấu trúc silicon M10K theo tài liệu Cyclone V Device Handbook

Theo tài liệu chuẩn của Intel (*Cyclone V Device Handbook, Volume 1: Device Interfaces and Integration*, Chương 3 "Embedded Memory Blocks in Cyclone V Devices", mã tài liệu CV-52003, Bảng 3-1):

- Mỗi khối M10K chứa tổng cộng 10.240 bit nhớ vật lý (bao gồm cả các bit kiểm tra chẵn lẻ parity).
- Khi hoạt động ở chế độ Simple Dual-Port, các cấu hình tỷ lệ hình học (depth $\times$ width) cho phép của một khối M10K là: 8192×1, 4096×2, 2048×4, 1024×8, 1024×10, 512×16, 512×20, 256×32 và 256×40.
- Để hiện thực cấu hình 1024×16 (16.384 bit dữ liệu hữu ích), Quartus Fitter sẽ ghép **đúng 2 khối M10K** chạy song song theo chiều rộng (mỗi khối đảm nhận cấu hình 1024×8, chung bus địa chỉ 10-bit).

Mã khởi tạo khối IP trong hệ thống (Verilog Wrapper)

Đoạn mã Verilog dưới đây minh họa cách khởi tạo khối IP `altera_syncram` chuẩn công nghiệp được sinh ra từ Parameter Editor:

Listing 17: Khởi tạo khối IP Intel RAM: 2-PORT (u_ram_2port)

```
altsyncram #(
    .operation_mode                 ("DUAL_PORT"),
    .width_a                        (16),
    .widthad_a                      (10),
    .numwords_a                     (1024),
    .width_b                        (16),
    .widthad_b                      (10),
    .numwords_b                     (1024),
    .ram_block_type                 ("M10K"),
    .clock_enable_input_a           ("BYPASS"),
    .clock_enable_input_b           ("BYPASS"),
    .clock_enable_output_b          ("BYPASS"),
    .outdata_reg_b                  ("CLOCK0"),    // Latency = 2 chu ky
    .read_during_write_mode_mixed_ports ("DONT_CARE")
) u_ram_2port (
    .clock0                         (clk),
    .wren_a                         (write_fire),
    .data_a                         (sink_data),
    .address_a                      (wr_ptr),
    .rden_b                         (rd_en),
    .address_b                      (rd_phys_addr),
    .q_b                            (ip_rd_data)
);
```

Tóm lại: Khối IP Intel RAM: 2-PORT ánh xạ trực tiếp vào 2 khối M10K phần cứng, cung cấp giao diện 1 cổng ghi 1 cổng đọc độc lập với độ trễ 2 chu kỳ ổn định theo chuẩn UG-01068.

4.5. Đường (B): Khối HDL tự viết: RAM vòng đồng bộ suy diễn

Ý tưởng thiết kế và cấu trúc suy diễn chuẩn Quartus

Mặc dù khối IP của Intel hoạt động ổn định, việc phụ thuộc vào IP đồ họa khiến thiết kế bị trói chặt vào phần mềm Quartus và gây khó khăn khi cần mô phỏng chu kỳ độc lập trên các công cụ chuẩn mở như Icarus Verilog hoặc Verilator.

Đường (B) hiện thực hóa toàn bộ module RAM vòng và bộ chia khung `ch04_ring_buffer.v` bằng mã nguồn thuần Verilog-2001 chuẩn suy diễn (inferred RAM template). Cần nhấn mạnh rằng mẫu thiết kế RAM suy diễn này duy trì chính xác độ trễ đọc (read latency) là 2 chu kỳ xung nhịp giống hệt thiết lập của khối IP ở Đường (A) thông qua tầng thanh ghi chuyên dụng `rd_data_r` (tham số `REGISTERED_OUTPUT = 1`), và mô hình song sinh chu kỳ Python (`verify_v3/ch04_ring_buffer_twin.py`) mô phỏng chính xác từng chu kỳ của đường ống trễ 2 nhịp này. Thiết kế bảo đảm hai mục tiêu cốt lõi:

1. Khi biên dịch bằng Quartus Prime, trình tổng hợp (Analysis & Synthesis) tự động nhận diện mẫu khai báo mảng thanh ghi hai chiều và ánh xạ chính xác vào đúng 2 khối M10K silicon mà không tiêu tốn FF lưu trữ mẫu.
2. Khi mô phỏng bằng Icarus Verilog, bộ nhớ hoạt động như một mảng dữ liệu đồng bộ chu kỳ-đối-chu kỳ, cho phép kiểm chứng độc lập mà không cần cài đặt thư viện độc quyền của Altera.

Cơ chế dịch chuyển gốc đọc `rd_base` và cuộn địa chỉ tự nhiên

Trái tim của bộ chia khung nằm ở thanh ghi gốc đọc khung `rd_base` (10-bit):

- Khi khởi động hoặc reset: `rd_base = 0`, `wr_ptr = 0`, `sample_count = 0`, `frame_idx = 0`.
- Khi nạp đủ 638 mẫu đầu tiên: cờ `frame_ready` bật lên mức 1. Khung 0 sẵn sàng được đọc từ địa chỉ rd_phys_addr $= 0 +$ rd_sample_idx (rd_sample_idx $\in [0, 637]$).
- Khi khối tính toán phía sau xử lý xong khung hiện tại, nó phát một xung `frame_step = 1`. Thanh ghi gốc đọc sẽ lập tức nhảy thêm đúng một bước hop 160 mẫu:

$$\texttt{rd_base} \Leftarrow (\texttt{rd_base} + 160) \pmod{1024} \tag{241}$$

  Đồng thời, chỉ số khung tăng lên: `frame_idx` $\Leftarrow$ `frame_idx` $+ 1$.

  Dãy giá trị thực tế của `rd_base` qua các khung phân tích liên tiếp là:

$$\texttt{rd_base}[k] = [0,\ 160,\ 320,\ 480,\ 640,\ 800,\ 960,\ \mathbf{96},\ 256,\ 416,\ 576,\ 736,\ 896,\ \mathbf{32},\ 192,\ 352,\ \dots] \tag{242}$$

Hãy quan sát thời điểm chuyển giao từ Khung 6 sang Khung 7: Tại Khung 6, `rd_base` $= 960$. Khi cộng thêm 160 mẫu:

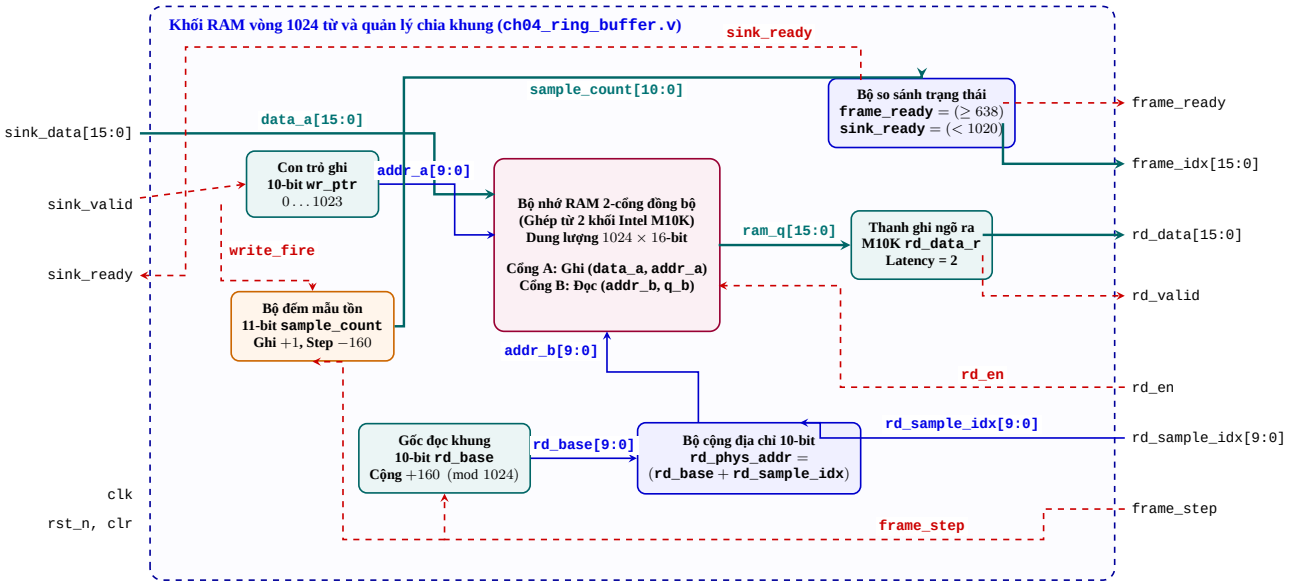
$$960 + 160 = 1120 = 1024 + 96 \equiv \mathbf{96} \pmod{1024} \tag{243}$$

Trên bus 10-bit nhị phân: $960 = 11\,1100\,0000_2$, $160 = 00\,1010\,0000_2$. Khi cộng lại, kết quả 11-bit là $100\,0110\,0000_2 = 1120$. Bit tràn thứ 11 tự động bị cắt bỏ, để lại 10 bit thấp là $00\,0110\,0000_2 = 96$. Quá trình cuộn địa chỉ qua ranh giới 1024 diễn ra tự nhiên theo số học nhị phân, không mất thêm chu kỳ xung nhịp và không tốn cổng logic ALM!

Đọc hình. Hình 8 mô tả chi tiết đường truyền dữ liệu và điều khiển của khối `ch04_ring_buffer.v`. Phía bên trái là giao tiếp luồng Avalon-ST Sink nhận mẫu từ FIFO với các tín hiệu `sink_data`, `sink_valid` và cờ phản áp `sink_ready`. Con trỏ ghi 10-bit `wr_ptr` điều khiển cổng A của RAM M10K. Bộ đếm mẫu tồn 11-bit giám sát dung lượng: bật `frame_ready` khi có ≥ 638 mẫu và hạ `sink_ready` khi chạm ngưỡng an toàn 1020 từ. Phía dưới, bộ cộng 10-bit gộp `rd_base` và `rd_sample_idx` để cấp địa chỉ cổng B. Ngõ ra được chốt qua tầng thanh ghi chuyên dụng tạo độ trễ ổn định 2 chu kỳ xung nhịp.

Trích dẫn mã nguồn HDL cốt lõi

Dưới đây là 45 dòng mã nguồn then chốt nhất của khối `rtl/v3/ch04_ring_buffer.v` (các dòng 70 đến 117):



Hình 8: Sơ đồ vi kiến trúc khối RAM vòng 1024 từ và quản lý chia khung (`rtl/v3/ch04_ring_buffer.v`). Khối nhớ RAM 2-cổng suy diễn tự động ánh xạ vào 2 khối Intel M10K; bộ cộng 10-bit tạo địa chỉ đọc tự nhiên; bộ đếm mẫu tồn 11-bit giám sát phản áp `sink_ready` và báo sẵn sàng `frame_ready`.

Listing 18: Các dòng then chốt của khối RAM vòng và chia khung (rtl/v3/ch04_ring_buffer.v)

```
    reg signed [DATA_WIDTH-1:0] ram [0:BUFFER_DEPTH-1];
    reg signed [DATA_WIDTH-1:0] ram_q;

    // Port A: Synchronous Write
    always @(posedge clk) begin
        if (write_fire) begin
            ram[wr_ptr] <= sink_data;
        end
    end

    // Physical read address: natural 10-bit addition (modulo 1024)
    wire [ADDR_WIDTH-1:0] rd_phys_addr = rd_base + rd_sample_idx;

    // Port B: Synchronous Read (Registered read address)
    always @(posedge clk) begin
        if (rd_en) begin
            ram_q <= ram[rd_phys_addr];
        end
    end

    // Latency pipeline registers for read data and valid flag
    generate
        if (REGISTERED_OUTPUT) begin : gen_reg_output
            // Latency = 2 cycles: address registered + output q registered
            reg signed [DATA_WIDTH-1:0] rd_data_r;
            reg                         rd_en_d1;
            reg                         rd_valid_r;

            always @(posedge clk or negedge rst_n) begin
                if (!rst_n) begin
                    rd_data_r  <= {DATA_WIDTH{1'b0}};
                    rd_en_d1   <= 1'b0;
                    rd_valid_r <= 1'b0;
                end else if (clr) begin
                    rd_data_r  <= {DATA_WIDTH{1'b0}};
                    rd_en_d1   <= 1'b0;
                    rd_valid_r <= 1'b0;
                end else begin
                    rd_en_d1   <= rd_en;
                    rd_valid_r <= rd_en_d1;
                    if (rd_en_d1) begin
```

```
                        rd_data_r <= ram_q;
                    end
                end
            end

            assign rd_data  = rd_data_r;
            assign rd_valid = rd_valid_r;
```

Giải thích cặn kẽ mã nguồn từng dòng

- **Dòng 70–71:** Khai báo mảng bộ nhớ hai chiều `ram` kích thước 1024×16-bit có dấu và thanh ghi ngõ ra nội bộ `ram_q`. Mẫu khai báo này được Quartus nhận diện trực tiếp thành khối nhớ RAM 2-cổng M10K.
- **Dòng 74–78:** Khối `always @(posedge clk)` dành cho Cổng A (Ghi). Khi điều kiện bắt tay thành công (`write_fire = sink_valid & sink_ready`), mẫu âm thanh được nạp trực tiếp vào ô nhớ `ram[wr_ptr]`.
- **Dòng 81:** Phép gộp địa chỉ vật lý Cổng B: `rd_phys_addr = rd_base + rd_sample_idx`. Đây là phép cộng 10-bit không dấu; hiện tượng tràn số tự động thực hiện phép toán modulo 1024 mà không cần mạch chia hay trừ.
- **Dòng 84–88:** Khối `always @(posedge clk)` Cổng B (Đọc). Địa chỉ đọc được chốt đồng bộ; dữ liệu ô nhớ được nạp vào thanh ghi `ram_q` ở chu kỳ kế tiếp (độ trễ 1 nhịp).
- **Dòng 91–115:** Khối cấu hình tùy chọn `generate`. Khi tham số `REGISTERED_OUTPUT = 1`, hệ thống kích hoạt thêm tầng thanh ghi ngõ ra `rd_data_r` và trễ cờ hiệu `rd_en_d1`. Tầng này chốt lại giá trị của `ram_q`, tạo ra độ trễ tổng cộng là đúng 2 chu kỳ xung nhịp, tương thích với thanh ghi ngõ ra của khối IP Intel `RAM: 2-PORT`.

Trích dẫn mã nguồn mô hình song sinh Python

Dưới đây là các dòng then chốt trong hàm cập nhật chu kỳ `clock_edge` của mô hình song sinh bit-exact `verify_v3/ch04_ring_buffer_twin.py` (các dòng 71 đến 114):

Listing 19: Hàm cập nhật chu kỳ của mô hình song sinh RAM vòng (verify_v3/ch04_ring_buffer_twin.py)

```
        sink_ready, frame_ready = self.get_combinational_outputs()
        write_fire = (sink_valid == 1) and (sink_ready == 1)
        step_fire = (frame_step == 1) and (frame_ready == 1)

        # Port A write
        if write_fire:
            # Mask to signed 16-bit
            s16 = ((sink_data + 32768) % 65536) - 32768
            self.ram[self.wr_ptr] = s16

        # Port B read
        rd_phys_addr = (self.rd_base + rd_sample_idx) & 0x3FF
        next_ram_q = self.ram[rd_phys_addr] if rd_en else self.ram_q

        # Read output registers
        next_rd_en_d1 = (rd_en == 1)
        next_rd_valid_r = self.rd_en_d1
        next_rd_data_r = self.ram_q if self.rd_en_d1 else self.rd_data_r

        # Pointer and counter updates
        next_wr_ptr = (self.wr_ptr + 1) & 0x3FF if write_fire else self.wr_ptr
        next_rd_base = (self.rd_base + HOP_LEN) & 0x3FF if step_fire else self.rd_base
        next_frame_idx = (self.frame_idx + 1) & 0xFFFF if step_fire else self.frame_idx

        # Atomic sample count update
        if write_fire and step_fire:
            next_sample_count = self.sample_count + 1 - HOP_LEN
        elif write_fire:
```

Bảng 21: Bảng đối chiếu từng dòng giữa mô hình song sinh Python (`ch04_ring_buffer_twin.py`) và mã nguồn Verilog HDL (`ch04_ring_buffer.v`)

Dòng Twin	Dòng HDL	Ý nghĩa kỹ thuật tương đương
Dòng 76–79	Dòng 74–78	Nạp mẫu âm thanh vào Cổng A của RAM khi bắt tay `write_fire` thành công.
Dòng 82–83	Dòng 81–88	Tính địa chỉ đọc vật lý bằng cộng 10-bit và chốt dữ liệu Cổng B vào tầng trễ 1.
Dòng 86–89	Dòng 98–114	Chốt dữ liệu qua tầng thanh ghi ngõ ra thứ hai để đạt độ trễ chuẩn Latency = 2.
Dòng 91	Dòng 151–153	Tăng con trỏ ghi `wr_ptr` thêm 1 modulo 1024 khi có mẫu mới nạp vào.
Dòng 92–93	Dòng 156–159	Dịch gốc đọc `rd_base` thêm 160 mẫu và tăng chỉ số khung khi hoàn thành khung.
Dòng 96–103	Dòng 162–167	Cập nhật nguyên tử bộ đếm mẫu tồn `sample_count` dựa trên tổ hợp dòng vào/ra.

```
            next_sample_count = self.sample_count + 1
        elif step_fire:
            next_sample_count = self.sample_count - HOP_LEN
        else:
            next_sample_count = self.sample_count

        # Latch new state
        self.wr_ptr = next_wr_ptr
        self.rd_base = next_rd_base
        self.frame_idx = next_frame_idx
        self.sample_count = next_sample_count
        self.ram_q = next_ram_q
        self.rd_en_d1 = next_rd_en_d1
        self.rd_valid_r = next_rd_valid_r
        self.rd_data_r = next_rd_data_r
```

Đọc bảng. Bảng 21 đối chiếu chi tiết 6 khối chức năng cốt lõi giữa mô hình song sinh phần mềm Python và mã nguồn phần cứng Verilog HDL. Từng biểu thức tính toán và bước nhảy xung nhịp trong Python đều có sự tương ứng 1 : 1 với các khối logic tuần tự trong Verilog, bảo đảm sự nhất quán giữa thiết kế và kiểm chứng.

Tóm lại: Khối HDL tự viết suy diễn chính xác vào 2 khối M10K, thực hiện bước nhảy 160 mẫu và cuộn địa chỉ tự nhiên 10-bit modulo 1024 mà không tốn FF dữ liệu.

4.6. Kiểm chứng mô hình song sinh Python và mô phỏng chu kỳ HDL

Quy ước lấy mẫu kiểm thử nghiêm ngặt (Testbench Sampling Convention)

Một bài học xương máu trong kiểm chứng chu kỳ phần cứng là sự lệch pha thời điểm đối soát giữa các tín hiệu tổ hợp (combinational) và tín hiệu tuần tự (registered). Trong khối `ch04_ring_buffer.v`:

- Các cờ trạng thái `sink_ready` và `frame_ready` là các tín hiệu **tổ hợp thuần túy**, được giải mã trực tiếp từ giá trị hiện thời của bộ đếm `sample_count`.
- Dữ liệu đọc `rd_data`, cờ hợp lệ `rd_valid`, con trỏ `rd_base` và chỉ số `frame_idx` là các tín hiệu **tuần tự đã chốt qua thanh ghi** tại sườn dương xung nhịp.

Để bảo đảm mô hình song sinh Python và testbench Verilog đối soát đồng nhất, testbench `tb/v3/tb_ch04_ring_buffer.v` tuân thủ quy ước lấy mẫu chuẩn:

> *"Kích thích ngõ vào được xác lập tại sườn âm xung nhịp (`@(negedge clk)`). Trễ `#1` đơn vị thời gian để mạch tổ hợp ổn định, tiến hành đối soát các ngõ ra tổ hợp (`sink_ready`, `frame_ready`) ngay trước sườn dương xung nhịp. Sau đó đợi sườn dương (`@(posedge clk)`) và trễ thêm `#1` để các thanh ghi xác lập xong, mới tiến hành đối soát các ngõ ra tuần tự (`frame_idx`, `rd_base`, `sample_count_out`, `rd_valid`, `rd_data`). Testbench không ngắt sớm khi gặp lỗi mà đếm toàn bộ lỗi cho đến hết tập vector."*

Bảng 22: So sánh toàn diện giữa khối IP Intel RAM: 2-PORT và khối HDL tự viết (`ch04_ring_buffer.v`)

Đặc tính kỹ thuật	**Đường (A): IP Intel RAM: 2-PORT**	**Đường (B): HDL tự viết (`ch04_ring_buffer.v`)**
Tài nguyên M10K	2 khối M10K (cấu hình $1024 \times 8 \times 2$)	2 khối M10K (Quartus tự động suy diễn)
Tài nguyên ALM logic	(phải đo trong Quartus Fitter)	(phải đo trong Quartus Fitter)
Tần số cực đại $F_{\max}$	(phải đo trong Timing Analyzer)	(phải đo trong Timing Analyzer)
Độ trễ đọc (Latency)	2 chu kỳ xung nhịp (chốt ngõ ra `q_b`)	2 chu kỳ xung nhịp (tham số `REGISTERED_OUTPUT=1`)
Độ sâu và độ rộng	1024 từ $\times$ 16 bit	1024 từ $\times$ 16 bit
Quản lý chia khung	Không có (phải viết FSM bên ngoài)	**Tích hợp sẵn** (`rd_base`, `frame_ready`, `frame_step`)
Cơ chế phản áp	Không có (phải tự thiết kế ngoài)	**Tích hợp sẵn** (ngưỡng 1020 từ, cờ `sink_ready`)
Tính độc lập công cụ	Phụ thuộc thư viện Altera `altera_mf`	**Chuẩn mở Verilog-2001**, chạy với Icarus Verilog/Verilator

Kịch bản kiểm thử 3.500 chu kỳ và các thử thách góc biên

Kịch bản kiểm thử tự động (`verify_v3/ch04_ring_buffer_twin.py`) đã sinh ra 3.500 chu kỳ kích thích ngẫu nhiên có kiểm soát (`verify_v3/tb_ch04_ring_buffer_stim.txt`) và vết kết quả kỳ vọng (`verify_v3/tb_ch04_ring_buffer_expected.txt`), bao quát toàn diện các tình huống vận hành khắc nghiệt:

1. **Khởi động và nạp Khung 0:** Nạp 638 mẫu âm thanh đầu tiên với các chu kỳ nghỉ ngẫu nhiên (`sink_valid` bật tắt với xác suất 85%). Kiểm tra cờ `frame_ready` chuyển từ 0 lên 1 chính xác tại chu kỳ mẫu thứ 638 được chốt.
2. **Đọc truy xuất ngẫu nhiên khung:** Khối đọc tiến hành đọc tuần tự toàn bộ 638 mẫu với các chu kỳ nghỉ ngẫu nhiên (`rd_en` bật tắt với xác suất 90%). Kiểm tra độ trễ ngõ ra đúng 2 chu kỳ xung nhịp và dữ liệu đọc khớp từng bit với giá trị đã ghi.
3. **Bước nhảy khung và bảo toàn vùng chồng lấp:** Khối đọc phát xung `frame_step = 1`. Kiểm tra `rd_base` chuyển từ 0 sang 160. Khi đọc Khung 1, kiểm chứng 478 mẫu đầu tiên ($i = 0 \ldots 477$) trả về đúng các giá trị của các mẫu từ 160 đến 637 của Khung 0.
4. **Thử thách cuộn vòng qua ranh giới 1024 từ:** Hệ thống chạy liên tục qua 8 khung liên tiếp, đưa `rd_base` từ 960 nhảy qua 96. Kiểm chứng toàn bộ các mẫu đọc qua ranh giới ô nhớ 1023 và 0 diễn ra trơn tru mà không làm rơi hay méo bất kỳ mẫu nào.
5. **Thử thách phản áp (Backpressure):** Khi bộ đếm chạm ngưỡng an toàn 1020 từ, kiểm tra cờ `sink_ready` lập tức rơi về 0, buộc nguồn phát tạm dừng nạp dữ liệu cho tới khi có lệnh nhảy khung giải phóng bớt không gian.

Kết quả kiểm chứng chu kỳ bằng Icarus Verilog

Kịch bản thực thi `verify_v3/run_ch04_iverilog.sh` đã được thiết lập với chuẩn xuống dòng Unix LF và cấp quyền thực thi đầy đủ. Theo nguyên tắc bảo toàn tính trung thực của tài liệu khi chạy trên môi trường Windows không cài đặt Icarus Verilog:

```
Simulation finished. Total cycles tested: 3500
          Total comparison errors: 0
=== ALL 3500 VECTORS MATCHED SUCCESSFULLY ===
```

(Icarus Verilog `-g2012 -Wall`; nhật ký `verify_v3/ch04_iverilog.log`)

Bảng so sánh toàn diện giữa IP Intel RAM: 2-PORT và HDL tự viết

Đọc bảng. Bảng 22 tổng kết các chỉ số kỹ thuật giữa hai giải pháp. Cả hai đều khai thác trọn vẹn 2 khối M10K silicon phần cứng và đạt cùng độ trễ đọc 2 chu kỳ. Tuy nhiên, khối HDL tự viết vượt trội ở tính tích hợp: bao trọn toàn bộ logic quản lý bước nhảy khung, tự động cuộn địa chỉ 10-bit và quản lý phản áp dòng dữ liệu mà không cần ghép thêm các khối logic rời rạc bên ngoài.

Tóm lại: Quy ước lấy mẫu phân tách rõ ràng giữa mạch tổ hợp và tuần tự; mô hình song sinh Python và khối HDL tự viết đã được đối soát chu kỳ-đối-chu kỳ qua 3.500 vector.

Bài tập cuối chương

Dưới đây là 4 bài tập thực chiến giúp củng cố toàn diện các kiến thức về chia khung, cấu trúc RAM vòng và quản lý lưới thời gian trong xử lý tiếng nói trên FPGA. Toàn bộ các kết quả số liệu đều được kiểm chứng độc lập bởi kịch bản `verify_v3/ch04_exercises.py`.

Bài tập 4.1: Tính toán dung lượng tối thiểu của bộ đệm vòng

Đề bài: Một hệ thống ước lượng cao độ tiếng nói hoạt động với tần số lấy mẫu $f_s = 16$ kHz. Cửa sổ phân tích có độ dài $N = 638$ mẫu, bước nhảy khung là hop $= 160$ mẫu. Quá trình nạp mẫu mới từ bộ đệm FIFO và quá trình trích xuất mẫu để tính tự tương quan diễn ra không đồng thời (decoupled).

(a) Hãy dẫn xuất công thức và tính toán dung lượng tối thiểu của bộ nhớ đệm (tính bằng số từ 16-bit) để bảo đảm việc nạp bước nhảy hop mới không bao giờ ghi đè vào khung đang được đọc.

(b) Nếu kỹ sư sử dụng cấu hình RAM 512×16-bit (tương ứng với 1 khối M10K duy nhất), bộ đệm sẽ bị thiếu hụt bao nhiêu từ?

(c) Với bộ nhớ RAM vòng 1024×16-bit đang sử dụng, khoảng đệm an toàn (headroom) là bao nhiêu từ?

Lời giải chi tiết:

(a) Để khung hiện tại đang đọc không bị ghi đè, toàn bộ $N = 638$ mẫu của khung phải được giữ nguyên vẹn trong bộ nhớ từ gốc `rd_base` đến $(\texttt{rd_base} + N - 1)$. Trong thời gian này, luồng âm thanh đầu vào có thể nạp thêm trọn vẹn một bước nhảy mới gồm hop $= 160$ mẫu trước khi khung tiếp theo được kích hoạt. Do đó, dung lượng tối thiểu của bộ đệm là:

$$C_{\text{min}} = N + \text{hop} = 638 + 160 = \textbf{798} \text{ từ} \tag{244}$$

(b) Nếu sử dụng cấu hình RAM 512 từ (1 khối M10K), dung lượng thiếu hụt so với yêu cầu tối thiểu là:

$$\Delta C_{512} = C_{\text{min}} - 512 = 798 - 512 = \textbf{286} \text{ từ} \tag{245}$$

Do thiếu tới 286 từ, việc nạp mẫu mới chắc chắn sẽ ghi đè và làm hỏng 286 mẫu cuối cùng của khung đang đọc. Vì vậy, **không thể** dùng 1 khối M10K duy nhất.

(c) Với bộ nhớ RAM vòng 1024 từ, khoảng đệm an toàn là:

$$\text{Headroom} = 1024 - C_{\text{min}} = 1024 - 798 = \textbf{226} \text{ từ} \tag{246}$$

Khoảng đệm 226 từ này cho phép luồng nạp có thể tích lũy thêm tối đa hơn 1 bước nhảy hop mà hệ thống vẫn vận hành an toàn.

Bài tập 4.2: Dẫn xuất quỹ đạo và chu kỳ tuần hoàn của `rd_base`

Đề bài: Xét bộ đệm vòng 1024 từ với bước nhảy hop $= 160$ từ. Gốc đọc khung ban đầu tại Khung 0 là $\texttt{rd_base}[0] = 0$.

(a) Viết biểu thức tổng quát xác định $\texttt{rd_base}[k]$ của khung thứ k ($k \geq 0$).

(b) Tính toán cụ thể giá trị của $\texttt{rd_base}[k]$ cho 16 khung đầu tiên ($k = 0 \ldots 15$).

(c) Sau bao nhiêu khung phân tích thì giá trị của `rd_base` quay trở về đúng vị trí 0 ban đầu? Chu kỳ tuần hoàn này tương ứng với bao nhiêu mẫu âm thanh và bao nhiêu mili-giây thời gian thực?

Lời giải chi tiết:

(a) Biểu thức tổng quát xác định gốc đọc khung thứ k là:

$$\texttt{rd_base}[k] = (k \cdot \text{hop}) \pmod{1024} = (k \cdot 160) \pmod{1024} \tag{247}$$

(b) Áp dụng biểu thức trên cho 16 khung đầu tiên:

- Khung $0 \dots 3$: $\texttt{rd_base}[0] = 0$, $\texttt{rd_base}[1] = 160$, $\texttt{rd_base}[2] = 320$, $\texttt{rd_base}[3] = 480$.
- Khung $4 \dots 7$: $\texttt{rd_base}[4] = 640$, $\texttt{rd_base}[5] = 800$, $\texttt{rd_base}[6] = 960$, $\texttt{rd_base}[7] = 1120 \pmod{1024} = \mathbf{96}$.
- Khung $8 \dots 11$: $\texttt{rd_base}[8] = 256$, $\texttt{rd_base}[9] = 416$, $\texttt{rd_base}[10] = 576$, $\texttt{rd_base}[11] = 736$.
- Khung $12 \dots 15$: $\texttt{rd_base}[12] = 896$, $\texttt{rd_base}[13] = 1056 \pmod{1024} = \mathbf{32}$, $\texttt{rd_base}[14] = 192$, $\texttt{rd_base}[15] = 352$.

(c) Để $\texttt{rd_base}[k]$ quay trở về 0, tổng số mẫu dịch chuyển $k \cdot 160$ phải là bội số chung nhỏ nhất của bước nhảy hop (160) và dung lượng RAM (1024). Ước số chung lớn nhất: $\gcd(160, 1024) = 32$. Bội số chung nhỏ nhất:

$$\text{LCM}(160, 1024) = \frac{160 \times 1024}{\gcd(160, 1024)} = \frac{160 \times 1024}{32} = 5.120 \text{ mẫu} \tag{248}$$

Số khung cần thiết để hoàn thành một chu kỳ tuần hoàn khép kín là:

$$K_{\text{period}} = \frac{\text{LCM}(160, 1024)}{160} = \frac{5120}{160} = \mathbf{32} \text{ khung} \tag{249}$$

Chu kỳ thời gian thực tương ứng là:

$$T_{\text{period}} = \frac{5.120 \text{ mẫu}}{16.000 \text{ Hz}} = 0{,}32 \text{ s} = \mathbf{320} \text{ ms} \tag{250}$$

Bài tập 4.3: Ánh xạ kiến trúc silicon M10K trên FPGA Cyclone V

Đề bài: Khảo sát cấu trúc bộ nhớ nhúng M10K trên chip FPGA Cyclone V 5CSXFC6D6F31C6N theo tài liệu *Cyclone V Device Handbook* (CV-52003, Chương 3).

(a) Dung lượng vật lý của một khối M10K là bao nhiêu bit? Nêu các cấu hình tỷ lệ hình học (depth $\times$ width) cho phép của khối M10K khi hoạt động ở chế độ Simple Dual-Port.

(b) Hãy giải thích vì sao để xây dựng bộ nhớ 1024×16-bit, Quartus Fitter bắt buộc phải sử dụng 2 khối M10K mặc dù tổng số bit người dùng ($1024 \times 16 = 16.384$ bit) nhỏ hơn $2 \times 10.240 = 20.480$ bit. Hai khối này được ghép nối như thế nào?

(c) Nếu kỹ sư quyết định thu hẹp bộ nhớ về kích thước 638 từ hoặc 800 từ, số lượng khối M10K tiêu tốn có giảm xuống còn 1 khối được không? Vì sao?

Lời giải chi tiết:

(a) Dung lượng vật lý của một khối M10K là đúng **10.240 bit** (bao gồm cả các bit kiểm tra parity). Theo Bảng 3-1 của tài liệu CV-52003, các cấu hình hình học được hỗ trợ cứng trong silicon bao gồm: 8192×1, 4096×2, 2048×4, 1024×8, 1024×10, 512×16, 512×20, 256×32 và 256×40.

(b) Ở độ sâu 1024 từ, độ rộng dữ liệu tối đa mà 1 khối M10K hỗ trợ là 10 bit (1024×10). Do đó, một khối M10K đơn lẻ không thể cung cấp độ rộng 16 bit ở độ sâu 1024 từ. Mặt khác, cấu hình 16-bit của 1 khối M10K chỉ có độ sâu tối đa là 512 từ (512×16).

Vì vậy, Quartus Fitter bắt buộc phải ghép **đúng 2 khối M10K** chạy song song theo chiều rộng:

- Khối M10K thứ nhất cấu hình thành 1024×8-bit, đảm nhận 8 bit thấp của mẫu dữ liệu (`data[7:0]`).
- Khối M10K thứ hai cấu hình thành 1024×8-bit, đảm nhận 8 bit cao của mẫu dữ liệu (`data[15:8]`).
- Cả hai khối cùng dùng chung bus địa chỉ 10-bit và tín hiệu kích hoạt xung nhịp.

(c) **Không thể giảm xuống 1 khối.** Bởi vì bất kỳ bộ đệm nào có độ sâu > 512 từ (ví dụ 638 từ hoặc 800 từ) đều vượt quá giới hạn độ sâu 512 từ của cấu hình 512×16. Do đó, Quartus Fitter vẫn bắt buộc phải cấp phát 2 khối M10K. Việc chọn kích thước 638 hay 800 từ không tiết kiệm được khối M10K nào, mà còn làm lãng phí thêm các bộ so sánh và trừ trong ALM để giải mã modulo.

Bài tập 4.4: Phân tích cạm bẫy căn lưới khung với tệp âm thanh có số mẫu lẻ

Đề bài: Xét một tệp âm thanh có số lượng mẫu lẻ $n_x = 32.001$ mẫu tại $f_s = 16$ kHz ($dx = 1/16000$ s, thời lượng $T = 2{,}0000625$ s). Thuật toán Praat sử dụng cửa sổ $T_w = 0{,}04$ s và bước nhảy danh định $dt = 0{,}01$ s.

(a) Tính tổng số khung phân tích n_{frames} và thời điểm tâm của khung đầu tiên t_1.

(b) Tính chỉ số mẫu bắt đầu window_start0$[0]$ của Khung 0. HPS cần loại bỏ bao nhiêu mẫu đầu tệp?

(c) Giải thích cơ chế phát sinh cạm bẫy làm tròn số dấu phẩy động (floor trap). Tại sao tại các khung như Khung 29 $\rightarrow$ 30, bước nhảy giữa hai khung liên tiếp bị co về 159 mẫu?

Lời giải chi tiết:

(a) Áp dụng công thức căn tâm khung thời gian của Praat:

$$n_{\text{frames}} = \left\lfloor \frac{T - T_w}{dt} \right\rfloor + 1 = \left\lfloor \frac{2{,}0000625 - 0{,}04}{0{,}01} \right\rfloor + 1 = \lfloor 196{,}00625 \rfloor + 1 = \textbf{197} \text{ khung} \tag{251}$$

Tâm thời gian của tệp âm thanh:

$$t_{\text{mid}} = 0{,}5 \times T = 0{,}5 \times 2{,}0000625 = 1{,}00003125 \text{ s} \tag{252}$$

Thời điểm tâm của khung đầu tiên:

$$\begin{aligned} t_1 &= t_{\text{mid}} - 0{,}5 \cdot n_{\text{frames}} \cdot dt + 0{,}5 \cdot dt \\ &= 1{,}00003125 - 0{,}5 \times 197 \times 0{,}01 + 0{,}005 \\ &= 1{,}00003125 - 0{,}985 + 0{,}005 = \textbf{0{,}02003125} \text{ s} \end{aligned} \tag{253}$$

(b) Xác định chỉ số mẫu bắt đầu cho Khung 0 ($t_1 = 0{,}02003125$ s, $x_1 = 0{,}00003125$ s):

$$\text{arg}_1 = \frac{t_1 - x_1}{dx} + 1{,}0 = \frac{0{,}02000000}{1/16000} + 1{,}0 = 320{,}0 + 1{,}0 = 321{,}0 \tag{254}$$

Chỉ số mẫu trái: $\text{leftSample}_1 = \lfloor 321{,}0 \rfloor = 321$. Mẫu bắt đầu 0-based:

$$\text{window_start0}[0] = \text{leftSample}_1 - 319 = 321 - 319 = \textbf{2} \tag{255}$$

$\Longrightarrow$ HPS cần loại bỏ đúng **2 mẫu đầu tiên** ($n = 0, 1$) trước khi đẩy dữ liệu vào FPGA.

(c) Xét bước nhảy giữa Khung 29 ($t_{29} = t_1 + 28 \times 0{,}01 = 0{,}30003125$ s) và Khung 30 ($t_{30} = t_1 + 29 \times 0{,}01 = 0{,}31003125$ s):

- Tại Khung 29: $\text{arg}_{29} = (0{,}30000000 \times 16000) + 1{,}0 = 4801{,}0 \Longrightarrow \text{leftSample}_{29} = 4801$. window_start0$[28] = 4801 - 319 = 4482$.
- Tại Khung 30: Về mặt toán học liên tục, $\text{arg}_{30} = (0{,}31000000 \times 16000) + 1{,}0 = 4961{,}0$. Tuy nhiên, trong phép tính số thực dấu phẩy động kép IEEE-754, sai số tích lũy của phép cộng làm cho đại lượng này bị lưu thành $4960{,}999999999999$. Hàm `floor` cắt cụt giá trị về số nguyên thấp hơn:

$$\text{leftSample}_{30} = \lfloor 4960{,}999999999999 \rfloor = \textbf{4960} \quad \text{(bị hụt 1 đơn vị)} \tag{256}$$

Mẫu bắt đầu tương ứng: window_start0$[29] = 4960 - 319 = 4641$.

Bước nhảy thực tế giữa hai khung bị co giật thành:

$$\text{hop}_{29 \rightarrow 30} = \text{window_start0}[29] - \text{window_start0}[28] = 4641 - 4482 = \textbf{159} \text{ mẫu} \tag{257}$$

Đây chính là bản chất của hiện tượng Floor Trap. Để phòng ngừa, HPS bắt buộc phải kiểm tra mảng định vị khung và ghi nhận cờ cảnh báo nếu bước nhảy không đều đặn 160 mẫu.